bitGen Studio

Instructions and shortcut keys

Version 2.780.0 · 14 November 2026

Local AI video, image, music and editing studio - ComfyUI, MiniMax H3, Wan, LTX, local LLMs

Produced and Developed by Tim Paul

Programmer - Claude Opus 5.5

Part 1 - Shortcut keys at a glance

Part 2 - Instructions (every docker, step by step)

Credits - at the end of Part 2

Shortcut keys at a glance

Keys work when the docker they belong to has the pointer or the focus, and never while you type in a box. In the image editor every menu shows its keys too, and **Ctrl+K** finds any command by name.

Anywhere in the app

Ctrl+K	Find anything - a docker, a setting, a button, a style, the guide, a shortcut (the in the menu bar too). Over the timeline's layers Ctrl+K still splits, in the image editor it is its own palette
Ctrl+Alt+D	Show / hide ▣ Docker - an empty docker the others drop into (side by side or stacked; ⇱ on a docker's title floats it, then drag it in by its title)
Ctrl+Alt+S	Open Settings (works while typing too - also when a theme hid the ⚙ button)
Ctrl+Alt+Shift+T	Back to the bitGen Default theme (Dark, no element styles, UI size 100 %); Settings ▸ Themes ▸ ↶ The theme before undoes it. ?theme=default in the address does it before the page paints
P	Show / hide the prompt docker (not while typing in a box; over the image editor P is its Pen)
Ctrl+P	Bring the prompt docker to the front: opened, unfolded, over whatever covers it, the prompt box ready to type in (typing in a box too; in Artist mode too - over the full-screen canvas)
S	Show / hide Basic Settings - a floating docker (not while typing; over the image editor S is its Clone Stamp)
L	Show / hide LLM tokens - a floating docker (not while typing; over the image editor L is its Lasso, over the timeline the shuttle)
1	Show / hide the timeline Tools & Properties window
2	Show / hide the negative prompt docker (not while typing; while the multi-camera dialog is open 1 - 9 cut cameras)
Space	Pause / play the preview - the timeline canvas, the live preview or the preview video (not while typing in a box)
Esc	Leave a maximized docker
Delete	Delete the selected gallery picture (to the recycle bin)
Enter	In the prompt box: send it to the chat
Shift+Enter	In the prompt box: a new line
Ctrl+Shift+Alt+E	Edit theme mode with the dialogs too (an open Settings stays open to be styled) - again to leave
Ctrl+S (in edit theme mode)	Save the theme and leave edit theme mode

Ctrl+Alt+Shift+B	Bug report - a screenshot of the window first, then the docker (works while typing too); again hides it

Lightbox (a picture or clip opened full size)

← / →	Previous / next
+ / −	Zoom in / out
0	Fit again

Timeline canvas (pointer over the canvas)

V	Move
T	Text
H	Hand
Z	Zoom
M	Select (for Fill)
B	Mask brush
Shift+P	Path tool (path layers)
Shift+C	Camera tool: drag = pan / tilt, Alt+drag = roll, Shift+drag = height, wheel = dolly, Shift+wheel = zoom
Space (hold)	Pan with the hand
Space (tap)	Play / pause
Delete	Remove the selected clip
F2	Rename the selected layer
← / →	Nudge the selected clip (Shift: the bigger step - timeline ⚙)
Enter	Set the selected clip's start time
Delete / Backspace	Remove the selected keyframe (in its popup)
Esc / Enter	Path tool: back to the Move tool
Alt+click	Path tool: remove a point
Shift+Q	✎ Freehand shape: draw a path (Alt = into the picked shape layer)
Shift+M	✒ Mask Pen: click = a corner, drag = a curve, the first point / Enter = closed, Esc = done
Alt+drag (Blender view)	Zoom - drag up / forward = in, down / back = out (in the image editor's Blender scene tools too)
Z (Blender view)	Zoom to the picked object (nothing picked = the whole scene)
Shift + right-drag	Zoom - forward (up) in, back (down) out, round the point pressed

Timeline layers (pointer over the layers, the ruler or the menu bar)

V	▸ Select tool
B	⇤ Ripple edit tool
N	⇹ Roll edit tool
Y	⇆ Slip tool
U	↔ Slide tool
I	In - the ruler selection starts at the position marker
O	Out - the ruler selection ends at the position marker
Ctrl+K	Split at the position marker (the selected clip, else the highlighted layer's)
↑ / ↓	Previous / next edit point
Home / End	The start / end of the timeline
M	Add a marker at the position marker (on a marker: its name, colour, comment - double-click a flag does the same)
Shift+M / Alt+M	Next / previous marker
Ctrl+Shift+E	Export the frame at the position marker as a PNG (the canvas too)
Ctrl+Shift+M	A clip marker on the selected clip at the position marker (it stays on its frame)
J / K / L	Shuttle the preview: backwards / stop / forwards - J or L again plays at 2x, 4x, 8x (anywhere in the timeline docker, the canvas too)
K + J / K + L	Hold K: one frame back / on

3D layers (a 3D layer selected)

G	Move widget (again = off)
R	Rotate widget
S	Scale widget

Image editor - tools

V	Move
M (Shift+M cycles)	Rectangular Marquee · Elliptical Marquee
L (Shift+L cycles)	Lasso · Polygonal Lasso · Magnetic Lasso
W (Shift+W cycles)	Object Selection (Segment Anything) · Magic Wand · Quick Selection Brush
C	Crop
I	Eyedropper
J (Shift+J cycles)	Spot Healing Brush · Healing Brush · Patch · Content-Aware Move · Red Eye
B (Shift+B cycles)	Brush · Pencil · Mixer Brush
S	Clone Stamp
Y	History Brush
E (Shift+E cycles)	Eraser · Background Eraser · Magic Eraser
G (Shift+G cycles)	Gradient · Paint Bucket
R (Shift+R cycles)	Blur · Sharpen · Smudge
O (Shift+O cycles)	Dodge · Burn · Sponge
P (Shift+P cycles)	Pen · Freeform Pen · Curvature Pen · Add Anchor Point · Delete Anchor Point · Convert Point
T (Shift+T cycles)	Horizontal Type · Vertical Type · Type on a Path
A (Shift+A cycles)	Path Selection · Direct Selection
U (Shift+U cycles)	Shape (rectangle, ellipse, polygon, line) · Custom Shape
H	Hand
Z	Zoom
F5 (F5 again cycles)	Curve tools: Bézier · Freehand · 2-Point Line · Polyline · 3-Point Curve · B-Spline · Pen · Artistic Media
F10	Shape tool (nodes)
F6 (F6 again cycles)	Shapes: Rectangle · 3-Point Rectangle · Ellipse · 3-Point Ellipse · Polygon · Star · Complex Star · Spiral · Graph Paper · Common Shapes
F8 (F8 again cycles)	Parallel · Horizontal / Vertical · Angular Dimension · 3-Point Callout · Connector

Image editor - File menu

Ctrl+N	New
Ctrl+O	Open
Alt+Ctrl+O	Place as Layer
Ctrl+S	Save to Images
Shift+Ctrl+S	Save as PSD
Alt+Ctrl+S	Save Project (.bgi)
Shift+Delete	Delete Layer
Ctrl+;	Guidelines
Alt+Ctrl+;	Lock Guidelines
Shift+Ctrl+;	Snap
Alt+Shift+Ctrl+W	Export As
Ctrl+W	Close
Alt+Ctrl+W	Close All

Image editor - Edit menu

Ctrl+Z	Undo
Shift+Ctrl+Z	Redo
Ctrl+X	Cut
Ctrl+C	Copy
Shift+Ctrl+C	Copy Merged
Ctrl+V	Paste
Shift+Ctrl+V	Paste in Place
Shift+F5	Fill
Ctrl+T	Free Transform
Alt+Shift+Ctrl+C	Content-Aware Scale
Ctrl+,	Preferences

Image editor - Image menu

Shift+Ctrl+L	Auto Tone
Alt+Shift+Ctrl+L	Auto Contrast
Shift+Ctrl+B	Auto Color
Alt+Ctrl+I	Image Size
Alt+Ctrl+C	Canvas Size

Image editor - Layer menu

Shift+Ctrl+N	Layer
Ctrl+J	Layer via Copy
Shift+Ctrl+J	Layer via Cut
Alt+Ctrl+G	Create / Release Clipping Mask
Ctrl+G	Group Layers
Shift+Ctrl+G	Ungroup Layers
Shift+Ctrl+]	Bring to Front
Ctrl+]	Bring Forward
Ctrl+[	Send Backward
Shift+Ctrl+[	Send to Back
Ctrl+/	Lock All
Ctrl+E	Merge Down
Shift+Ctrl+E	Merge Visible
F12	Outline Pen
Ctrl+Q	Convert to Curves

Image editor - Select menu

Ctrl+A	All
Ctrl+D	Deselect
Shift+Ctrl+D	Reselect
Shift+Ctrl+I	Inverse
Shift+F6	Feather
Q	Edit in Quick Mask Mode
Alt+Ctrl+R	Select and Mask

Image editor - Channels panel

Ctrl+2	RGB (all channels)
Ctrl+3	Red channel alone
Ctrl+4	Green channel alone
Ctrl+5	Blue channel alone
Ctrl-click a channel	its lightness / the saved selection as a selection

Image editor - curve tools (F5) and Shape tool (F10)

click / drag	Bézier: a cusp node / a smooth node (curve)
C / S while dragging	Bézier: the node cusp / smooth
Ctrl	curve tools: 15° steps
click the first node	close the curve
double-click · Enter · Space	done
Esc	drop the line being drawn
C / S / Y	Shape tool: cusp / smooth / symmetrical node
+ · Insert	Shape tool: add a node
- · Delete	Shape tool: delete the picked nodes
double-click the line / a node	Shape tool: add / delete a node
arrows (Shift ×10)	Shape tool: nudge the picked nodes

Image editor - Filter menu

Ctrl+F	Last Filter
Shift+Ctrl+X	Liquify

Image editor - View menu

Ctrl+=	Zoom In
Ctrl+-	Zoom Out
Ctrl+0	Fit on Screen
Ctrl+1	100%
Ctrl+R	Rulers
Ctrl+'	Grid
\	Mask Overlay
F	Full Screen Mode
Ctrl+Alt+0	Zoom to Selection

Image editor - Window menu

F7	Layers
Tab	Show / Hide Panels
Alt+F9	Actions
Alt+F7	Dockers: Transform
Shift+Ctrl+A	Dockers: Align and Distribute
Alt+Enter	Dockers: Object Properties
Ctrl+Tab	Next Document
Shift+Ctrl+Tab	Previous Document

Image editor - Help menu

F1	Image Editor Help
Ctrl+K	Command Palette

Image editor - single keys and clicks

Q	Quick Mask on / off
X	Swap the foreground / background colours
D	Default colours (black / white)
F	Full screen
K	Selection brush
1 … 0	Brush opacity 10 … 100 % (paint tools)
[/]	Brush size smaller / bigger
Tab	Show / hide the panels
Shift+Tab	Show / hide the panel groups
****	Mask overlay: red where the mask hides (always while painting a mask)
Space (hold)	Pan
Enter	Apply the shown result / close a polygon
Esc	Discard results / cancel a polygon / leave the mask view
Shift / Alt / Shift+Alt	Selection tools: add / subtract / intersect
Click / Shift / Alt	Object Selection: the thing / add one / take one away (drag = a box)
Ctrl+click a layer	Pick it for Compose / → References (in order)
Ctrl+click a thumbnail	Load its transparency (or mask) as the selection
Alt+click a mask	View the mask itself
Shift+click a mask	Disable / enable the mask
Alt+click the eye	Show only this layer
O	Sketch: the Offset tool (F = the other side, Enter = the typed distance)
Shift + middle-drag	Orbit the picked 3D body (or 3D view layer)
Middle-drag	Pan the canvas
Shift+right-click	In Artist mode: back to normal (Ctrl+Esc / Ctrl+Shift+F too - on Windows Ctrl+Esc can open the Start menu); no context menu opens
Alt+P	In Artist mode: the colour palette at the pointer (Alt+P again closes it)
Ctrl+B	In Artist mode: the brushes at the pointer
Alt+G	In Artist mode: the Layers panel + Pending over the canvas - pick a layer, Ctrl+P for the prompt docker, Generate / ▾ Edit; Alt+G again hides them
Shift + right-drag	Zoom - forward (up) in, back (down) out, round the point pressed (Artist mode too; a Shift+right-CLICK leaves Artist mode)

Sketcher - over the docker

Space	Select
L / R / C / N	Line / Rectangle / Circle / Polygon
P	Push/Pull
M / Q / S	Move (Ctrl: a copy) / Rotate / Scale
F	Offset
T / B / E	Tape / Paint / Eraser
O / H	Orbit / Pan (the middle / right button always)
→ ← ↑ / ↓	Keep the red / green / blue axis / let it go (Shift: the axis shown)
G	Group what is picked (a group picked: explode it)
Shift+Z	Everything in view
U / W	Room / Wall
D / I	Door / Window (on a wall)
J / K	Floor / Ceiling on-off
Y	Roof (Shift+click: more rooms under it)
X	Light (the type in the panel)
V	Stairs
Alt+G	Terrain - the ground: Raise / Lower / Smooth / Flatten with a brush, or draw contour lines the ground follows (the mode in the panel)
[/] (Terrain)	The brush smaller / bigger; Shift+drag: the other way (raise ⇄ lower); Alt+click (Flatten): take the height there
Drag a room (Select)	Its wall snaps onto a neighbour's - one shared wall
Delete	Erase what is picked
Esc	No tool and nothing picked - Orbit, in the 3D view, the floorplan and a camera's view (a wall run is kept); in Orbit: ➤ Select. Never docks the maximized Sketcher
a number	Into the Measurements box - Enter applies it (3,2 · 150cm · 36s)
Ctrl+Z / Ctrl+Y / Ctrl+S	Undo / redo / save
Double-click a picked camera	Move arrows ⇄ rotate rings
Shift + right-drag	Zoom - forward (up) in, back (down) out, round the point pressed (the floorplan too)
Ctrl+click	➤ Select: add to what is picked - faces, lines and building parts (a picked part: takes it out); drag one of them to move them all, Delete erases them all
Shift+click	Paint: every wall on that side (a room's insides / every outside wall) - a click paints the one wall side

Instructions

Contents

The window

One clip from a prompt

Characters

A film on the scene board

A music video

LTX 2.5 (first / last frame, lipsync, audio to video)

LTX 2.3 and 3D render → real (Blender → photoreal)

Sketcher - 3D modelling, rooms and cameras (2.619 - 2.653)

Editing on the timeline: speed, colour, sound, mix and captions (2.406.0)

Edit tools, colour panel, export queue and exchange (2.407.0)

Edit by transcript, scene cuts, karaoke word fixes (2.408.0)

Override style for every video model (2.409.0)

Effects and ◐ adjustment layers (2.409.0)

Karaoke: word nudges, the line under the marker (2.433.0)

Karaoke and Tools as dockers beside the canvas (2.410.0)

⌨ Timeline keys, markers, In / Out export, frames (2.410.0)

Text effect presets (2.411.0)

J / K / L, AV1, 10-bit, the graphics card (2.411.0)

Smaller editing jobs (2.412.0)

Partly-done editing items finished (2.413.0)

Nested sequences, the fuller XML, FCPXML, whole-sequence transcripts (2.414.0)

Track mixer, distort effects, the transitions library, colour match (2.415.0)

Masks + tracking, multi-camera, proxies (2.416.0)

HDR, AAF, the sound of reversed clips, motion graphics templates (2.417.0)

Real-time playback: the render bar, ⚡ Render, , playback resolution (2.418.0)

One prompt docker, ⊖ the negative prompt docker (2.419.0)

AI inside the timeline (2.420.0)

44 more filters - the image editor's gap, round B (2.481.0)

More adjustments and Apply Image - the image editor's gap, round A (2.480.0)

JPEG XL (.jxl) pictures (2.479.0)

Search in every dropdown list (2.478.0)

The public beta: the repository, what is experimental, reporting a problem (2.477.0)

Unreal lights, volumetric lights and fog - in the image editor and the timeline (2.476.0)

The Unreal camera in the image editor: move it, lens, f-stop, exposure, presets (2.475.0)

No tool, the selection box and several layers at once (2.474.0)

The Feature Reference PDF (2.473.0)

The Unreal Engine 5.8 bridge: Unreal layers, MetaHuman layers, the Unreal camera (2.472.0)

Parametric 3D bodies: Extrude, Revolve, Fillet, Chamfer, Shell, Split Body (2.471.0)

Picking like a drawing program + the CAD toolbar (2.470.0)

Linked smart objects, PSD smart objects kept, .atn actions (2.469.0)

Actions: brush strokes + hand-made selections recorded, conditionals, Image Processor, droplets (2.468.0)

Editing single colour channels; spot channels (2.467.0)

OpenType features, optical kerning, type to outlines, text in a shape, round corners on any path (2.466.0)

Puppet pin depth + rotation; the Patch tool's Content-Aware mode (2.465.0)

Sketch layers: typed sizes, the Measure tool, units, DXF / SVG files (2.464.0)

Sketch modify tools: trim, extend, fillet, offset, mirror, patterns, polygon, slot, splines (2.463.0)

Sketch layers: a CAD sketch with constraints and dimensions (2.462.0)

Poses on the timeline, and the Daz live link (2.461.0)

3D layers: a Daz / Blender scene in the image editor (2.460.0)

Tools named by what they do (2.459.0)

Auto, floorplan and chain dimensions (2.458.0)

Dimension groups (2.457.0)

The StoryBoard (2.456.0)

Edit theme mode leaves dialogs working (2.455.0)

The GPU compositor (2.420.0)
Wan 2.1 and Wan 2.2
VR World (a performer inside a 360° / 180° picture)
Character swap
The H3_Viggle style
The SFX dock
Save and Save as (preview dialog and gallery)
Transitions between song segments (bridge scenes)
Render size on the scene board
The timeline's preview canvas
Render size with H3
Scenes and the song
Nudging timeline clips
Override style and the StoryBoard
Recycle bin and resetting a project
Karaoke: lyrics with a bouncing ball
Saving and opening a project
Starting a new project
Saving and clearing the prompt chat
Screenshots and screen recordings (promos, tutorials)
CFG on MiniMax H3
Speech with emotion in a character's voice (no song)
How bitGen writes a MiniMax H3 prompt with speech
Lip sync on a music-video board
The prompt LLM and your references
Reference videos and sounds in a prompt render
Append / Prepend a reference video (2.388.0)
The floating prompt docker (2.389.0)
Basic Settings and LLM tokens float (2.393.0)
Choosing the frames of a reference video
Images, ControlNet and face fix
The image editor (with AI)
Music, stems and voices
Timeline and export
The menu bar
Path layers (2.387.0)
The panel column is a docker; every tab group floats (2.454.0)
Frames lock their contents (2.453.0)
Move tool: scale / rotate handles, transform options (2.452.0)
The editor's Edit / Generate tabs drive the prompt docker, Del, quick prompts (2.451.0)
Warp grid + split lines, Liquify extras (2.450.0)
Per-character type (2.449.0)
The Content-Aware Fill workspace (2.448.0)
Artboards (2.447.0)
Rulers, guidelines and snapping on the timeline's preview canvas (2.446.0)
Grid buttons, the Grids & Guides settings (2.445.0)
The image editor's Edit / Generate run from the prompt docker, ⚙ editor settings (2.444.0)
Edit theme mode: style anything on screen (2.443.0)
Frames: layers inside a shape, Edit Frame, generate into a shape (2.442.0)
CAD dimensions: exact points, live ends, a scale (2.441.0)
Rulers, guidelines and snapping (2.440.0)
The property bar, corner radii, Add / Subtract into a shape, line width and style (2.439.0)
Floating dockers, named dockers, dialogs in the middle (2.438.0)
Generate fill in a shape or a mask, the pattern library (2.437.0)
Project files: .bgi, .bgv, .bgp (2.436.0)
Saved themes and PDF fonts (2.435.0)
Themes and the UI size (2.432.0)
PDFs: the guide and the brochure (2.392.0)
3D text and 3D models on the timeline canvas
Generate progress
Projects, queue and gallery
Grouping and deleting assets
Adding files with the + tiles (2.394.0)
The file dialog
The guide and help in Chinese and Japanese (2.703.0)
The app itself in Chinese and Japanese (2.705.0)
Language packs (2.715.0)
Faster first frames and the speed-up fixes (2.717.0)
Models not found - checked before a render (2.719.0)
Free VRAM only when the model changes (2.722.0)
Floating dockers: the one you used last is on top (2.724.0)
A song in the preview: a visualization, and Next plays the next song (2.725.0)

The camera path: pan, tilt, roll, dolly, height, a 3D path (2.399.0)
Upscaling a video with SeedVR2; the queue's running card (2.400.0, Enhance 2.401.0)
The video style in Basic Settings (2.401.0)
The statusbar's docker buttons, the 3D toolbar (2.386.0)
Tools & Properties, keyframes, 3D widgets, lights, materials, Blender (2.385.0)
Seeds and render details
The position marker, layer colours
Layer names
Dock sizes, pins and maximize

Docker configs: a saved layout for each set of dockers (2.392.0)
▾ Which picture the prompt docker works from: references or layers, and Compose (2.543.0)
A Blender layer: the timeline's scene tools in the image editor (2.542.0)
▾ Generate, Edit, Refine, Fill, Replace background, Remove, and Blender mode (2.541.0)
Update bitGen Studio from GitHub (2.540.5)
Settings ▸ Add-ons: optional engines installed in the app (2.540.4)
Blender and Unreal renders made real with the reference docker's pictures (2.540.0)
Reference descriptions, the wall / floor / furniture / light types, references in the image editor (2.539.0)
Civitai downloads into each type's folder, the folders, Sync (2.538.0)
Maximize beside the side dockers (2.538.0)

Models: pages, folder tabs, card size, the model info page and Civitai info for every model (2.537.0)
Veda sparse attention - a faster MiniMax H3 (2.536.0)
FLUX 3 Klein needs ComfyUI 0.41 or newer (2.535.0)
Test mode, and new renders joining the open preview (2.534.0)
Dropdown lists inside a dialog open in front of it (2.533.0)
Tests tick themselves, docker title bars (2.532.0)

A missing model file says where to get it (2.531.0)

Find anything - the Ctrl+K command palette and Help (2.530.0)
The Style doctor - which styles work, and why the others don't (2.527.0)
Workspaces - Basic, Image editing, Video editing, Music video, Expert (2.526.0)

The Neural blue theme and the bitGen brain picture (2.726.0)
The brain comes alive; upscales that came back as the source (2.727.0)
1920 × 1080 exactly, and RelayVSR (2.728.0)
NitroFusion2: pictures in one step (2.729.0)

Motion from a reference video: FlowHMR (2.730.0)

Sheet music on the staff, saved as a PDF (2.731.0)
The paper under the sheet music (2.732.0)
Enhance on a picture: upscale an image (2.733.0)
▶ Playing the sheet music, the notes lit as they sound (2.733.0)
The words under the sheet music (2.734.0)

The words lit as they are sung (2.735.0)

The Sheet Music tab's layout (2.736.0)

Parts and the mix: each instrument and the vocals (2.737.0)
The song in its layers, as waveforms (2.738.0)
Create: a song's layers and its sheet music in one press (2.739.0)
⬇ Missing parts download when they are needed (2.740.0)
ACE-Step 1.5, and the music styles on the Audio tab (2.741.0)
Every Basic Settings tab keeps its own settings (2.742.0)
☁ Rendering on a RunPod Serverless endpoint (2.743.0)
☁ On RunPod Serverless, only the styles it can render (2.744.0)
Basic Settings ▸ GPU: what runs on the remote GPU (2.745.0)
Remote styles: each style says where it runs (2.746.0)
GPU on Local by default, and an update check that finishes (2.747.0)
Your API keys are kept, encrypted, in the settings folder (2.748.0)
☁ Ready-made remote styles for RunPod's worker images (2.749.0)
The splash is the bitGen brain, with the logo (2.752.0)
The LLM on OpenRouter; "Remote GPU" is now "Remote generation" (2.753.0)
☁ Use remote generation, the Remote generation docker and pod shortcuts (2.754.0)
☁ Pods keep working after 15 November; loading pod, pod loaded, stopping pod (2.755.0)

A model in another ComfyUI folder is still found (2.525.0)
A style's model can be a .gguf or a .safetensors (2.523.0)
Reporting a bug (2.522.0)
Fine-tunes in the trainer: checkpoints, the assistant (LLM) and the vision model (2.521.0)
The LoRA trainer, peak / off-peak prices, the usage log, Remote GPU, packages in Settings (2.520.0)
Setup asks for your hardware and picks a package; the daily supply charge (2.519.0)
Queue presets, the Model manager, Civitai downloads, energy cost, the Testing docker (2.518.0)
Power draw and clock speeds, the energy a job used; Anima image styles (2.517.0)
Keyframes on any setting; Krea 2 image styles (2.516.0)
Cut out a subject, solve the camera, remove an object; theme edit mode for dialogs (2.515.0)
Edit theme mode: every button and its words (2.623.0)
Edit theme mode: resize, move, hide, rename; leaving saves; folding dockers (2.514.0)
Load UE project, the timeline's frame size bar (2.513.0)
Lottie (JSON) export (2.512.0)

Variable mask feather, data-driven animation and template fields (2.511.0)
2D layers in 3D: cards, a moving camera, depth of field, shadows (2.510.0)

Motion sketch, the wiggler and the smoother (2.509.0)
Audio → keys, waveform and spectrum layers (2.508.0)
Puppet pins and mesh warp (2.507.0)

Particles and light rays (2.506.0)

Generator layers and distortions: fractal noise, gradient ramp, turbulent displace, displacement map (2.505.0)
DLSS 5: enhance, upscale and frame generation for videos (2.504.0)
RTX VFX: NVIDIA RTX Video finishing and upscaling (2.708.0)
16 GB mode: MiniMax H3 on a 16 GB card (2.708.0)

The bitGen Community: help and friends (2.710.0)

Long video: one MiniMax H3 run up to 30 seconds (2.503.0)

Remote servers, RunPod Serverless or Pods, and the PODS docker (2.756.0)
Every RunPod template as a card (2.756.1)

Joining the Community is for people (2.756.3)
Settings ▸ API - render on an online service (2.757.0)
P_ pod styles and a pod's loading bar (2.758.0)

⬇ Models onto a pod, and pods for a style (2.759.0)

The Audio docker, prices in your currency, and My pods (2.760.0)

Pods as items: what a pod is for, files onto it, its cost, and pods made again (2.761.0)
One checkpoint per pod - the pod becomes the style (2.762.0)
Your styles on your pods (2.763.0)

Your custom nodes on a pod, and what the pod is doing (2.763.1)
The PODS docker opens for a style with no pod (2.763.3)
The PODS docker: what a pod is doing, and every detail before it is made (2.763.4)
The RunPod docker, and dockers inside your own dockers (2.763.5)
Pod presets, and several styles on one pod (2.764.0)
Basic Settings ▸ GPU on Local shows nothing remote; a model sent to a pod shows how far it is (2.764.1)
Only the custom nodes your workflows use go on a pod (2.764.2)
A style's custom nodes come from their git address (2.764.3)
Joining the community: why the email code (2.764.4)
Dock a docker above or below another one; floating dockers stay in front (2.764.5)
Pod downloads that stop at 0%, and gated models (2.764.6)

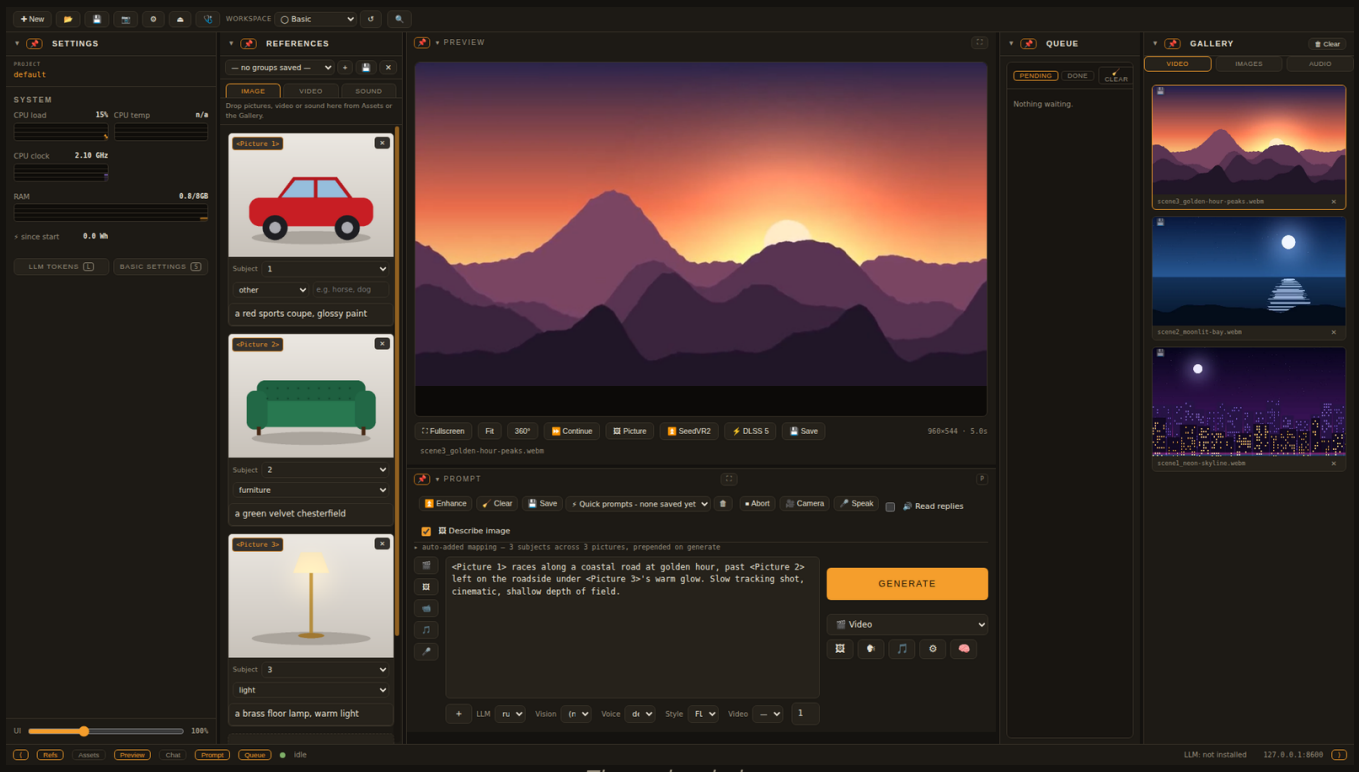
Checkpoints and LoRAs from Civitai straight onto the pod (2.765.0)
Remote models: the list your pods use, ticked pods and a bar for each transfer (2.766.0)
Sending to a pod that is busy, Override in remote mode, Civitai filters (2.766.1)
Local or Remote in Basic Settings switches everything over (2.766.2)
Files the pod's ComfyUI can see, and an old ComfyUI on the pod updated (2.766.3)

FLUX 3 Image Klein, and edit image mode lists only edit styles (2.502.0)
Time remapping on a curve (2.501.0)
Motion blur, Posterize Time, Echo (2.500.0)
Tracking and corner pin (2.499.0)
Bézier masks (2.498.0)
Expressions (2.497.0)
Freehand shapes, Merge Paths, text on a path, roving keyframes (2.496.0)
Colour blend modes, and video layers that follow a parent (2.495.0)
The CAD workings refined: the CAD button, Offset, the view cube, picking profiles (2.494.0)
Text animators (2.493.0)
Shape layers and their operators (2.492.0)
Blend modes and track mattes (2.491.0)
Parenting and null objects (2.490.0)
The Graph Editor, Easy Ease, the anchor point and curved motion paths (2.489.0)
PDF pages added, taken out and moved; Select Subject without a download - round H (2.488.0)
Image ▸ Mode, Proof Colors, Gamut Warning, CMYK TIFF - round G (2.487.0)
Auto-Align Layers, Auto-Blend Layers, Photomerge, Merge to HDR - round F, part 2 (2.486.0)
More file types, File Info, Print, Contact Sheet, Crop and Straighten Photos - round F (2.485.0)
Brush Settings, brush presets and .abr brushes - round E (2.484.0)
Navigator, Histogram, Info, measuring tools, Grow / Similar / Focus Area, Matting - round D (2.483.0)
The Develop Filter and Lens Correction - the image editor's gap, round C (2.482.0)
A pod that still lacks a node stops the render first (2.766.4)
The LLM, Vision LLM and video style in remote mode (2.767.0)
Remote options in their docker, and the prompt docker's model lists in remote mode (2.768.0)
A new pod's ComfyUI is updated automatically; missing files are looked for (2.769.0)
Pods bitGen makes set themselves up (2.769.1)
Only your live pods in the "no pod yet" box (2.769.2)
LLM Local / Remote, OpenRouter lists and All (2.770.0)
A pod whose ComfyUI crashed (2.770.1)
Costs docker and Statistics (2.771.0)
ComfyUI installed on a blank pod, and what a pod is doing (2.772.0)
Remote mode: the pod's stats, one mode's styles, uploads that finish (2.773.0)
A render that failed on its pod goes again; a template's ComfyUI waited for (2.774.0)
Text to speech, speech to text and song separation on a pod (2.775.0)
Every type Local or Remote, in sections (2.776.0)
Use model and Use temporary LoRAs (2.777.0)
A file that did not get onto the pod: Try again? (2.778.0)
When a pod cannot be created (2.779.0)
A pod is a bitGen pod (2.780.0)
Tips
Credits

A tour of the studio, from a single prompt to a finished film. If it is not installed yet, start with INSTALL.md.

- The window
- One clip from a prompt
- Characters
- A film on the scene board
- A music video
- Character swap
- Choosing the frames of a reference video
- Images, ControlNet and face fix
- Music, stems and voices
- Timeline and export
- Sketcher - 3D modelling, rooms and cameras
- Projects, queue and gallery
- The file dialog
- Tips
- Credits

The window



The main window

Area	What it is
Left	system stats, and two buttons that show the floating **LLM tokens** (L) and **Basic Settings** (S) windows. Basic Settings holds the generation mode (Ref · F/L · Text · Swap), resolution, aspect, seconds, steps, CFG, seed, format, LoRAs, ControlNet, face fix
References	pictures, videos and sounds that ride the next generation (drag them in from Assets or the Gallery)

Assets	the project's images, videos and sounds — drop files here to import them
Centre dockers	Preview, Timeline, **Video** (the scene board), Music, Stems, Image editor, ComfyUI, **Prompt**, Prompt chat, Console
Queue	pending and finished jobs, with their logs
Gallery	everything you have rendered

The buttons along the bottom show and hide each docker; every docker can be collapsed (▾) or maximized (— Esc restores). The layout is remembered.

One clip from a prompt

1 In **Basic Settings** (press **S** to show it) pick a mode:

– **Ref** (ref2va) — up to 9 reference pictures; each tray slot is a `<Picture n>` you can refer to in the prompt (click the slot's badge to insert its tag). Videos and sounds in the References dock ride too.

– **F/L** (fl2va) — one picture = the first frame, two = animate from the first to the last.

– **Text** (t2va) — no references.

– **Swap** — see Character swap.

– **Type** under each Ref picture (person, face, clothing, … , 2.340.0: **first frame**, **last frame**): set a picture's Type to **first frame** and Generate renders **First/Last** (fl2va) from it — to the picture typed **last frame** when there is one. The other pictures are not used (the console says so). Without a first frame the render stays **Ref** (ref2va, at most 9 pictures) and a lone last frame counts as an ordinary reference. LTX 2.5 styles use the same Types: the first frame goes first, the last frame last.

2 Choose resolution, aspect and **Seconds**. MiniMax H3 renders 24 fps; a clip longer than your card can do in one run is split and joined seamlessly.

3 Write the prompt and press **GENERATE** (the dropdown under it says what Generate makes: Video, Image, Music, Speech, ⚙ Graph, Chat).

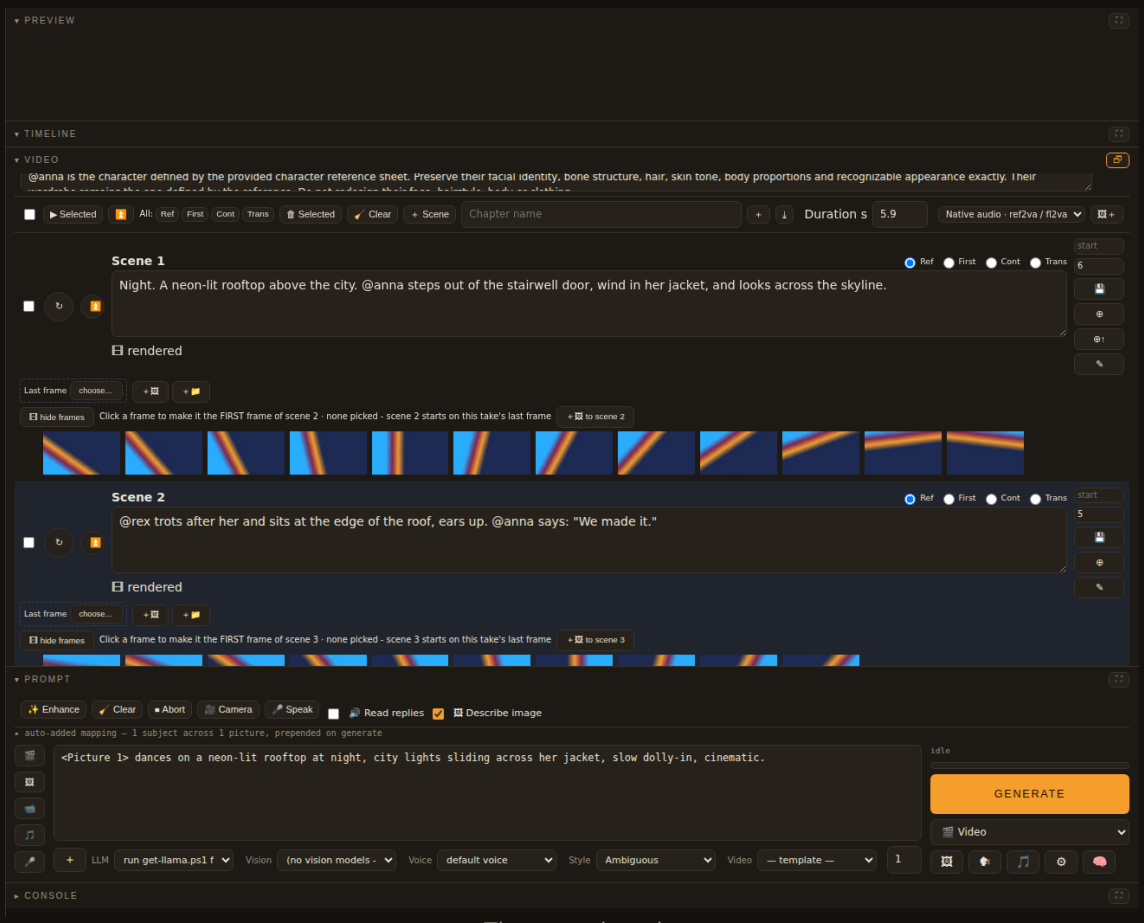
`/br/` on its own line splits the prompt into several jobs that run one after another. Press **Enter** in the prompt to have the local LLM improve it. **Enhance** upscales the selected video (see *Upscaling a video with SeedVR2*).

Characters

In the **Video** docker's **Characters** section:

- **NEW** — a character with a name, details, a **character sheet** (front / side / back picture) and optionally a **voice clip**. Its `@tag` (e.g. `@anna`) is how prompts and the screenplay refer to it.
- **NEW** — make a character *from one picture*: upload a photo or a render and a reference sheet is generated (your own pose sheet from the pose_sheets folder, photo or illustrated style), then **Keep** it.
- **Actors** — choose who is in the current scene; only they ride the references and the screenplay.
- Each character keeps a dataset folder for **RefMod** (identity models), clothing entries that follow the character from scene to scene, and its own voice for native-audio dialogue.

A film on the scene board



The scene board

1 **Workflow** (top of the StoryBoard): *Native audio · ref2va / fl2va* — each scene is its own clip with video **and** sound.

2 **Scenes**: type how many scenes you want, or leave it blank — then `projects\<name>\scenes.txt`, the number in your request, or the LLM decides.

3 In the prompt, press (*create screenplay*) and describe the story, e.g. *create screenplay: Anna and Rex reach the rooftop at night...* The LLM writes the scenes onto the board, with the characters speaking their lines (`@anna says: "We made it."`). No song = a film: the model makes the dialogue, effects and ambience itself, in each character's cloned voice.

4 For each scene choose how it follows the one before:

Ref	a new shot from the references (character sheets, location pictures); on Native audio the previous take's end rides as a transition video
First	starts **exactly** on a frame of the previous take (fl2va — no character sheets)
Cont	opens on that frame **with** the cast sheets, so the character stays consistent
Trans	travels from that frame to the new location's picture

All: sets every scene at once.

1 Press a scene's round ▶ to render it, or tick scenes and ▶ **Selected**. Rendering continues in the background; you can keep editing.

2 Under each take is its **filmstrip** — click a frame to make it the first frame of the next scene. ✎ **First frame (Qwen 2.1)** (2.337.0) — open the accordion under a scene's First / Last frame slots to *make* its first frame with Qwen Image 2.1: tick the **characters** (their sheets), pick a **location** picture and up to three **props**, type a line of direction (or leave it empty to use the scene's own prompt) and press ✎ **Make first frame**. **Start from** chooses the picture Qwen edits: the location (the characters are placed into it), **the frame the scene continues from** (same pose and position, the location changed — for Cont / First / Trans scenes), or the scene's own First picture;

Auto picks the continue frame when there is one. The result lands in the scene's **First** slot (a Ref scene becomes First). It needs the Qwen Image 2.1 models (Setup → Models → Image editing).

3. The take bar under the board: ▶ watch · ✔ keep · ↻ redo · ⇄ swap the character. upscales a take you like with the ⚙ upscaler (no new render).
4. **Compile** (at the top of the StoryBoard) joins the rendered takes into one film, in scene order. From 2.391.0 the board no longer has its own Produce and ⇥ Timeline buttons.

The **info line** at the bottom of the StoryBoard is the same height as the status bar. It shows the docker's latest message; hover it to read the whole text.

The Scenes dropdown (2.391.1). The scene board (the shared prompt, the scene menubar and the scene cards) sits in a **Scenes** dropdown like Characters and Locations. Click its title to open or close it; the title counts the scenes and how many are rendering.

Inside it, the **scene picker** above the cards shows **All scenes** or just one scene's card. Each scene is listed with its state (rendering, ● rendered, ✔ kept, ✕ failed) and the first words of its prompt. ◀ and ▶ step to the previous or next scene. The picker remembers your choice.

While a scene renders, its card says **rendering step n of total**, with a progress bar and the time so far. Before the first step it says **rendering - loading the models**.

A full **PDF guide** with every shortcut key is in `docs\bitGen-Studio-Guide.pdf`.

⚙ (render settings) sets the scenes' size, steps, LoRAs, the Sage / Spectrum / Comfy Kitchen accelerators, the upscaler and the tail frames used for continuity. **New** / **Save** / **Load** at the top of the StoryBoard start, save and reload whole movies (cast, chapters, scenes and renders).

A music video

1. **Load** a song (or pick it in the **Audio** dropdown). Its waveform opens auto-segmented; adjust the cuts and — one scene per segment, each with its start and duration.
2. Press (*create music video*) in the prompt dock and describe it; the writer uses the song's segments as the scenes and makes the characters sing their lines.
3. Every scene lip-syncs its own window of the song.

LTX 2.5 (first / last frame, lipsync, audio to video)

LTX 2.5 renders through Tim's **LTX 2.5 All-In-One** canvas (`comfy_workflows/LTX25_All_In_One.json`), one mode per clip. Pick the **LTX 2.5** video style (Settings → Video styles has it as a starter).

From the prompt dock. With the LTX 2.5 style picked, two dropdowns appear beside the template list:

- **LTX mode** — *Auto* picks from what the reference dock holds:

reference dock	mode
a video (+ picture 1)	video to video (style transfer, pose from the video)
audio + 3 pictures	first / middle / last (pictures 1, 2, 3)
audio + 2 pictures	first / last lipsync
audio + 1 picture	lipsync (picture 1 sings the audio)
audio only	audio to video
2 pictures	first / last frame
1 picture	image to video (picture 1 is the first frame)
nothing	text to video

Or choose the mode yourself; a mode missing its pictures / audio says what to load. Only the first audio and the first video in the dock are used.

- **Transition** — the *ltx2.3-transition* LoRA. *Auto* turns it on only for the modes that travel between pictures (first / last, first / last lipsync, first / middle / last).

The **Seconds**, **Width / Height** (rounded to multiples of 32) and **Seed** fields apply; clips are 24 fps, 1–30 s.

On the scene board. Set the StoryBoard's **Workflow** to **LTX 2.5**. Each scene's card decides the mode:

scene	no song	with the board's song
Ref (nothing set)	text to video	audio to video
First / Cont (or a First picture)	image to video from that frame	lipsync on the scene's slice of the song
Trans (or a Last picture)	first / last frame: from the continue frame to the Last picture (else the location picture)	first / last lipsync

Which workflow runs (2.401.5). *Text to video*, *image to video* and *first / last frame* run on **Comfy-Org's own LTX 2.5 templates** (`comfy_workflows/ltx25_official_t2v.json`, `_i2v`, `_flf2v`). They use only nodes built into ComfyUI, so nothing has to be installed from the Manager. The app fills them in for you: your prompt, seconds, size, seed, the picture(s), your style's LoRAs and negative. If your model is the GGUF, it is loaded with ComfyUI-GGUF. They need ComfyUI's LTX 2.5 files: the transformer (int8 or the GGUF), the Gemma 4 12B text encoder, the video and audio VAEs, and the x2 upscaler (first / last frame does not need the upscaler). If one of them is missing, the job log says which one and the canvas renders the clip instead. The canvas keeps the modes only it has (audio to video, lipsync, first / last lipsync, first / middle / last, style transfer), LTX Ripple, the timeline camera's exact path, and the *Transition* LoRA when you set it to *On*. The job log shows which workflow ran: "on Comfy-Org's own template …" or "converted LTX25_All_In_One.json …". Keep ComfyUI up to date: the templates use newer core nodes (TextGenerateLTX2Prompt, ComfySwitchNode, ResizeImageMaskNode).

✎ First frame (Qwen 2.1) works here too — make the opening picture, then render. @tags in the prompt become the characters' names (LTX has no reference sheets; the look comes from the first frame). upscale is not available on LTX takes yet.

Models and nodes. *Settings → Models → Model catalog* (or the setup wizard's Models page) → *Video - LTX 2.5*:

- the Q4_K_M GGUF (15 GB, what the canvas loads) or the int8 model;
- the Gemma 4 12B text encoder and the Gemma 4 E2B one;
- the video / audio VAEs, the TAE preview VAE and the spatial upscaler;
- (2.341.0) the canvas's LoRAs: OmniNFT RL, Transition, IC-LoRA Union Control (style transfer), IC-LoRA Detailer, and LTX Ripple.

The Lightricks repos are gated: accept their licences on Hugging Face and put your token in the catalog. **Download with** the Hugging Face CLI (**Install hf CLI** adds it) for the big files. The canvas needs the node packs ComfyUI-GGUF, KJNodes, VideoHelperSuite, rgthree, mxToolkit, Easy-Use and controlnet_aux (Setup → Node packs); a missing one is named, with its pack, when the job starts. Since 2.401.2 the canvas no longer needs ComfyMath (it has gone from GitHub) or the pysssss math node: its number conversions and formulas run on ComfyUI's own **Math Expression** node. If you re-export the canvas from your own ComfyUI with those nodes still in it, the app converts them when the job starts. Since 2.401.3 it no longer needs ComfyUI-MediaMixer either: its "last frame" pick runs on KJNodes (Get Image Range From Batch), which the canvas already uses.

Every video job (MiniMax H3, Wan, LTX, the swap, Viggle, SeedVR2) now shows ComfyUI's live progress: which node is running and "step n of total" while it samples. A long silence with "running UNETLoader" or a loader node means ComfyUI is still loading the model; Wan 2.2's two 14B models can take several minutes the first time.

LTX 2.3 and 3D render → real (Blender → photoreal)

Pick the **LTX 2.3** video style (Settings → Video styles has it as a starter). It runs **Comfy-Org's own LTX 2.3 templates**, which use only nodes built into ComfyUI (`comfy_workflows/ltx23_official_*.json`). It has four modes. The LTX mode dropdown shows only these, and *Auto* picks by what's in the reference dock:

reference dock	mode
a video	**3D render → real**. The video is a render (Blender, a game viewport, any CG clip); fal's **3DREAL** IC-LoRA turns it photoreal with the same layout and camera move. A picture as well = a real first frame (optional).
2 pictures	first / last frame
1 picture	image to video (first frame)
nothing	text to video

Lipsync, audio to video, first / middle / last and style transfer are LTX 2.5 canvas modes. Pick the LTX 2.5 style for those.

3DREAL (huggingface.co/fal/LTX-2.3-3DREAL-LoRA): the select beside the mode picks the variant:

- *strong v2*: the newest (several scenes, audio, better faces);
- *strong*: a harder photoreal push;
- *light*: faithful and gentle;
- *Auto*: the newest one ComfyUI has.

bitGen puts the LoRA's trigger at the front of your prompt ("3DREAL. Make it photorealistic. …"). Describe what the shot should look like: materials, light, weather, time of day.

From Blender (bitgen-blender-addon 1.1): lay the scene out in Blender, then use **sidebar (N) → bitGen → Render scene (LTX 2.3 · 3DREAL)**. Grey boxes and a camera move are enough. The settings:

- **Looks like**: the prompt.
- **Engine**: *Solid* (the fastest; a blockout is enough), *EEVEE*, or the scene's own engine.
- **3DREAL**: the variant.
- **Strength**: how strongly the render guides LTX (1 = the LoRA's own).
- **Max seconds**: the length cap.
- **Render in bitGen**: when ticked, the job is queued straight away.

Render scene → bitGen renders the frame range through the **scene camera**. The long side is at most 1280, both sides are multiples of 32, and the output is H.264 .mp4. It's sent to the open project as **Video assets → blender_<name>.mp4**, and when *Render in bitGen* is ticked the LTX 2.3 job starts at once. Every render setting in Blender is put back afterwards. Untick *Render in bitGen* to send only the render, then put it in the reference dock yourself (add a real first picture if you like).

Models (*Settings → Models → Model catalog → Video - LTX 2.3*), in ComfyUI's folders:

- **checkpoints**: `ltx-2.3-22b-distilled-fp8` runs every mode. Add `ltx-2.3-22b-dev-fp8` + `ltx_2.3_22b_distilled_1.1_lora…` (loras) to run text / image the way Comfy-Org's template does.
- **text_encoders**: `gemma_3_12B_it_fp4_mixed`.
- **latent_upscale_models**: `ltx-2.3-spatial-upscaler-x2-1.1` (text / image).
- **loras**: `3DREAL-strong-v2` / `3DREAL-strong` / `3DREAL-light`, and optionally the Gemma "abliterated" LoRA (left out when it is missing).

The job log names the checkpoint, the encoder and the 3DREAL file it used, and says what it left out. Keep ComfyUI up to date: the templates use its newest LTX nodes (GetICLoRAParameters, LTXVAddGuide, LTXAVTextEncoderLoader).

A .blend in the preview canvas: look around, edit, render a segment to real

Click in the timeline canvas's tools and pick a `.blend`. Blender (headless) reads it and the scene appears over the preview canvas, framed at the scene's render size. bitGen works on a copy: `<project>/blender/<name>.tmp`. The original isn't touched until you save to it.

- **Scene camera** looks through the scene's active camera, with its animation playing. **Free view**: drag to orbit, right-drag or Shift-drag to pan, the wheel to zoom, **F** to frame everything.
- **Zoom** (2.546.0): **/** , or **Alt+drag** - forward (up) zooms in, back (down) zooms out. **Zoom to object** (or **Z**) brings the view to the picked object from the side you are looking from; with nothing picked it frames the whole scene. Zooming from the Scene camera switches to the free view at the camera's place first.
- **Select** an object by clicking it, or in the object list in **Tools & Properties** (key 1). **G / R / S** (or ✥) switch the widget between move, rotate and scale. The Properties also take X / Y / Z (Blender's axes) and a size.
- **Edits**:
  - Moving an object that Blender animates puts a **key** at the current frame. Moving any other object gives it a new place. Tick *Auto-key every move* to key everything.
  - ◆ **Key** keys the object here, × **Key** removes this frame's key, and the key row (◇ Blender's, ◆ yours) jumps to a key.
  - duplicates, ◌ **Hide** / hides it in the render, deletes, ↺ drops its edits.
  - **Ctrl+Z** undoes. The preview plays between keys in straight lines; Blender eases them.
- **Segment**: scrub the frame bar, then **[In** and **Out]**. The bar shows the length; LTX takes up to 30 s.
- **Save to** (in the Properties):
  - *Only in bitGen*: `<project>/blender/<name>.blend`.
  - *Write back to the original .blend*: Blender keeps the old one as `.blend1`.
  - *Copy to a new file*: `<name>_bitgen.blend` beside the original.

Save always writes the project's `<name>.blend` as well, and bakes your edits into the working copy.

- **Render to real**: pick the camera (*the scene camera*, or *this view* for a still camera where you are looking), the engine (*Solid* is enough for a blockout), the look, the 3DREAL variant and strength, then **Render to real**. Blender renders the segment to **Video assets → blender_<name>_<in>-<out>.mp4**, and with *Render in bitGen* ticked the LTX 2.3 3DREAL job is queued, as with the add-on's Render scene.
- × returns to the timeline canvas. The scene stays open; brings it back. **Reload** drops your edits and reads the .blend again.

Blender must be installed (Blender settings has its path).

Build a scene with AI, and 3D models from pictures

The local LLM can lay out a 3D scene for you. Open (with no scene open you get **Open a .blend** and **Build a scene with AI**; with one open, **New scene with AI** ▸ sits at the top of the Properties). Describe the shot (what is there, what moves, how the camera moves), set the length, and press **Build**:

1. The LLM writes a **plan**: every object as a known kind (car, truck, tree, house, building, road, person, dog, lamp post, bench, table … 58 in all) with its size in metres, its place, its turn and its colour, what moves when, and the camera's path. It never writes code. A reply that isn't a plan is asked for again once.
2. Things no kind fits (an animal, a statue, a special vehicle, a character) become **3D models**. Each is first drawn as a picture with the image style picked in Basic Settings (alone, on white), then turned into a .glb in ComfyUI. *Max 3D* caps how many; untick *3D models* to draw everything from simple shapes. These are normal jobs in the queue.

3 Blender builds `<project>/blender/ai/<title>.blend` (the plan is saved beside it as `<title>.plan.json`) and it opens in the preview: look around, move things, key them, then **Render to real**. Building the same title again makes `<title>_2.blend`; nothing is overwritten.

If a picture or a 3D model can't be made, that object becomes a simple stand-in of its kind and the log says why; the scene is still built. ■ **Stop** stops a build.

Image to 3D (ComfyUI's own nodes, Comfy-Org's templates):

choice	what you get	files (Settings → Models → Model catalog → **3D models**)
Hunyuan3D 2.1 (default)	the shape, no colours; 3DREAL paints the real look anyway	`hunyuan_3d_v2.1.safetensors` (checkpoints), `birefnet.safetensors` (background_removal - cuts the object out first, optional)
TRELLIS.2	shape + baked texture, slower	`trellis_2_int8_convrot` (diffusion_models), `trellis_2_shape_vae_bf16` + `trellis_2_texture_vae_bf16` (vae), `dino_v3_L_naf_fp32` (clip_vision), `birefnet`

Auto uses Hunyuan3D 2.1 when it's there, otherwise TRELLIS.2. A missing file is named in the job's log.

on any picture: every picture in the Gallery (Images) and the Assets dock (Images) has a . Pick Hunyuan3D or TRELLIS.2 and the .glb lands in the project's `models3d/`. From there, **Model** in the .blend preview's Properties adds it to the open scene (1 m tall, standing where the view looks; scale it with S or the Size box) and Save puts it into the .blend. The timeline's 3D layers can use the same files.

Licences: Hunyuan3D 2.1 is under Tencent's Hunyuan 3D community licence (it excludes the EU, the UK and South Korea); TRELLIS.2's code is MIT (some of its parts carry their own licences). Check each model's licence before commercial use.

Daz Studio → bitGen (the bitGen Bridge script)

Install `bitgen-daz-bridge-v1.0.0.zip`: copy its `Scripts` folder into one of your Daz content libraries (for example *Documents/DAZ 3D/Studio/My Library*). In Daz Studio, open **Content Library → (your library) → Scripts → bitGen** and double-click **bitGen Bridge**. To keep it handy, drag it onto a toolbar with *Window → Workspace → Customize*. It works with Daz Studio 4.2x and Daz Studio 2025 / 6. bitGen Studio must be running on the same computer; **Test connection** checks it.

The bridge window (it has its own title bar) offers five sends:

send	what bitGen does
Render the animation → real	Daz renders the timeline's play range with *your* render settings (Iray, Filament or 3Delight; Filament is quick). The frames become `daz_<scene>.mp4` in Video assets (long side 1280, up to 30 s). With *Render in bitGen right away* ticked, LTX 2.3 · 3DREAL renders it real, using your *Looks like* text, the 3DREAL variant and the strength.
Render this frame → a picture	Goes into the Gallery and Images assets, ready for references, first frames or 3D.
The scene as 3D → open in bitGen's preview	Daz exports FBX: figures with their rig and animation, props, and lights / cameras as ticked. Blender turns it into `blender/daz/<scene>.blend` and it opens in the preview, where you can edit it and render it to real.
The scene as 3D → merge into the scene open in bitGen	bitGen opens the **Merge** dialog (see below).
The selected figure / prop → a 3D model	Saved to `models3d/<figure>.glb`, to use with Model in the preview. Tick *also add it as a 3D layer* to put it on the timeline too.

The Merge dialog pops up in bitGen when a Daz scene arrives to be merged while a .blend is open in the preview. You can drag it by its title bar; × or Esc closes it and keeps the scene waiting. It lists the Daz scene in three groups: **Figures & objects**, **Lights** and **Cameras**. Each item has a tick, and each group has *all* / *none*. Then choose:

- **Scene camera**: keep the scene's own camera, or use one of the ticked Daz cameras.
- **Stretch the frame range**: extend the scene to cover the Daz animation.

Merge appends the ticked items, each with everything under it (a figure's body and clothing come with its rig), into a "Daz" collection in the open .blend. Daz animation is respaced to the scene's frame rate, for example 30 fps into a 24 fps scene, and the preview reloads. **Open on its own** opens the Daz scene by itself instead. **Cancel** drops it.

The 3D sends need Blender (bitGen finds it; * Blender settings* has its path). The bridge sends only the paths of the files Daz wrote, so Daz and bitGen must run on the same computer. It uses curl, which Windows 10+ and macOS include.

Sketcher - 3D modelling, rooms and cameras (2.619 - 2.653)

The **Sketcher** docker (its button is in the status bar - it starts hidden) is for modelling in 3D the quick way: draw a shape on the ground or on a face, then push or pull it into a solid - and for building rooms and whole houses to use as film sets. It replaces the House builder of older versions. Its has the whole list.

What it does, in short:

- **Modelling**: lines, rectangles, circles, polygons, push / pull, offset, move / rotate / scale (copies too), groups, faces extruded or twisted, edges moved, extruded or filleted, exact sizes typed at any time.
- **Building** (the Build tools): rooms with shared walls, free-standing walls, doors and windows (open or shut), floors, ceilings (flat or following sloped walls), roofs, stairs and lights (day / night); walls dragged across their line or freely, sloped wall tops.
- **Floorplan** (2.647.0): the view straight down in the app's theme colours (2.649.0), one storey at a time - walls cut solid, doors with their swing, windows, stairs, furniture and lights as plan symbols, each room's name, Width × Length and area. Every tool draws as in 3D; a click picks a symbol. **Esc** anywhere: no tool, nothing picked, Orbit.
- **Outline**: every room and its parts (walls, floor, ceiling, skirting, cornice, doors, windows) listed, hidden or shown, rooms in groups. A part clicked in the view or the outline shows **its own properties** (a wall's length, height and colour; the ceiling's type; the skirting's profile).
- **Profiles**: skirting, cornices, door frames, window frames and window sills from a picture dropdown, or drawn in the ✎ **Profiles** dialog and kept in the project.
- **Terrain** (2.653.0): the ground round the building - raised, lowered, smoothed and flattened with a brush, shaped to contour lines you draw at their heights, made from a heightmap picture, levelled under the rooms; its contour lines on the ground and on the floorplan.
- **Furnishing**: the furniture library and people, materials with textures, Furnish (the LLM fills a room), the project's 3D models.
- **Cameras**: picked, moved (arrows) and turned (rings, double-click), lens / focal length / field of view, Look through (and move or turn it from its own view), shots with depth and normal maps, 360° locations for the video renders.
- **Out**: a real photo of the model through the prompt docker, a picture or a 3D layer to the image editor and the timeline, Blender, Unreal, .glb / .obj / .stl.

1. **R** (Rectangle), click a corner on the ground, move, and type `4,3` + Enter - a 4 × 3 m face.
2. **P** (Push/Pull), click the face, move up, type `2.7` + Enter - a box 2.7 m high.
3. Draw a rectangle **on** the box's side and push it in - a window opening; push it all the way through - a hole.
4. Or: **F** (Offset) on a floor face, 0.2 + Enter, then push the ring up - four walls round a room.

Sizes are typed at any moment (no need to click the box): lengths (`1.5`, `150cm`, `1500mm`, `5'`), two values (`3,2`), sides (`36s`), degrees. The marker shows what the point snaps to (Endpoint, Midpoint, On Edge, On Face, On Ground) and a dotted line when you are on the red / green / blue axis - **Shift** or → ← ↑ keep that axis. Middle drag orbits, right drag pans, the wheel zooms, **Shift+Z** shows everything.

Other tools: **L** Line (a line across a face splits it), **C** Circle, **N** Polygon, **M** Move (**Ctrl** = a copy, type `x,y,z`), **Q** Rotate, **S** Scale, **T** Tape, **B** Paint, **E** Eraser, **Space** Select (double-click a group to edit inside it, **G** makes a group). Models are kept in the project (projects/<project>/pushpull) and save themselves; ↶ ↷ undo / redo. → **Send** puts the model in the image editor (a picture of the view), on the timeline (a 3D layer), in Blender or Unreal; **Export** writes .glb / .obj / .stl; **Import** brings a project 3D model in as a group.

Building: rooms, shared walls, doors, windows, roofs, lights (2.620.0)

The **Build** tools under the modelling tools make a building that stays editable:

1. **U** (Room), click a corner, type `4,3` + Enter - a 4 × 3 m room with walls, a floor and a ceiling.
2. Draw a second room a little way off, then with **Space** (Select) press on it and drag it toward the first. When its wall comes near the first room's wall it snaps onto it - the two rooms now have **one shared wall** (and their corners line up).
3. **D** (Door), click on the shared wall - the door goes right through it. **I** (Window), click on an outside wall. Drag a door or window along its wall with Select; its size, style and sill are in the panel on the right.
4. **X** (Light), pick the type in the panel - downlight, ceiling-mounted, pendant, chandelier, spot, wall light, floor or table lamp - and click in a room: it goes on that room's ceiling (the middle of the room snaps). **Night** turns the sun down so the lights light the rooms.
5. **Y** (Roof), click a room - a gable roof over it; **Shift+click** the other room to put it under the same roof. Gable, hip, shed or flat, with its pitch and overhang, in the panel. ⌂ **Cut-away** hides roofs and ceilings.

Also: **W** Wall (free-standing walls; click the start again for a room), **J** Floor (a floor without walls), **K** Ceiling (a room's ceiling on / off), **V** Stairs. Click anything built for its settings - a room's size, wall height and thickness, its colours inside, 90°, Duplicate. **B** paints walls / floors / ceilings / doors / roofs, **E** or **Delete** erases. Everything is saved, undone, exported and sent with the model (the lights go to Blender and Unreal as glTF lights).

(the docker's title bar) fills the whole bitGen window between its title bar and its status bar (2.625.0); it is then ⇲ **Dock** - that or **Esc** puts it back. × beside it closes the docker (2.624.0). **File** has New, Open, Save, Save as, Rename, Import (a project 3D model or a file from this computer), Export, Send, Delete and Close; → **Image editor** and → **Timeline** are buttons on the bar.

Walls on a floor, sloped walls, follow ceilings and roofs (2.624.0)

- **bitGen's theme colours**: the docker, its 3D view's background, grid and selection take the theme's colours (and change with it).
- **Offset from floor edge** (the Room and Wall tools' panel): a room or wall drawn along a floor's (J) edge has its walls' outside face that far in from the edge (0 = on the edge).
- **Drag one wall** (Select, press on a wall of a room): it moves square to itself and the room stretches. The floors' edges show their **snap regions** while you drag; let go in one and the wall's outside face is on the edge (with the offset). A dragged room snaps to a floor's edge too. Shift = no snap.
- **Double-click the top of a wall** = a cut there; drag the cut up / down and the top slopes to it from both corners (a gable end). Click a **top edge** and drag it (or type its ends' heights in the panel) to slope the wall - a corner takes the wall beside it along. A picked cut or edge + **Delete** takes it away.
- A room's **Ceiling**: None, **Flat** or **Follows the walls' slope**. The **Roof** tool's type **Follows the walls' slope** sits on the sloped tops, with an **Eaves width**.
- **Double-click empty space**: everything is deselected and the tool is **Orbit** (Space = Select again).

Editing faces, edges and corners; doors and windows by hand (2.625.0)

With **Select**:

- **A face**: the panel shows **Extrude** (+ out, - in = a cut into the object) and **Twist**. **Drag the face** to extrude it with the pointer (like Blender); **Ctrl+drag** turns it round its normal - the face just extruded twists its extrusion to match. Type a distance in the Measurements box to change the last extrusion.
- **An edge**: drag it to move it (Shift: one axis). **Shift+click** picks more edges, **Ctrl+click** every edge of the loop round its face. The panel: **Length**, **Extrude edge** (a new face - up, out from its face, along red / green), **Fillet** (radius + pieces; an edge between two faces rounded).
- **A corner**: click and drag it (Shift: one axis; it snaps onto another corner) or type X / Y / Z. Faces bent out of flat are split into triangles.
- **A door or window**: grips on its sides, top and (not a door) bottom - drag them for the width, the height, the elevation; drag a window itself along the wall and up / down. A window's panel has **Elevation (height from floor)**.

The outline docker, skirting + cornice, walls dragged, room groups (2.639.0)

- **Outline** (the Sketcher's bar) - a docker left of the view listing every object: Room groups, Rooms ▸ Room 1 ▸ Wall 1 … n, Floor 1, Ceiling 1, Skirting 1, Cornice 1, Floor slab 1, Door 1, Window 1 …; free-standing Walls, Roofs, Floors, Stairs, Lights, Furniture, Cameras. A click picks; hides / shows (undo-able, saved with the model).
- **Room groups** - makes one, ⊞ on a room puts it in (or out); a group's hides / shows all its rooms.
- **Skirting and cornice** - new rooms have both; the room's panel switches them, sets their size and colour.
- **Walls** - a dragged wall moves across its own line (the walls beside it stretch or shorten; the floor, ceiling, skirting and cornice follow); **Alt+drag** moves it freely; **Shift+click** picks the whole room.

Picking, moving and turning a camera (2.641.0)

- A click on a camera picks it: it lights up (no box round it) and shows the **move arrows** - red X, green Y, blue Z.
- **Drag the camera** to move it at its height; **drag an arrow** to move it along that axis only (the blue one lifts / lowers it); 1 cm steps, Shift = free. The arrow under the pointer turns yellow.
- **Double-click the camera** for the **rotate rings** (blue turn, red tilt, green roll - as in 2.633.0); double-click again for the arrows. Its panel has ✥ Move / Rotate too.
- A click on any other object picks that one and lets the camera go; a click just beside a camera's lines picks what is there (they used to catch clicks up to a metre away).
- **Looking through the camera** (2.643.0) - with the picked camera's Look through on, ✥ Move / Rotate in its panel show the widget in front of you (lower right): drag an arrow or a ring and the camera - and the view - move or turn with it. **Esc** deselects the camera (the view stays as it is).

Each part's own properties; profiles for skirting, cornices, frames and sills (2.640.0)

- **A part picked = its own panel** - click a wall, floor, ceiling, skirting or cornice in the view (or its row in Outline) and the panel shows just that part:
  - – **Wall n** - Length (a rectangle room keeps its shape), Height (only this wall), Thickness (the room's walls), its own **Inside colour** (↺ back to the room's), Shown, the doors / windows in it.
  - – **Floor n** - On, Colour, Level, Floor slab; **Ceiling n** - None / Flat / Follows the walls, Colour, Height.
  - – **Skirting n** / **Cornice n** - On, **Profile**, Height / Size, Colour, ✎ Profiles…; **Floor slab n** - On.
  - – ⌂ *Room*'s settings goes back to the whole room. **Delete** on a part turns it off (a wall: hidden) - the room stays.

- **Profiles** - a picture dropdown (each profile drawn as its cross-section): 7 skirting (Square, Pencil round, Bullnose, Chamfered, Ogee, Torus, Lamb's tongue), 6 cornices (Square, Cove, Quadrant, Ogee, Stepped, Cove + band), 6 door frames and 6 window frames (Flat, Bullnose, Chamfered, Splayed, Colonial, Lamb's tongue), 5 window sills (Square, Bullnose, Round nosing, Chamfered, With an apron), then **This project**'s own.
  - – Skirting is swept round the room with mitred corners and stops at a door's frame; a cornice runs under the ceiling.
  - – A door's or window's panel: **Door frame** / **Window frame** (an architrave round the opening - a door on both faces, a window on the room's side) and **Window sill** (under the window, reaching back to its frame), None to start, and a Frame / sill colour.
- ✎ **Profiles…** (the dropdown's last row, or the part's button) - the profiles of a kind down the side; draw on the canvas: a click adds a point on the nearest side, drag moves it, Delete / right-click removes it (1 mm grid, Shift = free); type a Width / Height to scale it; **New** copies the one shown. **Save** keeps it in the project (projects/<project>/pushpull/profiles.json) and uses it; a model keeps a copy of every profile it uses.

Rotate rings, roll, lenses (2.633.0)

- A picked camera or piece of furniture shows three rings: **blue** turns it (yaw), **red** tips it (pitch), **green** rolls it. Drag round a ring: 5° steps, **Shift** for free turning, **Ctrl+Z** takes it back.
- In numbers in its panel: a camera's **Turn (yaw)**, **Tilt (pitch)**, **Roll**; a piece's **Turn (yaw)**, **Pitch**, **Roll**.
- A camera's **Lens**: Fisheye (8 mm), Ultra wide (14 mm), Wide angle (24 mm), 35 mm, 50 mm (normal), 85 mm (portrait), 135 mm, 200 mm (telephoto) or Custom; **Focal length** (mm, full frame) and **Field of view** (°) follow each other. The fisheye gives its widest view (the curve itself isn't drawn). The Camera tool's panel has the same Lens / Focal length / Field of view.
- **Zoom extents** takes the cameras in too.

Cameras picked, moved, turned; deselecting (2.632.0)

- A camera is drawn over everything - one inside a room (under its ceiling and roof) is picked with a click, not the room. Drag it to move it; **R** turns it 15°, **Shift+R** back (a piece of furniture: R 90°, Shift+R back).
- **Esc** or a **double-click on empty space** deselects everything (2.647.0: one Esc also drops the tool - Orbit; 2.648.0: Esc in Orbit = ➤ Select, and Ctrl+click picks several things).
- **Orbit**'s pointer is an orbit cursor (two arrows round a dot), not the hand (Pan keeps the hand).

Camera views, open doors and windows, auto roof + floor (2.631.0)

- **View ▾** lists the model's cameras (Camera 1, …) as they are added, renamed or deleted; pick one for the view from it - its place, its way, its lens (its own glyph out of the way). Changing its turn / tilt / lens in its panel moves the view with it; an orbit, pan or zoom (or a standard view) leaves it.
- A door or window: **Open (%)** and a direction tick - a door swings into the room (**Opens outward**: out), a window's sashes swing out (**Opens inward**: in), a sliding door / window slides a pane (**The other pane slides**).
- The Room tool: **Auto roof** (the Roof tool's type) and **Auto floor** (a 20 cm slab under the walls) for each new room; a room's panel: **Floor slab under the walls** and **Roof** on / off.

People; houses from older versions (2.630.0)

- ▸ **People**: standing, sitting, walking or waving mannequins - click on a floor; picked: Name, Pose, Height, Turn.
- **File ▸ A house from the House builder…**: a house saved by the House builder of older versions becomes a Sketcher model - rooms (each floor at its height), doors, windows, furniture, people, cameras, lights, stairs; the not-carried parts (a 3D-model piece, the roof) are named in the status line.

- The **Sketcher replaces the House builder** of older versions. SolidCraft's **Send to Sketcher** brings a part in as a group.

Materials, cameras + 360° locations, Furnish (2.629.0)

- ▦ **Materials** ▾ (beside the colours): materials - brick, stone, concrete, oak / walnut floors, tiles, marble, carpet, grass … with their textures; pick one and paint (B) faces, walls and floors.
- **Z Camera**: click where it stands, click where it looks (eye height + lens in the panel). Picked: **Look through**, **Shot + depth + normal** (three pictures in the Image assets) and **360° location** (a panorama from it becomes a Location - its @tag, the prompt's location). Select drags it; the panel turns / tilts / names it.
- **Furnish** (a room's panel): fills the room with furniture - the LLM picks when one is running, else a room's usual set; optional words ("a reading corner"). Ctrl+Z takes it out.
- The furniture now goes into the .glb / .obj exports too.

Furniture and the project's 3D models; maximized = alone (2.628.0)

- **A Furniture** (the Build tools): the furniture library - pick a **Category** (Living, Dining, Kitchen, Bedroom, Bathroom, Lights, Outside) and a piece, click on a floor to put it there (it stands on the room's floor, with its materials and textures). **R** turns the next piece 90° (or type degrees). With **Select**: drag a piece, R turns it; its panel has Width / Depth / Height, Turn, Up from the floor, its own colour, Duplicate, Delete. The pieces are saved with the model.
- The **3D models** category lists the project's 3D models (.glb / .obj - SolidCraft's, imported); a click brings one in as a group (move, scale, paint it).
- **Maximize** now closes every other docker (the title bar and the status bar stay); ⇲ **Dock** / Esc opens the same dockers again.

A real photo of the model; Sketcher layers (2.626.0)

The docker is called **Sketcher** now (it was PushPull; the **Push/Pull** tool keeps its name).

- **With the prompt docker open** (and the image editor not on screen - it has its own Edit link - unless the Sketcher is maximized): the prompt docker goes to **Image** with an **edit model** (Qwen Image Edit, FLUX Kontext, FLUX.2 Klein edit … - the one picked there when it edits, else the default), an empty prompt gets **create a real photo of the model in view**, and its button reads ✏ **Edit (Sketcher)**. Generate sends the Sketcher's view to that edit model and the photo comes back as a **real image** - an ordinary picture in the project's **Image assets** (and a plain layer in the image editor when it is open). Change the words for another look ("a rainy evening street, cinematic"). 2.627.0: what goes to the edit model is a **clean render** of the view (no outlines, grid or axes; shadows; a daylight sky) as picture 1 and its **depth map** (near white, far black) as picture 2, and the prompt starts with "Convert image 1: Turn this 3D render into a photorealistic photograph of a building: real brick, timber, glass and concrete, natural daylight, soft shadows, no outlines, DSLR photo." + a line saying picture 2 is its depth map (the picture words are the edit model's own - <image1> for Qwen 2.1). Your words are added after it. Closing the Sketcher puts the prompt docker's mode, style and words back.
- **→ Image editor** sends the view as a **Sketcher layer** (named after the model); sent again, that layer is updated instead of a new one. **→ Timeline** puts the model in as a **Sketcher layer** (3D).

Editing on the timeline: speed, colour, sound, mix and captions (2.406.0)

Select a clip on the timeline canvas; its **Properties** get three folds. Everything is saved with the timeline, the canvas previews it, and ⚙ **Compile** renders it.

Speed

control	what it does
Speed %	50 = half speed (twice as long on the timeline), 200 = double speed (half as long); 10 – 1000 %
⇄ reverse	the clip plays backwards
Ramp	the speed changes along the clip: *slow → fast*, *fast → slow*, *slow middle*, *fast middle* (it replaces Speed %)
Frames	how slowed frames are made: *repeat* (crisp, steppy), *blend* (neighbours mixed), *optical flow* (smooth; slow to compile)

The clip's length on the timeline follows its speed. The trim edges still trim the *source*, and keep / cut on a retimed clip keep the right frames. In the canvas a reversed or ramped clip plays silently (a browser cannot play sound backwards); the compile renders its sound at the speed (the pitch is kept).

Colour

- **Look**: teal & orange, warm film, cool night, bleach bypass, vivid, faded, mono - a starting point for the sliders.
- **Sliders**: exposure (stops), contrast, saturation, temperature (blue ↔ amber), tint (green ↔ magenta), hue.
- **LUT**: any 3D `.cube` LUT (from a camera maker, a colourist or a LUT pack). **LUT** imports one into the project's `luts/` folder; LUTs in the app's own `luts/` folder are offered in every project. **LUT mix** blends it.

The LUT is applied first, then the sliders. The canvas shows the grade with the very table the compile uses (the whole look is baked into one 33-point LUT), so what you see is what you get.

Sound

On a video clip or a sound clip: **voice clarity** (cuts the rumble, adds a little presence and evens the level), **denoise** (steady background noise: hum, hiss, fans) and **gain** (dB). **before / after** plays a few seconds from the position marker, first as it is, then cleaned up.

Mix

The timeline's opens the Mix dialog:

- **Roles**: each layer is *voice*, *music* or *sfx* (V1 and video layers start as voice, sound layers as music).
- **Ducking**: the music dips by the chosen amount (3 – 30 dB) whenever a voice speaks, then comes back.
- **Loudness**: the finished movie is measured and evened out to a target: -14 LUFS (YouTube, Spotify), -16 (podcasts, Apple), -23 (broadcast, EBU R128) or -24 (US broadcast). The true peak stays under -1.5 dB.

Captions

The timeline's opens the Captions dialog.

1. **From speech** transcribes everything that speaks (V1, the video layers, and any sound layer set to voice) on this computer with faster-whisper (*Settings → Audio* installs it) and makes the captions - each clip's words placed where the clip sits, its trim and speed included. Each file is transcribed once and remembered. *max chars* sets how long a caption may be; *language* can be left on auto.
2. Edit them: the start and end times, the words, a caption at the position marker, ✕ to remove one, ▶ to jump there.
3. **Style**: font, size (% of the picture's height), colour, bold, an outline or a **box** behind the words (colour and opacity), bottom / middle / top and the margin.

4 **Output**: *burn into the movie* and / or *.srt beside it* (the movie's name with `.srt` - players and YouTube take it as a subtitle track). ⬇ **.srt** downloads them now. Untick *on* to keep them without using them.

The canvas draws the captions where the compile burns them.

Edit tools, colour panel, export queue and exchange (2.407.0)

Edit tools

The timeline's tool buttons: ▸ **Select** (move, trim - as before), ⇤ **Ripple** (drag an edge: trim it and move everything after it on that layer), ⇹ **Roll** (drag the edge between two touching clips: the cut moves, the total stays), ⇆ **Slip** (drag a clip: it shows other frames of its source, same place and length) and ↔ **Slide** (drag a clip: it moves, the clips either side give and take). A tip beside the pointer shows what the drag will do. Zoom in so each edge has its own handle. A reversed clip can't be edited with these (its edges run backwards).

▯ **Frame** (F over the layers) picks single frames: click a layer's track (on a clip or between clips) and the frame under the pointer on THAT layer is selected - the position marker jumps to it and the frame is outlined on its row. Drag across for several frames, Shift+click to select from the selected frame up to the one clicked, ← / → move the selection a frame (Shift: its end) - exact at any zoom. **Del** (or ⌫) removes the selected frame(s) from that layer and closes the gap (always, whatever the ripple box says); ↶ / Ctrl+Z puts them back; Esc drops the selection. Clips don't move while ▯ Frame is on - press V for ▸ Select.

A cut clip stays one clip. When ✂ Cut (or Del on frames) goes through the middle of a clip with ripple on, the part before and the part after the cut are joined into ONE continuous clip again: a video (or a sound) is made into one new file - "<name>-join1.mp4" in the project's Video assets (a sound: "-join1.flac" in Sound) - with the cut part gone, frame-exact; a picture just gets shorter. The cut shows at once and the join follows a moment later (a few seconds for a long video); one ↶ undoes both. Left as two clips (the status line says why): a retimed clip (speed / reverse / ramp), a cut-out, a text / shape / 3D / generated clip, and a cut with ripple off (the gap stays). The preview also loads the next clip a second before it starts, so a cut between two clips no longer flashes black.

▼ **Ranges with Ctrl while it plays.** Highlight a layer, play the preview and TAP Ctrl (press and let go) where a range should start - a ▼ appears in the time ruler at the position marker - and tap Ctrl again where it ends (a second ▼). Playback carries on, so you can mark as many ranges as you like in one pass; each gets its own colour, with a band on its layer's row. Paused, a Ctrl tap with the pointer on the timeline works the same. Esc forgets a started range. Ctrl taps count whatever is focused (▶, the speed list, a ticked box, the zoom slider) - only a box you type in holds them back - and also while the preview docker's player plays a clip that is on the timeline (the range lands at that moment of the clip on the timeline). Drag a ▼ to move that end (whole frames) - the preview shows the frame under it. A click on a ▼ PICKS its range (Ctrl+click or Shift+click picks more; Esc unpicks); right-click a ▼ removes its range, Del removes the picked ones' markers. The ▼ group in the timeline's menu bar: the count, ✂ **Cut** (the picked ranges - "✂ Cut 2 picked" - or, none picked, every range: out of its layer, ripple as its box says; a clip cut several times becomes one clip again; the selection bar's ✂ Cut also cuts the picked ranges), **Speed up** with 2× / 4× / 8× / 10× (the picked ranges, or all, on video layers play that much faster - like fast-forwarding through a render you don't want to wait for; what follows moves up with ripple) and × (remove the picked markers, or all). Ctrl+Z undoes any of these.

Hover time and playback speed. Hovering over the timeline shows the time under the pointer in a small box (timecode · seconds · frame). The speed list beside under the preview plays at 0.125×, 0.25×, 0.5×, 1×, 2×, 4× or 8× (▶ and Space use it; changed while playing it applies at once; it is kept). J / K / L still shuttle.

✂ **to new layer** (the checkbox in the timeline's menu bar, kept): ✂ Cut, Del on frames and Ctrl+K split put the part AFTER the cut on a new layer right above ("<layer> ✂"; V1's goes on the first video layer). The part before stays where it is. One ↶ undoes it.

J / L cuts (V1): in a V1 clip's Sound, *J … s early* starts its sound before its picture, *L … s late* runs it on after; the neighbour's own sound is silenced for that stretch.

Timecode: HH:MM:SS:FF beside the position. The start (for example 01:00:00:00) and the frame rate are set in Export or ⇄ Exchange.

Colour panel

The timeline's (or * more…* in a clip's Colour fold) opens the Colour dialog for the selected clip:

- **Wheels**: Lift (shadows), Gamma (mid-tones), Gain (highlights). Drag the puck toward a colour; the slider under each wheel makes that range darker or brighter; double-click a wheel to centre it.
- **Curves**: Master, R, G, B. Click to add a point, drag to move it, double-click to remove it.
- **HSL secondaries**: up to six hue ranges, each shifted, (de)saturated or lightened on its own (bluer skies, warmer skin, calmer greens) - the rest of the picture is untouched.
- **Scopes**: waveform, RGB parade, vectorscope and histogram of the picture at the position marker, as the movie gets it.

All of it is baked into the same colour table as the looks and LUTs, so the canvas and the compile match.

Export

Add every format the movie should come out in - one compile makes them all, beside `<project>.mp4`:

format	for
MP4 · H.264	everywhere
MP4 · H.265 / HEVC	about half the size; newer players
MOV · ProRes 422 HQ	editing and finishing (large)
MOV · DNxHR HQ	Avid, Resolve (large)
WebM · VP9	the web
GIF	previews (no sound, 15 fps, 640 wide at most)
WAV, MP3	the sound only

Each has a **size** (the short side), a **reframe** (9:16, 1:1, 4:5, 21:9 - *crop* the middle, *pad* with black or a *blur*: a blurred copy behind), a **frame rate** and a **quality**. The packs add a few at once (*social*, *for editing*, *web*). *make them on every* ⚙ *Compile* keeps the list on; ⚙ **Compile + export** runs it now. **Delivered** lists the files to download. The start timecode is written into MP4 and MOV files; the captions' `.srt` goes beside each movie.

⇄ Exchange

- **Send to another editor**: *Final Cut Pro 7 XML* (opens in DaVinci Resolve, Avid and most other video editors), *CMX3600 EDL* (V1 only - the format holds one picture track) or *OpenTimelineIO*. The clips point at the files in this project. Kept: clips, trims, layers, stills, speed and reverse, opacity, volume, the timecode. Not carried: moves and keyframes, the colour grade, captions, 3D layers.
- **Bring in from another editor**: pick an XML (most editors: *File → Export → Final Cut Pro XML*; Resolve: *Export Timeline → FCP 7 XML*) or an EDL. It opens as a new named timeline; clips are matched by file name in this project's assets, and any that are missing are listed (add them, then import again).

Edit by transcript, scene cuts, karaoke word fixes (2.408.0)

Transcript

Select a video or sound clip on the timeline, then press the timeline's . The clip's words appear: its file is transcribed once on this computer (faster-whisper, *Settings → Audio*) and remembered, the same as for Captions.

- **Choose words**: click them, shift-click for a run, or drag across them. Double-click a word to jump there.

- ✂ **Cut them**: the chosen words are taken out and the gap closes; the clips after it on that layer move up.
- ⇥ **Keep only them**: everything but the chosen words is taken out.
- **Remove pauses over … s**: every silence longer than that (the end of the clip included) is cut down to 0.3 s.
- **Remove filler words**: um, uh, erm, hmm and the like (shown in italics) are taken out.
- ↺ **Undo** puts back the last cut. The clip becomes pieces of the same file, and the transcript keeps following them, so you can keep cutting.
- ✂ **Scene cuts** splits the clip wherever its shots change. Lower the *sensitivity* number to find softer changes.

A clip with a changed speed () has to go back to 100 % before it can be cut by words.

Karaoke: wrong words

- ✎ **Edit lyrics**: if a song has no lyrics on file, the box now shows the words speech-to-text heard - the same words the karaoke showed - so you can correct the wrong ones (or paste the real lyrics). **Generate** saves them and times them against the song again.
- **Words & timing**: every word the overlay will show, with its start and end. ▶ plays a word. Correct a single word or nudge its timing here; **Generate** keeps your corrections for that song (↺ *undo my corrections* removes them). Changing the lyrics later starts again from the new lyrics.
- **Save words** keeps the corrections for the song without making an overlay now.
- **From the StoryBoard**: the ✎ beside Karaoke opens this dialog on the board's song. After **Save words**, every scene that already has a lyrics overlay gets it remade with the corrected words (and the new one replaces the old on the timeline's Karaoke layer).

Override style for every video model (2.409.0)

With **Override style** ticked at the top of Basic Settings, Basic Settings decide for **every** video model - MiniMax H3, Wan 2.1 / 2.2 and LTX 2.3 / 2.5 - whatever the style says:

setting	what it does with Override style on
Steps	that many steps (0 = the model's own). LTX's 8-step schedule is re-drawn with that many steps on the same curve; its short refine pass stays
CFG	the guidance (LTX: its CFG guiders, LTX 2.5's video CFG - the audio CFG stays)
Seed	-1 = random, any number = that seed
Model	the new *Model (override)* box: any model ComfyUI lists; blank = the style's own. Wan 2.2: the high-noise model (the low-noise one stays). A `.gguf` file is loaded with ComfyUI-GGUF's loader
LoRAs	the *Temporary LoRAs (video)* are added on top of the style's

The line under the switch says what it is deciding right now. Unticked, each style's own settings rule, as before. Generate takes the prompt docker's text; the chat's ⬇ To prompt / ▶ buttons and Enhance's **Generate** send theirs the same way (Enhance's Generate used to fail with an LTX style). The scene board's ▶ also uses the model, steps and CFG (its scenes keep their own seed rule from the StoryBoard's ⚙). LTX models are distilled for 8 steps and CFG 1 - more steps or a higher CFG changes the look and takes longer.

Effects and ◐ adjustment layers (2.409.0)

Effects (a video clip's Properties): blur, sharpen, glow, vignette, grain; **crop** (each edge in %, made see-through - the picture stays in place); **key** (a green or blue screen: pick the colour, *match* = how close still keys, *edge* = a soft edge, *spill* = removes the screen's cast from the edges - the layer under it shows through); **stabilise** (steadies a shaky shot in the compile). The canvas shows the key and the crop exactly and the others closely.

◐ **Adjustment layer** (the timeline's toolbar): adds a clip on a new top layer at the position marker. Its Colour (looks, sliders, LUTs, wheels, curves, HSL) and Effects (blur, sharpen, glow, vignette, grain) work on everything under it, only while it is on the timeline. Trim, move and fade it like a picture; its opacity mixes the adjustment in.

Karaoke: word nudges, the line under the marker (2.433.0)

In Karaoke ▸ **Words & timing** every word has ◀ ▶ (earlier / later) - the word's start and end move together:

	moves	by
click	that word	the **nudge** (0.05 s)
Shift+click	that word	the **larger nudge** (0.25 s)
Ctrl+click (⌘ on a Mac)	that word **and every word after it**	the nudge
Shift+Ctrl+click	that word and every word after it	the larger nudge

The two amounts are the boxes above the list (**Nudge** / **Shift**) - kept for next time. Nothing moves before 0 s: a move that would is cut short for every word it moves, so their spacing stays (the note says by how much they moved). Generate (or Save words) keeps the new times for the song.

The line under the marker is lit - every word of the line being sung where the preview marker is (or the next line when it starts within 1.5 s); the word being sung is lit brighter, and on canvas draws the same line.

Karaoke and Tools as dockers beside the canvas (2.410.0)

Both are **dockers** now, not dialogs. Drag one by its titlebar to the preview canvas's **left or right edge** (that side lights up) and let go - or click ◧ / ◨ in its titlebar. Docked, it is as tall as the canvas and the canvas makes room; drag the column's inner edge to make it wider. Drag the titlebar away (or click the lit ◧ / ◨) and it floats again. The first time Karaoke opens it docks on the right (when the canvas is wide enough), so the song can play while its words are corrected:

- in **Words & timing** the word being sung is lit and the list follows the preview;
- a word's **time** is a button that puts the preview on it;
- beside a start / an end sets it to where the preview is;
- **on canvas** draws the corrected words over the preview (the canvas only, never the movie).

Space pauses the preview anywhere in the app - the canvas, the live preview or a video in the Preview docker - unless you are typing in a box. Paused, it plays again.

⌨ Timeline keys, markers, In / Out export, frames (2.410.0)

With the pointer over the timeline's **layers** (the canvas keeps its own keys):

Key	Does
V / B / N / Y / U / F	▸ Select / ⇤ Ripple / ⇹ Roll / ⇆ Slip / ↔ Slide / ▯ Frame
Del / ⌫	(▯ Frame) remove the selected frame(s) from their layer - the gap closes
Ctrl (tap)	while it plays: ▼ a range starts at the position marker on the highlighted layer - tap again where it ends
I / O	In / Out of the ruler selection at the position marker
Ctrl+K	split at the position marker (the selected clip, else the highlighted layer's)
↑ / ↓	previous / next edit point
Home / End	the start / the end
M	a marker (on a marker: its name, colour, comment)
Shift+M / Alt+M	next / previous marker

Ctrl+Shift+E	the frame at the position marker as a PNG (over the canvas too)

Markers are flags on the ruler: click = go there, drag = move, double-click = name / colour / comment / . ✂ Scene cuts **as markers** drops one at every shot change instead of cutting. They travel in ⇄ Exchange (XML `<marker>`, EDL `* LOC:`, OTIO markers) both ways.

Export's **Range**: *the whole movie* or *In / Out only* - every format then holds just the ruler selection (`<project>_<format>_io.<ext>`), its timecode starts where the part starts, its .srt is cut to the part. **Export frame** saves the picture at the position marker as `<project>_frame_HHhMMmSSsFF.png` beside the movies.

Text effect presets (2.411.0)

A text layer's Properties (timeline) and the image editor's Type tool / text Properties have **Preset**: text effects made from layered PSD text effects - **Arena Gold 3D** and **Trend Candy 3D** come with the app - plus the classic looks. The words are drawn with the preset **live as you type** (a quick picture while they change, the sharp one when you stop); ⚙ Compile always uses the sharp one. **Import PSD text effects or a zip…** turns your own PSD (the word as a smart object with layer effects) into a preset; backgrounds and decorations are left out. Pick several PSDs at once, or a **zip** of them (a text effect pack): every .psd / .psb in it - in any folder - becomes a preset, a bitGen preset .json in it is added as it is, and the fonts it carries (.ttf / .otf) go into the studio's own font folder (`fonts\` in the install folder - every font list then shows them). Previews, read-me files and macOS's `__MACOSX` copies are left out; a PSD that is not a text effect is named in the line that says what went in, the rest still go in. Fonts are not inside a PSD itself: the note under Preset names the one it used. From the command line: `node scripts/textfx_build.js <file.psd> "<Name>"` writes a built-in preset to `textfx/`.

J / K / L, AV1, 10-bit, the graphics card (2.411.0)

Key (anywhere in the timeline docker)	Does
L	play forwards; again 2×, 4×, 8×
J	play backwards; again 2×, 4×, 8× (stops at the start)
K	stop
K + L / K + J	one frame on / back (hold K)

Export adds **MP4 · AV1** and **MP4 · H.265 10-bit** (and the *smallest* pack). **use the graphics card's encoder when it has one** makes H.264 / H.265 / AV1 with NVENC, Quick Sync or AMF when one works here, else the software encoder - the log says which.

Smaller editing jobs (2.412.0)

- **Drop-frame timecode** (29.97 / 59.94): Export / ⇄ Exchange → *drop-frame*; HH:MM:SS;FF in the files, XML (DF), EDL (FCM: DROP FRAME) and the timeline readout.
- **Marker ranges**: the marker editor's *length* (or ⟦ ⟧ = the ruler's In / Out); drag the band's end. **Clip markers**: Ctrl+Shift+M on the selected clip - they stay on their frame through trims, slips, speed and reverse.
- ◎ **Track targets**: Ctrl+K with no clip selected splits every targeted layer.
- **Linked sound**: Properties → Sound → as its own clip; linked selection (menu bar) moves the pair together; Unlink / ↩ Sound back.
- **J / L cuts** on V2+ layers and on retimed clips.
- **EQ / multiband / reverb** in a clip's Sound section.
- **H/H and H/S curves** in the colour panel.
- **Effect keyframes**: ◆ key here, then move the sliders elsewhere on the clip.

- **ProRes 4444 (alpha)** (the *overlay* pack), **H.264 10-bit**, **VAAPI** among the graphics-card encoders.

Partly-done editing items finished (2.413.0)

- Ripple / roll / slip / slide, the edge handles and Ctrl+K on **reversed** and **ramped** clips, frame-exact.
- **Edit by transcript** and **scene cuts** on retimed clips.
- fit **follow** = smart reframe (the crop follows faces, people, what stands out).
- **Stabilise**: *smooth* and *locked*; the compile measures the shake (vidstab); the canvas shows a steady preview.
- ⇄ XML speed in the usual editors' convention (speed-adjusted in / out), verified with OpenTimelineIO.
- Text presets: the bevel's profile contour kept (re-import a PSD made before 2.413 for it).

Nested sequences, the fuller XML, FCPXML, whole-sequence transcripts (2.414.0)

- **Nest** (the timeline's menu bar): pick another named timeline - it is compiled into ONE clip on a new layer " name" at the position marker. Trim it, move it, grade it like any clip. When that timeline is saved again the clip's badge says **↻ out of date**: Properties → **↻ Re-render** (the clip keeps its place and trim) or **Open** (switch to it to edit). A timeline cannot hold itself, nor one that already uses it.
- ⇄ **The XML carries more**: position / scale / rotation (Basic Motion) and opacity / volume, each with its ◆ keyframes; the captions as text titles; the **grade** - each graded clip's whole look is baked into a .cube beside the export (`exports/<name>_luts/`), named in the clip's comment; apply it as the clip's input LUT (Resolve: a node's LUT; Final Cut: Custom LUT). bitGen reads its own grade back exactly.
- ⇄ **FCPXML 1.10** for Final Cut Pro X, both ways: V1 is the primary storyline, the other layers connected lanes, sound layers below; speed, moves, keyframes, markers, captions, the start timecode.
- **Transcript scope**: *this clip*, *its whole layer* or *the whole sequence* (V1, the video layers and the voice sound layers as one text, in the order they play). A cut takes the words out of whichever clip says them; for the whole sequence every layer after it moves up too, so the music and the pictures stay in step. ↺ undoes it.

Track mixer, distort effects, the transitions library, colour match (2.415.0)

- **Mixer** (the timeline's menu bar): a strip per track - V1, every video layer, every sound layer - and the **master**: a fader (−∞ .. +6 dB), **pan** (L 100 .. C .. R 100), **M**ute, **S**olo and a live **level meter** (left / right, the peak held) while the canvas plays. Double-click a fader for 0 dB, a pan for the centre. The movie pans and mixes the same way (the balance law: panned right, the left side is turned down).
- **Distort** in a clip's Effects: **lens** (right = barrel / fisheye, left = pincushion), **mirror** (left → right, top → bottom, four ways), **warp** (*wave* - a sideways ripple, its size and speed; *twirl* - a swirl at the middle). The canvas shows exactly what the movie gets.
- **Transitions**: 59 now (wipes to the corners, soft wipes, covers / reveals, squeezes, irises, barn doors, slices, wind, zoom …). A video or picture layer's **entry / exit** is drawn as its shape (a wipe, a slide, an iris) - on the canvas and in the movie - instead of always a dissolve.
- **Match**: pick another clip on the timeline - this clip's R / G / B curves are set so its tones match that clip's frame (each clip's frame at the position marker when it is there, else its middle). Strength 10 .. 100 %. The curves are ordinary curves: tweak them, or reset to undo.

Masks + tracking, multi-camera, proxies (2.416.0)

- ◯ **Mask (power window)** in the Colour panel: an *ellipse* or a *box* - centre, size, rotation, **feather** (the soft edge), **invert**. The clip's grade and the effects after it (blur, glow, vignette, distort …) then apply only inside it (or only outside). Its outline shows on the canvas while the panel is open. Works on adjustment layers too.
- **Track**: put the position marker where the mask sits on the thing, press - the mask follows it to the clip's end (the canvas and the movie). If it loses it (it left the frame) it holds there and says so; × **track** removes it. Moving a tracked mask's centre shifts the whole track.
- **Multi-camera**: put every angle of the same moment on the timeline, open , tick them (the first is the reference) and pick whose sound plays → ⇆ **Sync by sound**. V1 becomes the program; play it and press **1 - 9** (or click an angle in the grid) to cut to that angle there. ⇲ **Flatten** keeps the cuts as ordinary clips.
- **Proxies**: heavy footage (over 1920 wide, HEVC / ProRes / DNx, over 40 Mbit/s) is listed; **Make for the heavy ones**, then tick **play proxies on the canvas**. The movie, the deliverables and the exchange files still use the originals.

HDR, AAF, the sound of reversed clips, motion graphics templates (2.417.0)

- **HDR**: the 10-bit presets (H.265 10-bit, AV1, ProRes 422 HQ, ProRes 4444) have a **colour**: *SDR* (as the master), **HDR10** (PQ, BT.2020, with HDR10 metadata), **HLG** (broadcast HDR) or **Rec.2020 SDR** (wide gamut). The master's white lands at 203 nits (the HDR reference white), so it looks as graded on an HDR screen. The **HDR pack** adds an HDR10 and an HLG file in one go. An **HDR clip** (a phone's HDR video) on the timeline is tone-mapped into the movie - no more washed-out grey.
- ⇄ **AAF** (Exchange → *AAF - Avid Media Composer, Pro Tools*): the cut as an .aaf - every video layer a picture track, the sound as mono tracks, gaps, the start timecode; the media is linked where it is. Avid / Pro Tools get the cut, not opacity, volume, moves or the grade (use the FCP 7 XML for Resolve and other editors). WebM / MP3 media is linked as it is - Avid may want it as .mov / .mxf. Needs `pip install pyaaf2` (requirements.txt).
- **Reversed / ramped clips are heard on the canvas**: their sound is rendered once (the movie's own retime) as soon as the position marker sits on them, then plays in step with the picture.
- **Motion graphics templates** (the menu bar): pick a template, fill in its texts, **Insert at the playhead** - its layers land on the timeline sized to your picture, with their moves and keyframes. Built in: *Lower third*, *Title card*, *End card*, *Corner tag*. **Save as a template**: tick the picture / text layers (the ones under the position marker are ticked), name the texts, save - every project sees it (★ in the list; deletes a saved one).

Real-time playback: the render bar, ⚡ Render, , playback resolution (2.418.0)

- **The render bar** (a thin line under the ruler): no line = the canvas draws that stretch live in real time; **yellow** = it may drop frames; **red** = too heavy to draw live (many layers, grades, blurs / glows, masks, reversed or ramped clips, 3D, adjustment layers - hover it for why) or it **did** drop frames while playing; **green** = rendered.
- ⚡ **Render** (the menu bar): the timeline is compiled once (the real movie, into the project's cache) and every stretch turns green; while the canvas **plays** over a green stretch it shows that render - the exact movie at full speed. An edit turns only the stretch it touches red / yellow again; ⚡ again to refresh. **Shift+click** ⚡ turns the previews off / on.
- beside ▶: how many frames were drawn late since ▶ (green 0 = smooth).
- **Full / ½ / ¼** beside it: the playback resolution - the clips' grades, effects and masks are worked on a half / quarter size picture while playing (paused frames stay full size).

One prompt docker, ⊖ the negative prompt docker (2.419.0)

- **One prompt docker for every tool**: the image editor's AI, the timeline's Fill, a path layer's subject, AI scene, the 3DREAL look, the character creator, an environment and a VR world no longer have prompt boxes of their own - they use what is in the **prompt docker** (P). Where the box was, a line shows the text the tool will use; click it to edit in the prompt docker. With the prompt docker empty a tool keeps its own default. (A scene's own prompt, the scene editor, lyrics, speech and captions stay where they are - they are that scene's / song's text.)
- ⊖ **Negative prompt docker** - press **2** (not while typing; while the multi-camera dialog is open 1 - 9 cut cameras) or **Negative** in the statusbar. Write what the picture must NOT have. It goes with every image and video generation and the AI edits; empty = the style's own negative. Its status line says whether the selected model uses one: SDXL / Pony / Illustrious (and ComfyUI image styles at cfg > 1), LTX, Wan 2.1 / 2.2 and MiniMax H3 at cfg > 1 do; FLUX / Qwen / Z-Image (distilled), music and speech do not. ↺ **style** puts the style's own negative there to edit.
- **The LLM's negatives land there**: a reply's "Negative: …" line no longer goes into the prompt - ⬇ To prompt, a ``image /`` video reply and Generate itself move it into the ⊖ docker (untick *put the LLM's "Negative:" lines here* to keep yours). With an LTX or Wan video style selected the LLM now writes for THAT model (with a Negative line).

AI inside the timeline (2.420.0)

in the timeline's menu bar. Every render takes its words from the **prompt docker** and its negative from the ⊖ **negative docker**, with the selected video style (MiniMax H3, Wan, or LTX). The dialog shows both before you press.

- **New take**: select a video clip, press - it is rendered again in its own slot (same start, length, place, keyframes); "start from its first frame" keeps the shot's opening picture. The old file stays a take: ⇆ switches.
- ⇥ **Extend**: N more seconds continuing from the clip's last frame, right after it; the clips after it move on.
- **B-roll**: put the position marker in an empty stretch of V1 → a clip fills it (optionally continuing from the clip before); or select a range on the ruler and choose *a cutaway* → a new "B-roll" layer over it, its sound off so V1's sound carries on.
- ✂ **Cut to the beat**: pick the song (a sound layer's clip or V1), ♪ **Find the beats** (its tempo is shown), then ⚑ **Beat markers** (every N beats) or ✂ **Cut the selected clip on the beats** (pieces of the same file).

The GPU compositor (2.420.0)

While the canvas **plays**, the layers are put together on the graphics card: each clip's picture is a texture drawn in one pass (moved, scaled, turned, its opacity), and a clip that only has a grade is graded in that same pass. A paused frame, a layer's wipe and an adjustment layer use the 2D path as before. **GPU** beside switches it off / on.

Wan 2.1 and Wan 2.2

Pick **Wan 2.1** or **Wan 2.2** in the Basic Settings video style list. Then render from the prompt docker as usual.

Mode	What Wan does
Text	text to video
F/L, one picture	the picture is the first frame (image to video)
F/L, two pictures	first frame and last frame
Ref	the first picture is the character (Wan VACE). Without a VACE model, the picture becomes the first frame instead, and the log says so

- **Models:** bitGen builds the Wan graph itself from ComfyUI's own Wan nodes, so no workflow file is needed. It picks the model file ComfyUI has for each mode by name (t2v, i2v 480p / 720p, flf2v, vace). For Wan 2.2 it picks the high-noise + low-noise pair. If a file is missing, the job tells you its name. Download from Comfy-Org's *Wan_2.1_ComfyUI_repackaged* / *Wan_2.2_ComfyUI_Repackaged* into `ComfyUI/models/diffusion_models`, with `umt5_xxl_fp8_e4m3fn_scaled.safetensors` (text_encoders), `wan_2.1_vae.safetensors` (vae) and, for Wan 2.1 picture modes, `clip_vision_h.safetensors` (clip_vision).
- **Forcing a file:** set a model in the style's phase (Settings → Video styles) to use that file whenever it fits the mode.
- **Steps and CFG:** blank uses Wan's own (2.1: 30 steps, cfg 6; 2.2: 20 steps, cfg 3.5). With a Lightning, lightx2v or CausVid LoRA it uses 4 steps, cfg 1.
- **Sound:** Wan makes no sound. The reference dock's first sound is laid under the clip.
- **Frame rate:** Wan renders 16 fps. The clip is motion-interpolated to 24 fps so it joins your other clips smoothly. Up to 10 s per clip; Wan is trained on 5 s, and longer clips can drift.

VR World (a performer inside a 360° / 180° picture)

The StoryBoard's **VR World** button opens the dialog.

1. **World:** choose a 360° or 180° picture, or drop one or an Image asset. The type is read from the name (`_360`, `_360_TB`, `_360_LR`, `_180_LR`, `SBS`, `TB`...) or the shape. Check the **Type** if the dialog says so.
2. **Aim:** click the panorama where the performer stands. **View °** sets how wide the camera is; smaller makes the performer bigger and sharper. **Shape** and **Size** set the render (Size 0 = matched to the picture's sharpness). *What the camera sees* shows the view.
3. **The performer:** add up to three pictures, a sound (song or voice) with its start, and a prompt.
4. **Engine:**
   - – **MiniMax H3** makes its own sound and speech.
   - – **H3 + the sound lip-synced** uses MMH3_NativeAudio_MV, so the performer sings or speaks your sound.
   - – Any **Wan** style lays your sound under the clip.
5. ✎ **First frame** (optional, recommended): the image-edit style (Qwen Image 2.1 edit) puts the performer into the view. The render then starts on that frame, so the place stays the place. Without it the render starts on the empty view and the prompt brings the performer in.
6. ▶ **Render VR.**

The finished `…_vrworld_360` / `_360_TB` / `_360_LR` / `_180_LR` mp4 lands in the gallery with VR metadata. Copy it to your headset and open it in its player (DeoVR, Skybox, Meta Quest TV...).

Good to know:

- **Only the performer moves.** The area around the performer is animated; the rest of the world is your still picture.
- **Lock the background:** outside the performer, both eyes show the picture itself. This removes any shimmer or seam, but also loses shadows on the floor.
- **Stereo depth:** the place keeps its own depth (measured from the picture's two eyes). The performer has no second eye, so they stand as a flat card at the depth of the ground at their feet. This looks right from a couple of metres away.
- **Best results:** a static camera (added to the prompt for you), the performer framed with some margin, a View ° of 50–80, and a ✎ first frame.

Character swap

Put a new person into existing footage — the performance, camera, background and **the original sound** stay; only the character changes.

From the Prompt docker



Swap mode

1. Basic Settings → **SWAP**.
2. References → **Image**: one picture of the new character (`<Picture 1>` — H3's name for it; a clear, full-body picture works best).
3. References → **Video**: the footage (the first video is used — trim it to the part you want).
4. Optional: **Swap from** / **Swap length** (seconds) and extra direction in the prompt box (the swap instructions always ride; the box may stay empty).
5. **GENERATE**.

Getting the new character to stick (2.334.0)

- Swap **4–5 second pieces**. The swap dialog's ✂ Auto segment cuts 5 s pieces by default; one long 15 s run tends to slide back to the original performer after a couple of seconds.
- Install the **H3 Character Swap LoRA** (Setup → Models → Video extras, or put `h3_character_swap_pro4500_1000.safetensors` anywhere under ComfyUI's `models/loras` folder and restart ComfyUI). The swap adds it to the workflow automatically when ComfyUI has it, and the job log says so ("swap LoRA: … stacked"). Without it H3 alone rarely holds the new character.
- **Quality** (the swap dialog's menubar, 2.336.0): **Full** (default — no turbo LoRA, 20 steps, ComfyUI's own H3 reference settings), **Best** (28 steps — how the swap LoRA was tested) or **Fast** (the turbo LoRA at 4 steps — quickest, weakest swap). Full and Best take roughly 5–7× longer than Fast. The job log names the quality it used.
- The picture goes to H3 at full detail (`ref_image_size` max) and the prompt names it `<Picture 1>`, H3's own tag.

On a rendered scene



Swap on a scene

Select the scene, press ⇄ on the take bar, pick a cast sheet (or choose a picture file), optionally a window and some direction, then ⇄ **Swap**. The result becomes the scene's next take — the old one stays in your assets.

One run covers up to about 15 seconds; a longer window is cut there and the log says so (swap the rest with a later start).

A whole movie — the Swap dialog

The StoryBoard's ⇄ **Swap** button opens the character swap dialog, for footage longer than one run (about 15 s):

- **Left** — click the box to choose the picture of the new character (or drop a picture / an Image asset on it); the prompt box below, with ✎ **Write prompt** under it (see next).
- **Middle** — the preview plays only the box. The menubar under it: **Load video** (or drop a Video asset on the preview), **Start**, **Duration** (the box), **Max** (the longest segment), ✂ **Auto segment** (cuts the *whole* video into back-to-back segments of Max length), ⇄ **Swap** (the box) and ⇄ **Swap all** (every segment not done yet). Below it the filmstrip with the box and a coloured band per segment; the chips under it select a segment, ▶ swaps just that one, × drops it.
- **Right** — the result preview with **Save** (a copy to the Video assets), ↻ **Retry** (same window, new seed), **Upscale** (same window and seed with the StoryBoard ⚙ upscaler) and **Compile**. Finished segments land in `projects/<name>/vidTemp`; ✓ **Keep** moves one to `projects/<name>/vidSwap`; Compile joins the ticked kept segments in time order into one video in the Video assets (`<video>_swapped.mp4`).

Engine: LTX Ripple (2.339.0) — the dialog's **Engine** dropdown switches between **MiniMax H3** (the swap above) and **LTX Ripple**. Ripple works in two steps per segment:

1. ✎ **First frame.** Qwen Image 2.1 puts the new character into the frame at the segment's start: same pose, framing, light and background. A card per segment appears under the segment chips. The first ⇄ **Swap** (or ✎ **First frames** in the menubar) makes them. On each card:
   - – ✎ redoes it.
   - – opens it in the Image editor. The dialog closes; touch it up and **Save**, then reopen ⇄ and press (the picture you saved last in the Image assets).
   - – uses any picture file.
   - – × removes it.
2. **The render.** By default the segment renders as soon as its first frame is made, so one ⇄ **Swap** does both steps (2.343.0). To check or touch up the first frames first, untick ⚙ → ▶ **render as soon as the ✎ first frame is made**, then press ⇄ **Swap** again when you're happy. The segment

renders through Tim's own workflow, **LTX_Ripple_FFAF_Edit_v11** (2.342.0; it ships in the full zip's `comfy_workflows` folder, together with its API version). It carries your first-frame edit through the clip, keeping the motion, timing and camera of the original.

The take is trimmed to the segment exactly and gets the original sound, so ✓ Keep and Compile work as with H3. (Tim's workflow puts the edited still at frame 0; the app renders one frame more and drops it, so the take lines up with the sound.)

Several first frames (2.348.0): **✎×N** in the menubar sets how many first frames each ✎ makes (1-4, a new seed each). Every one is kept on the segment's card as a small strip: **click** one to use it, **double-click** to see it big. × removes the one in use. With ✎×2 or more, ⇄ Swap makes the frames and waits for you to pick before rendering.

✎ **First frames** (2.349.0) always makes another ✎×N first frames for the segment the box is on - if the box isn't on a segment yet, it becomes one first. ⇄ Swap makes a first frame only for a segment that has none, and otherwise renders from the one you picked. A segment only has a card while it has first frames: × on its last one removes the card and the others move up.

If a render comes back **all black**, the segment fails with that reason instead of saving a black video (retry with a new seed; if it stays black, restart ComfyUI).

Looking at what was made (2.347.0): double-click a ✎ first-frame picture, or a finished segment in the Finished / Kept lists, and it opens in the big preview in the middle (a single click on a segment makes it the Result, so Save and ↻ Retry act on it). × **back to the video** (top left of the preview), Esc, or touching the box goes back to the video you are swapping.

⚙ **Settings** (the icon beside Load video, 2.342.0):

- **Video size:** "from the video" (its aspect at 768 on the short side, in multiples of 32), or your own width × height (from video fills it).
- **Duration:** the segment length (the same as **Max**; ✂ Auto segment cuts the video into these).
- **Steps** (LTX Ripple; 8 is Tim's workflow) and the **Ripple strength** (1.35).
- **LoRAs:** Add LoRA, the same LoRA rows as Basic Settings. They go into the Ripple render after the Ripple LoRA.
- **Seed:** Random, or Fixed with its number (picks one). A fixed seed also holds for ↻ Retry.
- **LTX Ripple: ▶ render as soon as the ✎ first frame is made** (on by default, 2.343.0).

The app matches the model files a Ripple or Viggle workflow names to the ones your ComfyUI lists (the same file in a subfolder, or the closest file of the same kind), and says so in the job log (2.345.0). Loading a video that doesn't look like VR switches VR off.

If a segment's render fails, its row says why (the worker's own words, for example a missing node pack or LoRA), and the message line under the menubar announces it.

The size and seed apply to MiniMax H3 too; H3's steps are its **Quality**.

Ripple needs the LTX 2.5 models, `LTX25_Ripple_v11.safetensors` in ComfyUI's `models/loras`, the conv video VAE, and the node packs **ComfyUI-LTXVideo** and **RES4LYF** besides KJNodes and VideoHelperSuite (*Settings → Models → Model catalog*; the packs in Setup). A missing node is named, with its pack, when the render starts.

Engine: Viggle (H3) (2.344.0) — Viggle-Animate, a MiniMax H3 finetune made for character replacement. It works exactly like LTX Ripple: each segment gets its ✎ first frame (the character repainted into the segment's first frame by Qwen Image 2.1), and Viggle carries that frame through the clip, keeping the motion, timing and camera. The workflow is Tim's upload, `viggle_animate_h3_api.json` (in the full zip's `comfy_workflows` folder).

- **Size:** "from the video" is the video's aspect at **480** on the short side (Viggle's own canvas is 480 × 832); set your own in ⚙.
- **Viggle steps** (⚙): **3** is the fast DMD schedule the workflow uses ("a 3 step Viggle Animate"); 5 and 7 are its slower, finer presets.
- **Viggle DMD** (⚙): the DMD LoRA's strength (1).

- **Speed patches** (⚙, off by default): the workflow's Sol-Attn attention patch and its fp16 accumulation. Both are faster when they work, but Sol-Attn crashed ComfyUI in a real test and fp16 accumulation can turn a take black, so the app leaves them off unless you tick the box.
- **Safe mode** (⚙, 2.350.0): renders with everything that can make a black take switched off. That means no speed patches, no fp16 accumulation, ComfyUI's own attention instead of the workflow's attention backend, and the fp16 H3 VAE instead of the int8 one when you have both. It is slower.
- **A black take:** when a Viggle take comes back all black, the app renders it once more in safe mode with the same seed. If that works, the take is filed as done, the line says , and ⚙ safe mode stays on for the next renders (untick it to try the fast settings again). If it is black in safe mode too, the row says so: then it is the model files or ComfyUI itself. Restart ComfyUI, update it and its custom nodes, and check that the model downloads finished.
- ⚙ LoRAs are added after the DMD LoRA. The Direction box is not used (Viggle has a fixed text conditioning).
- The take is every frame of the segment with the original sound.
- **Frame counts** (2.351.0): MiniMax H3 only renders 17 × n + 5 frames, at least 124 (about 5.2 s) and at most 362 (15.08 s). A shorter or odd-length segment is rendered at the next valid count and trimmed back afterwards. Before 2.351.0 the segment's own count (for example 120 or 48) was sent, which the H3 video VAE cannot encode. That was the likely cause of the black takes.

Engine: Viggle (native) (2.351.0; its own resident engine since 2.352.0) is the same Viggle-Animate model run by bitGen's own engine, with no ComfyUI graph. It is optional: ComfyUI stays the engine for everything else, and the plain **Viggle (H3)** engine above works without it. It uses Viggle's frozen prompt embedding, so no text encoder is needed, and renders 4 steps (3 passes) at flow shift 3, Viggle's own settings.

- **Install:** ⚙ → *Viggle native* → **Install**. It builds its own Python environment (`venvs/native`) and downloads only what is still missing.
- **Use my ComfyUI files:** the engine reads only each file's header and uses your ComfyUI Viggle model, its DMD LoRA and the H3 VAE when it can (bf16, int8, int8-ConvRot or fp8; nvfp4 cannot be read). The ⚙ block lists where each part comes from: your file by name, "ComfyUI file found - to use it", or missing. Only what is still missing is downloaded.
- **Memory** (2.352.0): the 2.351 engine loaded the 66 GB model in full precision *and* a pinned copy, more RAM than a 128 GB machine has, so the render sat at "no steps started". The model now stays packed as int8 in RAM (about half the size). Each block is unpacked on the GPU just before it runs, while the next one is already being copied (Wan2GP's technique, written from scratch). A small local server keeps the model loaded between segments, so only the first render of a session pays the load, and it hands the RAM and GPU back when a ComfyUI job starts. The log shows the load in steps ("packing the weights N%").
- **Installed on start** (2.353.0): every start checks whether Viggle (native) is installed and records it in `components.txt` in the install folder. A 0 starts the install straight away (follow it in the queue). The install reads the model files your ComfyUI workflows name (`viggle_animate_h3_api.json`, then the H3_Viggle and other workflows): the Viggle model, its DMD LoRA and the H3 video and audio VAEs. When those files are in ComfyUI's model folders they are used, not downloaded. Once the Python environment is built, each one is checked; only a file the engine cannot read is downloaded (one extra install, and the DMD LoRA is now its own 2.5 GB download). The rest is small: the configs and Viggle's prompt embedding.
- **`components.txt`** (2.355.0): `viggle native=0` installs it on the next start, `1` means it is installed (nothing happens), `2` means do not install it. The setup wizard writes 0 when you tick **Viggle (native)** and 2 when you don't; the box shows the download size. A 2 can still be installed with ⚙ → Viggle native → Install, and becomes 1 once it is installed.
- **Native steps / shift** (⚙): 4 and 3. More steps are not better for this distilled model.
- The ✎ first frames are still made by Qwen Image in ComfyUI. The native render itself does not need ComfyUI running.
- ⚙ LoRAs and the ComfyUI speed / safe-mode switches do not apply to the native engine.

- A take is 5 to 14.4 s (124 to 345 frames): a shorter segment is rendered at 124 frames and trimmed back.

Viggle needs the node pack **ComfyUI-Viggle-Animate-H3** (Setup), the Viggle model (`minimax_h3_ref2va_viggle_pruned_int8_convrot.safetensors`, 21 GB, in `models/diffusion_models`), the DMD LoRA (`viggle_animate_dmd_lora_r64.safetensors`, `models/loras`), the text conditioning (`fixed_embed_fwd_anyframe.safetensors`, `models/text_cond`) and the MiniMax H3 video VAE (either the int8 or the fp16 one). All of them are in *Settings* → *Models* → *Model catalog* under **Video - Viggle-Animate**. The workflow's speed patches (Sol-Attn, Chunk FeedForward) are used when your ComfyUI has them and skipped when it doesn't.

Some limits:

- **Quality** and **Upscale** are H3-only (not for Ripple or Viggle takes). For a bigger Ripple take, raise the size in ⚙ and ↻ **Retry**.
- VR is not supported on Ripple or Viggle.
- The Direction box's plain words ride along (e.g. "rain on her coat"); an H3 prompt with `<Video 1>` tags is ignored.

✎ **Write prompt** (2.335.0) — the vision model looks at the new character's picture and at the video in the middle of the box, and the LLM (the one picked under the Prompt docker's prompt) writes a swap prompt into the box: who to replace ("the man in the purple shirt"), the character's face, hair, outfit and art style, keep the motion, keep everything else. Anything already in the box rides along as your wishes ("keep his red cap", "photoreal"), so type a wish first and press it. It takes about a minute (the two models load in turn). Edit the result freely. A prompt that names both `<Video 1>` and `<Picture 1>` is sent as the WHOLE prompt; plain words are added under the built-in swap instructions instead. Without a vision model the LLM still writes it (the dialog says the picture was not described). The rules the LLM follows are the `minimax_h3_swap` guide in `lib/promptGuides.json` — copy it into `styles/prompt-guides.json` to change them.

Segments are cut on H3's frame grid so they join without a gap; the last fraction of a second that doesn't fill a grid step is left unswapped (the dialog says how much). Every segment is its own job in the queue, so you can close the dialog while they render.

VR video (360° and 180° stereo)

Set **VR** on the line under the video menubar: *360° mono*, *360° stereo (top-bottom)* or *180° stereo (side by side)*. A file whose name says so (`_360_TB`, `_180_LR`, `_360` …) switches it on by itself.

- The swap never renders the whole sphere. It renders a **virtual camera** aimed at the person: click the person in the preview (or type **Yaw** / **Pitch**), set **View** (how wide, in degrees) and **Shape** (3:4 or 9:16 for a standing person). **View** shows exactly what the swap will see — frame the person with a little room around them. A narrower view is sharper.
- For stereo, only the left eye is swapped; the right eye is built from it through the depth of your original footage, so both eyes show the same character at the right distance.
- The swapped view goes back into the untouched full-resolution frame with a soft edge. The result keeps the VR layout and size, is H.265 above 4K, is named with the players' suffix and carries VR (spherical) metadata. Output is 24 fps — the model's frame rate.
- Stereo needs `numpy` and `opencv-python-headless` in the app's Python (`start.bat` installs them).

Loading long movies: there is no length limit. Load video opens the app's file dialog and brings the file in *by path* (a hard link or a copy — nothing is uploaded through the page). If the player cannot show a movie (H.265 / 8K), the dialog makes a small preview copy; the swap always uses the original.

The swap style — `mmh3_chr_swap`

Swaps render through the video style **mmh3_chr_swap** (Settings → Video styles; *Used for: Character swap*, the swap workflow in its ref2va slot). Picking it in the prompt bar's style dropdown switches to Swap mode. Edit it like any video style — a different H3 model, load precision, text encoder, VAE, LoRAs or Sage attention — or **Duplicate** it to keep variants (the copy stays a swap style). Swap mode uses the style picked in the dropdown when it is a swap style, otherwise mmh3_chr_swap. A swap style can't be used for a normal render (the app says so).

The H3_Viggle style

H3_Viggle (2.352.0) renders a scene the Viggle-Animate way, through ComfyUI. The scene's **motion video** drives the movement, and a picture says who performs it:

- **ref2va:** the character sheet or picture performs it (`comfy_workflows/H3_Viggle_ref2va.json`);
- **fl2va:** the scene's first frame performs it (`comfy_workflows/H3_Viggle_fl2va.json`).

Both are Tim's Viggle-Animate graph with the side-by-side comparison taken out. They need the same ComfyUI files as the swap dialog's Viggle (H3) engine (see *Character swap* above).

From the prompt docker: pick the video style **H3_Viggle** and put the motion clip in the reference dock's **Video** tab (its first video is the motion). Then:

- **Ref** (ref2va): the first reference picture performs it. Viggle takes one picture; the others are left out and the log says so.
- **F/L** (fl2va): the first frame performs it.
- **t2va** is refused, because there is no one to animate.

On the scene board: set the StoryBoard's *Workflow* to **H3_Viggle**. Each scene then shows a **Motion** slot. Choose… picks a video from the project, another scene's take, or a file; × removes it. The scene's type picks the variant:

- **Ref:** the first cast sheet (or the scene's first picture) performs, ref2va.
- **First / Cont / Trans:** the frame the scene starts on performs, fl2va. That is its First picture, else the frame it continues from.

A scene without a motion video is refused in words.

How the length and the render work:

- **Length:** the board song's window for the scene, else the scene's own length, else the motion clip's length. If the clip is shorter, its last frame is held.
- **Frames:** H3 renders 17 × n + 5 frames, and Viggle at least 124. A shorter scene is rendered at 124 frames and trimmed. Longer than 15 s is chunked: chunk 2 onward continues from the hand-off frame (fl2va) and cuts the motion from where the last chunk stopped.
- **The motion clip** is cut per chunk at the render size (scaled to fill, centre crop, multiples of 32) at 24 fps, and its own sound stays on the take.
- Viggle's 3-step DMD schedule is used, not the ⚙ step count. The style's LoRAs are chained after the DMD LoRA. fp16 accumulation is off, because it can turn a Viggle take black. An all-black take is refused in words.
- The H3 accelerators, RefMods and the upscale stage do not apply.

The SFX dock

A dock for the project's sound effects, kept in projects/<project>/sfx as 48 kHz stereo WAV files. That means every cut is sample-exact and cutting again never loses quality.

- **Add** takes audio files (wav, mp3, flac, ogg, m4a, aac, opus, wma, aiff) and video files (.mov, .mp4, .m4v, .mkv, .webm, .avi, .wmv, .flv, .mpg, .3gp, .mts/.m2ts, .ts, .mxf). For a video, its audio track is extracted. A video with no audio track is refused, with a message saying so.
- **The waveform:** drag across it to select a part, or click it to move the play position. + zooms in on the selection and shows the whole sound again. Start and End take exact seconds. ▶ plays the selection, or the whole sound when nothing is selected.
- **Duration** gives the selection a fixed length. Press on the waveform to drop a box of that many seconds centred on the pointer, then drag to slide it. Changing the Duration resizes the box from its start, and typing Start or End moves the whole box. Leave Duration blank to drag-select any length. Any selection can be slid by grabbing it.

- **The gallery:** every saved cut (Keep or Delete) is also copied into the gallery's **Audio** tab as sfx_<name>, and the tab opens to show it. Cutting the same sound again, or ↶ Undo, updates that copy.
- ✂ **Keep selected** leaves only the selected part. **Delete selected** cuts it out and joins the two sides. Every cut edge gets a 5 ms fade so there is no click.
- **Where cuts go:** a cut changes the file itself, and **↶ Undo** brings back the version before the last cut. Tick **as a new file** to write name_keep.wav or name_cut.wav and leave the original alone.
- → copies the sound effect into the Sound assets, where the timeline and the references can use it. deletes the file.

Speech & dialog (2.621.0)

Under the sound effects, the SFX dock reads text aloud - one voice, or a dialog with a voice per speaker - with emotion, using OmniVoice or Ming-3.0 TTS, and saves it into the **Audio assets** (projects/<project>/sound).

1. **File name** (the box above the text) - the file is saved as that name .wav or .mp3; a name already there is never written over (_2 is added).
2. **The text** - type it, **Paste** it (or Ctrl+V), or write a brief and press **Write** for the LLM to write the lines (Ctrl+Z in the box brings the old text back). **tag** puts a sound such as `[laughter]` or `[sigh]` where the cursor is.
3. **Description** (the box below the text) - what it is for.
4. A dialog is one line per line: `Anna (happy): We made it!` - `Ben: Already?` - `(sad) It is over.` A line with no name goes on with the speaker above; a blank line is a longer pause. The **Cast** lists the speakers found - give each a voice (from Settings ▸ Audio ▸ Voices) and an emotion for the lines that name none. **One voice reads it all** reads the whole text in the chosen **Voice**.
5. Pick the **Engine**, **Emotion**, **WAV / MP3** and the **Pause** between lines, then **Read & save to Audio**. A player under it plays the result.

Emotions: neutral, happy, sad, angry, fearful, disgusted, surprised. **Ming-3.0 TTS** reads each line with its EMOTION task (with a cloned voice too). **OmniVoice** has no emotion switch: the emotion moves the pace a little, a voice saved as `Anna.happy` (Anna recorded happy) is used for Anna's happy lines, and the sound tags are read as sounds.

The file carries its own **metadata**: the title, the date and time it was made, the description, the whole text and the voices (WAV: the INFO list; MP3: ID3 - the text as its lyrics).

Save and Save as (preview dialog and gallery)

- **Save as** is the small icon at the top left of every gallery tile (on a song it is in the name row); it brightens when you hover the tile. It opens the file dialog in Save As mode: pick a folder, change the name if you like, and press Save. If a file with that name is already there, the dialog asks once, and pressing Save again replaces it. That folder becomes your **last save-as location**.
- **Save** in the preview dialog (next to ↗ Open externally) saves a copy straight into the last save-as location under the file's own name, with no dialog. If a file with that name is already there, the copy is named "name (2)" and so on: Save never overwrites. The first time, or if the folder is gone, it asks with Save as. Shift-click Save to choose a folder anyway. The button's tooltip names the folder it will use.
- The location is remembered for the whole install (workspace/save_as.json), across projects.
- From a phone or another computer (the remote page), both buttons download through the browser, because saving writes to the computer running bitGen Studio.

Transitions between song segments (bridge scenes)

On the StoryBoard's song strip, tick **Transition** (before), set its length in seconds (0.1 - 4) and a type, then press . Every segment except the last gives up that many seconds at its end to a **bridge** scene, so each segment still starts on the cut you placed. The board then reads, for three segments: scene 1, bridge, scene 2, bridge, scene 3. The song's .seg file keeps your cuts unchanged.

A bridge goes from the scene before's **last frame** to the scene after's **first frame**, so render both neighbours first (Render selected queues bridges after the scenes around them). Each bridge card has a type picker instead of Ref / First / Cont / Trans:

- **AI (fl2va)** - the native-audio board (MMH3_NativeAudio_MV) renders the move with the model, starting on the last frame and ending on the first frame. H3 renders 17n+5 frames, so a 0.5 s bridge renders 22 frames and is then fitted to exactly 0.5 s with the song under it.
- **A standard transition** - cross dissolve, dissolve, dip to black/white, wipes, slides, circle, clock, pixelize, blur (the timeline's list). Drawn with ffmpeg between the two frames: no model, and it renders in seconds on any workflow.

The music-video writer only writes the segments; the bridges stay where they are. Saving again with the same type keeps each card's own pick; a new type on the strip applies to every bridge; unticking Transition and saving removes the bridges.

Render size on the scene board

Board renders (▶ on a card, ↻ redo, Render selected - the native-audio music video included) are rendered at:

1. the StoryBoard's ⚙ **Resolution**, when it is set to 480p/540p/720p/1080p;
2. otherwise the left **Basic Settings** width x height (its Resolution/Aspect pickers, or the custom fields) - sent with every press and kept for the project, so queued and auto-started scenes use it too;
3. otherwise the app default size (Settings).

The job's notes say which one was used ("render size 1280x720 (Basic Settings, the left dock)"). Larger is sharper but slower and needs more VRAM; 720p plus the ⚙ upscaler (on a take you like) is a good balance.

The timeline's preview canvas

The timeline docker now has a preview canvas at the top, a toolbar on its left and a **Layers** panel on its right. in the timeline's menu bar shows or hides them, and the bar under the canvas makes it taller or shorter. Drag the timeline docker taller if the canvas looks cramped.

- **Drop** videos, pictures or sounds onto the canvas, from the Assets, the gallery or your computer. Each one lands on a **new layer** at the position marker, where you dropped it. A video dropped on an empty timeline goes on V1.
- **Toolbar:**

  – ✥ **Move (V)** — click a layer to select it, then drag to move it. Drag a corner to scale it, or the round handle on top to rotate it. Hold Shift to rotate in 15° steps or to move along one axis. V1 is the base picture: it always fills the frame and can't be moved.

  – **T Text (T)** — click to add a text layer, then type in the Layers panel. Font, size, colour, bold, italic, outline, shadow and alignment are all there. Click a text layer with T, or double-click it, to edit it.

  – **Hand (H, or hold Space)** pans the view. **Zoom (Z)** zooms in; Alt-click (or Ctrl + wheel) zooms out. fits the picture.
- **Play:** ▶ / under the canvas, or tap Space with the mouse over the canvas. Every layer plays together, with sound, and the timeline's position marker follows. loops the ruler selection, or the whole timeline.

- **Layers panel:** shows or hides a layer, ▲ ▼ change its order, deletes it, and double-clicking renames it. The selected clip's X, Y, Scale, Rotation, Opacity and length are underneath.
- **Compile** draws the moved, scaled and rotated layers and the text exactly where the canvas shows them.

Generating inside a selection (Fill)

1. Pick ⬚ **Select** (M) and drag a box over the area, or **Mask** (B) and paint it (Alt erases).
2. In the Layers panel's **Fill**, write what should be there, choose **Image** or **Video** (and seconds), then **Generate**.
3. The result lands on a new layer exactly over the area, with a soft edge (the feather) so it blends into the scene. An image lasts the ruler selection, or 3 s; a video starts at the position marker.

Animating a layer with keyframes

Keyframes make a layer move, grow, turn, fade or change colour over time. Between two keyframes the canvas works out every frame for you (a tween), and Compile draws the same animation.

1. Select the layer and move the position marker to where the animation should start.
2. Click ◇ **Key** in the Layers panel, under X / Y / Scale / Rot. This saves the layer's position, size, rotation, opacity (and a text layer's colour) at that moment. A ◆ appears on the clip in the timeline.
3. Move the position marker later, then change the layer: drag it, scale it, rotate it, or type new X, Y, Scale, Rot, Opacity or colour values. A second keyframe is made there automatically.
4. Press ▶. The layer animates from the first keyframe to the second.

Add as many keyframes as you like. Once a property has keyframes, changing it always edits the keyframe at the position marker, or makes a new one there. Properties without keyframes stay as they are.

- ◀ ▶ next to ◇ Key jump to the previous or next keyframe. On a keyframe, the menu beside them sets how it moves to the next one: **linear**, **ease in** (starts slowly), **ease out** (ends slowly), **ease in+out**, or **hold** (stays put, then jumps at the next keyframe). ×◆ deletes that keyframe.
- Click a ◆ on a clip in the timeline to edit that keyframe directly: opacity, volume, X, Y, scale, rotation, colour and the ease. Diamonds with an ease are blue; held ones are white.
- Sound clips can have volume keyframes too, and Compile now applies those as well.

Render size with H3

H3 renders in steps of 32 pixels. In the StoryBoard's ⚙, **H3 size** picks what happens to a size between steps: **704** rounds down (1280×720 renders as 1280×704), **736** renders 1280×736 and crops the take back to exactly 1280×720 (about 4% slower). Takes rendered before 2.379.2 have a dark strip at the bottom; render them again.

Scenes and the song

A scene renders against its own part of the song: a scene starting at 15.5 s uses the song from 15.5 s. With a song on the board, each rendered take is placed on the timeline at that same time, so picture and song stay in sync. To place each take straight after the previous one instead, untick "Place each rendered scene at its song time" in ⚙ Timeline settings.

Nudging timeline clips

- ⚙ at the right end of the timeline's menu bar opens **Timeline settings**. Under **Nudge**, set how many frames ← / → move a clip (default 1) and how many Shift + ← / → move it (default 24 = 1 second). The dialog shows each value in seconds too.

- Click a clip to select it, then press ← or → to nudge it earlier or later on its layer. A clip never goes before 0. V1 plays its clips one after another, so a V1 clip stops at the clip ahead of it. Use a video layer to overlap clips.
- The arrows are left alone while you type in a field or while a dialog is open.

Override style and the StoryBoard

With **Override style** ticked in Basic Settings, the StoryBoard's renders use Basic Settings' **Steps** and **CFG**. They take priority over the StoryBoard's ⚙ and the workflow's own count. With it unticked, the ⚙ decides (blank = the workflow's own). A render's notes say which one it used. A Steps value of 0 ("auto") leaves the count to the ⚙ or the workflow.

Recycle bin and resetting a project

- **Recycle bin** (left dock menu bar): opens the project's recycle bin. Files you delete from the gallery or the assets go there. Each file shows where it came from and when it was deleted.
  - ↩ **Restore** puts the selected files back in their folders (double-clicking a file does the same).
  - × **Delete** removes the selected files for good. It asks first.
  - **Empty** deletes everything in the bin for good. It asks first.
- ↺ **Reset project** (left dock menu bar): deletes the whole project folder, including its recycle bin, and creates it again, empty, with the same name and description. It asks you to confirm first. Every docker then starts empty, as if it were a new project: the queue's finished jobs for this project and their cached files are cleared too, and any of the project's renders still queued or running are stopped (the confirm box says how many).
- If a file in the project is still open in another program (a media player, Explorer's preview pane, ComfyUI), Windows won't let it be deleted. The reset still makes the project new, then tells you which files were in use. Those files are deleted the next time bitGen Studio starts.
- After a reset you can add characters (with pictures) straight away; uploads always go to the open project.

Karaoke: lyrics with a bouncing ball

- **Lyrics file:** every song bitGen makes saves its lyrics as `lyrics/<song name>.txt` in the project folder. For a song you brought in yourself, put its lyrics in a txt with the same name in that folder.
- **Karaoke** (StoryBoard menu bar): when a scene finishes rendering, bitGen makes a second video for it. That video has a transparent background and shows the scene's part of the song's lyrics: the current line turns from white to yellow as it's sung, the next line waits underneath, and a ball bounces onto each word. The scene's own video has no lyrics.
- The lyrics video goes on the timeline's **Karaoke** layer, lined up with its scene. That layer always stays on top, and Compile shows the scene through the transparent parts.
- **Timing:** if speech-to-text (faster-whisper, Settings → Audio) is installed, it times your lyrics against the singing. Without it, the lyrics are spread evenly over the song as a rough guide.
- Turning on also makes lyrics videos for scenes that are already rendered.
- **A bar or captions at the bottom of a scene?** Once the song is timed, each scene's prompt includes the words sung in that part (for lip sync). The prompt tells the video model not to show them as text. If a scene still has a bar, hide the Karaoke layer () and look at the scene on its own in the gallery: if the bar is there too, the model painted it into the video, so re-render that scene.
- **in the timeline's menu bar** makes a karaoke overlay for any song: pick a sound clip on the timeline (or a sound asset), the whole song or the ruler selection, check the lyrics, then **Generate**. The overlay goes on the Karaoke layer, lined up with the song.

- **Karaoke style** (in the StoryBoard's ⚙ and the timeline's ⚙): font, font size, colours, opacity, outline, text shadow, a background band (colour, opacity, border, box shadow, height, position from the top) and the ball's size and colours. New overlays use it.

Saving and opening a project

- **Save** keeps everything in the project: the StoryBoard's song and scenes, the prompt, the prompt chat, characters, environments, references, assets, the gallery, the timeline and each docker's fields (for example the music title and the enhanced prompt). Keep the name as it is to save in place.
- **The reference docker is kept as you change it** (2.548.0): a picture, video or sound added, removed or described is written to the project within a couple of seconds, so closing the app keeps it even without Save. All of them are kept - a save used to keep only 9 pictures, and a 10th refused the whole save. Pictures with the same name (pasted pictures are all "image.png") get their own names (image_2.png …).
- **F/L, Swap and Text no longer throw pictures away** (2.548.0): the dock keeps them all; F/L sends the first two (the rest show faded), Swap the first one, Text none. Back in Ref they are all there.
- **The StoryBoard's shared prompt is kept as you type** (2.548.0) - a moment after you stop, when you leave the box, and when the app closes; beside it still saves at once. Its @tags, the chapter references and each scene's own pictures are kept with the project through Save, Save as and closing.
- **Save as**: type a new name in the same box. The whole project is copied under that name, and the app opens the copy. The original stays as it was. An existing project's name is refused, so nothing is overwritten.
- **Open**: the project you're leaving is saved first. The app then reopens on the one you picked, with every docker exactly as it was saved.

Starting a new project

New project gives you a clean slate. The project you are leaving is saved first, exactly as it was. Then the app reloads on the new project, so every docker starts empty: references, characters and actors, environments, scenes, the prompt chat, assets, the gallery, the timeline and the music dock. The queue shows only this project's finished jobs. Anything still rendering for another project stays listed until it finishes. **Open** a project to go back to it; everything is where you left it.

Saving and clearing the prompt chat

- **Save** stores the conversation as **`<subject> - <date>`**, for example `A woman walking in rain - 2026-10-05`. If an LLM is already running, it names the subject; otherwise the name comes from the first words you asked. A name already taken gets `(2)`. The chat carries on in a fresh copy.
- **The list** next to it shows every saved chat, newest first. **Load** opens one.
- (right after the list) clears the current conversation: both the window and what the model remembers. Saved chats are kept. If the chat has messages, it asks first.

Screenshots and screen recordings (promos, tutorials)

The menu bar (the row of buttons at the top of the window) has two icon buttons, just before ⚙:

- **Screenshot** (Alt+Shift+S): saved to the gallery's Images as `screenshot_<date-time>.png`.
- **Record** (Alt+Shift+R): click to start. The button turns into a pulsing red ■, and hovering it shows the time. Click again to stop. The recording is saved as an H.264 mp4, `screenrec_<date-time>.mp4`, in the gallery's Videos, so it is ready for any editor or phone. Recording sends the video to the server every second, so long tutorials are fine.

Right-click either button for the options. They are remembered on this computer.

Option	What it does
App window / Whole screen	Capture just bitGen Studio (the cleanest frame for tutorials) or the whole screen it is on
Computer sound	Records the sound your computer plays, such as the renders and songs you preview. In the app this works on Windows; a browser tab asks you to tick "share audio"
Microphone	Records your narration, mixed with the computer sound
Countdown	Off, 3 s or 5 s before it starts, so you can get the mouse in place

In the app the window or screen is picked for you. In a browser tab, the browser asks what to share.

CFG on MiniMax H3

MiniMax H3 is distilled to render at **CFG 1**: one model pass per step, no negative prompt. The H3 graphs use a guider without a CFG input, so before 2.366.0 the Basic Settings CFG never reached them.

Set CFG **above 1** and the render switches to true CFG: a second, negative pass runs with the same pictures, voice and video references and a negative text (mumbling, babbling, extra words, repeating the reference audio). The guidance then strengthens only what your **prompt text** adds, such as the words in `<d>...</d>` and the acting. Each step runs twice, so the render takes about twice as long.

- Start at **2 to 3**. Higher values over-sharpen and over-saturate.
- Turn **Settings → ComfyUI → 4-step LoRA off** and use **20+ steps**. The turbo LoRA is made for CFG 1, and the job log warns when it is on.
- The job log line `true cfg 3: 1 BasicGuider -> CFGGuider ...` confirms CFG was applied.

Speech with emotion in a character's voice (no song)

On a board without a song, the characters speak their lines, each in their own voice. A character's voice clip is used for its **timbre only**: pitch, accent, texture and emotional colour. The model is told never to say the words heard in the clip.

- **How the writer writes lines:** every line carries its emotion and manner, e.g. `@anna whispers, voice trembling with fear: "Is someone there?"`. The matching face and body go in the beat before the line. It never quotes or reuses what a voice clip says.
- **What the app sends:** at render time each spoken line becomes MiniMax H3's own dialogue syntax. The delivery stays outside the marks and the words go inside, e.g. `whispers, voice trembling with fear, in the timbre of <Audio 1>: <d>[English] Is someone there?</d>`. The prompt ends by saying that only those lines are spoken. Lines in plain prose are what made the model babble.
- **Voice clips:** a clip longer than 8 s goes to the model as an 8 s sample with any silence at the start removed. That gives it the voice with far less speech to copy. Your file is unchanged, and the scene's notes say when a clip was trimmed.
- **Other languages:** write `"[Spanish] ..."` in the quotes and that language is used instead of English.
- **The prompt docker:** the prompt chat (Enter) is taught the same syntax. Local LLMs don't always follow it, so the app also fixes their output itself. The enhanced prompt comes back with its spoken lines marked, and the console says so. Every prompt render marks any line still in plain quotes, e.g. `says with a bright smile: "Thank you!"` becomes `says with a bright smile: <d>[English] Thank you!</d>`. Quoted names and signs ("a hotel called "The Palace"") are left alone. The voice is the `<Audio n>` the prompt calls her voice, or the one reference sound whose file name says voice.

How bitGen writes a MiniMax H3 prompt with speech

Every LLM that writes a video prompt (the prompt chat, Enhance, the reference dock) is taught this shape, with a worked example:

```
<Subject 1> is the woman in <Picture 1>, the charismatic host of a luxury travel ad.
<Subject 2> is the reception area and desk in <Picture 2>.
<Audio 1> is the voice timbre for <Subject 1> - pitch, accent and tone only.

Medium shot. ... She looks into the camera and says with a bright, welcoming smile, in the timbre of
<Audio 1>, <d>[English] Thank you!</d> She then holds her warm smile in silence as the camera slowly
pushes in. ... Sound: only her line in <d>, then a soft sea breeze and gentle room tone.
```

- Definitions come first, one per line: subjects, then the voice.
- Each spoken line sits inline, with its emotion outside the tags, its words inside, and the language marked.
- After a short line, the speaker does something silent.
- The prompt ends with one Sound line, and never stacks "do not speak" negatives.

Local LLMs drift from that, so the LLM's prompts and every prompt render are put into the shape by the app itself. The console or job log lists what was changed:

- `<subject n>` becomes `<Subject n>`.
- Quoted speech becomes `<d>`, and a `<d>` without a language gets `[English]`.
- Every `<d>` line goes "in the timbre of" the voice.
- The voice is defined once, after the subject lines.
- Stray "the voice is only the timbre of…" sentences and repeated "do not speak / no murmurs / DO NOT SPEAK AUDIO 1" instructions are removed. One closing line takes their place: only the `<d>` lines are spoken.
- A story beat like "she does not speak until the door opens" is kept.

Lip sync on a music-video board

On the **MMH3_NativeAudio_MV** board the song is the voice: a scene lip-syncs only when its prompt has the singing line (`… sings the exact supplied vocal audio with natural, precise lip synchronization …`). A scene that says `lips remain fully closed` renders **without** lip sync. From 2.356.0:

- **The song's timed transcript decides.** Transcribe the board song's lyrics once (Voice / Stems → Transcribe, lyrics mode). It is saved in the project's `transcripts` folder as `<song>.srt` / `.lrc`, or you can put your own `.srt` / `.lrc` beside the song. Then every scene's window is checked when it renders:
  - **words in the window:** a closed-lips line becomes the singing line, and the words go into the prompt (`The words sung in this clip: "…"`) to help the lips;
  - **no words** (an intro, a solo, a break): a singing line becomes the closed-lips line.
- **The writer** sees the same list (which scenes are sung, with their words). It must sing every scene and may close the lips only on a stretch the transcript (or your brief) says is instrumental. A guitar or a quiet mood is no longer a reason to close them.
- **No transcript:** a closed-lips scene stays as written, and the render note says it will have no lip sync.

The prompt LLM and your references

When you chat about a **video**, the LLM is given the reference dock the way the video model will receive it (2.354.0):

- the pictures in tray order as `<Picture 1>`, `<Picture 2>` …, each with its subject number, its role and a one-line description from the local vision model (described once, then remembered; Describe off skips it);

- the videos as `<Video 1..3>` and the sounds as `<Audio 1..3>`.

It writes prompts like: *`<subject 1> is the woman in <Picture 1>.` Keep the head, face and hair of the woman in `<Picture 1>`, keep the body of the woman in `<Picture 2>`. She does the action in `<Video 1>`, her voice is `<Audio 1>`.* If a model still writes a file name as a tag (`<photo.png>`), or `<sound 1>`, the reply is corrected to the right tag before you see it. MiniMax H3 calls sounds `<Audio n>`.

Reference videos and sounds in a prompt render

The reference dock's **Video** and **Sound** tabs feed MiniMax H3's reference render (ref2va) as `<Video n>` and `<Audio n>`: up to 3 of each. From 2.353.0:

- **Slots are added when needed.** If the style's ref2va workflow (for example **H3_Video_Ref**) was saved with picture slots only, the render adds the video and sound slots itself. A video's own sound goes in with it. The log says `the graph had no video reference slot - added 1`.
- **No picture needed.** Reference mode renders with only a video or a sound. Text mode with a video or sound loaded is rendered as ref2va so they are used, and the queue says so.
- First / Last on a reference workflow (H3_Video_Ref) sends the video and sound along too.

Append / Prepend a reference video (2.388.0)

Under every video in **References → Video** there is a **Use** dropdown:

- **Reference** (default): the model follows the video, as before.
- **Append**: **Generate** (prompt docker) takes the video's **last frame**, upscales it to the video's size (the ⚙ PNG upscaler, or lanczos without one) and renders a new part **starting** on it. The result is **video + new part** as one clip; the new part alone is kept as `<name>_part.mp4`.
- **Prepend**: the same with the **first frame**: the new part **ends** on it and goes **before** the video.

Always first / last frame (2.395.0). Append and Prepend always render **fl2va** (MiniMax H3's first / last frame model), never References or a plain image-to-video:

- **Picture 1** is the video's frame: its last frame for Append (the start of the new part), its first frame for Prepend (the end of the new part).
- **Picture 2** is the first picture in **References → Image**: the end of the new part for Append, its start for Prepend. A picture whose Type is **Last frame** (Append) or **First frame** (Prepend) is used instead.
- The Image tab labels that picture **Picture 2 · last frame** (or **· first frame**). fl2va takes two frames, so the other pictures are dimmed and marked **not used**. The video's own card says what it gives.
- With no picture, the new part starts on the video's last frame (Append). For Prepend it is made forward from the first frame and played backwards, silent, so it lands on the video.
- No `<subject n>` lines are added to the prompt, because the pictures are frames, not subjects.

The LLM is told. The prompt chat gets the details:

- that this is an fl2va Append or Prepend of that video;
- that **Picture 1** is its last (or first) frame. The frame is taken out and described by the local vision model, like any reference picture;
- that **Picture 2** is your picture and where the new part starts or ends.

They write one continuous shot that carries on from the video, or leads into it, with no mapping lines and no cuts.

One video at a time can be Append / Prepend. It works with MiniMax H3 styles and Wan styles. **Refmods** (a character's extracted identity mod) also work in fl2va renders - experimental, used by the board's continuation chunks.

The floating prompt docker (2.389.0)

The prompt docker floats over the app as a window of its own.

- **Move it:** drag its titlebar. Its edges stick to the edges of the other dockers and of the window.
- **Resize it (2.398.0):** drag any edge or corner. A side docker's bottom edge sizes it there too. Docked in the column, its handle below sizes it. If nothing open is below it, the dockers above give it the room.
- **Collapse it:** click its titlebar.
- **Dock it:** drag it over another docker and the edge you are near lights up. Drop it there:
  - top or bottom puts it into the column above or below that docker;
  - left or right puts it beside that docker. A splitter between them sets its width.
  - A side docker (Settings, References, Assets, Queue, Gallery) takes it at its top or bottom.
- **Float it again:** drag its titlebar out of the dock.
- **P** shows and hides it, anywhere in the app. It does nothing while you are typing in a box, while a dialog is open, or with Shift held (Shift+P is the timeline's Path tool).
- Where it is, and its size, are kept in docker_status.json.

Basic Settings and LLM tokens float (2.393.0)

Basic Settings and **LLM tokens** are no longer in the left docker. Each one is a floating docker of its own.

- **S** shows and hides Basic Settings. **L** shows and hides LLM tokens.
- The keys do nothing while you type in a box, while a dialog is open, or with Ctrl, Alt or Shift held.
- Over the image editor and the timeline canvas, S and L stay those tools' keys (Clone Stamp and Lasso in the editor, the 3D Scale widget on the canvas). Move the pointer off them first.
- The left docker has two buttons where the sections were: **LLM tokens L** and **Basic Settings S**. Click one to show or hide its window. A button is lit while its window shows.
- **Move a window:** drag its titlebar. Its edges stick to the dockers' edges, to each other and to the window's edges.
- **Resize a window:** drag any edge or corner.
- × hides it.
- Whether each window shows, where it is and its size are kept in docker_status.json (`floats`).

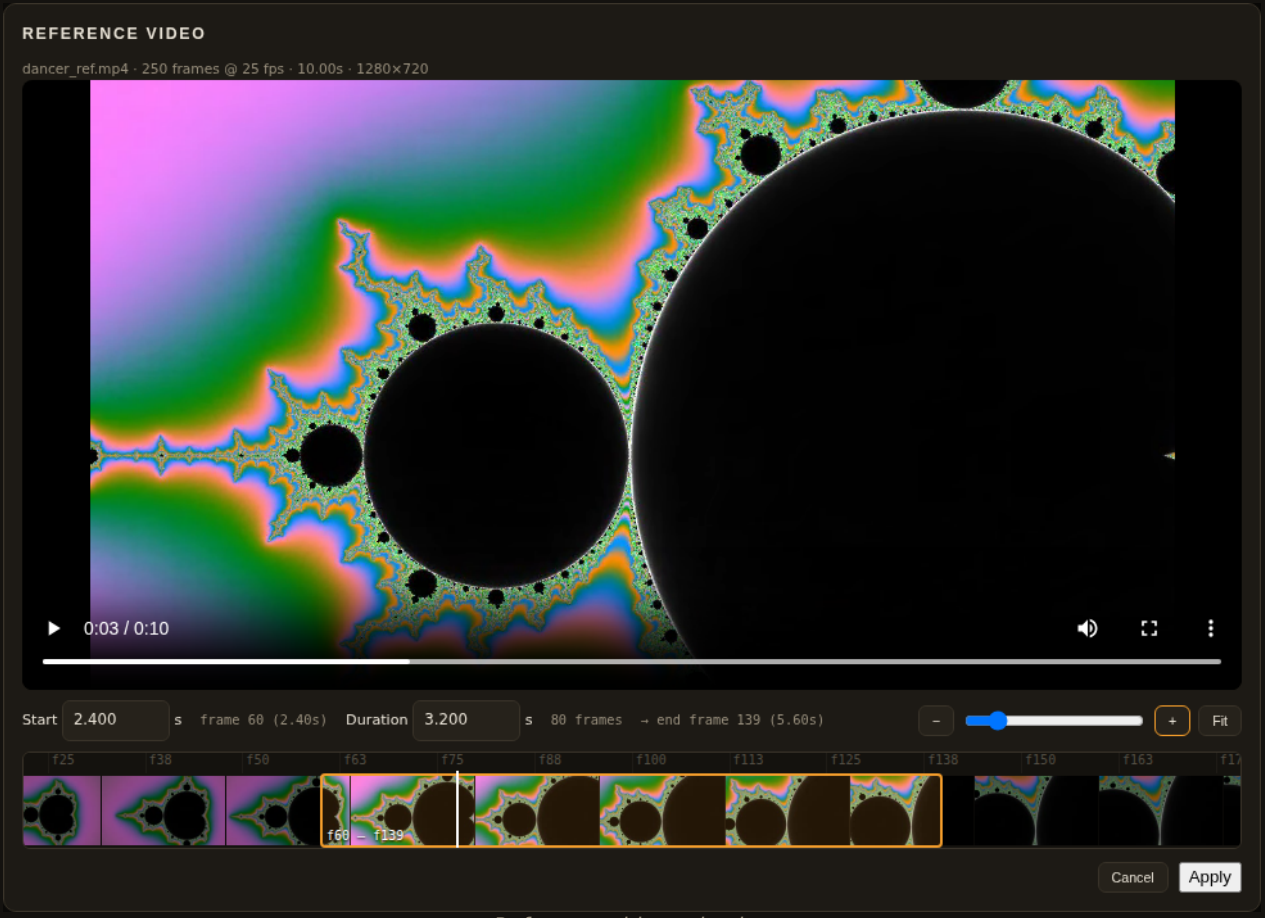
They are dockers (2.398.0). Drag Basic Settings or LLM tokens by its titlebar and the place it will go lights up:

- **Inside a docker, as a tab:** drop it on that docker's titlebar. The docker gets a row of tabs: its own name, then each panel in it. Click a tab to switch. This works for any docker, including Settings on the left.
- **Above or below a bottom docker** (Preview, Video, Music…): drop it near that docker's top or bottom edge. It takes its own place in the column, and its bottom edge sets its height.
- **Beside a bottom docker:** drop it near the left or right edge. Its inner edge sets its width.
- **Stacked in a side docker** (Settings, References, Assets, Queue, Gallery): drop it near the top or bottom edge. Its bottom edge sets its height.
- **In the left docker's list** (2.401.4): drag it over the Settings list (System, the panel buttons…). A line lights up between two sections and the label says where, for example "Dock Basic Settings in Settings - above the panel buttons". Drop it and it sits in the list at its full height and scrolls with it, like the sections did before. Put LLM tokens and Basic Settings in any order. Its titlebar still drags it out again.
- Let go anywhere else and it floats there.
- **Float it again:** drag its titlebar, or its tab, out.

- **S**, **L** and × still hide and show it wherever it is docked.

Where each one is docked, its sizes and which tab is showing are kept in docker_status.json, so they come back the same after a restart.

Choosing the frames of a reference video



Reference video selection

Click any video in the **References → Video** tab:

- the **preview** plays only the selected frames, looping;
- the **filmstrip** zooms with **− / + / the slider / Ctrl+wheel** (from the whole video down to one thumbnail per frame); **Fit** shows it all;
- the **selection box** is as long as the **Duration** — drag it, click the strip to centre it, or nudge it with ← → (one frame; Shift = one second). Start and end frames are shown under the preview.

Apply cuts exactly those frames into `<name>_sel_f<first>-f<last>.mp4` beside the original and puts it in the reference slot — Ref, F/L and Swap then use only that part. Clicking the cut again reopens the original with the box where you left it. **Cancel** changes nothing.

Images, ControlNet and face fix

Set the Generate dropdown to **Image** and pick an image style (FLUX.2 Klein, SDXL, Z-Image, Qwen-Image…; manage them in **Settings → Image**). In Basic Settings:

- **ControlNet** (2.593.0: an add-on - switch it on in Settings ▸ General) — renders a first picture, turns it into a control image (lineart, depth, pose…) and renders the final with a **second model** (e.g. a photoreal SDXL or Klein), with its own prompt, strength and end. *Final image only* hides the intermediate pictures.
- **Face fix** (2.593.0: an add-on - switch it on in Settings ▸ General) — FaceDetailer re-draws each face of the final picture.
- The **Image editor** docker edits pictures with AI; see the next section.

- **Qwen Image 2.1 reads the reference dock** (2.340.0). With **qwen2_1** (up to 10 pictures) or **Qwen Image 2.1 edit** (8), Generate Image sends the dock's pictures in order as `<image1>`, `<image2>`, ...
  - Write `<image2>` etc. in the prompt to point at them. The dock's own `<Picture n>` tags are translated for you.
  - If the prompt names none, one line is added saying what each picture is, from its Type (e.g. "<image1> is the person, <image2> is the clothing").
  - **qwen2_1** with an empty dock renders from text alone.

The image editor (with AI)

From 2.389.0 the editor has a menu bar, an options bar, a tool column and panels (the same layout as PhotoCraft). Open it with **Editor** in the bottom bar. Drop a picture on it (from the gallery, assets, references or a file) - 2.549.0: with a picture already open it asks ▦ **Add as a layer** or **Open as a new document** (Enter = a layer, Esc = neither, Shift while dropping = a layer at once) - or use **File ▸ Open** (Ctrl+O) or **File ▸ New** (Ctrl+N). or **F** gives it the whole screen. **Tab** shows or hides the panels; **Shift+Tab** hides only the panel groups.

Type along a path, warped and 3D text (2.397.0)

The **T** button now holds three tools; **Shift+T** switches between them:

- **Horizontal Type**, as before.
- **Vertical Type**: one letter under the next. Each new line is a new column to the right.
- **Type on a Path**: drag a line across the picture, or click points and double-click (or press **Enter**) to finish, then type. The letters follow the line. With this tool, drag a point of the path to reshape it. **Start** moves the text along the path and **⇅ Flip** puts it on the other side. **Esc** cancels a path you are drawing.

The Type options bar has four new buttons: **TT** (all caps), **U** (underline), **S** (strikethrough) and **↕T** (vertical).

Shape / 3D... (in the options bar, in Properties for a text layer, or **Layer ▸ Text Shape**) changes a text layer live:

- **Warp**: Arc, Arc Lower, Arc Upper, Arch, Bulge, Flag, Wave, Fish, Rise, Squeeze or Inflate, with a Bend from −100 to 100.
- **Round a circle**: the text on a circle. Set the radius, where the text is centred, and whether it runs over the top or reads left to right along the bottom.
- **Along a drawn path**: for text made with Type on a Path.
- **3D**: the letters are extruded back. Set the depth, the direction, the shading to the back and the side colour (the text's own colour, darker, unless you pick one). Add **Bevel & Emboss** (below) to round the front.

The text stays editable the whole time: click it with the Type tool to change the words. **Cancel** puts it back as it was.

Layer styles: Bevel & Emboss, Satin, Pattern Overlay, blend modes, Styles (2.397.0)

The Layer Style dialog (**fx**, or **Layer ▸ Layer Style**) now lists The ten layer styles:

- **Bevel & Emboss**: Inner Bevel, Outer Bevel, Emboss or Pillow Emboss. Set the depth, up or down, size, soften, the light's angle and altitude, and the highlight and shadow colours and opacities.
- **Satin**: a silky sheen inside the shape.
- **Pattern Overlay**: ten patterns (checkerboard, stripes, diagonal stripes, polka dots, grid, bricks, waves, carbon fibre, noise, halftone) in two colours you choose, with a scale.
- **Color, Gradient and Pattern Overlay** each have a **blend mode** (Multiply, Screen, Overlay, Soft Light and twelve more).
- **Gradient Overlay** has five styles (linear, radial, angle, reflected, diamond), plus reverse and scale. It lines up with the layer's own content, not the whole picture.

- **Styles** at the top of the dialog gives one-click looks: Gold, Chrome, Neon, Embossed, Glass, 3D Pop, Retro, Fire, Ice and Letterpress. Each one replaces the layer's styles. You can change any setting afterwards.

Bevel & Emboss and Satin are saved to and read from PSD files. Pattern overlays in a PSD are still left out.

New filters (2.397.0)

- **Filter ▸ Render ▸ Lens Flare**: four lens types (50-300mm Zoom, 35mm Prime, 105mm Prime, Movie Prime), brightness and a tint. **Click the picture while the dialog is open to place the flare.** On an empty layer it makes its own light.
- **Filter ▸ Distort ▸ Swirl**: a whirlpool around a centre. Set the strength, radius and falloff, and click the picture to place the centre.
- **Filter ▸ Stylize ▸ Drop Shadow**: a shadow cast by the layer's pixels, with angle, distance, blur, spread, opacity and colour. It needs transparency around the pixels, for example text or a cut-out.
- **Filter ▸ Stylize ▸ 3D Extrude**: extrudes any layer's pixels with depth, direction and shading. The sides are a darker copy or a solid colour.
- Clicking the picture also places the centre for **Radial Blur**.

Colours, New, the Paint Bucket and gradient fills (2.396.0)

- **File ▸ New** fills the new picture with the **background colour** (the back swatch in the tool column) unless you pick White, Black or Transparent. The colour is shown beside the choice.
- **⇄ swap (X)** swaps the foreground and background colours. If the picture's **Background** layer is still one plain colour, it is repainted with the new background colour. That is one step, so **Ctrl+Z** brings the old colour back. A Background you have painted on is left alone. The Color panel has a ⇄ button too, and ◩ **(D)** sets black / white.
- The colour swatches are bigger and outlined, so a dark colour still shows on the dark theme.
- The **Paint Bucket** has its own button under the Gradient tool. **G** picks the one you used last and **Shift+G** switches between them. Its options bar:

  – **Fill**: Foreground, Background or **Gradient**.

  – With Gradient, a click fills the area from top to bottom. Drag to set your own direction. It uses the Gradient tool's colours and shape (linear, radial…) and fills only the clicked area.

  – **Tolerance**, **Contiguous** (off = every matching pixel in the picture), **All layers** (find the area in everything you see) and **Opacity**.

  – A click inside a selection fills the selection. Alt-click picks a colour.

It looks like bitGen (2.393.0)

- The editor uses the **bitGen** theme: the app's dark colours and its orange accent. Its menu bar is the same height and colour as bitGen's menu bar. Its status line is the same height and colour as bitGen's status bar.
- **Full screen (or F)** keeps bitGen around the editor. bitGen's title bar, menu bar and status bar stay where they are; the editor fills the space between them. The editor's status line moves into bitGen's status bar ("Editor · …"). **Esc**, **F** or again puts it back in its docker.
- **Window ▸ Theme** still has Pro (dark), Studio (light) and Classic.

The layout

- **The menu bar** holds File, Edit, Image, Layer, Select, Filter, View, Window and Help. Every item has its usual shortcut.
- **Ctrl+K** opens the command palette: type part of a command and press Enter.
- **Help ▸ Keyboard Shortcuts** lists every key.
- **The options bar** under the menus shows the settings of the current tool.
- **The tool column** has the usual groups and keys:

  – selection and move: Move (V), Marquee (M), Lasso (L), Magic Wand and the quick selection brush (W), Crop (C), Eyedropper (I);

  – retouching and painting: Spot Healing and Healing Brush (J), Brush and Pencil (B), Clone Stamp (S), History Brush (Y), Eraser (E), Gradient and Paint Bucket (G), Blur, Sharpen and Smudge (R), Dodge, Burn and Sponge (O);
  – type and shapes: Type (T), Shape (U);
  – view: Hand (H), Zoom (Z).
- A small triangle on a tool means it has more tools in its group. Press Shift with the key to cycle through them, or right-click or long-press the button to pick one.
- At the bottom of the column: D sets the default colours, X swaps them, and Q (◘) turns Quick Mask on or off.
- **P is not a tool here.** P shows and hides the prompt docker everywhere.
- **The panels** on the right come in three groups:
  – **Color** and **Swatches**;
  – **Properties**, **Adjustments**, **AI** and ◈ **Control**;
  – **Layers** and **History**.
- **Window ▸ Theme** picks Pro (dark), Studio (light) or Classic.
- **View** turns on the Rulers (Ctrl+R) and the Grid (Ctrl+').

Layers

- **Kinds of layer:**
  – pixel layers;
  – **adjustment layers** (the Adjustments panel, or Layer ▸ New Adjustment Layer). They change everything below them without touching any pixels. There are 14: Brightness/Contrast, Levels, Curves, Exposure, Vibrance, Hue/Saturation, Color Balance, Black & White, Photo Filter, Channel Mixer, Gradient Map, Invert, Posterize and Threshold. Their settings are live in Properties.
  – **fill layers** (solid colour or gradient);
  – **text layers** (T: click and type; Esc or Ctrl+Enter commits; click the text again to edit it);
  – **shape layers** (U: rectangle, rounded rectangle, ellipse, triangle, polygon, line), which stay editable;
  – **groups** (Ctrl+G groups the active layer; Shift+Ctrl+G ungroups; 2.549.0: the keys work right after a click in the Layers panel too). A group set to *Pass Through* lets its adjustment layers reach the layers under it. Any other blend mode composites the group on its own. 2.547.0: with **several layers picked** (Ctrl-click / Shift-click in the Layers panel) **Ctrl+G** puts them all in one new group, in their order; with **one group** selected **Ctrl+G** ungroups it. With several layers picked the button, Layer ▸ Delete Layer and Shift+Del delete them all (one Ctrl+Z brings them back).
- **Layer masks** (◫, or Layer ▸ Layer Mask) show where white and hide where black:
  – click the mask thumbnail to paint on it;
  – Alt-click it to view the mask itself;
  – Shift-click it to disable it;
  – Ctrl-click it to load it as the selection.
  – Properties sets its density and feather, and can invert, apply or delete it.
- **Clipping** (Alt+Ctrl+G) shows a layer only where the layer below it has pixels.
- **Layer styles** (fx, or double-click a thumbnail): Drop Shadow, Inner Shadow, Outer Glow, Inner Glow, Stroke (outside, inside or centre), Colour Overlay and Gradient Overlay. They can be copied and pasted.
- **Fill opacity** dims the pixels but not their styles.
- **Locks:** transparent pixels, image pixels, position, all.
- **Right-click a layer** to set a colour label and more.
- **Drag layers** in the panel to reorder them, or drop one onto a group to put it inside.
- **Arrange:** Ctrl+[and Ctrl+] move a layer down and up; add Shift for back and front.

- **Merging:** Merge Down (Ctrl+E), Merge Visible, Flatten Image, Rasterize.
- **History:** the History panel lists every step (the last 80). Click one to go back or forward to it.

Select first. Everything else respects the selection.

- **Combining:** Shift adds, Alt subtracts, Shift+Alt intersects (or use the buttons in the options bar). The options bar's **Feather** softens the next selection.
- **Select menu:**
  - All (Ctrl+A), Deselect (Ctrl+D), Reselect, Inverse (Shift+Ctrl+I);
  - **Color Range**;
  - **Modify:** Border, Smooth, Expand, Contract, Feather (Shift+F6);
  - load a layer's transparency or its mask as the selection.
- **Quick Mask (Q):** the selection shows red where it is not. The brush adds to the selection and the eraser removes from it. Press Q again to turn it back into the selection.

Paint and retouch

- **Brush (B)** uses one of the 20 presets (▾). The keys 1…0 set the opacity. Alt-click or Ctrl-click picks a colour. A tablet's pressure drives the size and/or flow.
- **Pencil:** a hard edge.
- **Clone Stamp (S):** Alt-click the source, then paint. Choose aligned or not, this layer or all layers.
- **Healing Brush:** the same, with the colour matched to the spot it is painted on.
- **Spot Healing (J):** paint over a spot and it is filled from around it.
- **History Brush (Y):** paints the layer back as it was opened. Use Set source to change that point.
- **Dodge, Burn, Sponge (O):** a tone range and an exposure.
- **Blur, Sharpen, Smudge (R):** a strength.
- **Gradient (G):** linear, radial, angle, reflected or diamond. The colours can be foreground to background, foreground to transparent, black and white, or a spectrum.
- **Paint Bucket (Shift+G):** fills an area of similar colour.
- **Crop (C):** drag a box, pick a ratio, Enter crops (Esc cancels).
- **Free Transform (Ctrl+T, or Transform on the Move tool's bar):** drag inside to move. A corner scales and keeps the proportions; Shift-drag a corner to scale freely. Drag outside the box to rotate (Shift for 15° steps). 2.551.0: a **middle handle skews** it (the cursor turns into a slanted arrow; top / bottom = sideways, left / right = up and down); **Shift or Alt + a middle handle** stretches that side. You can also type X, Y, W, H, the angle and the two skews (⇋ ⇵) in the options bar. Enter applies it. With a selection that holds none of the layer's pixels, the whole layer is transformed.
- **Move (V) and Paste (2.550.0):** Ctrl+V puts the picture in the middle as a new layer and drops the selection (it sat on the copied area, so the Move tool moved an empty part of the new layer). The Move tool moves the selected pixels, or the whole layer when there is no selection - or when the selection holds none of the layer's pixels.
- **Edit ▸ Transform** rotates or flips the layer.
- **Image:**
  - Adjustments (all 18, applied directly to the pixels);
  - Auto Tone, Auto Contrast and Auto Color;
  - Image Size and Canvas Size (with an anchor);
  - Image Rotation;
  - Crop to the selection;
  - Trim.
- **Filter:** Blur, Distort, Noise, Pixelate, Render, Sharpen and Stylize submenus, 23 filters in all. Each previews live. Ctrl+F repeats the last one.
- Painting, filters and fills go onto the mask while the mask thumbnail is selected, and onto the selection while in Quick Mask.

Saving

- **File ▸ Save to Images** (Ctrl+S) saves the visible picture into the project's Images.
- **File ▸ Export As** picks PNG, JPEG or WebP, the quality and the scale. It shows a preview and the file's size, and can save to Images or download.
- Copy and paste use Ctrl+C, Ctrl+X and Ctrl+V. Paste in Place is Shift+Ctrl+V and Copy Merged is Shift+Ctrl+C. Pictures from the system clipboard paste in too.

AI works like Krita AI Diffusion. The model sees the selection plus a **context** margin around it, scaled to at most the **MP** budget. The answer comes back as a **new layer** that holds the new pixels only inside the **feathered** selection.

- **Generate** regenerates the selection from the prompt, using an inpaint style (FLUX Fill inpaint, SDXL…).
- **Refine** reworks it at a **strength** below 100 %: the shapes stay and the detail changes.
- **Remove** removes what is selected and continues the surroundings. The prompt is optional.
- **Edit** follows an instruction with an edit model: **Qwen Image Edit 2511**, **FLUX.1 Kontext** or FLUX.2 Klein edit ("make her jacket red", "remove the glasses"). With nothing selected it edits the whole picture.
- **Background** keeps the selection and regenerates everything else.
- **Results** makes 1–4 versions. Click one to compare it, then use ✔ **Apply** (Enter) to keep it as a layer, or ✕ **Discard** (Esc).
- **Expand** grows the canvas on one or all sides and selects the new strip. Then Generate continues the picture into it.

If the prompt box in the panel is empty, the main prompt is used. The models download in setup, under *Image editing*.

◈ **Control layers** work like Krita AI Diffusion's. A control layer guides Generate, Refine, Remove and Background through a ControlNet. It is never part of the picture itself: saving and the AI both skip it.

- Pick a type and press **layer**. The types are scribble, line art, soft edge, canny, depth, normal map, pose and segmentation.
- For the line types, draw dark lines on the layer with the brush.
- Or press **From image** to have the type's preprocessor make the layer from the picture. You can then touch it up with the brush or eraser.
- ◈ **active** turns the layer you are on into a control layer.
- Each control layer has a **strength**, a **start / end** range (the part of the sampling where it guides) and its **ControlNet model** from ComfyUI's `models/controlnet`. *auto* picks a model whose name says the type, or else a union / ProMax model.
- Use up to four at once. Untick one to leave it out of the next run.
- Edit models take the picture itself, so control layers are not sent with an Edit.
- From image needs the *comfyui_controlnet_aux* node pack. Canny uses ComfyUI's own node.

Adjust and ✦ Filter are menus that work on the active layer, and only inside the selection when there is one (blended by its soft edge).

- **Adjust:** Brightness/Contrast, Hue/Saturation, Levels, Color Balance, Exposure, Vibrance, Black & White, Sepia, Invert, Posterize, Threshold and Auto Contrast.
- **Filter:** Gaussian Blur, Motion Blur, Unsharp Mask, Add Noise, Median, Pixelate, Emboss, Find Edges, Vignette, Glow, Chromatic Aberration and Oil Paint.
- The dialog previews live. **OK** makes the change one undo step, and **Ctrl+F** repeats the last filter.

Segment Anything, Compose, References and PSD (2.390.0)

Segment Anything finds the outline of a thing for you. It runs on this computer. The first time, open **Select ▸ Segment Anything Settings…** and press **⬇ Install**. It downloads the models once, then works offline. The SAM ViT-B model is 375 MB; ViT-H is slower but has better edges. If ComfyUI already has a SAM model in `models/sams`, it is used.

- **Object Selection Tool (W):** click a thing to select it. **Shift**-click adds another thing; **Alt**-click takes one away. Drag a box around something to select what is inside the box.
- **Find:** in the tool's options bar, type words such as *the person's head* or *dog, car*. **Select** selects them; ◐ **Mask** makes them the active layer's mask.
- **Select ▸ Subject…** does the same from a dialog. It can also put each match on its own layer.
- **Layer ▸ Segment Layer…** splits the layer into its parts, each on its own layer in a group. Tick **cut out of the layer** to take them out of the original. Fill in **Only** to keep just the things you name.
- In the **AI** panel, **Select from prompt** and ◐ **Mask from prompt** use the words in the prompt box.

Compose (layers as references)

1. **Ctrl**-click the layers you want in the Layers panel. The order matters: **1** is the scene, **2** and **3** are the things to bring in. Each picked layer shows its number.
2. In **AI**, pick **Compose** and an Edit model (Qwen Image Edit 2511 is the default).
3. Write what to make, for example *put the dog from Layer 2 on the sofa in Layer 1*. Layer names in your text are passed to the model as Picture 1, 2, 3.
4. Press **Compose**. ✓ **Apply** keeps the result as a new layer.

→ References:Layer ▸ Send to References (or the **→ References** button) puts the picked layers into the reference dock. **Generate** in the prompt docker then uses them, for videos too.

Seeing the mask while you paint it (2.391.0)

When a brush paints on a layer mask, the area the mask hides turns red as you draw. The red goes when the stroke ends. **View ▸ Mask Overlay** (**) keeps the red showing the whole time the mask is selected.

PSD files

- **File ▸ Open PSD…** opens a PSD, or drop one on the editor.
- **File ▸ Save as PSD…** (Shift+Ctrl+S) saves a PSD. It is kept in the project's `psd` folder and can also be downloaded. **File ▸ Open PSD from Project…** opens it again.
- These come across: layers, groups, masks, opacity, blend modes, visibility and layer styles.
- Text layers and smart objects open as pixels.
- Adjustment layers are left out (the editor says how many). The saved PSD still shows the finished picture.
- Vector masks and path shape layers come across as paths both ways (2.421.0) - see below.

Pen, paths, vector masks and custom shapes (2.421.0)

The first part of the professional editing features.

The tools.P (Shift+P cycles) - the **Pen**: click = a corner anchor, drag = a smooth anchor (its handles mirror), click the first anchor = close, Alt = a cusp (one handle), Shift = 45° steps, Ctrl = Direct Selection for a moment, Enter / Esc leaves the path open. Over the picked path the Pen adds an anchor on a segment and deletes one on an anchor (**Auto Add/Delete**). The **Freeform Pen** draws by hand and fits Béziers (**Curve Fit** px). **Add / Delete Anchor Point** and **Convert Point** (click: curve ⇄ corner; drag an anchor: new handles; drag a handle: a cusp). **A** - **Path Selection** (whole components: drag moves, Alt-drag copies, Delete removes, their operation, **Merge Components**, **Transform…**) and **Direct Selection** (anchors and handles, Shift adds, a box picks several, arrows nudge, Delete removes, Corner / Smooth). **U** - the Shape tool and the new **Custom Shape** tool (heart, star, burst, arrow, chevron, plus, cross, check, lightning, speech bubble, drop, moon, house, cloud, ring, frame - and yours: **Edit ▸ Define Custom Shape…** saves the picked path).

Shape or Path. The options bar's **Mode**: *Shape* draws a shape layer (the foreground fills it, the background colour + W strokes it), *Path* draws the **Work Path**. The operation buttons: ▣ New Layer, ⊕ Combine, ⊖ Subtract Front Shape, ⊗ Intersect, ⊘ Exclude - anything but New Layer adds the new part to the active shape layer. The plain Shape tool (rectangle, ellipse …) has the same buttons. Every shape layer becomes an editable path the moment you move one of its anchors.

The Paths panel (next to Layers and History): the Work Path, your saved paths, and the active layer's Shape Path / Vector Mask. Click one to pick it (its anchors show with the path tools); **Ctrl-click** = load it as a selection (Shift adds, Alt subtracts). Double-click the Work Path to save it, a saved path to rename it. The buttons: ● fill it with the foreground, ○ stroke it with the brush, ⬚ as a selection, make a work path from the selection, ◇ a vector mask from it, new, duplicate, more, delete (Alt-click ●, ○, ⬚, for their options: opacity / feather, brush / pencil / eraser + simulate pressure, feather + add / subtract, tolerance).

Vector masks.Layer ▸ Vector Mask: Reveal All / Hide All / Current Path / Disable / Rasterize / Delete. A vector mask shows its layer inside the path (Reveal All: everywhere except the shapes you draw in it). Its thumbnail sits beside the layer mask's: click it to edit the path, Shift-click turns it off. Properties has its Density, Feather, Invert, Link and Rasterize (into the pixel mask). A linked vector mask moves with the layer.

Everything follows the picture: crop, canvas size, image size, rotate and expand move / scale the paths. Every edit is one undo step.

SVG and PSD.File ▸ Export Path as SVG…, **File ▸ Place SVG as Shape Layer…** (and ▸ Import SVG as Path…). A PSD keeps vector masks and path shape layers (PSD vector shape layers: the path, the fill, the stroke) both ways.

Smart objects, smart filters and the transforms (2.422.0)

Smart objects.Layer ▸ Smart Objects: Convert to Smart Object (or **Filter ▸ Convert for Smart Filters**) turns the active layer - a pixel, text, shape or fill layer, or a whole group - into a smart object (▣ on its thumbnail). Its layers go inside, still editable; the smart object shows how they look. It is always re-drawn from its contents, so you can scale it down and back up, turn it, warp it, with nothing lost.

- **Edit Contents** (double-click the thumbnail): the inside opens in the editor with a banner - change anything (a text stays a text), ✔ **Done** puts it back into the smart object and every copy of it, × **Cancel** leaves it. A smart object inside a smart object opens the same way.
- **Duplicate Layer** shares the contents (edit one, all change); **New Smart Object via Copy** makes an independent one.
- **Replace Contents…** (a picture file; it keeps the place and size), **Export Contents…** (a PNG), **Convert to Layers** (the inside back as a group - at its own size, unturned), **Rasterize** (plain pixels).
- Properties shows its size, whether it is warped, how many copies share it, and buttons for all of the above.

Smart filters. Any **Filter** or **Image ▸ Adjustments** item (and Ctrl+F, Auto Tone …) on a smart object is ADDED as a smart filter instead of changing the pixels. The filters are listed under the layer: turns one off, double-click opens its settings again, right-click sets its opacity, ▲ moves it later in the order, × deletes it. One **filter mask** (its thumbnail on the Smart Filters row) limits them all: click it and paint - black hides the filters there, white shows them; Shift-click turns it off.

Transforms.Edit ▸ Transform: Skew / Distort / Perspective / Warp (and **Ctrl+T** on a smart object). The options bar switches between Free (move, scale - Shift keeps proportions -, rotate outside the box - Shift = 15° steps), Skew (corners along their sides, sides slide), Distort (corners anywhere), Perspective (a corner and its partner move in mirror) and Warp (a 4 × 4 grid: drag its points, or drag anywhere inside to bend it there; presets Arc, Arc Lower / Upper, Arch, Bulge, Flag, Wave, Fish, Rise, Inflate, Squeeze with a Bend %). Enter applies, Esc cancels. A pixel layer (or the selection on it) is re-drawn once; a smart object keeps the transform.

Puppet Warp (Edit ▸ Puppet Warp): click to place pins on the layer's content (three or more hold its shape), drag a pin to bend - everything between the pins follows naturally; Alt-click or Delete removes

a pin; Density and Show Mesh in the options bar. On a smart object it is a smart filter.

Content-Aware Scale (Edit ▸ Content-Aware Scale…, Alt+Shift+Ctrl+C): the layer's content made narrower / wider / shorter / taller by taking out (or doubling) the least important lines of pixels - faces, objects and edges keep their shape. **Protect: the selection** keeps what you selected exactly. The dialog shows a small live preview.

Liquify (Filter ▸ Liquify…, Shift+Ctrl+X) opens its own window: **W** Forward Warp (drag to push), **R** Reconstruct (paint changes back), **C** Twirl (Alt = the other way), **S** Pucker, **B** Bloat, **F** Freeze Mask (red = nothing moves there), **D** Thaw Mask; Size ([and]), Pressure, Restore All. OK applies (Enter), Cancel (Esc). On a smart object it is a smart filter - double-click it to carry on.

Selections: Select and Mask, Magnetic Lasso, Quick Selection, Sky, Channels (2.423.0)

Select and Mask (Select ▸ Select and Mask…, Alt+Ctrl+R) opens a workspace of its own. Paint with **Quick Selection** (W - it grows to the edges; Alt takes away), **Refine Edge** (R - paint over hair or fur: the edge is searched there) and **Brush** (B - by hand); ☁ Sky and ⇄ Invert at the top. On the right: the **View** (Overlay red, On Black, On White, Black & White, On Layers) and its opacity; **Edge Detection**: Radius (the band the real edge is looked for in - a sloppy selection snaps onto the picture's edge) and Smart Radius; **Global Refinements**: Smooth, Feather, Contrast, Shift Edge; **Output**: Decontaminate Colors (the coloured fringe of the old background taken out of the edge) and Output To: Selection, Layer Mask, New Layer, New Layer with Layer Mask. Enter = OK, Esc = Cancel.

- **Magnetic Lasso** (L, Shift+L cycles): click, then move along an edge - the line snaps to it, points fall every Frequency px (click to set one, Backspace takes the last back); double-click, Enter or the first point closes it. Width, Contrast, Frequency in the options bar.
- **Quick Selection** (W): paint - the selection grows to the edges under the brush; the first stroke makes a new selection, the next ones add, Alt takes away. Tolerance; Auto-Enhance smooths the edge after each stroke.
- **Select ▸ Sky**: the sky (blue, overcast or a sunset) reaching the top edge, soft along the skyline.
- **Channels** (a tab next to Layers / Paths): RGB, Red, Green, Blue - click one to see it alone (Ctrl+2 … Ctrl+5), Ctrl-click for its lightness as a selection. **Alpha channels** are saved selections: **Select ▸ Save Selection…** (new, or into a channel: replace / add / subtract / intersect), **Select ▸ Load Selection…** (invert, the operation); click one to see it, Ctrl-click to load it, double-click to rename. They undo, and follow crop / image size / rotate.

Retouching: Patch, Content-Aware Move and Fill, Red Eye, Mixer Brush, erasers, healing (2.423.0)

None of these use AI - the picture is filled from itself (PatchMatch) and blended in (Poisson), all in the page.

- **Spot Healing Brush** (J): Type **Content-Aware** (filled from what is around it) or Proximity Match (the old way).
- **Healing Brush** (J): Alt-click the source, paint - the copy now takes the colour and light of where it lands (only its texture comes from the source).
- **Patch** (J): draw round what to fix (or select it), then drag it onto clean texture: it takes that texture, blended (Patch: Source). Destination does it the other way round.
- **Content-Aware Move** (J): select a thing, drag it - it moves, its old place is filled from around it (Mode: Move), or it is copied (Extend).
- **Red Eye** (J): click the red pupil; Pupil Size, Darken Amount.
- **Edit ▸ Content-Aware Fill…** and Content-Aware in **Edit ▸ Fill…**: the selection filled from the rest of the picture, onto the layer or a new one.
- **Mixer Brush** (B): the paint picks up the canvas's colour as it goes - Wet, Load, Mix, Clean after each stroke.

- **Background Eraser** (E): erases what is like the colour under the brush's centre (Continuous / Once, Tolerance, Protect Foreground Color).
- **Magic Eraser** (E): a click erases the area of that colour (Tolerance, Contiguous).

The Gradient Editor, Layer Comps, smart filter blend modes (2.434.0)

- **Gradient Editor** (Edit ▸ Gradient Editor…, the Gradient tool's gradient button, a Gradient fill layer's Properties, Object Properties for a vector shape): 12 presets (Foreground to Background, Foreground to Transparent, Black / White, Spectrum, Transparent Rainbow, Chrome, Copper, Gold, Sunset, Violet / Orange, Blue / Red / Yellow, Transparent Stripes) + your own (**New** keeps one; right-click deletes it). The bar: **click under it = a new stop** (the colour there), **drag a stop** = move it, **drag it down off the bar = delete it** (two stay at least). The picked stop: **Colour** (or "foreground" / "background" - it follows the swatches), **Opacity**, **Location**; Delete, ⇄ Reverse, Distribute evenly. The Gradient tool / Paint Bucket then use Colours: **Custom**; a fill layer or a shape keeps the stops (a shape's SVG too).
- **Layer Comps** (Window ▸ Layer Comps, the Comps tab beside History / Actions): keeps the layers' state - name, what it applies (Visibility, Position, Appearance = Layer Style / opacity / blend mode) and a comment; ▶ on a comp applies it (one undo step), ◀ ▶ the previous / next, ↻ updates the picked one from the layers now, deletes; double-click = its options. **Last Document State** brings back the layers as they were before. Each document keeps its own comps.
- **Smart filter blend modes**: right-click a smart filter (under a smart object in Layers) = **Blending Options**: Mode (24: Normal, Dissolve, Darken, Multiply, Color Burn, Linear Burn, Lighten, Screen, Color Dodge, Linear Dodge, Overlay, Soft / Hard / Vivid / Linear / Pin Light, Difference, Exclusion, Subtract, Divide, Hue, Saturation, Color, Luminosity) + Opacity; the row says the mode.

The image editor and the prompt docker: Image mode, generated pictures onto the canvas (2.431.0)

2.549.0: **a Generate in image-edit mode is always a NEW LAYER**. With no selection the prompt docker's own Generate runs with the style picked there (Krea 2 …) and the picture comes in as a new layer; with a selection the editor fills it, as a new layer too. It no longer paints into the selected layer.

2.550.0: **a selection is filled SEAMLESSLY**, the way the Krita AI plugin does it: the prompt docker's style (Krea 2, the FLUX.2 / FLUX 3 Klein styles, Qwen …) runs its own graph with the picture around the selection encoded and a noise mask on the selection (plus differential diffusion for a soft edge), so the new part continues what is there instead of being a new picture. Checkpoint styles (SDXL …) keep the SDXL inpaint export; a style's own inpaint workflow still wins. Refine (a strength) works the same way.

2.551.0, closer to the Krita AI plugin: the region sent around a selection is bigger (at least 3/4 of the selection's size on every side, about 512 px across), the selection is **pre-filled with the colours around it** before it is encoded, the mask is grown and **blurred** (a soft edge), and a style that was not trained to inpaint (Krea 2, Klein, Qwen …) starts just below full noise (0.9) so the new part keeps the light and colours of what is around it. For the best fills use a real inpaint model - FLUX Fill inpaint, or the SDXL inpaint export (the Krita plugin uses inpaint models too: Fooocus inpaint for SDXL, a ControlNet inpaint for SD 1.5, Flux Fill).

2.552.0, from the Krita AI plugin's own code:

- **Generate** in a selection runs at **full strength** again (as Krita does with a model that was not trained to inpaint - the picture around the selection is kept while the model draws, so the new part is drawn into it). **Remove** starts at 0.9 from the colours around the selection.
- A small selection's region is **rendered bigger** (at least about 0.75 megapixels, at most 4x) and scaled back - a 512 px region of a model trained at 1 megapixel comes out muddled.
- The mask's soft edge **grows with the selection**: feathered by 10 % of the selection's diagonal (at least 32 px) and grown by 4 px + half of that, as in Krita. Refine uses less (times its strength).
- When the model kept the picture around the selection, the result is **blended a little past the selection's edge** (no visible line) and **colour-matched** to the picture around it (Krita's Color Match).

- **Edit** (Qwen Image Edit, FLUX Kontext, FLUX.2 / FLUX 3 Klein edit) inside a selection now changes **only the selection**: the picture is the model's reference and also its starting point under the same soft mask, so everything around the selection stays exactly as it was.

2.553.0 - **fill with the edit model (sees the picture)**, in the AI panel, on unless you turn it off: Generate, Remove and New background in a selection run through the **edit model** (the prompt docker's style when it can edit - FLUX.2 / FLUX 3 Klein, Qwen Image Edit - else the panel's Edit model). A model that was not trained to fill (Krea 2 …) cannot see the picture around a selection and draws a picture of its own into it; the edit model looks at the whole region, so the new part belongs in the scene. As in the Krita AI plugin, the selection is painted **green** and the model is told "Fill the green spaces according to the image." followed by your prompt (Remove: "Remove the object.", New background: "Replace the background while keeping the main subject."), and only the selection changes. A real inpaint style (FLUX Fill inpaint, the SDXL inpaint export) is always used as it is.

- ▾ **Edit** with a selection changes **only the selection** (a new layer over it). Without a selection it edits the selected layer - a layer with transparent parts keeps its outline: only its own pixels change, nothing is added around it.
- 2.554.0: a style that cannot see around a selection (Krea 2 …, with off or no edit model) gets the **selection's own box** as its whole picture (plus a little room for the soft edge), rendered at the model's size and scaled back - so what it draws is the size and place of the selection, not a bigger picture cut down.

2.555.0 - the Krita AI plugin's fill, per model family. Each family gets the helper the Krita plugin uses - a part that was *trained* to fill - when ComfyUI has it:

- **SDXL / Pony**: the Fooocus inpaint patch (the style's OWN checkpoint fills - Pony stays Pony).
- **Illustrious / NoobAI**: the NoobAI inpainting ControlNet (else the Fooocus patch).
- **SD 1.5**: the SD 1.5 inpaint ControlNet. **FLUX.1**: the Alimama FLUX inpainting ControlNet.
- **Remove** (any style): MAT / LaMa takes the object out first.
- **FLUX.2 / FLUX 3 Klein, Qwen, Krea 2**: no helper - with on they fill through the edit model (as before).

Also as in Krita: the region the model sees (the selection padded by its feather, then by max(the picture's long side / 16, half its size), at least 512 px), and the **fill** row for Generate - **Add content** (the default: something new goes there; the selection starts grey, with the edit model "Add the object to the scene.") or **Fill** (the picture continues there; the selection starts as a blur of what was there). A selection across a whole side of the picture expands it. **What to install** (the setup wizard lists both): the node pack **comfyui-inpaint-nodes** (in the setup's node list) and, from the setup's model list group **Selection fill - the Krita AI way**: the Fooocus inpaint head + patch (models/inpaint), MAT (models/inpaint), and the inpainting ControlNets you need (models/controlnet). If the Krita plugin already downloaded them, add Krita's ComfyUI models folder in Settings → ComfyUI → Folders instead of downloading them again.

2.556.0 - Pending: queue as many as you like. Generate, Edit, Refine, Fill, Remove, New background and Compose never wait for the last one any more - press again (another selection, another prompt, another layer) and the new renders queue behind the others. Every result lands in **Pending**, a small docker that floats *inside* the image editor (drag it by its title; ✕ hides it, the AI panel's **Pending** button shows it again, and every new render brings it back). Each card shows the result over the area it was made for (click the picture to preview it over the canvas), what it was and which layer it belongs to:

- ✔ **Apply** puts it on a new layer **above the layer that was selected when you asked for it** - inside one group per layer, **" changes: <layer>"**, made the first time and reused after that (each Apply is one undo step).
- **Delete** throws it away. ✔ **all** / **all** in the title bar do every card.

The References docker in every mode (2.556.0). Name a reference picture in the prompt by its number in the docker - "picture 1", "image 2", "ref 3" - and it goes along:

- **Edit**: up to two pictures (the ↳ reference dropdown still works as before: that picture is "Picture 2").

- **Generate / Fill / Remove / New background / Refine**: the run goes through the **edit model** (only an edit model can take pictures), the selection is filled using them, and the numbers are rewritten for the model ("picture 3" in the docker becomes the model's "Picture 2"). With no edit model the pictures' descriptions are added to the prompt instead.
- **Compose**: write "ref 1" / "reference 2" (in Compose "picture N" means the N-th picked layer); the picture is added after the picked layers (three pictures in all).

Also fixed: the editor's settings from 2.552.0-2.555.0 (the mask sizes, fill with the edit model, Add content / Fill) were dropped on the way to the server - they reach it now.

2.557.0: the files the Krita-way fill needs download from **Settings ▸ Add-ons ▸ AI fill models** (tick, Download ticked).

- While the image editor is **open** (on screen, not collapsed) the prompt docker's Generate mode is **Image**, and a → ▦ **canvas** chip beside it says where the pictures go. When the editor closes the mode it was in comes back (a mode picked while it was open is kept). Opening / closing the editor is what switches it; the project keeps the mode you picked, not the editor's Image.
- **Every picture a Generate makes while the editor is open lands on the canvas**: a new layer on the document in front, named after the prompt (a red fox …), centred - scaled down when it is bigger than the canvas. A render that makes several pictures gives a layer each. No document open = the first picture opens as one. Each layer is an undo step; the gallery keeps the pictures as always. With the editor closed nothing is added.

Documents (floating windows / tabs), the colour palette, ⊠ transparent, auto-fill (2.430.0)

- **Every New / Open is a document of its own** (File ▸ New, Open, Open PDF, a PSD, a picture dropped in, Paste into an empty editor). The first one docks - it fills the canvas room, with a **tab** above it; the next ones open in their own **floating window** (cascaded) - a "modular docker" over the canvas room. Window ▸ Arrange ▸ Open Documents as Tabs makes every new one a tab instead.
- Each document keeps its own layers, undo / redo (History), selection, paths, channels, zoom and a PDF's pages; the one in front is the one the tools, the panels and AI work on.
- **Tabs**: click = to the front; × or a middle-click = close; ● = changes not saved; drag a tab off the strip = it floats there.
- **Windows**: drag the title bar (dropped on the tab strip = docked as a tab; a double-click on it docks too), ⇲ = dock, × = close, the bottom-right corner resizes. A window in the back shows its picture as it was left - a click brings it to the front (the canvas moves into it).
- File ▸ **Close** (Ctrl+W) / **Close All** (Alt+Ctrl+W): unsaved changes ask - Save and close / Close without saving / Cancel. Window ▸ Arrange: Consolidate All to Tabs, Float in Window, Float All in Windows, Tile, Cascade, **Next / Previous Document** (Ctrl+Tab / Shift+Ctrl+Tab). File ▸ Automate ▸ Batch closes each picture when it is done (the documents you had open stay).
- **The colour palette**: the strip down the right edge of the editor - ⊠ first, then the colours; ▲ ▼ / the wheel scroll it; ▾ picks the palette (Default, Greys, Your Swatches, Document Colours); Window ▸ Color Palette shows / hides it. On a curve, a shape or type (every picked one - Ctrl-click in Layers - or the active one): **click = its fill, right-click = its outline**, ⊠ = none - one undo step. On anything else: click = the foreground colour, right-click = the background colour.
- ⊠ **transparent** (the palette and the Swatches panel): as the foreground a brush paints transparency, the Paint Bucket and Edit ▸ Fill clear; the swatch shows it with a red line.
- **Auto-fill**: a curve closed with the Bézier, Pen (F5), Freehand, Polyline, 3-point curve, B-spline or Curvature tool, a new F6 shape, a Pen / Shape tool shape - is filled straight away with **the colour picked last** (the palette, a swatch, the colour pickers, the eyedropper). ⊠ picked last = not filled. The curve tools' property bar: ▨ = auto-fill on / off (on by default).

Dockers: Transform, Shaping, Align and Distribute, Object Properties (2.429.0)

Window ▸ Dockers opens a panel group of their own (✕ closes it). They work on the **picked** layers (Ctrl-click in Layers picks several) or the active one.

- **Transform** (Alt+F7): Position (x / y, Relative position), Rotate (angle, + = counter-clockwise), Scale & Mirror (%, proportional, ⇆ ⇅), Size (w / h, proportional), Skew (x° / y°) - about one of the 9 points of the box (Relative center), **Copies** (each copy one more step - 2 copies of a 30 px move = a row of three). Vector shapes / lines / paths exactly (a live shape stays live when only moved / scaled), pixel layers re-drawn, type moved and sized. Each Apply is one undo step.
- **Shaping**: Weld, Trim, Intersect, Simplify, Front Minus Back, Back Minus Front, Boundary - on the picked vector shapes; the active one is the target, the result takes its look; Leave original source / target.
- **Align and Distribute** (Shift+Ctrl+A): align left / centres / right / top / middle / bottom to the active object, the first picked, the selection's box, the page edge or the page centre; distribute left / centres / right / spacing, top / centres / bottom / spacing - over the selection or the page.
- **Object Properties** (Alt+Enter): the active vector shape's fill (none, uniform, linear / radial gradient with two colours and an angle - kept in SVG too), outline (width, colour, style, F12), transparency (opacity, blend mode).

Drawing tools: shapes (F6), Artistic Media, dimensions and connectors (F8) (2.428.0)

Everything they draw is a vector line layer - the outline pen (F12), the fill, the Shape tool (F10) and Convert to Curves (Ctrl+Q) work on all of it.

- **F6** (again = the next): **Rectangle** (corner radius per corner - changes all four - and Round / Scallop / Chamfer corners), **3-Point Rectangle** (drag one side at any angle, let go, click the height), **Ellipse** (Ellipse / Pie / Arc with start and end angles), **3-Point Ellipse**, **Polygon** (sides), **Star** (points, sharpness), **Complex Star** (points, sharpness - its middle open), **Spiral** (revolutions, symmetric / logarithmic, expansion), **Graph Paper** (columns x rows), **Common Shapes** (basic shapes, arrows, flowchart, banners, callouts). Ctrl = square / circle, Shift = from the centre. They stay **live**: pick one and the property bar changes it, until F10 edits a node or Ctrl+Q converts it to curves.
- **Artistic Media** (in F5): draw a stroke and it becomes a filled shape - **Preset** (Taper, Teardrop, Flare, Uniform) and its width, **Calligraphic** (nib width and angle), **Pressure** (a pen's pressure; with a mouse, slower = wider), **Sprayer** (stars, dots, leaves, hearts, bubbles or confetti along the stroke - size, spacing). It stays live too.
- **F8** (again = the next): **Parallel Dimension**, **Horizontal / Vertical Dimension** (click the two points, then where the dimension line goes - above / below = horizontal, beside = vertical), **Angular Dimension** (the vertex, a point on each arm, where the arc goes), **3-Point Callout** (the point, the elbow, the end - the text from the bar). Units px / mm / cm / in at a dpi, decimals, text size. **Connector** (Straight / Right-angle / Rounded right-angle): drag from one object to another - it snaps to the middle of their sides (or their centres) and **follows them when they move**.

Actions and Batch (2.427.0)

The **Actions** tab sits next to History (Window ▸ Actions, Alt+F9). Sets () hold actions (▶), actions hold steps.

- **Record**: pick an action (or New Action - recording starts at once), then work: every menu command, shortcut or palette command is a step, with the settings of the dialog it opened when you press OK (Cancel = no step); filters and adjustments with their settings; the Layers panel's buttons, blend mode, opacity and fill. ■ stops. Painting and selecting by hand are not recorded - Select menu commands are.

- **Play**: ▶ plays the picked action, or from the picked step. ☑ switches a step off; ▢ (the dialog toggle) makes a step stop in its dialog so you can change it (OK carries on, Cancel stops the action). A step that cannot run (no layer, no selection) stops the action and says which step and why.
- **Edit**: ▲ ▼ move, deletes; double-click a step to record its dialog's settings again, an action for its name and function key (F2 … Shift+F12 - the keys the editor does not use), a set to rename it. : Insert Stop (a message, with or without Continue), Insert Menu Item, Duplicate, Button Mode (a button per action), Load / Save Actions (.json - .atn files cannot be read), Reset Actions, Batch. A **Default Actions** set comes with it (Vignette, Sepia Toning, Black & White Punch, Soft Glow, Sharpen + Flatten). Actions are kept with the app.
- **File ▸ Automate: Batch…** plays an action on many pictures: pick files or a folder, what to do on an error (stop, or log it and go on), where the results go (the project's batch folder, the image gallery, a download each, or nowhere), the folder name, a suffix for the file names, PNG or JPEG. A progress box (■ Stop) and a report at the end.

Character and Paragraph panels, paragraph text, type styles (2.426.0)

Two tabs next to Color / Swatches (Window ▸ Character, Window ▸ Paragraph). They edit the active text layer - every change is one undo step - or, with no text layer picked, the settings the next text starts with.

- **Character**: font and style (Regular / Bold / Italic / Bold Italic), size, leading, kerning (Metrics / 0), tracking, vertical and horizontal scale, baseline shift, colour; Faux Bold, Faux Italic, All Caps, Small Caps, Superscript, Subscript, Underline, Strikethrough.
- **Paragraph**: left / centre / right, Justify last left / centred / right, Justify all; indent left, right and first line; space before and after a paragraph; Hyphenate.
- **Paragraph text**: drag with the Type tool (T) to draw a box - the text wraps inside it (a click still makes point text). Justified lines need a box. Text that does not fit is hidden and the box's corner shows ⊞ (the Paragraph panel says so - make the box larger with W / H). Layer ▸ Type: Convert to Paragraph Text / Convert to Point Text (lines that did not fit are left out). Image Size scales the box and indents too.
- **Character Styles / Paragraph Styles** (the lists in the two panels): makes a style from the text, a click applies it (the text remembers it), + after its name = the text was changed since (⌫ Clear Override puts the style back), Redefine updates the style from the text and every text using it, deletes, double-click renames. Styles are kept with the app.

Vector lines: the Bézier, curve tools and Shape tool (2.425.0)

The keys: **F5** picks the curve tools (F5 again = the next one), **F10** the Shape tool, **F12** the Outline Pen, **Ctrl+Q** Convert to Curves. Every curve tool draws a **vector line** - a layer that stays sharp at any size, with an outline pen - or carries the active line on when you start on one of its open ends.

- **Bézier** (F5): click = a cusp node (straight segments), drag = a smooth node (a curve; while dragging **C** makes it a cusp so the next segment can turn, **S** smooth again). Ctrl = 15° steps. Click the first node to close it; double-click, Enter or Space = done; Esc drops the line being drawn.
- **Freehand**: drag = a freehand line (Smoothing in the bar); click, click = a straight line.
- **2-Point Line**: drag from start to end. **Polyline**: clicks = straight segments, a drag = a freehand stretch, double-click ends. **3-Point Curve**: drag the chord, let go, click where it bends through. **B-Spline**: click control points - the curve follows them smoothly, and stays a B-spline for the Shape tool. **Node Pen**: the Bézier with the next segment shown, and nodes added / deleted with a click on the active line.
- **The property bar** (every curve tool and the Shape tool): outline width, colour, style (solid, dashed, dotted, dash-dot, long dash, custom), start and end arrowheads (arrow, triangle, open, half, circle, square, diamond, bar), fill for closed curves, ✒ F12 and ⇢ (to a path).
- **Shape tool** (F10): click / Shift-click / drag a box round nodes; drag nodes, handles (cusp, smooth and symmetrical nodes - square, round and diamond markers) or a segment (it bends).

Double-click the line = add a node, a node = delete it; + / - / Delete; C / S / Y = cusp / smooth / symmetrical; arrows nudge. The bar: add, delete, join two nodes, break curve, to line, to curve, the node types, reverse direction, extend curve to close, extract subpath, auto-close, reduce nodes (with a smoothness slider), select all. It works on any shape layer and on paths; a B-spline shows its control points.

- **Outline Pen** (F12, Layer menu): colour, width, style + custom dashes (in widths), corners and miter limit, caps, position (outside / centred / inside), arrowheads + size + swap, the fill, behind fill, scale with object. With no shape layer active it sets the outline for the next lines.
- **Convert to Curves** (Ctrl+Q): a rectangle, ellipse, polygon, line or custom-shape layer becomes an editable vector line (its look kept).
- **Vector line ⇄ path**: Layer ▸ **Vector Line: Convert to Path…** puts the line in the Paths panel as a saved path that remembers its outline pen (keep the layer or not); **Vector Line: Make from Path…** (or the Paths panel's) turns a path into a vector line - one that came from a line comes back exactly (name, width, dashes, arrowheads), any other path takes the curve tools' outline. Every step undoes.
- **Files**: Export Path as SVG… writes a vector line with its dashes and arrowheads (and its outline pen, so Place SVG as Shape Layer… brings it back as a line). In a PSD it is a vector shape layer with a PSD stroke (width, colour, dashes, caps, corners, position); a PSD has no arrowheads, so bitGen keeps the outline pen in the file's XMP and restores it when it opens the PSD.

The path side of it: the **Curvature Pen** (P, Shift+P cycles) - click points and the curve bends smoothly through them; double-click a point = corner / smooth; drag one to move it; Enter / Esc ends. The Pen's **Rubber Band** option shows the next segment before you click. **Path Selection** (A) can **align** the picked components (left, centres, right, top, middle, bottom - to each other or to the canvas) and **distribute** them evenly.

PDF files: open, edit, save (2.424.0)

File ▸ Open PDF… (or drop a .pdf on the editor) opens a PDF in the editor; **File ▸ Open PDF from Project…** opens one saved before. The **Pages** strip comes up on the left (✎ = a page you changed); click a page to open it. A page opens as:

- a **Background** - the page's picture *without its text* (the text is taken out of the page itself, not painted over), and
- one **type layer per line of text**, where it was, with its font matched (serif / sans / mono, bold, italic), its size, its colour and its width.

Edit with every tool - retype a line, move it, paint, add layers. Each page keeps its own layers and undo while you move between pages - also when you open another picture in between (File ▸ PDF Pages brings the strip back). dpi in the strip sets how sharp pages not opened yet open (72 - 300, 150).

Save PDF (File ▸ Save PDF) saves the changed pages back into the original - the project's copy of it in its pdf folder - and downloads it; **Save as New…** makes a new file. Pages you did not change stay exactly as they were (still vector). A changed page becomes its picture with its type on top as **real PDF text** (Helvetica / Times / Courier in the matching weight and style, colour, size and spacing) - selectable and searchable. Type those fonts cannot write (other alphabets, warped / 3D / path text, layer styles, masks, a blend mode) is drawn into the picture instead - the save says how many. pdf.js and pdf-lib do the reading and writing in the page; nothing goes to the internet.

Music, stems and voices

- **Music** docker — a song from lyrics and a style (MiniMax Music3 through ComfyUI, or YuE2), covers and sheet music.
- **Stems** — split a song into vocals / drums / bass / other (demucs).
- **Voices** — clone a voice from a short clip or a waveform region, then speak text with it (OmniVoice); transcribe speech with faster-whisper; **Speak** dictates into the prompt.

Timeline and export

Drag clips, songs and pictures onto the **Timeline** layers, trim and move them, add transitions, and compile the timeline into one video in the project folder.

The menu bar

The row of buttons under the title bar: **✚ New**, load, save, screenshot, record, recycle bin, ↺ reset project, ⚙ settings, ⏏ eject every loaded model. It stays put whichever docks are shown or hidden.

Path layers (2.387.0)

- **Path layer** → draw the path with **(Shift+P)** (click points, drag to move, Alt-click to remove) → in its panel write what follows it (animal / person / vehicle), Start / Length, Size at start → end, pace, facing → **Generate**.
- **In place + follow path**: picture on green → video on the spot (your video style) → green cut away → clips moved along the path with keyframes (mirrored when it turns back), looped as needed. **Place again** after editing.
- **Along the path (Wan ATI)**: needs a Wan 2.1 style and `Wan2_1-I2V-ATI-14B_fp8_e4m3fn.safetensors`; the model makes it travel the path; one full-frame see-through clip.
- **Cut out**: AI matte (⬇ Install AI matte: venvs/matte, rembg + BiRefNet) or chroma key. Each layer is see-through.

The camera path: pan, tilt, roll, dolly, height, a 3D path (2.399.0)

Camera on the timeline bar opens the Camera panel. Like Basic Settings, it is a docker of its own: float it, or drop it beside or inside another docker. The tool in the canvas toolbar (**Shift+C**) draws the camera's frame on the picture at the position marker.

Making the move

- **Preset ▸ Apply**: push in, pull out, truck left / right, pedestal up / down, pan, tilt, roll, orbit, crane up / down, zoom in / out, fly in.
- **Camera / Top / Side** shows what the camera sees, and the path from above and from the side. In Top and Side, drag a point to move it, or click empty space to add one. **Point** and **× Point** add and remove points.
- ▶ and the slider play and scrub the move. The timeline's marker follows.
- **Pan, Tilt, Roll, Height, Dolly, Lens**: the values at that moment. Changing one writes a **◆ key** there, and the camera moves smoothly between keys. **◆ Key**, **× Key** and **◀ ▶** add, remove and step between keys.
- **Aim**: straight ahead, at the picture's centre (an orbit keeps looking in), or along the path. **Ease**, **Length** (= **Seconds** matches Basic Settings) and **From** (where it starts on the timeline).
- **With the tool on the canvas**: drag to pan and tilt, Alt+drag to roll, Shift+drag to change the height, the wheel to dolly, Shift+wheel to zoom. Each one writes a key at the marker.

Telling the video models

Tick **Use in Generate**, and the prompt docker's Generate passes the camera on. None of the three models takes a free 3D camera path, so each gets what it can follow:

- **MiniMax H3** gets its own camera sentence, for example "The camera pushes in with large amplitude at slow speed." It takes **one move per shot**, so it is told the main move.

- **Wan** gets the moves in words. With a **Fun Camera** model in ComfyUI (`wan2.2_fun_camera_high/low_noise_14B`, or a Wan 2.1 Fun Camera model) and a picture in the reference dock, it also gets that model's move: Zoom In/Out, Pan Left/Right/Up/Down, or a clockwise or anticlockwise roll.
- **LTX** gets the moves in words. With the matching **LTX-2 camera-control LoRA** in ComfyUI's loras, it also gets that LoRA: dolly in, out, left or right, jib up or down, or static. For example, `ltx-2-19b-lora-camera-control-dolly-in.safetensors` comes from `Lightricks/LTX-2-19b-LoRA-Camera-Control-Dolly-In`.

The panel's **What the models are told** shows all of this, plus what ComfyUI has installed.

The exact path (guide video)

1. **Use picture** takes the first reference picture, or the timeline canvas at the marker. It is kept in Assets.
2. ◐ **Depth** makes its depth map with Depth Anything V2 through ComfyUI. This needs comfyui_controlnet_aux. **Relief** sets how far the near parts stand out.
3. **Make guide** flies the picture along the path and saves two videos to Assets: `camguide_<id>_rgb.mp4` (what the camera sees) and `_depth.mp4` (its depth).
4. Tick **Exact path**. Generate then sends the guide too:
   - – **MiniMax H3**: the RGB guide becomes a reference video, and the prompt says to follow its camera. This works in References renders only (a First / Last render takes no video) and is **experimental**.
   - – **Wan**: the depth guide becomes **VACE**'s control video, with the first picture as its reference. This needs a VACE model.
   - – **LTX**: the depth guide runs through the **style-transfer** mode as its depth guide, with the first picture as the first frame.

If you change the path or the picture, the guide is marked ⚠ and Generate leaves it out until you make it again.

Upscaling a video with SeedVR2; the queue's running card (2.400.0, Enhance 2.401.0)

SeedVR2 on the preview bar upscales the video you are watching (a gallery video, a Video asset or an export) with ComfyUI's own SeedVR2 nodes:

- **Size**: the short side (720p, 1080p, 1440p, 4K, keeping the shape) or a factor (×2, ×3, ×4). The panel shows the result size before you start.
- **Colour match**: lab (the most faithful), wavelet, adain or none.
- **Temporal split**: auto cuts a long clip into pieces that fit your graphics card's free memory and blends them back together; or a fixed piece length; or off.
- **Model**: *best available* uses `seedvr2_7b_sharp_fp16.safetensors` when ComfyUI has it, else another SeedVR2 7B, else the 3B. **Setup → Models → Upscaling** downloads it (16.5 GB), the smaller 3B int8 and the SeedVR2 VAE.
- **Denoise**: 1 restores fully; lower keeps more of the original texture.

Upscale queues it as its own job. The result keeps the original's sound and lands in the gallery as `<name>_seedvr2_<size>p.mp4`. Your choices are kept for the project.

For the scene board: in the StoryBoard's ⚙, set **Upscaler** to **SeedVR2 (video)**. A scene's then sends its take (the lossless master when there is one) through SeedVR2 at the ⚙'s **Upscale resolution**, and files it as the scene's upscaled take. This works for any video style. With **Upscale every render** ticked, each take is queued for SeedVR2 as soon as it lands.

Enhance in the prompt docker opens the same panel for the **selected video**: the last one you clicked in the gallery, the Video assets or the reference dock. The tile gets a dashed ring, and the button's tooltip names the video and where it is. If nothing is selected, the panel asks you to click one. Click another video while the panel is open and it switches to that one. (The LLM prompt rewrite

moved to **Enter** in the prompt box.)

The queue docker shows a render that is running as **one card**: its status and ▲ ▼ × buttons, the prompt, the job's id, mode and size, a progress bar, and "chunk 1/2 · step 3 of 8 · 1:11 · 19%". Click the card to follow it in the console and the preview. The job list below no longer repeats it.

The video style in Basic Settings (2.401.0)

The top of Basic Settings shows the video style the prompt docker's Video dropdown has picked. Pick another style there and it changes in both places. The card shows:

- what kind of style it is (MiniMax H3, Wan 2.1 / 2.2, LTX 2.5) and its workflows;
- **Model**, with one row per phase (Wan 2.2: high noise and low noise), and the model it actually renders with. A blank model means the workflow's own, and the card names that file. Wan picks its own files from what ComfyUI lists;
- each phase's steps, cfg and load precision, plus the text encoder, VAE, LoRAs and negative prompt.

Pick another model in a row and it is **saved into the style**, so the next render uses it. browses ComfyUI's model folders. ⚙ opens the full style editor in Settings → Video.

The statusbar's docker buttons, the 3D toolbar (2.386.0)

- Every docker has a statusbar button (Stems, SFX and Chat now too). A button is **lit while its docker is on screen** (open or collapsed); a click hides the docker, another shows it again.
- Select a 3D layer: a **3D toolbar** appears along the top of the preview canvas - widget (✥ ○), snap 15°, views (front / left / right / top / bottom / back, 0), spin, reset depth + scale, material preset, ☀ studio lights + brightness, add a light, ◇ Key, Blender, ⚙, Bake.

Tools & Properties, keyframes, 3D widgets, lights, materials, Blender (2.385.0)

- **Tools & Properties** float in their own window: **1** shows / hides it (or in the timeline bar, × on its titlebar); drag it by the titlebar, resize by the corner.
- ◆ **keyframes** drag along their clip (never past their neighbours: first ◆ 0 .. next − 1 frame, last ◆ previous + 1 frame .. the clip's end). Right-click / Alt-click / Delete removes one.
- **3D widgets** (Move tool, a 3D layer selected): **G** move (X / Y across, blue Z toward the camera), **R** rotate (ball = tumble, rings = X / Y only, outer ring = roll), **S** scale (its own X / Y / Z, centre = even). Same key again hides it.
- **Material**: presets (gold, chrome, glass, neon …) or colour / metal / rough / glow / opacity / coat / glass / wireframe / flat / texture; a model's parts can each keep their own, use the layer's or a preset.
- **Lights**: studio lights on/off + brightness, and Point / Spot / Sun / Ambient / Sky with colour, brightness slider and position.
- **Blender**: install the **bitgen-blender-addon** zip in Blender (Preferences → Add-ons → Install from Disk) → sidebar (N) → bitGen: Send to bitGen (glTF, "Add as 3D layer"), Live link (re-send on Save), Get from bitGen. A 3D layer's **Render with: Blender** renders it in Blender (Eevee / Cycles) for the compile; ⚙ sets Blender's path. (2.402.0) **Render scene → bitGen** renders a whole Blender layout through its camera and has LTX 2.3 + 3DREAL make it photoreal - see *LTX 2.3 and 3D render → real*.

Seeds and render details

- **Basic Settings → Seed**: -1 = a random seed every render; type a number to use that seed; = random again; ♻ = the last seed used (shown under the box).

- **Queue**: a running job shows *step n of total · time*. A finished job has — its seed, CFG, steps, sampler, model, LoRAs, size, render time and prompt; ♻ **Use** puts its seed in Basic Settings.
- **Preview dialog** (click a gallery picture or clip): the same details in a pane on the right (hides it).
- **StoryBoard ⚙ → Seed**: *Random for each scene* or *Same seed for every scene* (the seed box; blank = the last seed used); ♻ sets the last seed used as the seed.
- **Each scene card** has its own seed box (it wins over the ⚙); ♻ beside it reuses the seed its last take used. While a scene renders it shows *running step n of total*, a progress bar and the time; a finished one shows its render time and seed.
- The pins are lit until a dock is pinned, then dim. **on a pinned dock** sets its max-height back to 0: drag it to any height, then again to pin that height. A pinned dock can still be made smaller by its seams, never taller than its pin. A pinned video dock keeps its height (it stops taking the left-over room until unpinned).

The position marker, layer colours

- **Drag the red marker** (its head or its line) to move around the timeline at will: a playing preview carries on from where you drop it; stopped, the previews show that frame. **Double-click a time** on the ruler or a track to jump there and play from there. Play / pause / stop are under the preview canvas.
- Each layer has its own colour; the selected layer is outlined. The F/L and REF marker rows grow with the zoom.

Layer names

Double-click a layer's name in the timeline canvas's Layers list and type (Enter keeps it, Esc cancels), press F2 for the selected layer, or use the **Name** box in its properties. Clicking the name on a timeline track works too.

Dock sizes, pins and maximize

- Every dock remembers whether it is open, closed or hidden, its height and width, its pin and whether it is maximized — in `docker_status.json` beside the app, the same layout in every project, loaded before anything else appears.
- (the first button in each dock's header) pins the dock's height as it is now: it can be dragged smaller but not taller. Click again to unpin. On the side docks the pin keeps the width.
- Opening a dock puts the other pinned docks back at their pinned height; the room comes from the unpinned docks.
- **Maximize** gives a dock all the room the other open docks leave — they stay open at their own height (or their pin). Every splitter still works. **Esc** or restores.

Docker configs: a saved layout for each set of dockers (2.392.0)

⊞ (Docker configs) is the icon button at the right end of the menu bar. It keeps your docker layout tidy, so the dockers you open together never end up squashed or cut off.

1. Open the dockers you work with together, for example Music, Prompt chat and Prompt. Size them the way you like.
2. Click ⊞ **Docker layout ▸ Save layout**. The size of every docker is saved in **docker.cfg**, a file in the install folder - and (2.432.0) **which dockers are on screen** (open, collapsed to their header or hidden), which **side docks** are shown (left dock, References, Assets, queue, gallery), and the **places and sizes of the floating panels** (Basic Settings, LLM tokens, the ⊖ negative prompt, the camera, the floating prompt) and of Karaoke / Tools beside the timeline canvas (docked left / right or floating, shown or not, the columns' widths).
3. From then on, whenever exactly those dockers are open together, they take those sizes again.

Opening a fourth docker looks for a layout saved for those four. If there isn't one, the dockers stay as they are, so you can size them and save again. Closing the fourth docker brings back the layout for the three.

Different screens. Each set can keep a layout per window size (up to six). On a screen you haven't saved for, the nearest saved layout is used and fitted to the window, so every docker stays in view.

The panel lists every saved layout:

- **Load** opens that set of dockers with its sizes; the dockers that were on screen collapsed come back collapsed, the side docks and the floating panels go back to how they were saved.
- × forgets a saved layout.
- **Load saved layouts automatically** turns the automatic loading off or on.

The button is highlighted while a layout is saved for the dockers that are open now.

▾ Which picture the prompt docker works from: references or layers, and Compose (2.543.0)

The prompt docker's dropdown (Image mode) now picks the picture by where you are working. The AI panel in the image editor no longer has its own Generate / Refine / Edit / Background / Compose tabs; the dropdown chooses. A chip beside it says what the button will use.

With the image editor minimized or closed, the dropdown has Generate, Edit, Refine, Replace background and Compose, and they work from the **References docker's pictures** (up to 3). Fill and Remove leave the list, because they need a selection in the editor. Your choice comes back when the editor opens again.

- ✏ **Edit**: with one reference, the edit model changes it. With two or three, they are composed together.
- **Replace background**: the edit model is told to replace the background and keep the subject as it is.
- **Refine**: redraws reference picture 1 at the slider's strength.
- **Compose**: one picture made from the references ("put the dog from picture 2 on the sofa in picture 1").
- The results land in the gallery like any picture.

With the image editor open, Fill and Remove are back, and the picture comes from the **selected layer**:

- ✏ **Edit**, **Refine** and **Replace background** work from the selected layer, even if other layers cover it. An empty layer uses the picture as you see it. Each result comes in as a new layer.
- **Several layers picked** (Ctrl-click in the Layers panel) are all used: Edit and Replace background compose them, and **Compose** makes one picture from them.
- With a **selection**, Refine and Replace background work on the selection as before, and Fill and Remove need one.

A Blender layer: the timeline's scene tools in the image editor (2.542.0)

on the image editor's menu bar (beside), or **Layer ▸ New Blender Layer (Scene Tools)…**, puts the timeline's Blender scene tools over the canvas. They are the same tools, the same scene and the same edits as in the timeline:

- **Open a .blend**, or carry on with the scene that is already open. **Build a scene with AI** works here too.
- **Scene camera** looks through the .blend's camera. **Free view**: drag to orbit, right-drag to pan, use the wheel to zoom, and press F to frame everything. **/** or **Alt+drag** (forward = in, back = out) zoom, and **Zoom to object** (Z) goes to the picked object (2.546.0).

- In the image editor there is no player bar and no Auto-key: the scene is one still picture there. They are still in the timeline (2.546.0).
- **Click an object** to pick it, then ✥ (G, R, S) to move, rotate or scale it. You can also key it, hide it, duplicate it, delete it, or add a model with Model.
- 2.550.0: **a rigged figure** (a Daz character, anything with an armature) is picked by its **rig** - a click on its body (or its mesh in the object list) picks the armature, because a skinned body is drawn by its bones: moving the mesh alone only moved the widget. A figure the click ray missed (its bounds were the rest pose) is found by its posed box.
- **Save** writes the edits into the .blend, the same way as in the timeline.

Their settings are in the **Blender** tab. The scene becomes a **Blender layer** that holds the view as you see it. ✕ (or again, or Esc) closes the tools, and the layer keeps the view. It is an ordinary layer: hide it, move it or delete it. Pick it and press **Scene tools** to open the tools again.

Making a picture from the scene. Pick the Blender layer and choose ✏ **Edit** (or Refine) in the prompt docker's dropdown. The button reads "✏ Edit · Blender". Type what the picture should be and press it. The scene goes to the edit model, and **each result comes in as a new layer** above the Blender layer. Nothing is left over the canvas, and the Blender layer stays picked. The Blender tab's **Edit from** sets which picture of the scene is used:

- **The view as shown** (instant): the live view while the tools are open, else the layer's last view.
- **Blender render - Solid** or **EEVEE**: Blender renders that view at the picture's full size first.

The 3D layers from Layer ▸ New 3D Layer work the same way now: an Edit from the scene is a new layer.

The References docker's pictures with the scene (2.546.0). Name a reference picture in the prompt and it goes to the edit model together with the scene - for example "*photo real blonde woman on a beach with the face of the woman in picture 1*". Picture 1 is the first picture in the References docker, picture 2 the second, and so on ("image 2", "pic 2", "ref 2" work too). At most two go along with the scene, because the edit models take three pictures.

- The scene is always the model's first picture, so bitGen renumbers your references for the model, and it uses the model's own word: FLUX.2 / FLUX 3 Klein are told "image 2", Qwen Image 2.1 "<image2>", Qwen Image Edit 2511 "Picture 2". You write "picture 1" either way.
- The Blender tab's **References** setting: **When the prompt names them** (the default), **Always** (the ones named, else the first ones), or **Never**. It works for Blender layers and for 3D layers.
- A prompt that names no picture sends the scene alone, as before.

▾ Generate, Edit, Refine, Fill, Replace background, Remove, and Blender mode (2.541.0)

In Image mode, the prompt docker's menu bar has a dropdown that says what its button does. It works like the Krita AI plugin, on the picture open in the image editor:

- **Generate**: a new picture from the prompt, as before. With the editor open it lands on the canvas as a layer.
- ✏ **Edit**: the edit model follows the instruction ("make her jacket red") on the selection, else the whole picture.
- **Refine**: redraws the selection, else the whole picture. A slider appears on the menu bar for how much it changes (5 – 95 %). It is the same setting as the AI panel's strength.
- **Fill**: fills the selection from the prompt. Select the area first.
- **Replace background**: keeps what is selected and makes everything else new.
- **Remove**: takes out what is selected and continues the surroundings. The prompt is optional.

The button's words follow the choice (" Refine 40%"), and the AI panel's tabs follow it too. If the editor is closed, the button opens it. The results come into the AI panel: click one to compare, and its ✔ keeps it as a new layer.

Blender mode. Pick a 3D layer (Layer ▸ New 3D Layer, a Daz or Blender scene), and a Blender chip shows beside the dropdown. Edit and Refine then work from the scene: Blender renders it at the picture's full size with the layer's engine, and that picture goes to the edit model with the prompt. The 3D layer itself is left as it is.

The model. The 3D tab has a **Model** section under the camera:

- **Object**: pick one of the scene's objects (cameras are not listed).
- **Move m**, **Turn °** and **Scale %** move it on top of where the file has it. **hidden** leaves it out.
- **Drag the object**: click an object on the canvas to pick it, then drag it along the ground as the camera sees it. Shift + drag moves it up or down, and Ctrl + drag turns it.
- **Reset object** and **Reset all** put things back. Each change is one undo step.

The original .blend file is never changed.

Update bitGen Studio from GitHub (2.540.5)

Settings ▸ General ▸ Updates shows your version, how bitGen Studio was installed (a git clone, or the full zip) and the GitHub repository it updates from.

- **Check for updates** asks GitHub for the newest version and lists the new changes.
- **Update now** puts the newest version in place. It asks first, and the log appears in the section.
  - **A git clone** moves to GitHub's version. If you changed any of the app's own files, they are copied to `backups\update-<time>\` first.
  - **An install from the zip** downloads GitHub's copy and writes the files that changed.
  - **Either way**, your projects, settings, styles, models, ComfyUI and saved quick prompts are not touched. When the Node or Python package lists change, they are installed too.
  - A folder with commits of its own (a developer's copy) is not updated. Use git there instead.
- **Restart now** finishes the update in the desktop app. In a browser, close the window running `start.bat` and start it again.
- **A note on start:** a little after start, bitGen Studio checks quietly. If GitHub has a newer version, it says so once.

Settings ▸ Add-ons: optional engines installed in the app (2.540.4)

Settings ▸ Add-ons lists the engines that are not part of the main install. For each one it shows what it is for, what it needs, how big it is and whether it is installed. **Install**, **Reinstall** and **Remove** work there, and the install log appears in the tab. One install runs at a time.

- **YuE2 music**: the "YuE2" style in the Music docker (songs from lyrics plus a style caption).
  - Install makes its own Python environment in the app's `yue2` folder.
  - It then installs PyTorch 2.10 with CUDA for your card and the pinned YuE2 pipeline (yue2-infer 0.1.6), about 5 GB.
  - The model, m-a-p/YuE2-3B, downloads on the first song.
  - It needs an NVIDIA GPU with BF16, about 24 GB of VRAM, and the app's Python 3.10 - 3.13.
  - On Windows, PyTorch comes without FlashAttention. bitGen Studio gives YuE2 another attention kernel (cuDNN, then PyTorch's own, then the slower no-graph mode), says which one in the log, and remembers the one that worked (2.540.7).
  - Remove deletes the `yue2` folder.
- **AI fill models (the Krita AI way)** (2.557.0): what the image editor's Generate / Fill / Remove in a selection uses per model family (see the 2.555.0 fill). Tick what you need - each with its size, ✓ downloaded when it is there:

  - – Fooocus inpaint (SDXL / Pony), MAT (Remove) and the NoobAI inpainting ControlNet (Illustrious / NoobAI) are ticked; the FLUX.1 and SD 1.5 inpainting ControlNets and the FLUX.2 Klein 4B outpaint LoRA are there to tick.
  - – **Download ticked** installs the node pack comfyui-inpaint-nodes into your ComfyUI (and comfyui_controlnet_aux when a ControlNet is ticked), then downloads the files into ComfyUI's models/inpaint, models/controlnet and models/loras (resumable; your Hugging Face token from Settings is used). Restart ComfyUI once after the first install.
  - – **Remove models** deletes the downloaded files (the node pack stays).
  - – It needs ComfyUI's folder set in Settings ▸ ComfyUI.
- **Viggle (native)**: starts its own installer. Its progress shows in the Queue.

Blender and Unreal renders made real with the reference docker's pictures (2.540.0)

Each object in a Blender or Unreal scene can be matched to a picture in the References docker. The grey block called "Car" then becomes the red sports car in your reference, and 3DREAL animates it.

- **How objects find their picture**
  - – **By name**: an object called Car, SM_Car_01 or Vehicle.002 finds a picture whose description or name says car / sedan / truck…, and Sofa finds "a green velvet chesterfield". Walls and floors find wall and floor pictures.
  - – **By hand**: every object has a dropdown, where *auto* uses the name match, *none* leaves it as it is, or you pick a picture.
- **Timeline ▸ Blender scene ▸ Render to real ▸ References**
  - – Tick **Use them** and pick the edit model (the one that takes the most pictures comes first).
  - – renders the segment as before, plus its first frame. That frame goes to the edit model as Picture 1, with the references after it ("the grey shape on the left (the object "Car") becomes a red sports car in Picture 2").
  - – LTX 2.3 **3DREAL** then animates the render, starting from that real first frame.
- **Image editor**
  - – On a **Blender 3D layer**, the References section lists the layer's meshes. Generate then makes the layer's picture real with the matched references as a new layer.
  - – On an **Unreal layer**, the Unreal tab has **Find the objects** (Unreal lists the mesh actors this camera sees) and **Make it real**.
- **Timeline ▸ UNREAL ▸ Render to Real** works the same way for an Unreal clip: Find the objects, then OK. The first frame is made real, and 3DREAL follows the clip.
- Unreal needs **bitGen Bridge 1.3.0** (Load UE project updates it).

Reference descriptions, the wall / floor / furniture / light types, references in the image editor (2.539.0)

- **A description for every reference** - a short box under each picture in the References docker's Image tab, each video and each sound ("a green velvet chesterfield", "a slow waltz", "her voice, calm and low"). It is saved with the project and with a reference group, it goes into the mapping lines ("<subject 2> furniture is in <Picture 2> (a green velvet chesterfield)"), and the prompt chat and Enhance use it (it beats the vision model's guess).
- **New picture types**: wall, floor, furniture and light in the Type dropdown (Make it real uses them).
- **Image editor**
  - – drop a reference picture on an open picture: it comes in as a **new layer**, named by its description.
  - – AI → **Edit** has a **reference** dropdown: the edit model gets that picture as Picture 2 ("put the lamp from Picture 2 on the table").

Civitai downloads into each type's folder, the folders, Sync (2.538.0)

- **on a Civitai card** downloads the newest version's main file. The first time for a *type* (for example SDXL LoRAs, Pony models, Wan LoRAs) the **Select Folder** dialog asks where that type goes; after that every model of the type goes straight there - no dialog. **Shift+click** opens the dialog again to change the folder. The buttons in a model's versions list work the same way.
- **The types**: the main category (image, image-edit, video, music), the model type (sd1, sdxl, pony, illustrious, noobai, flux, flux2, flux2klein, flux3klein, krea, qwen, z-image, chroma, hidream, sd3, anima, wan, ltx, h3, hunyuan, music ...) and the kind (models, LoRAs, VAEs ...). They are kept in **model_folders.json** in the app's folder, so every project uses them.
- in the Models docker lists every type's folder: *Change...*, × forget (the next asks again), or pick a folder for any type before you download.
- A folder outside ComfyUI's model folders still downloads, but ComfyUI only lists it once it is in extra_model_paths.yaml - the downloads list says so.
- **Sync** fetches Civitai's details, thumbnails and example pictures for **every** model and LoRA on this computer (all tabs) that has none yet or is out of date. A model Civitai already named (a .civitai.info, .metadata.json or our card) is asked for exactly by its version id, then by SHA-256, then by the file name. Shift+click syncs all of them again.

Maximize beside the side dockers (2.538.0)

Maximizing a floating docker now fills the room between the side dockers that are open - the left docker, References, Assets, the Queue and the Gallery stay visible. Close one and the next maximize uses its room too.

Models: pages, folder tabs, card size, the model info page and Civitai info for every model (2.537.0)

The Model manager is built for hundreds of models now:

- **Pages** - a page holds as many cards as fit the docker (*auto*; resize the docker and it refits), or 24 / 48 / 96 / 200 a page; ‹ 1 2 … › under the cards.
- **Folder tabs** - under Models / LoRAs: one tab per folder you keep your models in (checkpoints/SDXL -> **SDXL**, loras/Pony -> **Pony**), so the types are apart. **This project** lists the models + LoRAs this project's renders used.
- ⇲ **Edit** - drag a card's corner (or the slider) to size the cards; the columns refill the docker. It is remembered.
- **A click on a card** selects it (the model to use, or a LoRA to add). **Hover** shows its description card - move onto it and **click it** for the **info page**: the picture, every detail, trigger words, the description and the **example pictures with their prompts** (*Use this prompt*), ✓ Use this model, ✎ Edit card, Civitai ↗. ‹ Back or Esc.
- **Civitai info** - details, a thumbnail and example pictures for every card on the tab that has none yet, found by the file's name (Shift+click: fetch them all again; By SHA-256 on the info page is exact). They are kept in the app's **model_info** folder, so every project sees them, and nothing is written into your model folders. It runs in the background, one model at a time (a few seconds each); ■ Stop ends it.

Veda sparse attention - a faster MiniMax H3 (2.536.0)

Veda (veda-sparse/Veda-on-ComfyUI) is a learned sparse attention for MiniMax H3: a small predictor decides which ~10 % of the attention matters and a Triton kernel skips the rest - about **2-3x faster end to end** at its trained sizes (1344x768, 768x1344, 768x768, 1024x768; 5 / 10 / 14 s), faster than Sage. It is a preview: faces can come out a touch softer, and other sizes use the nearest trained size's plan - compare a hero shot with Veda off.

1. Settings -> ComfyUI -> **Video accelerators -> ⬇ Install Veda** (it clones the pack and pip-installs its requirements; triton-windows on Windows). It needs **ComfyUI 0.38 or newer**.
2. Settings -> Models -> **Model catalog -> Veda predictor (MiniMax H3)** (275 MB, into ComfyUI/models/veda). Restart ComfyUI.
3. Settings -> Video styles -> a MiniMax H3 style -> **Veda sparse attention**: **On** (fastest, the authors' 90 %), or **Quality** (denser - 75 % / 50 % and full attention on the first step). The predictor field can stay blank: the app picks the one ComfyUI lists (an R2VA one for reference renders when you have it).

Veda replaces attention, so the style's Sage setting is skipped while Veda is on; it is skipped itself when Long video is on. If Veda fails in a render the app drops it and renders again without it (the log says so).

FLUX 3 Klein needs ComfyUI 0.41 or newer (2.535.0)

FLUX 3 Klein's text encoder loads with a CLIPLoader *type* (`flux3`) that only ComfyUI **0.41.0 or newer** has (0.41.1 or newer is best). On an older ComfyUI the job now says so - '*flux3*' *is a CHOICE of CLIPLoader.type, not a file: this ComfyUI does not offer it*, which version you have and how to update (the portable build: `update\update_comfyui.bat`; a git install: `git pull` + `pip install -r requirements.txt`; or ComfyUI-Manager -> Update ComfyUI) - then restart ComfyUI. The Style doctor says the same. The model catalog now downloads FLUX 3 Klein under its real file names (`flux3-image-klein-int8-convrot`, `flux3-image-klein-base-int8-convrot`).

Test mode, and new renders joining the open preview (2.534.0)

Test mode (Settings -> General, under Testing - or the box at the top of the Testing docker). While it is on:

- a small **Try next** box (bottom right) suggests the next test *where you are* - in the image editor it starts with Gaussian Blur, then the brush, the eraser, selections, layers ...; in Settings the Settings ones. ✓ **Tried it** asks whether it worked, **– Skip** marks it not tested, **Later** shows the next one, ✕ stops the reminders in that area for this session;
- when the app sees a test work by itself (you applied the blur, a job of that style finished, you picked a workspace ...) a **Did it work?** box asks you: ✓ **It worked** / ✗ **It didn't** / Later, with *what happened* and **Suggestions to change this feature**. Your answer and suggestion go in the test list and at the top of the log you save (Save log) under SUGGESTIONS;
- the docker's ↺ on a row tests it again.

While it is **off** nothing is ticked, nothing is kept and nothing pops up; your own ticks stay.

The preview follows new renders. If the picture preview is open when a render finishes, the new picture joins its strip at once (the one you are looking at stays) and a **1 new** button in its top bar takes you to it.

Dropdown lists inside a dialog open in front of it (2.533.0)

A long dropdown opens its own list with a search box (Settings -> General -> Dropdown lists). Inside a dialog - the *Select a LoRA* / *Select a ComfyUI model* browser, Settings, any other - that list now opens in front of the dialog and takes the mouse and the keyboard; before, it opened behind it. An open list also no longer closes by itself while a video preview plays or the log scrolls.

Tests tick themselves, docker title bars (2.532.0)

The test list ticks itself. When the app sees something on the list work - a job of that style finishing, a workspace picked, the palette used, a room furnished, a docker minimized - its line is ticked and the tick is **kept** (it shows *auto* ✓), so it is still there after a restart. Your own ✓ / ✗ / – always win; a note on an auto tick keeps it.

_ × on every floating docker. _ shrinks it to its title bar (▭ or a double-click on the title opens it again), fills the window between the menu bar and the statusbar (puts it back where it was, docked or floating), × closes it.

In the Sketcher, cameras are picked, moved and turned with their own widgets - see *Picking, moving and turning a camera (2.641.0)*.

A missing model file says where to get it (2.531.0)

Before a job is sent, every file a style loads - the model, the text encoders, the VAE, LoRAs, ControlNets - is looked up in ComfyUI's own lists. One sitting in another subfolder (for example `Anima\qwen_3_06b_base.safetensors`) is used under the name ComfyUI gives it. One that is in no list at all makes ComfyUI refuse the job; the message now also says when the app's **model catalog** has it: *Settings -> Models -> Model catalog -> "Anima text encoder (Qwen3 0.6B base)" (1.2 GB) downloads it to ComfyUI/models/text_encoders* - then restart ComfyUI. The Style doctor says the same.

Find anything - the Ctrl+K command palette and Help (2.530.0)

Press **Ctrl+K** (or in the menu bar) and type what you are looking for - words in any order:

- every **docker** (a hidden one comes back on screen) and every **floating window** (LoRA, Style doctor …);
- every **button** of the menu bar and the statusbar, even those the current workspace hides;
- every **Settings field** - the dialog opens on its tab with the field lit;
- the **workspaces**, the **Generate modes** and your **picture / video styles**;
- every **section of this guide** - it opens in the **Help** window, where any other section can be picked;
- every **shortcut key** - its group's key table opens in Help.

↑ ↓ pick, **Enter** runs it, **Esc** closes. What you run often comes first next time. Over the timeline's layers Ctrl+K still splits a clip, and in the image editor it is the editor's own palette.

The Style doctor - which styles work, and why the others don't (2.527.0)

in the menu bar opens the Style doctor. It lists every image style. ▶ **Check** makes a small test picture with each ticked style, one at a time, through the same path as Generate:

- **Quick check** - 512 × 512, 4 steps. Does the style load and run? A few seconds each.
- **Real render** - the style's own steps and size, slower.

Each row ends ✓ **works** (with its time and the test picture) or ✗ **failed** with the line from the log that matters and what to do, for example:

- a model ComfyUI does not list -> which models folder to put it in;
- a missing node pack -> its name, and Install Missing Custom Nodes;
- ComfyUI not running, out of video memory, a half-downloaded file, a text encoder that does not fit the model.

A style with a missing field fails at once, without rendering. An edit style (one that takes a picture) is given a small test picture. ■ **Stop** cancels the render running now. ↻ checks one style again. The test pictures do not stay in the gallery or the Image assets unless you tick **Keep the test pictures**. The last run is kept; **Report** downloads it as a .md file, clears it.

Workspaces - Basic, Image editing, Video editing, Music video, Expert (2.526.0)

The **Workspace** dropdown in the menu bar sets the app up for one kind of work. Each workspace shows the dockers that work needs and hides the statusbar and menu-bar buttons it doesn't. Nothing is disabled - Expert shows everything.

Workspace	Shows	Generate is set to
◯ Basic	Preview, Prompt, References, the queue and the gallery - the advanced buttons are hidden	(unchanged)
Image editing	The image editor (maximized), Prompt, References, Assets	Image
Video editing	Preview, Timeline, StoryBoard, SFX, Prompt	Video
Music video	Preview, Timeline, StoryBoard, Music, Stems, Prompt	(unchanged)
◆ Expert	Every docker and every button	(unchanged)

Your changes are kept per workspace. Hide a docker, resize one, open the Sketcher or Basic Settings - when you go to another workspace and come back, it is as you left it. The first time you leave Expert, the layout you had becomes Expert's, so nothing you set up before is lost. ↺ next to the dropdown puts the current workspace back to its default. The app starts in the workspace you were last in.

A model in another ComfyUI folder is still found (2.525.0)

ComfyUI's loaders each list their own folders: **Load Diffusion Model** (UNETLoader) lists `models/diffusion_models` and `models/unet`; **Load Checkpoint** lists `models/checkpoints`. A model downloaded as a Civitai "Checkpoint" often lands in `checkpoints`, where a diffusion-model style (Krea 2, Anima, Qwen Image…) could not see it.

Before a job is sent, the app now asks ComfyUI which list really holds the style's model:

- the same file in another subfolder -> the name ComfyUI lists;
- a `.safetensors` only Load Checkpoint lists -> that node becomes **Load Checkpoint** (it loads a model-only file too); the workflow keeps its own text encoder and VAE loaders.

The job log says so: `node 70.unet_name: Krea2/… -> as ComfyUI lists it: … loaded by Load Checkpoint`. If the file is in no list at all, the error says so and tells you where to put it (`ComfyUI/models/diffusion_models`, a subfolder is fine, or a folder added to extra_model_paths.yaml - then restart ComfyUI).

A style's model can be a .gguf or a .safetensors (2.523.0)

A picture style's **Model** can be either kind of file. The app picks the matching ComfyUI loader for it:

- a `.safetensors` model uses Load Diffusion Model (UNETLoader);
- a `.gguf` model uses the GGUF loader (the ComfyUI-GGUF pack).

So "Krea 2 Raw GGUF" can render a `.safetensors` fine-tune, and "Krea 2 Raw", "Krea 2 Turbo" and "Anima" can render a `.gguf`. The job's log says when it switched, for example `model: node 70 UnetLoaderGGUF -> UNETLoader`.

Both loaders read the same folders: ComfyUI's `models/diffusion_models` and `models/unet`, and any subfolder in them. A separate `unet` folder is not needed.

Reporting a bug (2.522.0)

Press **Ctrl+Alt+Shift+B** anywhere in the app (it works while typing too). The app takes a screenshot of its window first, as it was, then opens the **Bug report** docker. Press it again to hide the docker. You can also open it from Settings ▸ General ▸ **Report a bug…**.

1. Fill in:
   - **Title**: in a few words, what broke.
   - **Severity**: crash / freeze, something does not work, ⚠ wrong result, looks wrong, or idea.
   - **Where**: the docker you used last is picked; change it if needed.
   - **What happened** and **what you expected**.
2. **Steps to reproduce**: **Use the recorded steps** puts your last clicks, choices and shortcut keys in, with the docker each was in. Edit them as you like.
3. **The screenshot**:
   - ✏ **Pen** and ▭ **Box** mark the problem in red; ↶ **Undo** takes the last mark off.
   - **Again** takes a new screenshot (the docker hides for it).
   - **A picture…** uses one of your own.
   - In a web browser, asks which window to capture.
4. **Attached** (tick or untick each):
   - the screenshot;
   - the **logs**: the app console, ComfyUI, the vision model, and the last jobs with failed ones first;
   - the **system + settings**: version, OS, graphics card, RAM, ComfyUI version;
   - the **recorded steps**.
5. **What is included** shows all of it before you save.
6. **Save and send** (2.714.0) saves the report and sends it to the **bitGen Community**: it gets its own discussion on the website, under **Discussions ▸ Bug reports**, where you and others can follow it and reply.
   - Signed in to **Community** with a confirmed email, it goes **as you** and shows at once. The line above the button says who it goes as.
   - Not signed in (or the email not confirmed yet), it goes **anonymously** and shows once an admin approves it. Tick **Send anonymously** to send it that way even when you are signed in.
   - Everyone sees the title and what you wrote. **Only the admins** see the logs, the settings, the system and the screenshot. The screenshot goes with your ✏ / ▭ marks; your home folder is shown as `~` in the logs and settings.
   - **Earlier reports**: sends one you saved before; ✓ **sent** opens where it went.
7. **Save as a file** writes the report into the app folder's `bug-reports` folder and downloads **bitgen-bug-<date>-<title>.zip**, without sending it. It holds:
   - `report.md`, to read;
   - `report.json`, everything;
   - `screenshot.png`;
   - the `logs`.

Earlier reports lists what you saved: sends it, downloads it again, deletes it. A report you have started but not saved is kept in this browser until you save it.

Privacy:

- The recorder keeps only what was pressed and where. A text box only says "edited", never what was typed.
- Every setting that looks like a key, token or password is hidden as •••, and so are token-shaped text in the logs (`hf_…`, `ghp_…`, `sk-…`, `token=…`).
- The recorder also keeps page errors and requests that failed (the address and status, never what was sent).

Fine-tunes in the trainer: checkpoints, the assistant (LLM) and the vision model (2.521.0)

The statusbar's **LoRA** button opens the **LoRA & fine-tune trainer**. Every job now has a **Type**, the first row of the Job tab:

Type	What it trains	Engine	The result
LoRA	a small add-on for a picture / video model	ai-toolkit	ComfyUI's `loras/trained`
Checkpoint fine-tune	every weight of a picture model - a new model	ai-toolkit	ComfyUI's `diffusion_models/trained`
LLM fine-tune	the app's assistant, on your own chats	Unsloth	a `.gguf` in the LLM folder
Vision fine-tune	the app's vision model (captions, picture questions)	Unsloth	a `.gguf` + its `mmproj` in the vision folder

The base-model list marks the recommended ones with ★ and says how much graphics memory each needs compared with your card. A model your card is too small for can still be trained on a bigger machine: **Export job**, or Settings ▸ Remote GPU ▸ RunPod.

The recommended base models

The assistant (LLM):

- ★ **Qwen3.5 4B**: the best start (a 12 GB card, ~10 GB for a 16-bit LoRA).
- ★ **Qwen3.5 9B**: the strongest that fits a 24 GB card.
- **Gemma 4 E4B / E2B**: Google, good writing; E2B fits an 8-10 GB card.
- **Gemma 4 31B**: as a 4-bit QLoRA on a 24 GB card.
- **Qwen3.5 0.8B / 2B**: small and quick; the 0.8B can take a full fine-tune.
- **Qwen3.8 27B**: the newest, for a 64-80 GB pod.

All are Apache 2.0. Qwen3.5 trains in 16-bit: Unsloth advises against 4-bit for it.

The vision model: ★ **Qwen3.5 4B vision** (12-16 GB) or ★ **Qwen3.5 9B vision** (24 GB); Gemma 4 E4B / E2B.

Pictures:

- **A LoRA** stays the best way to teach a person, a style or an object, on any card. New: **FLUX.2 klein base 4B** (Apache 2.0). It fits a 24 GB card, learns fast, and its LoRA works with the fast distilled klein 4B.
- **A checkpoint fine-tune** (every weight) needs a big card:
  - Lumina-Image 2.0: 24 GB, one of ai-toolkit's own examples.
  - Flex.1-alpha: 48 GB, or less with **parameter swapping**.
  - FLUX.2 klein 4B and SDXL: experimental.
- **Bake** gives you a new checkpoint on any card: train a LoRA, then bake it into its base model.

LLM fine-tune (the assistant)

1. ⚙ **Install Unsloth** (once). A queue job makes Unsloth's own Python with PyTorch for your card, plus llama.cpp's GGUF converter (about 8 GB).
2. **Job tab**:
   - **Base model** and **Method**: LoRA (16-bit, merged into the model), QLoRA (4-bit, less memory) or Full fine-tune (every weight, about 4× the memory).
   - **LoRA rank / alpha**, **epochs** or **max steps**, **learning rate**, **batch**, **gradient accumulation**, **max length** (tokens) and **warm-up**.
   - **System prompt** (who the assistant is) and **Model file size** (Q4_K_M is the usual).

– **Config** shows Unsloth's run as JSON; edit it and press ✎ **Use mine**.

3 **Dataset tab**: each example is a short conversation.

– **Example**: type what someone asks and the answer you want. **a follow-up turn** adds more turns.

– **Import…** reads:

– JSONL / JSON: `{"messages": [...]}`, ShareGPT `conversations`, `instruction` / `input` / `output`, `prompt` / `completion`, `question` / `answer`;

– a CSV with a header row.

– **chat.jsonl** saves them all.

– About 100 good examples teach a style or a task; a few thousand teach knowledge.

4 **Run & samples**: ▶ **Start the fine-tune**. The queue shows the steps and the loss. The finished model is merged, converted to `.gguf` and put in the LLM folder (`<name>-Q4_K_M.gguf`); Settings ▸ LLM picks it.

Vision fine-tune (the captioner)

Add pictures in the Dataset tab and write, under each one, **the answer** the vision model should give. Write the question in the Job tab's **Instruction**, for example "Caption this for a storyboard."

Answer the empty ones lets the vision model you have now write a first answer for you to correct. Pictures without an answer are left out.

The `.gguf` and its `mmproj` projector land in the vision folder (`llm-models/vision/<name>/`, or the vision folder picked in Settings), where Settings ▸ LLM ▸ Vision model lists them.

Checkpoint fine-tune (pictures)

The same pictures and captions as a LoRA, but every weight of the model is trained.

- **Settings**: adafactor with a small learning rate, saved in diffusers format.
- **Parameter swapping** trains part of the weights each step; it needs much less memory but is slower.
- **The result**: the saved files are joined into one `.safetensors` in ComfyUI's `diffusion_models/trained`, with a Model manager card. Load it with Load Diffusion Model.

Bake a LoRA into a new checkpoint

When a picture LoRA has finished, the Run tab offers **Bake into a new checkpoint**:

1 Pick the base model it was trained for (ComfyUI's checkpoints and diffusion models; ★ = the same family).

2 Set the strength and the name.

3 Press **Bake**.

ComfyUI merges and saves it; the new file is moved into `checkpoints/trained` (or `diffusion_models/trained`) with a card. Baking needs this computer's ComfyUI, not a remote one.

The LoRA trainer, peak / off-peak prices, the usage log, Remote GPU, packages in Settings (2.520.0)

The LoRA trainer

The statusbar's **LoRA** button opens the LoRA trainer docker. It trains a LoRA from your pictures with **ai-toolkit** (by Ostris), the trainer many LoRAs are made with.

1 ⚙ **Install ai-toolkit** (once). A queue job downloads ai-toolkit into the app folder, makes its own Python environment and installs PyTorch for your card. It is about 10 GB and takes a while; follow it in the Queue.

2 **New job**: give the LoRA a name.

3. **Job tab**:
   - **Base model**: Z-Image Turbo (the easiest start), Z-Image, FLUX.1 dev / schnell, Qwen-Image, SDXL, SD 1.5, Chroma or Wan 2.1 (video). Each one says how much graphics memory it needs and whether your card has it.
   - **Your own model file** (optional): a path instead of the Hugging Face download.
   - **Trigger word**: a made-up word, e.g. `t1m_style`. It is put in front of every caption.
   - **Rank / alpha, steps, learning rate, batch, optimizer, sizes** (512 / 768 / 1024…).
   - **Caption dropout, flip, Low VRAM, quantize**.
   - **Save every / keep**, **samples every** and the **sample prompts** (`[trigger]` is your trigger word).
   - **YAML** shows the ai-toolkit config the form makes. Edit it and press ✎ **Use my YAML** to train with yours; ↺ **Back to the generated one** returns to the form's.
   - **Save** keeps the job.
4. **Dataset tab**: add 10-30 pictures with **Pictures…**, by dragging them in, or with **From the gallery**. Write a caption under each one, or let the assistant write them: **Caption the empty ones** / **all** (writes every caption again).
5. **Run & samples tab**: ▶ **Start training**. The queue shows step N / M and the loss, and the samples appear as ai-toolkit makes them. **Stop** cancels the run.

When the run finishes, the LoRA is copied into ComfyUI's `loras/trained` folder with a Model manager card (the trigger word as its keyword) and the last sample as its picture. It is ready to pick in the Model manager.

Export job downloads config.yaml plus the dataset as a .zip, to train on a bigger machine (a RunPod pod's ai-toolkit).

Peak and off-peak prices

Settings ▸ General ▸ Energy cost:

- **Peak price per kWh** and **Peak times**: days (Mon-Sun) plus from-to times. Add as many windows as your tariff has; a window like 22:00-02:00 runs through midnight.
- **Every minute outside the peak times is off-peak**, at the normal price per kWh.

The power meter files every reading as peak or off-peak, so a job that ran across both is split:

- its ⚡ card and its `[power]` log line show peak Wh at the peak price + off-peak Wh at the off-peak price + its share of the daily supply charge;
- "⚡ since start" says whether it is peak now.

The usage log

Usage… (Settings ▸ General ▸ Energy cost), or a click on the ⚡ figure in the statusbar, opens the usage docker:

- **Ranges**: today, 7 days, this month, last month, everything, or your own dates.
- **Totals**: the bill for the range (energy + supply + remote GPU), the energy while the app ran (peak / off-peak), the jobs and their energy and cost.
- **Days**: kWh, peak / off-peak, jobs, hours, energy, supply, remote, total.
- **Jobs**: every finished job with its mode, what it was, status, minutes, kWh, peak kWh, average watts, cost and the remote GPU it ran on.
- **CSV** downloads the table you are looking at.

The log is kept in the app folder's `usage` folder: one file a month.

Settings ▸ Remote generation

Generate on another machine's graphics card:

- **This computer** (the normal way).

- **Another machine's ComfyUI**: its address (e.g. `http://192.168.1.20:8188`, started with `--listen`) and what it costs an hour (optional).
- **RunPod**: your API key (runpod.io ▸ Settings ▸ API Keys) lists your pods with their card and price. Start / stop a pod and pick it; the app uses its ComfyUI through RunPod's proxy (the port, 8188 by default, must be exposed as HTTP).
- **SSH tunnel**: user@host, port and key file. ▶ **Open the tunnel** forwards the machine's ComfyUI to this computer.

✓ **Test** asks that ComfyUI for its card and version. **Use it** points the app at it (the app then does not start its own ComfyUI); **Use this computer** puts everything back.

The models a job uses must be on that machine. The power meters see only this computer, so a remote job's cost in the usage log is its time × the hourly price (a RunPod pod's own price).

Settings ▸ Models: This computer & package

The top of the Models tab shows:

- your graphics card, RAM and processor;
- the package installed and the one for this computer;
- the four packages, with what each lets you use (✓ ~ ✗, in it / add later).

✎ **Not right?** corrects the hardware, the same way setup asks for it.

⬆ **Upgrade to …** moves to a bigger package without running setup again. In the background it installs what the package adds (ComfyUI and its node packs when there is none yet, the assistant, voices, speech to text) and downloads its models; gated models need your Hugging Face token from Settings ▸ General. The log and the progress show under the button.

Grouped downloads

On the models page (setup's last step and Settings ▸ Models ▸ Model downloads), every group has its own checkbox. It ticks or unticks the whole group and shows how many files the group has to get and their size. A part-ticked group shows a dash.

Setup asks for your hardware and picks a package; the daily supply charge (2.519.0)

Setup's first page shows your graphics card, RAM and processor. When it cannot find one of them (a card that is not NVIDIA, a card whose memory Windows does not report, a processor without a name), it asks:

- **Graphics card**: pick it from the list (NVIDIA RTX 50 / 40 / 30 / 20 series and workstation cards, AMD Radeon, Intel Arc, Apple Silicon, "No graphics card", or "Another card" with its name and memory).
- **System memory (RAM)** and **Processor (CPU)**.

Windows' Task Manager ▸ Performance shows all three. **Not right? Tell setup your hardware** opens the same form when setup did find something but got it wrong.

Your package (setup's second page) offers four packages, with the one for your computer picked:

Package	For	Installs
Editor	any computer	the full video and image editor, CAD and 3D sketching, sound and delivery - no ComfyUI, no models
Editor + small AI	6-12 GB graphics card	+ ComfyUI, Anima and SDXL pictures, background removal, speech to text, voices, a small assistant

Editor + mid-size AI	12-20 GB	+ Krea 2 / FLUX 3 Klein pictures, Kontext and Qwen Image 2.1 editing, MiniMax H3 at a smaller size, songs, upscaling
Everything	20 GB+	the whole studio at full size

Under the packages, **What you'll be able to use** lists every feature for the package you picked:

- ✓ works well; ~ works, but slower or at a smaller size; ✗ not on this computer, with the reason (for example "needs about 14 GB of VRAM (this card has 12)").
- **in this package** / **add later** (Settings ▸ Models adds it any time).
- The editing features work on every computer. The AI's memory needs are estimates.

The Install button names the package and the download size. A package bigger than your computer runs well says **Install anyway**. The Editor package installs no ComfyUI and no models and opens the studio straight after.

From a terminal: `node scripts/setup-cli.js --packages` prints the same list, and `--package editor|light|standard|full` with `--gpu ID --vram GB --ram GB --cpu "NAME"` installs it.

Daily supply charge (Settings ▸ General ▸ Energy cost): the fixed amount per day on your electricity bill. A job's cost adds its share of it for the time it ran (a 30-minute job carries 1/48 of a day's charge):

- the job's ⚡ card shows the total (for example "⚡ 125 Wh · $0.06");
- its tooltip and its `[power]` line split it into energy and supply;
- "⚡ since start" adds the share since the app started.

Queue presets, the Model manager, Civitai downloads, energy cost, the Testing docker (2.518.0)

Queue presets (the menu bar, beside the project buttons) set how the job queue runs:

Preset	Power limit	Rest between jobs	Waits while a card is hot
Normal	the card's own	none	never
Quiet (overnight)	70 %	20 s	from 80 °C until 70 °C
Eco	60 %	10 s	from 78 °C until 68 °C
Cool-down between jobs	the card's own	60 s	from 75 °C until 65 °C
Max performance	100 % (the default put back)	none	from 90 °C until 80 °C
Custom	your four numbers (in)		

- says what the preset does, whether the power limit was set, a job it is holding ("rest between jobs: 12 s left") and warnings ("GPU 0 is 86 °C", "probably throttling").
- The badge beside the dropdown shows while a job is held and ⚠ with a warning.
- A power limit needs the app run as administrator (Windows) or with sudo (Linux), and lasts until the computer restarts. Without the rights it says so, and the rest of the preset still works. Picking Normal puts the cards' default limit back.
- The job that starts after a hold says so in its log: `[queue] waited 20 s - …`. A hold for heat gives up after 20 minutes, so a stuck sensor never stops the queue for good.
- Compiles (ffmpeg only) never wait.

The Model manager (Models in the statusbar) is a floating docker: float it, or dock it beside or inside another docker.

- **Tabs**: Models (ComfyUI's checkpoints and diffusion_models folders, with extra_model_paths.yaml), LoRAs, Civitai, Model paths (2.580.0, below).
- **Filters**: search (name, type, keywords), category, type, sort (A-Z, newest, rating, size). reads the folders again.

- **A card** shows the thumbnail, name, ★ rating, release date, the model type (SDXL, Pony, Illustrious, Flux, Qwen, Krea, Anima, Wan, LTX ...) and the category (image, image edit, video, music). LoRA cards add their keywords; click one to put it in the prompt.
- **Hover a card** to see its description card beside it: the description, base model, cfg, steps, sampler, VAE, text encoder, keywords, size and file. It closes when the pointer leaves.
- **Where a card's details come from**, later ones winning:
  - the file itself: its name, folder, safetensors header or GGUF header;
  - what other tools left beside it (`<name>.civitai.info`, `<name>.metadata.json`);
  - `<name>.bitgen.json`: the Civitai answer and your own edits.

The thumbnail is `<name>.preview.png` / `.jpg` / `.webp` beside the model.

- ✎ **on a card** edits it: name, your rating, type, category, cfg, steps, VAE, text encoder, keywords, release date and description. They are saved beside the model.
- **on a card** looks the file up on Civitai by its SHA-256. The file is read once and the hash is kept.
- **Select**: click a model card for the model, and click LoRA cards to add or remove them (each has a weight).
- ☐ **Override the style's model + LoRAs**: ticked, Basic Settings renders with the selection. This turns Override style on, sets the checkpoint override to the model, adds the LoRAs as temporary LoRAs, and uses the card's steps and cfg. Unticked, everything is put back as it was.
- **+ New style from it**: a new style from the selected model and LoRAs. The style that renders its type is copied (its workflow, text encoder, VAE, sampler and negative), or another you pick; the model, the LoRAs and the card's cfg, steps, VAE and text encoder replace those. A video model makes a video style, with the model in its first sampler phase.
- **A ComfyUI that is installed but not set in Settings** (2.622.0): when Settings ▸ ComfyUI ▸ Folder is empty, bitGen looks for a ComfyUI on this computer - $COMFYUI_PATH, beside bitGen Studio, your home folder (ComfyUI, the portable build, Documents, Desktop, Downloads), Pinokio, Stability Matrix, the top of each drive, ComfyUI Desktop - and its **checkpoints** and **diffusion models** (and unet) fill the Model manager, Basic Settings' checkpoint list and the video Model list. The status line says where it was found; Settings ▸ ComfyUI ▸ Folder shows it with **Use this folder** to set it for good.

Civitai (Settings ▸ General ▸ Civitai, then the Model manager's Civitai tab):

- Paste your API key (civitai.com ▸ Account settings ▸ API Keys) and **Test key**.
- Pick **civitai.red** (everything your account's content level allows) or **civitai.com** (safe for work only). **Mature results** adds mature results and pictures where your account allows them.
- **Search** by words, type, sort and base model, then click a result to see its versions. on a file downloads it straight into the right ComfyUI folder:

Civitai type	Folder
Checkpoint	checkpoints (a GGUF, or a file with no VAE / text encoder inside → diffusion_models)
LoRA / LoCon / DoRA	loras
VAE	vae
Embedding	embeddings
ControlNet	controlnet
Upscaler	upscale_models

A subfolder you type is kept. There is no moving and no Save As. Its card (picture, base model, keywords, the example pictures' cfg and steps) is written beside it. The same file already there is not downloaded again.

- The key is saved in `custom.settings` and sent only to Civitai.

Energy cost (Settings ▸ General): your electricity price per kWh and the currency. Every finished job's ⚡ card, its `[power]` line and the System stats' "⚡ since start" then show what it cost (for example

"⚡ 125 Wh · $0.04").

Testing (Settings ▸ General ▸ Testing…) opens a floating docker with the tests this version needs, release by release.

- **auto** items tick themselves as the app sees them work: a job of that style finished, nvidia-smi answered, a Civitai download landed in loras, and so on.
- The others have ✓ (works), ✗ (does not) and – (not tested) buttons, and every item has a note.
- **Save log** downloads `bitgen-test-log-<version>-<date>.txt`. It has the computer, every item with how it was decided, your notes, the failed jobs' last log lines, and the same as JSON. Upload that file to Claude.
- Ticks and notes are kept in `custom.settings`, and **↺ Clear** starts again.

Power draw and clock speeds, the energy a job used; Anima image styles (2.517.0)

System stats (the left dock):

- **Each graphics card's frame** has two new graphs:
  - **Power**: the card's draw in watts, scaled from 0 to its power limit.
  - **Clock**: its core clock, scaled from 0 to its top clock, with the memory clock beside it.
  - Its title adds the watts and the GHz.
- **CPU clock**: the processor's current speed.
  - On Windows this comes from the performance counter, so it follows boost and idle.
  - On Linux it comes from the cores themselves.
- **CPU power**: shown only when the computer reports it. Linux usually does (RAPL, readable by the app); Windows has no way to read it without a driver.
- ⚡ **since start**: the energy used since the app started.

The card readings come from nvidia-smi, which keeps running and reports once a second.

The energy a job used. When a job ends, the app works out the energy used from its start to its end. It records:

- the total in Wh or kWh: the graphics cards, plus the CPU where it can be read;
- the average and peak watts;
- how long the job ran.

This shows in three places:

- a **[power]** line at the end of the job's log;
- ⚡ **… Wh** on the job's card in the queue's Done tab (its tooltip has the details);
- the project's job list, so it is still there after a restart.

The figure covers the whole card and CPU while the job runs, so anything else using them then is counted too. A computer without nvidia-smi gets the CPU only, where the CPU can be read.

Anima image styles (in the image styles list): CircleStone Labs' 2B anime and illustration model, run through ComfyUI.

- **Anima** uses Aesthetic v1.1, the one to start with.
- **Anima base** uses Base v1.0.
- Both run 30 steps at cfg 4 with er_sde, and come with the model card's negative prompt.

Where the files go:

File	Folder
`anima-aesthetic-v1.1.safetensors` / `anima-base-v1.0.safetensors`	`models/diffusion_models`

`qwen_3_06b_base.safetensors` (the text encoder)	`models/text_encoders`
`qwen_image_vae.safetensors`	`models/vae`

All of them come from circlestone-labs/Anima, and the models setup lists them.

Prompting:

- Start the prompt with `masterpiece, best quality, score_7, safe,` .
- Danbooru tags, plain words or both work.
- About 1 MP works best: 1024 × 1024, 832 × 1216 and so on.
- The licence is non-commercial for the model and LoRAs, but the pictures it makes may be used commercially.

Keyframes on any setting; Krea 2 image styles (2.516.0)

◇ **Keyframes on any setting.** A clip's settings in the timeline's Properties, and in the Colour dialog, now have a small ◇ beside them.

1. Move the playhead to where the change starts, set the value, and click ◇. It becomes ◆: a key at that moment.
2. Move the playhead and change the setting. That makes a second key by itself, because once a setting has keys any change to it is keyed where the playhead is.
3. A keyed setting has an orange edge and shows its value at the playhead. Click ◆ to take away the key at the playhead, or **Alt+click** to take away all its keys.

What can be keyed this way:

- **Colour**: exposure, contrast, saturation, temperature, tint, hue, LUT mix, and in the Colour dialog the wheels' brightness and the HSL ranges.
- **Effects**: lens, warp amount, size and speed, and the keyer's match, edge and spill.
- ✒ **Masks**: feather, expansion and opacity. A mask's shape keys with its own ◇, as before.
- **Text**: size and outline.
- **Time**: shutter, samples, posterize, and echo copies, gap, first and fade.

Position, scale, rotation, opacity, blur, sharpen, glow, vignette, grain, crop, generators, rays, distortions, shapes, text animators and 3D could be keyed already. Every keyed setting is listed in the **Graph Editor** with its value and speed graphs, eases and Bézier handles.

How ⚙ Compile handles each kind:

- **Colour** keys: the look is re-made every quarter second between keys and the looks are cross-faded.
- **Lens, warp and keyer** keys are animated by the compile itself, on V1 and on video layers.
- **Mask** keys go into the mask's video.
- A **picture or text layer** with keyed settings, and a **video layer with keyed time effects**, are rendered by the app frame by frame.
- Not yet covered: adjustment layers, and the colour wheels' colour (the puck), which is not a number box.

Krea 2 image styles (in the image styles list). These need ComfyUI with Krea 2 support (0.26 or later):

- **Krea 2 Turbo**: 8 steps, cfg 1.
- **Krea 2 Raw**: the full model, 52 steps, cfg 3.5, with a negative prompt.
- **Krea 2 Turbo GGUF** and **Krea 2 Raw GGUF**: the same, from a GGUF file.

If they are gone from the list (2.547.0): they come back by themselves once, the next time bitGen starts. Any other built-in style that went missing: Settings ▸ Image styles (or Video styles) ▸ ↺ **Built-in**

lists the built-in styles that are not there and puts them back after you say yes. Your own styles and the ones you edited are never touched.

Where the files go:

File	Folder
`krea2_turbo_fp8_scaled.safetensors` / `krea2_raw_fp8_scaled.safetensors` (Comfy-Org/Krea-2)	`models/diffusion_models`
`krea2_turbo-Q5_K_M.gguf` / `krea2_raw-Q5_K_M.gguf` (vantagewithai's Krea-2-Turbo-GGUF / Krea-2-Raw-GGUF)	`models/unet`
`qwen3vl_4b_fp8_scaled.safetensors` (text encoder)	`models/text_encoders`
`qwen_image_vae.safetensors` (VAE)	`models/vae`

Notes:

- The models setup lists every file with its download.
- Any GGUF quant works; set the style's **Model** to the file you have. Q8_0 is 13.7 GB, Q6_K 10.6 GB, Q5_K_M 8.9 GB (fits a 12 GB card) and Q4_K_M 7.5 GB.
- The GGUF styles need the ComfyUI-GGUF custom nodes in a version that knows Krea 2. If ComfyUI says *Unexpected architecture type*, update ComfyUI-GGUF; if that doesn't fix it, use a fork that supports Krea 2 (molbal/ComfyUI-GGUF).
- Krea's style LoRAs (`krea2_*.safetensors`) work in the style's LoRA slots on both Turbo and Raw.

Cut out a subject, solve the camera, remove an object; theme edit mode for dialogs (2.515.0)

Three new sections are in a video clip's Properties on the timeline, for V1 or a video layer: **Cut out**, **3D camera track** and **Remove**. Each one works on the clip's whole length in the background, with a progress line and ■ **Stop**.

Cut out: a subject followed through the shot

1. Pick the clip and go to the frame where the subject is clear.
2. Press **✚ Subject** and click the subject on the picture. Press **✖ Not it** and click anything next to it that is not part of it. Or press ▭ **Box** and drag a box round it. Click a mark to take it away.
3. If the cut-out goes wrong at another frame, move the playhead there and add marks. Each frame follows the nearest frame that has marks.
4. Choose how the subject is followed:
   - **Auto** uses the best one that is installed.
   - **SAM 2** (Segment Anything 2) is the best. **⬇ SAM 2** installs it into the AI matte's venv: torch, the sam2 package and its checkpoint, about 1–3 GB.
   - **AI matte** (BiRefNet) uses the same install as Select ▸ Subject.
   - **Built in** uses OpenCV and needs nothing installed.
5. **Refine edge**:
   - **snap** fits the edge to the picture's own edges (hair, fur).
   - **feather** softens it.
   - **choke** shrinks the edge (or grows it, with a minus number).
   - **smooth** stops the edge flickering from frame to frame.

6 Press ▶ **Cut out**. The result is saved in the project's videos as `cut_<clip>_….webm`, with the subject's see-through edge and the clip's sound.

On a video layer, **use it** makes the layer see-through round the subject, on the canvas and in ⚙ Compile. On V1 the box only shows a preview, because nothing is below V1. The cut-out is also what **Remove** takes out and what **3D camera track** leaves out.

Remove: content-aware fill for video

1 Choose what goes: **the cut-out** or, on a video layer, ✒ **the masks** (drawn with the Mask Pen; they can move).

2 Set the options:

– **grow**: the hole made bigger, to take an edge or a shadow too.

– **feather**: the soft join.

– **look**: how many seconds before and after are searched.

– **Auto** fills the hole from the frames round it, lined up on this one. It paints in only what the shot never shows, and keeps that part still. **Paint in only** is faster, but it may shimmer.

3 **Reference** (optional): one frame you have filled yourself. ✎ **Paint one** opens the frame at the playhead in the image editor with the hole see-through. Fill it with Generative Fill or the Clone Stamp, press **Ctrl+S** to save it to the project's Images, then pick it here.

4 Press ▶ **Remove**. The clip now plays `<clip>_fill….mp4` (`.webm` for a webm clip). It is the same length and times as the original, with its sound. ↺ **Original** puts the clip back on its own file.

3D camera track: put layers into the filmed scene

1 Choose how the camera moved: **Auto**, **Moving camera** or **Tripod** (it only turns). Leave the lens at **0** to have it worked out, or type its across angle in degrees. Tick **leave out the cut-out** so that a person or a car moving on its own is not used.

2 Press ▶ **Solve**. The line shows how many frames were solved, the lens, the error in pixels and how many points were found. The points are drawn on the picture; when the solve is good they stay stuck to the scene as it plays.

3 **Pick**: click points (Shift adds more) or drag a box round them.

4 Choose a layer (or **New text**), then **flat on them**, **standing on them** or **facing the camera**, and a size (a share of the frame's width where it stands). Press **Place**. The layer becomes a 3D card in the scene.

5 **Drive the 3D camera** keys the 3D camera every frame from the solve: position, aim, zoom and the new **roll**. Place does this by itself, so every 3D card sits in the shot. The tracked clip should fill the frame across and be centred, as V1 is.

Theme edit mode for dialogs

- **Ctrl+Shift+Alt+E** turns edit theme mode on with the dialogs included. An open dialog such as Settings stays open, and its buttons, labels and boxes are picked and styled like the rest of the app. The Style panel, the outline and the handles go inside the dialog. Esc closes the Style panel, never the dialog.
- **Ctrl+S** in edit theme mode (either way in) saves the theme to `themes/<name>.json` and leaves edit mode. A theme with no name yet asks for one.
- Ctrl+Shift+Alt+E again leaves, with the usual box when there are changes.

Edit theme mode: every button and its words (2.623.0)

In edit theme mode () every docker, panel and button can be picked and its words changed:

- **Double-click any words** - a button, a docker's title, a tab, a label, a heading, a hint - and a box opens over them: type the new words and press Enter (an empty box = its own words again), or Esc to leave them. A button keeps its icon, a docker's title keeps its ▾ and its buttons.
- **Dropdowns**: click one and the Style panel lists its **Options** - rename any of them (what it does stays the same).

- **Placeholder and Tooltip**: the Style panel changes the words shown in an empty box and the help shown when the pointer rests on anything.
- **The floating tool panels** (the timeline's stage panels, DLSS 5, SeedVR2, the voice chat) are picked and styled like the dockers.
- **Shift+click** (or **Alt+click**) gives the click to the app - a button does what it does, and a collapsed section, a tab, a docker or a menu opens - without leaving edit theme mode, so the parts inside can be reached and edited too.

All of it is kept with the theme (Save theme), and ↺ Clear element styles puts every word back.

Edit theme mode: resize, move, hide, rename; leaving saves; folding dockers (2.514.0)

Edit theme mode () can now change the layout. Pick an element as before, then:

- **Resize** it with the eight handles round its outline. The left and top handles keep the opposite edge where it is.
- **Move** it by dragging inside it. The Style panel's **Layout** section shows the move as **Δx** / **Δy**.
- Type an exact **Width** / **Height** (4 to 4000 px) in the Layout section.
- **Hide** it. In edit mode a hidden element shows faint with a dashed outline so you can pick it again; once you leave edit mode it is gone.
- **Hidden in Expert mode** (2.544.0): the element is gone while Settings ▸ General ▸ Expert mode is on, and shown in Basic mode. In edit mode with Expert mode on, it shows faint with a blue dashed outline so you can pick it again.
- **Text**: change the words of a label or a plain-text button.
- ↺ **Layout** puts the size, the move, the hiding and the text back.

Your changes are kept as you go. Every change is written to `themes/<theme name>.tmp`. If the app closes before you save, the next time you start edit mode a box offers **Bring them back** or **Throw them away**.

Leaving edit mode (or Esc) with changes opens a box:

- **Save theme "<name>"**: saves into `themes/<name>.json`. A theme that has no name yet asks for one.
- **Save theme as…**: saves under a new name, which becomes the current theme.
- ↺ **Reset theme**: goes back to how the theme was when edit mode began.
- **Not now**: leaves edit mode with the changes showing but not saved. The `.tmp` keeps them.
- **Cancel**: stays in edit mode.

Folding the side dockers. Each side docker's title bar has a ▾ at its left. Click it to fold the docker to a thin strip that keeps its title, which gives the rest of the window more room. Click ▸ to open it again. The app remembers which dockers are folded.

The image editor's dockers.

- Each group of tabs (Layers, Properties, History …) starts with ▾. Click it to fold the group to its tabs, and click a tab to open it again.
- Each group's ✕ closes it.
- **View ▸ Dockers** in the image editor's menu lists every image editor docker with a tick. Click one to show or hide it, or use **All Dockers** to bring every one back.
- The ✕ on a floating docker window now works. It was covered by the corner you resize the window with.

Load UE project, the timeline's frame size bar (2.513.0)

Where to open an Unreal project. Open it in **Unreal**, as always. bitGen links to the Unreal Editor through the bitGen Bridge plugin, and Unreal must have the project open for the link to work.

Load UE project (in the menu bar, after):

1 The folder dialog opens. Pick your Unreal project's folder (the one with `YourProject.uproject`) and press **Select**.

2 bitGen then does the rest by itself:

– It puts the bitGen Bridge plugin into `<project>\Plugins\bitGenBridge`, or updates an older one.

– It switches on the plugins the bridge needs in the `.uproject`: Python Editor Script Plugin, Editor Scripting Utilities, Sequencer Scripting, Movie Render Queue and bitGen Bridge. The old file is kept as `.uproject.bitgen-bak`.

– It closes Unreal if it has that project open. Unreal asks you to save first, so answer it.

– It starts Unreal again with the project. Unreal's Output Log then says `[bitGen Bridge] listening`.

3 The **Unreal project** dialog lists each step. It also shows what the project has: the plugin version, the plugins switched on, whether Unreal is open, and where the editor is.

The dialog also has:

- **Restart Unreal**: restart Unreal with the project only, without touching the plugin or the `.uproject`.
- **Test connection**: check whether bitGen is linked to Unreal.
- **Also close other Unreal windows**: tick it if another project is holding the bridge's port.
- The projects you used before.
- If the editor is not found from your Epic Games installs, a field to type the path of `UnrealEditor.exe`.

The timeline's menu bar (at the top of the Timeline docker) sets the **frame**: the size the timeline is made at. It applies to the canvas, the bakes and ⚙ Compile.

- **width × height**: type a size.
- **Preset**:

  – **Auto** is the V1 video's own size, as before.

  – Or pick a size: 4K, QHD, Full HD, HD, SD, the vertical sizes (Reels, Shorts, TikTok), square, 4:5, 21:9, cinema 2.39:1, DCI 2K or 4:3.
- **Aspect**: Free, 16:9, 9:16, 1:1, 4:5, 4:3, 3:2, 2:3, 21:9, 2.39:1 or 1.85:1. The aspect is kept while you type: a new width works out the height, and a new height works out the width.
- **CAD toolbar**: shows or hides the Create / 3D / Sketch / Solid toolbar over the preview.

When the frame isn't the V1 video's size, V1 is fitted inside it with its own aspect, centred on black. The canvas shows it and the compile makes it the same way.

The CAD buttons are hidden. In the timeline, use the menu bar's **CAD toolbar** box instead. In the image editor, use **Window ▸ CAD Toolbar**. It still shows by itself while a sketch or a 3D body is picked.

Lottie (JSON) export (2.512.0)

Export → Lottie (JSON) turns the timeline's picture and text layers into a Lottie animation: a JSON file for web pages, apps and any Lottie player. It is saved as `<project>_lottie.json` beside the movies, and Export lists it to download.

- **Real animated layers.** A layer that only moves, scales, turns or fades stays an animated layer, with its picture inside. Its ◆ keys keep their eases: linear, in, out, in-out, a custom curve, or hold. Its blend mode and the layer's opacity are kept. A text goes in as its picture.
- **Frames.** A layer the app draws itself goes in as its frames: one picture a frame, with repeats shared. That covers masks, effects and looks, generators and particles, warps, 3D, parents, expressions, a keyed text colour, entry or exit transitions, and a curved motion path.

- **Left out.** Video layers, the base video and track mattes are left out, because a Lottie has no video. The message names what was left out.

The export was checked by playing the file in lottie-web, the standard Lottie player. It drew the same picture as the timeline at every moment tested.

Variable mask feather, data-driven animation and template fields (2.511.0)

✒ **Variable mask feather.** With the ✒ Mask Pen on a closed mask, **Shift+drag a point**. That point gets its own feather, as far as you dragged, and a dashed ring shows it. The edge softens by the feather of the nearest place on it: along each piece of the edge the feather glides from one point's to the next. The points without one keep the mask's feather. Animated mask paths glide the point feathers between keys too. **variable feather** × in the mask's row puts every point back to the mask's feather. The compile matches the canvas, video layers' mask videos included.

Data. Under **Data & fields** in any layer's Properties, **Import CSV / JSON**:

- A **CSV** (commas, semicolons or tabs; quoted values) uses its first line as the column names.
- A **JSON** file can be a list of records, an object holding one, or columns of values.

The file becomes a named **data set**, saved with the timeline. Each set has a current **row**; change the row and everything that reads it changes.

- **In a text**: `{{sales.region}}` shows the current row's region, and `{{sales.region.3}}` shows row 3's. A name the data doesn't have stays as written. The compile's text pictures carry the values.
- **In ƒx**: `data("sales", "amount")` gives the current row's amount as a number, and `data("sales", "amount", 3)` gives row 3's. For example, `data("sales", "amount", floor(t) + 1)` steps through the rows, one a second.
- **Data → keys**: pick the set, a column and a property (opacity, scale, x, y, turn), then **s per row**, the rows **from … to**, and **x** (a factor) **+** (an offset). **Make keys** keys one row every so many seconds, for bar charts, counters and the like.

Template fields. Expose a layer's properties with a label: its text colour, opacity, scale, x, y or turn. Every exposed field on the timeline is listed under **exposed on this timeline**, so you can change them in one place. A template saved from these layers offers the fields, as colours and numbers, when it is dropped in. A keyed property's keys move by the change.

2D layers in 3D: cards, a moving camera, depth of field, shadows (2.510.0)

3D layer. In a layer's Properties (a picture, text, shape, generator or video layer), tick **3D layer**. The layer becomes a card in 3D space:

- **x / y**: the layer's usual move.
- **depth z**: in pixels. A higher number is further away.
- **x turn °** and **y turn °**: tip it back or swing it round. Its usual turn is the turn about z.
- **casts shadows** and **accepts shadows**.

Depth and the two turns can be keyed (◇).

3D camera. All 3D layers are seen through one camera, set in the Properties of any 3D layer:

- **position x / y / z**.
- **aim at x / y / z**.
- **zoom**: the camera's lens. With the camera at z = −zoom, a card at z 0 is exactly its 2D size, so ticking 3D changes nothing until you move something.

Every number can be keyed at the playhead (◇). Once a number has keys, changing it makes a key, and between keys the camera glides. **reset** puts the camera back to straight ahead.

Orbit (Shift+O) moves the camera on the canvas:

- **drag**: orbit round where it aims.
- **Shift+drag**: dolly closer or further.
- **Alt+drag**: track (the camera and its aim together).

Depth of field. Tick **depth of field**, then set the **focus** distance and the **aperture**. A card at the focus distance is sharp. A card one focus distance away is blurred by the aperture in pixels.

Shadows. Tick **light (shadows)** and place the light (**x / y / z**). Cards that cast throw the shape of their picture onto the cards behind them that accept. A video layer casts the shape of its box. Set the **darkness %** and the **softness px**.

Stacking and the compile. Cards stack in their layer order, as the timeline's 2D layers do, so put the far ones lower. 3D picture layers are rendered by the page frame by frame. 3D video layers are baked whole with their sound kept. When the camera, the light or another card changes, they are rendered again, so the result matches the canvas.

Motion sketch, the wiggler and the smoother (2.509.0)

All three are under **Motion** in the Properties of a picture, text, shape, generator or video layer. Each one makes plain keys that you can edit in the graph editor afterwards.

✍ **Motion sketch.** Pick a layer and put the playhead where the move should start.

1. Press ✍ **Sketch**, or choose the ✍ tool (**Shift+K**).
2. Press on the canvas and move the pointer. The timeline plays from the playhead and the layer follows the pointer. It starts from where it was, so it doesn't jump to the pointer.
3. Let go to stop. The sketch also stops at the end of the layer.

The path becomes position keys. Keys it passed over are replaced.

- **speed %**: how fast the timeline plays while you sketch. 100 is real time. Lower gives you more time for a careful move; it plays back faster.
- **smooth px**: only keys that change the path by more than this are kept. Higher means fewer keys and a smoother path.

≈ **Wiggler.** Adds random movement to a property.

- **property**: position (x, y), x, y, scale, turn, opacity, or a keyed generator or light-ray number.
- **per s**: how often it changes.
- **amount**: how far, either way.
- **noise**: smooth (a flowing curve) or jagged (a new value each time).
- **x, y apart**: for position, x and y wiggle on their own. Off, they wiggle together.
- **seed**: a different wiggle for each number.

The wiggle works between the property's first and last keys, riding on the move already there. If the property has fewer than two keys, it covers the whole layer.

Smoother. Calms a property's keys between its first and last key. It needs at least three.

- **frames**: the move is averaged over this many frames either side.
- **tolerance**: only keys that change the value by more than this are kept.

The result is fewer keys with the same overall shape. The ends stay where they were.

Audio → keys, waveform and spectrum layers (2.508.0)

The app reads the timeline's sound: its loudness, its waveform and 64 frequency bands over time. The sound can be one clip or **All sound (the mix)**: every sound clip, and the sound of every video clip that plays, at its volume. A file is read once and remembered.

♪ **Audio → keys** is in the Properties of every picture, text, shape, generator or video layer.

1. Pick the **sound**.

2 Pick what to follow: **loudness**, **bass** (30–250 Hz), **mids** (250 Hz–2 kHz) or **treble** (2–12 kHz).

3 Pick the **property**: opacity (0..1), scale (1 = 100 %), x, y, turn, or the layer's own generator or light-ray numbers.

4 Set the value when it is **quiet** and when it is **loud**, and the **smooth** (frames either side to average over).

5 Press ♪ **Make keys**.

The property is keyed over the whole length of the layer. The quietest moment of that stretch gets the quiet value and the loudest gets the loud value. Only the keys that matter are kept, at most 400. Keys already on that property are replaced, and the keys can be edited afterwards like any others.

Waveform. With the Generator tool, **Alt+click** on the canvas. In the layer's Generator:

- **sound**.
- **style**: wave (filled), line or bars.
- **x / y** (from the frame's centre), **width**, **height**.
- **span s**: how many seconds of sound are shown around the playhead. Small is a close-up of the wave.
- **gain %**, **thickness**, and **colours** left → right.

Spectrum.Alt+Shift+click with the tool. Its frequencies run from low (left) to high (right). In the layer's Generator:

- **sound**.
- **style**: bars, line or circle (bars around a ring).
- **bands** (4–64), **gain %**, **gap %** between bars.
- **floor %**: anything quieter shows nothing.
- **thickness**, and **colours** low → high.

Every number can be keyed (◇) or given an expression (ƒx).

ƒx audio(). In any expression, `audio()` is the loudness of the mix at that moment (0..1). `audio("bass")`, `audio("mid")` and `audio("treble")` follow the bands. For example, `value * (1 + audio() * 0.5)` pulses with the music. Both are in the ƒx presets.

The compile. Waveform and spectrum layers are rendered by the page frame by frame, from the same reading of the sound, so the result matches the canvas. The page waits for the sound to be read before it renders.

Puppet pins and mesh warp (2.507.0)

Puppet pins. Pick a layer (a picture, text, shape, generator or video layer). Then choose the **Puppet** tool (**Shift+W**).

- **Click** on the layer to put down a pin. Put down a few: at the ends of an arm, the head, the feet.
- **Drag** a pin and the layer bends to follow it. The other pins hold the picture where they are. Near a pin the picture turns and shifts with it, without stretching.
- **Alt+click** a pin to remove it, together with its keys.
- The bent picture may reach past the layer's own edges.

Animating the pins. Press ◇ **key here** in **Warp** (in the layer's Properties). It keys every pin at the playhead. From then on, every drag makes a key at the playhead, and between keys the pins glide (the keys have the usual eases in the graph editor).

Mesh warp. Tick **Mesh warp** in Warp. A grid of points (**columns** × **rows**, up to 8 × 8) appears over the layer. Drag its points with the tool. The picture follows smooth curves through them, not straight lines. **reset** puts every point back. Mesh points are keyed by ◇ key here like the pins. Pins and the mesh can be used together.

Warp settings:

- **on**: turn the warp off and on.
- **falloff**: how far a pin's pull reaches. Low is wide and soft, high stays close to the pin.
- **detail**: how fine the triangle mesh it is drawn with. Higher is smoother on strong bends.
- ×: remove the pins, the mesh and their keys.

Pins and points are kept as fractions of the layer's own picture. A warp rides with the layer when it is moved, scaled or turned.

The compile. Warped picture layers are rendered by the page frame by frame. Warped video layers are baked whole by the page with their sound kept, the same as light rays (2.506.0). The result matches the canvas.

Particles and light rays (2.506.0)

Particles. With the **Generator tool**, **Shift+click** on the canvas. A particle emitter appears where you clicked: a fountain of soft sparks that rise and fall back. Select the layer. In **Generator** (kind **Particles**) you can set:

- **emitter**: a **point**, a **line** or a **box**. Set its **x / y** (pixels from the frame's centre) and its **width / height**.
- **per s** (how many are born a second), **life s**, and **pre-roll** (the air is already full at the start).
- **speed px/s** and its random **± %**; **direction** ° (0 = right, −90 = up) and its **spread** °; **gravity** and **wind**.
- **particle**: dot (soft), circle, square, star or spark (a streak along its motion).
- **size start → end** and its random **± %**; **colours** start → end; **opacity start → end %**; **seed**.

Every number can be keyed (◇) or given an expression (fx). Particles are worked out, not simulated: any frame looks the same whenever it is drawn, so scrubbing, playing and the compile all agree. Give the layer a blend mode such as Add or Screen to make it glow over the picture underneath.

☀ **Light rays** are in the Properties of any picture, text, shape, generator or video layer. The bright parts of the layer streak away from a source point. The rays can reach past the layer's own edges.

- **source x / y**: pixels from the layer's centre.
- **length %**: how far the rays reach.
- **intensity %**: how strong they are.
- **threshold** (0–255): only light brighter than this makes rays.
- **tint**: the colour of the rays.
- **rays only**: hide the layer and keep its rays.

Every number can be keyed (◇) or given an expression (fx).

Video layers. A video layer with light rays is baked whole by the page when you press ⚙ Compile. Each frame is taken from the clip at its own time and drawn exactly as the canvas draws it. The compile overlays those frames, and the clip's sound is kept. A long clip takes a little while to bake. It is baked again only when something about it changes.

Generator layers and distortions: fractal noise, gradient ramp, turbulent displace, displacement map (2.505.0)

The Generator tool (in the canvas toolbox) makes a layer that draws itself:

- **Click** anywhere on the canvas for a **Fractal Noise** layer covering the frame.
- **Drag** for a **Gradient Ramp** running from where you pressed to where you let go.

Select the layer to see **Generator** in its Properties. Every number can be keyed (◇) or given an expression (fx).

- **Fractal noise** settings:

  - **type**: basic, turbulent smooth or turbulent sharp (creased).
  - **contrast**, **brightness**, **invert**.
  - **scale** in pixels, and **complexity** (how many layers of detail).
  - **sub influence** and **sub scaling**: how strong and how fine each finer layer is.
  - **evolution** (°) and **evolution speed** (°/s): the noise moves by itself.
  - **offset** and **drift** (px/s): slide the noise.
  - **seed**, and two **colours** (dark → light).
  - **detail**: ¼ (fast) or ½. Noise is smooth, so it is worked small and drawn scaled up.
- **Gradient ramp** settings: **linear** or **radial**, the **start** and **end** points (pixels from the frame's centre) and the two colours.

Lower the layer's opacity or give it a blend mode to lay it over other layers.

Distort is in the Properties of any picture, text, shape or video layer:

- **Turbulent displace** pushes the layer's pixels around with fractal noise. Set the **amount** (pixels), the **size** of the swirls, the **complexity**, the **evolution speed** (it moves by itself) and a **seed**.
- **Displacement map** pushes them by another layer's picture: a Fractal Noise or Gradient Ramp layer, or any picture layer. Mid grey leaves a pixel where it is. **use brightness** moves both ways by the same amount; **red ↔ / green ↕** moves across by red and up and down by green. Set the **max across** and **max up-down** in pixels.
- Pixels move whole pixels, up to 127 either way, and the edges are smeared. The compile does exactly the same.

The compile. Generator layers and distorted picture layers are rendered by the page frame by frame. A distorted video layer gets two map videos (across and up-down, stored losslessly). The compile moves every frame's pixels with them, so the result matches the canvas.

DLSS 5: enhance, upscale and frame generation for videos (2.504.0)

⚡ **DLSS 5** runs a video through NVIDIA DLSS 5, using RunningHub's ComfyUI nodes (the same step Tim's "secret sauce" workflow ends with). It does three things:

- **Neural rendering.** It cleans up and sharpens the picture. Style picks the look: default, natural (closer to the source), cinematic (more contrast), or off (upscale only). Detail sets how much of the change is kept: 0 keeps the original, 1 is the full result, 2 is more. Structure sets the local detail reconstruction.
- **Upscaling.** The sizes are 1x (DLAA: same size, enhanced), NVIDIA's fixed factors 1.5x, 1.724x, 2x and 3x, or 1K / 2K / 4K / 8K. With 1K–8K, DLSS picks the smallest factor whose short side reaches 1080 / 1440 / 2160 / 4320; a video already that big stays at 1x. A size more than 3x away is refused in words; pick a smaller one.
- **Frame generation** (optional). Real in-between frames, not blends: 2x / 3x / 4x the frame rate, or a target such as 60 fps, up to 6 times the source.

Where.

- The preview bar's ⚡ **DLSS 5** button works on the video you are previewing. The panel says what it will make; ⚡ **Enhance** queues it, and the result lands in the gallery with the original's sound.
- In the StoryBoard's ⚙, set **Upscaler → DLSS 5**. A scene's then sends its take through DLSS 5, at the ⚙'s Upscale resolution (720 = DLAA, 1080 = 1K, 1440 = 2K, 2160 = 4K). With "Upscale every render" ticked, each take is sent as it lands.

Setup. NVIDIA RTX cards only. In Settings → ComfyUI → Video accelerators, press **Install RH DLSS 5**. Then copy NVIDIA's DLLs into ComfyUI/models/dlss5/ yourself; the pack's README names them, and it never downloads them. Restart ComfyUI. Frame generation needs its own worker files in models/dlss5/dlssg/ (also in the README).

RTX VFX: NVIDIA RTX Video finishing and upscaling (2.708.0)

RTX VFX runs a video through NVIDIA RTX Video, using the DENO ComfyUI nodes (DenoRTXVFXVideoFinisher). It makes up to two passes over the picture:

- **First pass** (optional): **Deblur** works on soft frames before the upscale. Off skips it.
- **Upscale pass**: **VSR** (Super Resolution) upscales and cleans up the picture; **High Bitrate** is NVIDIA's upscale mode for clean, high-bitrate sources such as a freshly rendered take.
- **Presets.** A = one VSR pass. B = Deblur first, then High Bitrate. Custom lets you pick each pass and its quality; the dropdowns list what your ComfyUI's node offers.
- **Scale** multiplies each side (1x cleans up at the same size, up to 4x). The panel says about what size it will make; the node rounds each side to a multiple of 32.
- **Frames per batch** is how many frames go through ComfyUI in one run (48 by default). A longer video runs in several pieces, which are joined without re-encoding; the queue shows "chunk 2/5" as they go. Lower it if ComfyUI runs out of memory; 0 runs every frame at once.

Where.

- The preview bar's **RTX VFX** button works on the video you are previewing. **Finish** queues it, and the result lands in the gallery with the original's sound.
- In the StoryBoard's ⚙, set **Upscaler → RTX VFX (NVIDIA RTX)**. A scene's then sends its take through it, scaled to the ⚙'s Upscale resolution (the take's short side up to 720 / 1080 / 1440 / 2160, at most 4x), with the passes from the panel. With "Upscale every render" ticked, each take is sent as it lands.

Setup. NVIDIA RTX cards on Windows only. In Settings → ComfyUI → Video accelerators, press **Install DENO nodes (RTX VFX)**. The node pack does not include NVIDIA's runtime; install it with the pack's own installer (https://deno2026.github.io/comfyui-deno-custom-nodes/rtx-vfx-install/): extract NVIDIA's official installer zip into ComfyUI/custom_nodes/deno-custom-nodes/tools/, close ComfyUI, run **install_rtx_vfx.bat**, then restart ComfyUI. The graph also uses ComfyUI-VideoHelperSuite. If the node fails, the job shows the pack's own error message.

16 GB mode: MiniMax H3 on a 16 GB card (2.708.0)

MiniMax H3 is built for a big card. **16 GB mode** renders it the way DENO's *R2V Turbo 8+2* workflow does, so a 16 GB card can make a scene:

1. The 32B **text encoder is unloaded** right after it has read the prompt, so it never shares the card with the sampler.
2. **Pass 1** renders the scene at Basic Settings' size in **8 steps** (the Ref2VA Acc 8-step LoRA). The sound is made here.
3. The video is **upscaled in latent space** (× the upscale, 1.0-2.0).
4. **Pass 2** redraws only the **last 2 steps** at the bigger size with the DMD + 2K refiner LoRAs and Sol sparse attention. The sound from pass 1 is kept as it was.

The take comes out at Basic Settings' size × the upscale. Start around 896 × 512 with an upscale of 1.2-1.8; a bigger upscale or a longer clip takes much longer.

Settings ▸ ComfyUI ▸ 16 GB mode: **Auto** (the default) turns it on when the card ComfyUI renders on has 16 GB or less; **On** uses it for every H3 render; **Off** never. **Upscale ×** sets pass 2; 1 = no second pass. **Pass 1 in 8 steps** can be turned off to keep your style's own steps for pass 1.

What it needs: a ComfyUI with MiniMax H3 and Sol attention (0.43 or later), the **DENO custom nodes** (Settings ▸ ComfyUI ▸ Video accelerators ▸ ⬇ Install DENO nodes), the **H3 latent upscaler nodes** (⬇ Install the latent upscaler nodes, in the 16 GB mode section) and the files (**The 16 GB mode files…** opens Settings ▸ Add-ons ▸ 16 GB mode: the two LoRAs of each pass and the latent upscaler). KJNodes, when installed, also unloads every model before the big decode. The status line under the switch shows what is there. A part this ComfyUI cannot run is left out and the render's log says which and what to install, so a render never fails because 16 GB mode is on.

Not with: a FastH3 Omni style (already 8 steps with sparse attention) and the ⚙ upscaler's stage (it upscales the render itself, so 16 GB mode keeps only the text encoder unload and pass 1). RunPod renders do not use it.

The bitGen Community: help and friends (2.710.0)

Community on the status bar shows / hides a small chat panel, like Facebook's. It is a docker of its own (2.711.0): drag its title bar to float it anywhere (every edge sizes it), drop it beside or below another docker, or onto a docker's middle to put it INSIDE that docker as a tab; **_** minimizes it, maximizes it, ✕ closes it. Where it is and whether it is open are remembered.

- **Join**: type a **user ID** (3-24 letters, numbers, _ . -) and a password (8 characters or more), and under **New here? Also** your **first name**, **last name** and **email address** (and a display name if you like), then **Join** (2.713.0). Only the community's admins see your name and email address - everyone else sees your user ID and display name. Already have an ID (on another computer)? Type it and the password, then **Sign in**.
- **Confirm your email address**: the community emails a **6-digit code**; type it and press **Confirm** (or Enter). The Help room and friends open after that. **Send a new code** sends another (4 an hour); **Wrong address?** changes the name or address. The code works for 15 minutes and 5 tries. No email? Look in the spam folder. Members who joined before 2.713.0 are asked for their name and email address the same way the next time they sign in.
- **Help**: the room every member shares. Ask a question there and anyone can answer. Click a name to find that person and add them as a friend.
- **Friends**: find people by user ID or name and press **+ Add friend**. They get a friend request; when they press ✓ **Accept** you are friends (× **Decline** says no). Requests to you are listed at the top. A friend has for a private chat (its own tab), **Remove**, and **Block** (they cannot message or ask you again, and their Help messages are hidden from you).
- ⚑ on a message reports it to the community's admins, with a reason. The badge on Community counts new messages and friend requests while the panel is closed.
- ⋯: **Discussions** (opens the website's discussions - see below), **Your details** (your name and email address; **Change** - a new address is confirmed with a new code), **Sign out** on this computer (your ID stays), **Leave the community** (deletes your ID and friendships - type your password), and the community server's address. Admins also see **Reports** there: delete the message, or ban its writer. A ban also bans the writer's **email address**: they cannot come back under a new user ID with it.

Joining is optional - bitGen Studio works the same without it.

The Discussions (2.713.0) are on the website, https://tigerart.com.au/discussions.php: longer questions, your work, ideas - in five categories (Help, Bug reports, Show your work, Ideas, General). **Bug reports** (2.714.0) are filled by Bug report ▸ Save and send and by the website's **Report a bug** form; each shows its status (Open, Being fixed, Fixed, Duplicate, Not a bug, Won't fix), set by the admins. Sign in there with the **same user ID and password** (or join there - it is the same account). Members' posts show at once. **Guests** (no account) can read and post too, with a name; a guest's post waits until an admin approves it. Admins see the waiting posts, approve or refuse them, remove posts and ban members there too.

What people write is shown exactly as written, never translated or turned into links or pictures. New messages arrive every few seconds while the panel is open. Your password is sent once to the community server (over https) and never kept; bitGen keeps only a sign-in token in community.json in the install folder, and the page never sees it.

The community server is a small PHP program (the `community/` folder of the repo, `community/README.md` explains the install on any PHP + MariaDB host; the emails go through the host's own mail server). The default address is https://tigerart.com.au/bitgen-community/api.php.

Long video: one MiniMax H3 run up to 30 seconds (2.503.0)

A MiniMax H3 clip used to be one generation of up to 15 seconds; anything longer was rendered in 15-second pieces and joined. **Long video** renders up to **30 seconds in one generation**. One run keeps the characters, the motion and the sound continuous, with no joins. The settings are the ones Tim's "secret sauce" workflow uses: three model patches in front of the sampler.

- **MiniMax low-VRAM attention**: the attention heads run in groups (4), and buffers are freed as soon as they are used.
- **Chunked feed-forward**: the feed-forward step runs in pieces along the frames (2 pieces once the sequence passes 4096 tokens). The result is the same; the memory peak is lower.
- **Comfy kitchen attention** (optional): ComfyUI's comfy-kitchen attention backend in front.

Turning it on.

- **The scene board:** tick **Long video** in the StoryBoard's ⚙. A scene then renders up to 30 seconds in one go. With **Comfy Kitchen** also ticked, the kitchen attention goes in too.
- **The prompt docker's Generate:** pick the **MiniMax H3 Long (30 s)** video style, or turn Long video on in any MiniMax H3 style (Settings → Video styles → Long video: On, or On + comfy kitchen attention). The plan line under Seconds then shows a 25-second clip as one 600-frame run.
- Clips longer than 30 seconds are still split and joined, in 30-second pieces.
- Sage attention is skipped while Long video is on, because Long video brings its own attention patch. Spectrum still works with it.
- It is for MiniMax H3 only; LTX and Wan styles ignore it.

Install. Long video needs a recent ComfyUI-KJNodes. Settings → ComfyUI → Video accelerators has an **Install / update KJNodes** button. The kitchen attention needs ComfyUI 0.32 or later with comfy-kitchen installed (the Comfy Kitchen section above it). If this machine has already learned a lower VRAM ceiling from a run that ran out of memory, that ceiling still applies.

The same panel can also install **Yusu H3 Unified** (a one-node front end for MiniMax H3, for imported workflows that use it) and **RH DLSS 5** (NVIDIA RTX enhance / upscale / frame generation). For DLSS 5, NVIDIA's DLLs go into ComfyUI/models/dlss5/ yourself; the pack never downloads them.

The export also ran its turbo LoRA at 8 steps. Add a turbo LoRA and set Steps in Basic Settings for the same speed.

FLUX 3 Image Klein, and edit image mode lists only edit styles (2.502.0)

FLUX 3 Image Klein is Black Forest Labs' 5.6B open image model (October 2026). Three new image styles use it:

- **FLUX 3 Klein**: the distilled model, 16 steps at cfg 1 (4 to 16 steps work). Fast.
- **FLUX 3 Klein base**: the base model, about 50 steps at cfg 3.5, with a real negative prompt. Slower, and the one LoRAs are trained on.
- **FLUX 3 Klein edit**: edits a picture from up to 4 reference pictures. Picture 1 sets the output size. In the prompt the pictures can be named `<ref_image_0>`, `<ref_image_1>` and so on, in order.

The text-to-image styles also take the reference dock: with pictures in it they switch to their edit graph by themselves (the base style to the base edit graph). They never switch to a FLUX.2 graph.

Models. Settings → Setup lists the three files: the distilled and base int8 weights and the Qwen3-VL 4B text encoder. The VAE is FLUX.2's `flux2-vae.safetensors`. The Hugging Face repo is gated: accept the FLUX Kommunity licence there first and add your token. bf16 and nvfp4 builds are in the same repo; set the style's Model to use one. FLUX 3 needs ComfyUI 0.41.0 or later. It works best with long, specific prompts.

Edit image mode. While the image editor is open and the prompt docker's ▾ is on **Edit**, **Replace background** or **Compose**, the image style dropdown lists only styles that can edit a picture: the FLUX 3 and FLUX.2 Klein styles, Qwen Image Edit, FLUX.1 Kontext and the like. 2.549.0: on **Generate**,

Refine, Fill or Remove EVERY style is listed (Krea 2, SDXL …), and picking one that cannot edit while the ▾ is on Edit puts the ▾ on Generate. Close the editor and the style you had picked before is back. The edit style you used last is picked again the next time you edit.

Time remapping on a curve (2.501.0)

A video clip's **Speed** section now has **Time remap**. It sets which second of the source shows at each moment of the clip.

- **Turn it on** and the clip gets a curve that plays it exactly as it plays now (its speed, ramp or reverse become keys). Speed, reverse and ramp are switched off while the remap is on, and come back when you turn it off.
- **The curve.** Across is the clip's own time on the timeline; up is the source second that shows.
  - Rising plays forward, falling plays backwards, flat is a freeze. Steeper is faster.
  - Drag a key to move it. Double-click the curve to add a key; double-click a key to remove it.
  - The last key's time is the clip's length on the timeline.
- ◇ **Key** adds a key at the playhead, on the curve where it is.
- ❄ **Freeze** holds the frame at the playhead for the seconds beside it; the rest of the clip moves later.
- **The key row** shows the chosen key's time and source second. Type new values, or × to remove it.
- **Trimming.** Shortening a remapped clip keeps the part of the curve inside. Lengthening it holds the edge frame.
- **Sound.** The clip's sound follows the curve: backwards where the picture runs backwards, silent during a freeze. The preview plays it the same way.

Motion blur, Posterize Time, Echo (2.500.0)

A layer's Properties now has a **Time** section.

- **Motion blur** blurs the layer's movement (its position, scale and turn, from keys, a parent or an expression).
  - **shutter** is how much of each frame the blur covers: 180° is half a frame, 360° the whole frame.
  - **samples** is how many times a frame the movement is drawn. More samples give a smoother blur but compile more slowly.
- **Posterize time** holds the layer at a lower frame rate (for example 8 fps) for a jerky, stop-motion look. Both its picture and its movement are held.
- **Echo** draws copies of the layer where it was a moment ago, behind it, as a motion trail.
  - **copies** and **apart** (seconds between copies).
  - **first**: the nearest copy's strength. **fade**: how much each further copy keeps.
  - On a video layer the compile also shows the earlier frames in the copies; the canvas draws the current frame there.

The compile. Picture and text layers are baked by the page. On video layers the compile does the same sums:

- Motion blur: the movement sampled several times a frame and averaged.
- Posterize: the held frames.
- Echo: delayed copies laid behind the layer.

Tracking and corner pin (2.499.0)

Track (a video clip's Properties: V1 or a video layer). Pick what to follow:

- ◎ **a point**: position.
- ◎◎ **two points**: position, scale and rotation.

- ▢ **corner pin**: 4 points followed separately.
- ▣ **planar**: a flat surface such as a screen, a sign or a page. Its 4 corners are carried by everything inside it, so it holds even when a corner is covered.

Tracking

1. Move the playhead to where the tracking should start.
2. Press ◎ **Place**, then drag the point(s) onto clear details. The square around each point is the patch the tracker looks for; **size** changes it.
3. Press ▶ **Track**. It follows them to the clip's end, to a fraction of a pixel, at the clip's own frame rate. The paths are drawn on the canvas, and any frames where a point wasn't found well are listed.

Apply to a layer, then **Apply**:

- With a point (or two points), the layer's position keys (and scale and rotation) follow the track. Apply it to a **null** to make other layers follow by parenting them to the null.
- With a corner pin or a planar track, the layer is **pinned** into the tracked corners on every frame. This is how to stick a picture or video onto a moving screen.

▢ **Corner pin (any layer's Properties).** Draws the layer into 4 corners.

- Tick it to start with the layer's own corners, then use ✥ **Edit corners** to drag them on the canvas.
- A pin with keys (from a track) gets a key at the playhead when you drag a corner.

The compile. Pinned picture and text layers are baked by the page. A pinned video layer is warped by the compile itself, with its corners as keys. Both match the canvas.

Bézier masks (2.498.0)

Any picture, text, shape or video layer on the timeline can have **masks**: shapes that say which part of the layer shows.

Drawing a mask with ✒ Mask Pen (Shift+M, in the canvas's tool column)

- Pick the layer first (Move tool).
- **Click** to add a corner point. **Drag** to add a smooth point; the drag pulls out its handles.
- Click the **first point** again, or press **Enter**, to close the mask. **Esc** stops drawing.
- To change a closed mask, drag its points or their handles with the Mask Pen. **Alt** moves one handle on its own.
- The points are stored as positions on the layer's own picture, so the mask travels with the layer when it moves, scales or turns.

✒ Masks (the layer's Properties)

Each mask has:

- **on / off** and **invert**.
- **mode**: how it meets the masks above it in the list.
  - **add** shows more of the layer.
  - **subtract** cuts away.
  - **intersect** keeps only where they overlap.
  - **difference** keeps where exactly one of them is.
- **feather** (px): a soft edge.
- **expansion** (px): + grows the mask, − shrinks it.
- **opacity** (%).
- ◇: keys the mask's path at the playhead. With two or more keys the points glide from one shape to the next; dragging a point at another time adds a key there.
- ✕ deletes the mask.

The compile

- Masked picture, text and shape layers are baked by the page.
- A video layer's masks become a grey mask video that the compile uses as the clip's transparency. It is a single frame when nothing in the mask moves.
- In both cases the movie matches the canvas.

Expressions (2.497.0)

A canvas layer's Properties now has a **ƒx Expressions** section. An expression is a small formula that works out a property's value on every frame.

Adding one

1. Pick a property: position x / y, scale (1 = the layer's size), rotation (°), opacity (0–1), or the anchor point. A shape layer also lists its shape numbers, and a text layer lists its animator and on-a-path numbers.
2. Pick a preset or keep the first one, then press **ƒx**.
3. Edit the expression in its box. A mistake is shown straight away; until it is fixed, the property keeps the last good expression (or what its keys say).
4. × takes the expression off.

The language (not JavaScript; nothing else can run)

- **Sums:** numbers, + − × ÷ (`* /`), `%`, `**`, brackets, comparisons, `&& || !`, and `a ? b : c`.
- **Several lines:** separate them with new lines or `;`, and use `var name = …` for a named value. The last line is the answer.
- **Values you can use:**
  - `value`: what the keys or the setting say now.
  - `time`: seconds on the timeline. `t`: seconds into the clip. `frame` and `fps`.
  - `inPoint` / `outPoint`, `width` / `height` of the picture, `index`, and `PI`.
- **Functions:**
  - `wiggle(freq, amount)`: smooth random motion, the same every time you play.
  - `loopOut()` / `loopIn()` with `"cycle"`, `"pingpong"`, `"offset"` or `"continue"`: repeat the keys after the last key, or before the first.
  - `valueAtTime(time - 0.5)`: the value at another moment (a delay).
  - `linear` / `ease` / `easeIn` / `easeOut(t, tMin, tMax, from, to)`: map one range onto another.
  - `clamp`, `random(min, max)` (new each frame), `noise(x)`, `posterize(value, step)`.
  - `layer("Null 1", "x")`: another layer's x, y, scale, rot or op. This links layers together.
  - The usual maths: `sin cos tan abs min max round floor ceil sqrt pow …`. `Math.sin` works too.
- **Examples:**
  - `time * 90` turns 90° a second.
  - `wiggle(2, 20)` shakes 20 px.
  - `loopOut("pingpong")` plays the keys there and back for ever.
  - `layer("Null 1", "x") + 100` follows the null, 100 px to its right.

The compile: a layer that an expression moves or fades reaches ⚙ Compile as keys. They are worked out every frame and thinned to where the motion bends. Shape layers and animated text are baked by the page as before, so the movie follows the canvas.

Freehand shapes, Merge Paths, text on a path, roving keyframes (2.496.0)

✎ **Freehand (Shift+Q, in the canvas's tool column).**

- Drag to draw a path. It becomes a **freehand shape layer**, thinned and smoothed.

- Ending where you began (or holding Shift) makes a closed, filled shape. Otherwise it is an open line with a 4 px stroke.
- **W / H** stretch it, and every shape operator works on it. On an open line, Trim Paths runs from one end to the other.
- **Alt** while drawing adds the drawing to the picked shape layer as another path.

Paths and ⊕ Merge Paths (a shape layer's Properties).

- **Path** adds another rectangle, ellipse, star or polygon to the same shape layer, with its own size, place and turn.
- Without Merge Paths, each path is filled on its own.
- With ⊕ **Merge Paths** on, the paths become one outline. Each extra path meets the ones before it by its mode: **add**, **subtract**, **intersect** or **exclude**. A path subtracted inside another makes a hole.
- The operators after it (Offset, Zig Zag, Trim and so on) work on the merged outline. Offset grows the outline and shrinks the holes.

Text on a path (a text layer's Text Animators section).

- Tick **On a path** and choose **an arc**, **a circle**, **a wave** or **a shape layer** (its outline, at the text's place).
- Set the radius, span, height or wavelength. **Start %** says where the text begins along the path; key it with ◇ to slide the text along.
- **Other way** runs the text the opposite direction (inside a circle).
- Animators still work: their x moves letters along the path, and y moves them across it.
- The compile bakes the text so it matches the canvas.

Roving keyframes (Graph Editor ▸ Rove).

- Pick a position key in the Graph Editor and press **Rove**. Its time then follows the motion path, so the layer moves at an even speed between the keys around it that don't rove.
- Moving a key re-times the roving ones. The first and last position keys never rove.
- Anything else a roving key sets (scale, opacity) stays at its old time.

Colour blend modes, and video layers that follow a parent (2.495.0)

Hue, Saturation, Color and Luminosity. A timeline layer's ◐ **Blend** list now has the four colour modes as well as the other 13. Each one splits a picture into its brightness and its colour, then takes one from the layer and the other from what is under it:

- **Luminosity:** the layer's brightness on the colours underneath (for example, a black-and-white texture that shades a scene without changing its colours).
- **Color:** the layer's colour on the brightness underneath (tinting or recolouring).
- **Hue:** the layer's colour family, at the strength and brightness of what is underneath.
- **Saturation:** how strong the colour underneath is becomes the layer's (a grey layer takes the colour out).

The canvas and the ⚙ Compile work these out with the same sums, so the movie matches the preview.

Video layers can follow a parent. Parent in a video layer's Properties now works like it does for picture, text and null layers:

- The video moves, turns and scales with its parent. Choosing the parent keeps the video where it is at the playhead.
- When you compile, the video's on-screen position, scale and rotation go into the movie as keyframes. They are checked every frame and only kept where the motion bends, so the movie follows the canvas.

The CAD workings refined: the CAD button, Offset, the view cube, picking profiles (2.494.0)

The CAD button. The image editor's tool column has a CAD button at its foot, and the timeline's tool column has one too.

- **Image editor:** the CAD toolbar (Sketch · Shapes · Solid · Unreal) shows by itself while a sketch layer, a sketch tool or a 3D body is picked. The rest of the time it stays out of the way. Press CAD (or Window ▸ CAD Toolbar) to keep it on.
- **Timeline:** CAD shows or hides the toolbar over the preview.

Sketching

- **Select** leads the Sketch tab (S).
- **Create Sketch** hides while a sketch is being edited. It comes back once you finish.
- **Finish Sketch** picks the Move tool and moves the toolbar on to the **Solid** tab, ready to extrude.
- **Draw on from a line:** pick one line (or arc) with Select. A appears beyond each of its ends. Click one and the Line tool carries on from that end; click the next points as usual.

Offset (O) is now a tool

1. Click a line, arc or circle. Everything joined to it end to end comes too; Shift-click adds more.
2. Move the pointer to the side you want. A dashed preview follows it and the Distance box shows how far.
3. Click to make it, or type the distance and press Enter.

- **F** (or ⇄ Flip) uses the other side.
- **Esc** drops the pick.
- Lines drawn in any direction now offset to the same side, and a rounded corner stays joined (its arc grows or shrinks with the rest).
- The Offset… dialog in the Sketch tab still works on what is picked. It now says **Out** or **In**.
- An offset (or any sketch edit) that the sketch cannot take changes nothing.

Extrude and Revolve: pick the profile

- When the sketch has more than one closed region, Extrude / Revolve first asks you to click the regions to use:
  - Each region lights up under the pointer. Click to add it, click again to drop it, or use **All**.
  - Then **Enter** or ✓ **OK** opens the settings with those regions ticked. **Esc** stops.
- A sketch with one region goes straight to the settings.
- If you already picked a region's lines with Select, it is used as it is.

The 3D viewport (a body picked)

- **Shift + middle-drag** orbits the body, with any tool. **Middle-drag** pans and the **wheel** zooms, as before.
- **The view cube** sits at the top right of the canvas:
  - Click a face for that view, or a corner for the view from that corner.
  - Drag the cube to orbit.
  - Under it: ⌂ **Home**, **Orbit** (drag the body itself), **Pan** and **Fit**.
- A 3D view layer orbits with Shift + middle-drag too. With on, middle-drag pans its camera.

Text animators (2.493.0)

A text layer's Properties panel has a **Text Animators** section. Animators move, turn, scale, fade, space out, blur or recolour the letters one by one, so a line can type itself, rise into place or ripple.

- **Animator** adds one. Pick a preset first if you like; each preset comes with its keyframes already set:

  - **Typewriter**: letters appear one after another.
  - **Fade up**: letters rise and fade in.
  - **Pop in**: letters grow from nothing.
  - **Slide in (words)**: words slide in from the left.
  - **Wave**: a ripple runs through the line.
  - **Blur in**: letters come into focus.
  - **Custom**: an empty animator to set up yourself.
- **What it changes:** x and y (px), scale (%), rotation (°), opacity (%), tracking (extra space, px), blur (px), and optionally a colour.
- **Which letters it changes (the range):**
  - **start**, **end** and **offset** (%) set the part of the text the animator affects. **amount** (%) sets how strongly.
  - It can work by **letters**, **words** or **lines**.
  - The shape sets how the effect is spread across the range:
  - **square**: on or off, with the edge letter partly in, so a typewriter runs smoothly.
  - **ramp up** / **ramp down**: a gradual build or fade across the range.
  - **triangle** / **round** / **smooth**: strongest in the middle of the range.
- **Animating:** every number has a ◇ that keys it at the playhead. Keying the offset from −(the range) to 100 % sweeps the effect through the text. The Graph Editor lists these keys too.
- Several animators can work on one text: their moves add up, and their scales and opacities multiply.
- × takes an animator out, along with its keys.
- **The compile:** before ⚙ Compile, a text layer with animators is baked by the page, so the movie shows exactly what the canvas showed.
- Animators work on plain text layers. A text with a text-effect preset keeps its preset.

Shape layers and their operators (2.492.0)

- ◆ **Shape tool** (Q, in the canvas toolbar): drag a box on the canvas to make a shape layer that size and in that place. Hold Shift for even sides. Afterwards the tool goes back to Move.
- A shape layer's Properties panel has these settings:
  - **The kind:** Rectangle (with a corner radius), Ellipse, Star (points, inner radius) or Polygon (points). Each has a width and height.
  - **Fill:** a colour, or a two-colour gradient, with its opacity.
  - **Stroke:** a colour, a width, and round, flat or square ends.
  - **The operators.** Tick one to turn it on and show its settings:
  - ✂ **Trim Paths**: start, end and offset (%). Only that part of the outline is drawn. Key the end from 0 to 100 % to draw a shape on.
  - **Repeater**: copies and an offset; each copy is moved (x, y), scaled (%) and turned (°) a step further. Opacity fades from a start value to an end value.
  - ⇲ **Offset Paths** grows the outline out, or in with a negative value.
  - ✳ **Pucker & Bloat** pulls the edges in or pushes them out.
  - **Twist** turns the outside more than the middle.
  - **Zig Zag** adds ridges, sharp or smooth.
  - ≈ **Wiggle Paths** makes the outline wobble, moving over time.
  - ◜ **Round Corners** rounds every corner.
- **Animating:** every number has a ◇ beside it, which keys that number at the playhead. Once a number is keyed, changing it keys it again at the playhead, the same as position. The Graph Editor lists the keyed shape numbers, so you can shape their eases.

- **The compile:** before ⚙ Compile, the page bakes each shape layer, frame by frame, exactly as the canvas draws it. An unchanged shape is not baked again.

Blend modes and track mattes (2.491.0)

Select a clip on a video or picture layer. The Properties panel has two new settings, which apply to the clip's whole **layer**:

- ◐ **Blend** sets how the layer mixes with what is under it: Normal, Multiply, Screen, Overlay, Darken, Lighten, Color Dodge, Color Burn, Hard Light, Soft Light, Difference, Exclusion or Add.
  - For example, Screen a light-leak video over a shot, or Multiply a paper texture over it.
- ▣ **Matte** shows the layer only where the layer **right above it** is (the matte layer is named beside the menu):
  - **Alpha**: where the matte layer has picture (its transparency).
  - **Alpha Inverted**: where it has none.
  - **Luma**: where it is bright.
  - **Luma Inverted**: where it is dark.
- The matte layer itself is no longer drawn. It still works as the matte even when it is hidden.
  - For example, put a text layer above a video layer and give the video layer an Alpha matte: the video plays inside the letters.
- The canvas shows both while you edit. Playback uses the drawing path for these layers.
- The compile draws the same picture.

Parenting and null objects (2.490.0)

- **Parent** (in the Properties of a picture, text or null layer) makes the layer follow another layer.
  - The child moves, turns and scales with its parent. It does not take on the parent's opacity.
  - Choosing a parent keeps the layer exactly where it is on screen: its position, rotation and scale (and its keyframes) are converted into the parent's space.
  - Choosing **none** does the same in reverse.
  - Chains work: a parent can have its own parent. A layer can never follow itself or one of its own children.
  - Video layers can be parents, but cannot follow one.
- **Null** adds an empty control layer, as long as the timeline.
  - It has no picture and is never compiled. The canvas shows it as a red dashed square with a cross.
  - Key the null (position, rotation, scale), then parent layers to it to move them all together.
- Dragging a parented layer on the canvas moves it the way you drag, whatever its parent is doing.
- **The compile:** before ⚙ Compile, each parented layer is baked. The page draws its frames exactly as the canvas shows them and saves them as a transparent video in the project (video/tlb_….webm), which the compile overlays. A layer that has not changed since its last bake is not baked again.

The Graph Editor, Easy Ease, the anchor point and curved motion paths (2.489.0)

Select a picture, text or video layer on the timeline's canvas. Its Properties panel now has these extras:

- ⊕ **Anchor X / Y**: the point the layer's position places, and that scale and rotation pivot on. It is measured in pixels from the layer's centre. Changing it keeps the layer where it is, because its position moves to match. If position is keyframed, the layer moves instead. The anchor can be keyframed too (in the Graph Editor, double-click the Anchor X or Anchor Y graph).

- **Easy Ease** (F9) eases the keyframe at the playhead in and out, so the layer slows into and out of it. **Ease In** and **Ease Out** (Shift+F9 / Ctrl+Shift+F9) do one side only.
- **Graph** opens the **Graph Editor**, a panel you can drag anywhere:
  - Pick a property: Position X / Y, Scale, Rotation, Opacity or Anchor X / Y. Keyed ones are marked ◆.
  - **value** shows the property over the clip's time; **speed** shows how fast it changes.
  - Drag a ■ keyframe to change its time and value. Click one to select it.
  - The selected keyframe's tween shows two ● handles. Drag them to shape the curve: a slow start, a fast finish, or even an overshoot. **Easy Ease**, **Linear** and **Hold** set the selected tween in one click.
  - Double-click the graph to add a keyframe there. Delete removes the selected one. F9 Easy Eases it.
- ↝ **Smooth path** gives every position keyframe smooth handles, so the layer travels along a curve through them.
  - The **motion path** shows on the canvas as a dotted line with a ■ at each keyframe.
  - Drag a ■ to move that keyframe. Alt-drag a ■ to pull out its handles.
  - Drag a ● handle to bend the path. Alt-drag a handle to bend one side only.
- The keyframe ease menu has a new **Bézier (graph)** entry.
- The compile draws the same curves, anchors and paths that the canvas shows.

PDF pages added, taken out and moved; Select Subject without a download - round H (2.488.0)

PDF pages. When a PDF is open, the Pages strip has a row of buttons under the pages. They act on the page that is open.

- **Page** adds a blank page, the same size as the open one, right after it.
- **PDF…** adds all the pages of another PDF right after the open one.
- ▲ / ▼ move the open page up or down. You can also drag a page in the strip to a new place.
- takes the open page out, and the page before or after it opens instead. If the page has changes, you are asked first.
- The same commands are under File ▸ Add / Remove PDF Pages.
- **Save PDF** writes the pages in their new order. Pages you didn't change are copied across as they were (text, vector shapes and all). Changed and new pages are saved as their picture plus real text.

Select ▸ Subject selects the picture's main subject in one step, with nothing to type.

- **Built in** works straight away, with no download. It learns the background from the picture's edges and the subject from what differs, then fits the selection to the real edges. It works best on a clear subject against a plainer background, and still works for subjects that stand on the bottom edge, like portraits.
- **AI matte** uses BiRefNet on this computer, the same install as the path layers' AI matte (about 1 GB, once). It handles hair, fur and busy backgrounds better.
- **Select ▸ Subject Settings…** picks the method: **Auto** (the AI matte when it is installed, built in otherwise), **Built in** or **AI matte**. It also picks the AI model and has **⬇ Install**.
- **Layer ▸ Remove Background** does the same thing but puts the subject into the active layer's mask, so the background is hidden, not deleted. Paint on the mask to fix any edges.
- Select Subject and Remove Background are each one undo step.
- Selecting by words (Segment Anything) is now **Select ▸ Subject by Words…**.

Image ▸ Mode, Proof Colors, Gamut Warning, CMYK TIFF - round G (2.487.0)

Image ▸ Mode sets what kind of picture the document is. Each mode change is one undo step, and the mode is saved in the document's .bgi file.

- **Grayscale**: every pixel layer turns grey. Anything you paint afterwards shows grey too, on screen and when exported.
- **Duotone…**: a grey picture printed with one to four inks (monotone, duotone, tritone, quadtone).
  - Pick each ink's colour. Its curve slider gives that ink more (right) or less (left) of the midtones.
  - Presets: Sepia, Blue duotone, Cyanotype, Gold tritone, Warm quadtone and Black.
  - The layers hold the grey picture, so you can change the inks later.
- **Indexed Color…**: at most 256 colours. The layers are flattened into one **Index** layer.
  - Palette: Local (made from the picture), Web (216), Uniform, Grey, or Black and white.
  - Dither (diffusion or pattern) with an amount; keep transparency or put the picture on white.
  - **Color Table…** shows the colours. Click one to change it everywhere it is used, or save and load .act colour tables.
- **Bitmap…**: black and white only. The picture is flattened onto white with a 50 % threshold, pattern dither, diffusion dither or a halftone screen (cell size and angle).
- **CMYK Color…**: colours are limited to what a printing press can reproduce.
  - Choose coated, uncoated or newsprint paper, black generation (none to maximum) and the total ink limit.
  - Colours the press cannot print, like bright screen greens and blues, move to the nearest colour it can.
  - **File ▸ Export As ▸ TIFF (CMYK, for print)** writes the four inks for a print shop. The Info panel's C M Y K readout uses the same model.
- **RGB Color** goes back to full colour. The look of the mode you leave is kept in the pixels.
- **8 Bits/Channel** is the only depth here, because the browser's canvas holds 8 bits per channel. There are no 16- or 32-bit documents.

View ▸ Proof Setup… picks the press to simulate (paper, black generation, total ink), whether to simulate the paper colour, and the gamut warning's limit and colour. These only change what you see on screen:

- **View ▸ Proof Colors** (Alt+Ctrl+Y) shows the picture as that press would print it.
- **View ▸ Gamut Warning** (Shift+Ctrl+Y) paints the colours the press cannot print in the warning colour (grey by default). In the Info panel, a "!" after the C M Y K values means the same.
- The press is a simple simulation of process inks on paper, not an ICC colour profile. Use it to spot trouble, and use your print shop's proof for exact colour.

Auto-Align Layers, Auto-Blend Layers, Photomerge, Merge to HDR - round F, part 2 (2.486.0)

These four work out how pictures fit together by themselves. They find small details (corners) in each picture, match them between pictures, and work out the move, turn, scale or perspective that lines them up. Pictures need some detail in common. Plain sky or a blank wall gives them nothing to match.

- **Edit ▸ Auto-Align Layers…** lines up the layers you have picked (Ctrl-click in Layers), or every visible pixel layer if you haven't picked any.
  - **Layout**: Auto or Perspective (move, turn, scale and perspective), Move, turn and scale, or Reposition (move only).
  - **Line up on**: the middle layer, the active layer or the bottom layer. That layer stays put.
  - Layers that cannot be matched stay where they are, and the status line says how many there were. It is one undo step.

- **Edit ▸ Auto-Blend Layers…** gives each layer a mask showing where it is used. You can paint on the masks afterwards.
  – **Panorama**: each spot comes from the layer it is deepest inside, with soft seams. **Seamless tones and colours** evens out brightness and colour between the pictures.
  – **Stack Images**: each spot comes from the layer that is sharpest there. Use it for focus stacking (the same shot taken several times, each focused at a different distance). Align the layers first.
- **File ▸ Automate ▸ Photomerge…** stitches a panorama. Choose files, or use the open documents.
  – The pictures should overlap by about a third.
  – The result is a new **Panorama** document with one layer per picture, placed and (with **Blend the pictures together**) masked.
  – Pictures that overlap nothing are left out, and the status line says so.
  – Very wide results are scaled to fit 8000 pixels.
- **File ▸ Automate ▸ Merge to HDR…** takes bracketed exposures (the same scene shot dark, normal and bright) and fuses them into one well-exposed picture.
  – Each spot takes most from the exposure where it is clearest: good contrast, good colour, neither too dark nor too bright.
  – With **Line the pictures up first**, slight camera movement between shots is corrected.
  – The result is an ordinary 8-bit picture, not a 32-bit HDR file.

More file types, File Info, Print, Contact Sheet, Crop and Straighten Photos - round F (2.485.0)

- **File ▸ Export As** now also writes **AVIF**, **GIF**, **TIFF**, **BMP** and **TGA**, next to PNG, JPEG, WebP and JPEG XL.
  – GIF keeps up to 256 colours (dithered, so gradients stay smooth) and a clear colour for transparent areas.
  – TIFF, BMP and TGA keep every pixel and the transparency. The preview shows the picture for TIFF and TGA, since a browser cannot show those files itself.
  – AVIF is made by ffmpeg on this computer (it needs an AV1 encoder: libaom-av1, libsvtav1 or librav1e) and has a quality slider. AVIF files are saved without transparency.
- **File ▸ File Info…** (Alt+Shift+Ctrl+I): title, author, description, keywords, copyright and a source URL. It is kept with the document (and in its .bgi file) and written into exported PNG files (as text chunks) and JPEG files (as XMP), so other programs show it.
- **File ▸ Print…** (Ctrl+Shift+P - Ctrl+P brings the prompt docker to the front): scale to fit the page, a size in cm or inches, or a percentage; portrait, landscape or as the picture. OK opens the system's print dialog, where you pick the printer, the paper and copies, or save as PDF.
- **File ▸ Automate ▸ Contact Sheet…**: puts pictures in a grid on a new document. Choose files, or use the open documents; pick the page (A4, Letter or 1920 x 1080), columns and rows, the margin and captions with the file names. Each picture is its own layer; more pictures than fit start a new page.
- **File ▸ Automate ▸ Crop and Straighten Photos**: for a scan with several photos on it. Each photo is found against the plain page colour, turned straight and opened as a document of its own (named "<scan> photo 1", "photo 2", …).

Brush Settings, brush presets and .abr brushes - round E (2.484.0)

The **Brush** tab (the Properties group; also Window ▸ Brush Settings) is where you set up the brush the Brush, Pencil and Eraser tools paint with.

- **The presets** at the top: the 20 that come with bitGen, yours, and the ones you have imported. Click one to use it.

  - **Import .abr…** brings in the sampled brushes from a brush file (.abr versions 1, 2, 6, 7 and 10). Each becomes a preset that paints its picture.
  - **New Brush Preset…** keeps the current brush, with all its settings, under a name. Your presets live in **brushes/<name>.json** in the install folder; copy them to share them.
  - **Delete** removes one of yours; **↺ Reset** puts a built-in preset back as it came.
- **Brush Tip Shape**: Angle, Roundness (a flat, calligraphic tip), Hardness, Spacing, Flip X / Y.
- **Shape Dynamics** (tick the section to turn it on): Size jitter + minimum diameter; Size control (off, pen pressure, or fade over N steps); Angle jitter + Angle control (off, the stroke's direction, pen pressure); Roundness jitter + minimum; Flip X / Y jitter.
- **Scattering**: Scatter (% of the size, across the stroke or both axes), Count, Count jitter.
- **Texture**: Noise, Canvas, Paper, or any pattern of the pattern library. Scale, Depth, Multiply / Subtract, Invert. The texture stays fixed to the picture.
- **Dual Brush**: a second tip (any preset) cuts into the first, with its size, scatter and count.
- **Color Dynamics**: Foreground / background jitter; hue, saturation and brightness jitter; Purity; per tip or once per stroke.
- **Transfer**: Opacity jitter, Flow jitter, Control by pen pressure.
- **Smoothing** (0-95 %): the brush follows the pointer on a string, for steadier lines.
- Each preset keeps its own settings: the built-in ones in the editor's settings on this computer, yours inside their file.

Navigator, Histogram, Info, measuring tools, Grow / Similar / Focus Area, Matting - round D (2.483.0)

- **Panels** (the Color group's tabs, and the Window menu):
  - **Navigator**: the whole picture small, with the view as a red box. Click or drag on it to move the view; the slider zooms.
  - **Histogram**: RGB, Luminosity, Red, Green, Blue or Colors, of the entire picture or the selected layer (inside the selection when there is one). It shows the Mean, Std dev, Median and Pixels, plus the level, count and percentile under the pointer.
  - **Info**: under the pointer, the RGB, HSB and CMYK values and the X, Y position. It also shows the document's size, the selection's size and place, the ruler's length and angle, and each colour sampler's colour.
  - **Notes**: every note on the picture; goes to it, ✕ deletes it, and a double-click edits it.
- **Tools** in the Eyedropper's group (**I**, Shift+I cycles):
  - **Color Sampler**: click to place up to 10 samplers and drag one to move it; Alt-click removes it. The Info panel reads them, and Sample size can be a point, 3 by 3 or 5 by 5.
  - **Ruler**: drag to measure (Shift = 45° steps). **↻ Straighten Layer** turns the active layer so the line is level (or upright); **↻ Straighten Picture** turns every layer.
  - **Note**: click to pin a note (with its author); click it to edit it, Alt-click to delete it.
  - **Count**: click to drop numbered markers in coloured count groups (Group, colour, size). The options bar shows the count; Alt-click removes the nearest marker.
- **Perspective Crop** (in the Crop tool's group, **C**): drag a box, move its four corners onto the edges of what should end up square (a sign, a document, a building's front), then press **Enter**. The whole picture is cropped and squared, every layer with it. One undo step.
- **Select ▸ Grow** (to touching pixels within the Magic Wand's tolerance), **Similar** (every pixel like the selected ones), **Focus Area…** (the sharp parts: In-focus range + Softness, previewed live), **Transform Selection…** (move, scale and rotate the selection itself).
- **Layer ▸ Matting**: **Defringe…** (edge pixels take the colour of the solid ones next to them), **Remove Black Matte**, **Remove White Matte**.
- Samplers, notes and count markers belong to each document and are kept in its .bgi.

The Develop Filter and Lens Correction - the image editor's gap, round C (2.482.0)

- **Filter ▸ Develop Filter…** (**Alt+Shift+Ctrl+A**) is a photo developer in one dialog. It previews live, works inside the selection, is one undo step, and becomes a smart filter on a smart object. Its tabs:
  - – **Basic**: Temperature, Tint, Exposure, Contrast, Highlights, Shadows, Whites, Blacks, Texture, Clarity, Dehaze, Vibrance, Saturation; **Black & White**; **Auto** sets exposure, contrast, whites and blacks from the picture.
  - – **Curve**: Highlights / Lights / Darks / Shadows sliders and a point curve (RGB, Red, Green, Blue).
  - – **Color**: the colour mixer: pick Red, Orange, Yellow, Green, Aqua, Blue, Purple or Magenta and change its Hue (±30°), Saturation and Luminance. In Black & White, Luminance sets how light each colour turns.
  - – **Grading**: tint the Shadows, Midtones and Highlights (hue, saturation, luminance); Blending and Balance.
  - – **Detail**: Sharpening (amount, radius, detail, masking: 0 = everywhere, more = only the edges), Noise reduction, Color noise reduction.
  - – **Optics**: Distortion, Vignetting (+ midpoint), Defringe purple / green.
  - – **Geometry**: Vertical / Horizontal perspective, Rotate, Scale, X / Y offset.
  - – **Effects**: Grain (amount, size, roughness) and a vignette after the crop (amount, midpoint, roundness, feather).
  - – **Presets**: Vivid, Matte, Warm film, Cool night, Portrait soft, Landscape punch, Black & White, Teal & orange. **Save as preset** keeps your own (they stay in the editor's settings), and **↺ Reset all** clears everything.
- **Filter ▸ Lens Correction…** (**Alt+Shift+Ctrl+R**): Remove distortion (barrel / pincushion), Fix red / cyan and blue / yellow fringes, Vignette (amount, midpoint), Vertical / Horizontal perspective, Angle, Scale, and what fills the **Edge** (edge extension, transparent, a colour). A **grid** over the picture while it is open helps straighten.

44 more filters - the image editor's gap, round B (2.481.0)

The **Filter** menu now has twelve submenus: Artistic, Blur, Blur Gallery, Distort, Noise, Other, Pixelate, Render, Sharpen, Sketch, Stylize and Texture. Every filter previews live, works inside the selection, is one undo step, can be repeated with **Ctrl+F**, recorded in an action, and becomes a **smart filter** on a smart object.

- **Blur**: **Surface Blur** (smooths while keeping edges: Radius + Threshold), **Smart Blur** (Normal / Edge Only / Overlay Edge), **Average**, **Lens Blur** (depth from transparency or brightness, a focal distance, the iris shape (circle, 3-8 blades), blade rotation, specular highlights for bokeh, noise).
- **Blur Gallery**: **Field Blur** (two or three pins, each with its own blur), **Iris Blur** (an ellipse that stays sharp and fades out; **click the picture to put its centre**), **Path Blur** (motion along a curve: start, bend, end, speed, taper), **Spin Blur** (round an ellipse by an angle; click to place).
- **Noise**: **Reduce Noise** (strength, preserve details, colour noise, sharpen details), **Dust & Scratches** (radius + threshold), **Despeckle**.
- **Sharpen**: **Smart Sharpen** (amount, radius, reduce noise; remove Gaussian, Lens or Motion blur at an angle; fade the sharpening in the shadows / highlights).
- **Other**: **High Pass**, **Minimum**, **Maximum** (square or round), **Offset** (wrap around / repeat the edge / see-through), **Custom** (your own 5 x 5 kernel with presets, a scale and an offset), **HSB / HSL**.
- **Distort**: **Displace** (a map from a layer or the merged picture: red moves pixels across, green up and down), **Polar Coordinates** (both ways), **Pinch**, **ZigZag**, **Shear** (bend through top / middle / bottom), **Glass** (frosted, blocks, canvas, tiny lens).
- **Pixelate**: **Color Halftone** (C / M / Y / K dot screens at their angles), **Crystallize**, **Pointillize**, **Facet**, **Fragment**, **Mezzotint** (dots, lines, strokes).

- **Render**: **Lighting Effects** (spot, point or infinite light; colour, intensity, where it is and where it aims, cone + hotspot, ambience, gloss; a height texture from the brightness or the transparency), **Fibers**, **Difference Clouds**.
- **Stylize**: **Diffuse** (normal, darken only, lighten only, anisotropic), **Tiles**, **Trace Contour**.
- **Artistic**: **Cutout**, **Poster Edges**, **Watercolor**, **Dry Brush**. **Sketch**: **Graphic Pen**, **Charcoal**. **Texture**: **Grain** (7 kinds), **Craquelure**.

More adjustments and Apply Image - the image editor's gap, round A (2.480.0)

New in **Image ▸ Adjustments**. Each one previews live, works inside the selection, is one undo step, and can be repeated with Ctrl+F, used as a smart filter or recorded in an action:

- **Shadows / Highlights**: lifts the shadows and pulls back the highlights *locally*. **Amount** sets how much, **Tone** sets how far into the midtones it reaches, and **Radius** sets the size of the neighbourhood it looks at. **Color correction** restores the colour in the areas it changed, and **Midtone contrast** works in between.
- **HDR Toning**: the "HDR look" on one picture. **Edge glow** (radius + strength) evens the light across the picture, then **Gamma**, **Exposure**, **Detail**, **Vibrance** and **Saturation** finish it.
- **Selective Color**: pick a range (**Reds, Yellows, Greens, Cyans, Blues, Magentas, Whites, Neutrals, Blacks**) and add or take away **Cyan, Magenta, Yellow and Black** in it. **Relative** changes the ink in proportion to what is there; **Absolute** adds the amount itself.
- **Replace Color**: **click the picture** to pick the colour (or set it), set **Fuzziness** for how alike a colour must be, then shift its **Hue / Saturation / Lightness**.
- **Match Color**: give the layer the colour of another: **Source** = another layer, the merged picture or a picture file (…). Then set **Luminance**, **Color intensity**, **Fade**, and **Neutralize** to take a colour cast out.
- **Color Lookup**: a 3D LUT. Pick one of the timeline's **looks** or a **.cube** from the LUT library (shared with the timeline's Colour panel); **.cube…** adds your own. **Amount** mixes it.
- **Image ▸ Apply Image…**: blends another layer (or the merged picture), or one of its channels (RGB, Red, Green, Blue, alpha, luminosity), inverted or not, onto the layer. There are 11 blend modes, plus Opacity and Preserve transparency.
- **Adjustment layers**: **Selective Color** and **Color Lookup** are new (Layer ▸ New Adjustment Layer, the Adjustments panel); you edit them in Properties.

JPEG XL (.jxl) pictures (2.479.0)

- **Open** a .jxl in the image editor like any picture: File ▸ Open, drop it on the canvas, Place, or open it from the project. Its pixels come in exactly as they are (8 bits per channel, sRGB; the first frame of an animated one).
- **Export As ▸ Format: JPEG XL**: **Quality** (1-100), or **Lossless** (every pixel kept exactly, and usually much smaller than PNG). Download, or **Save to Images**, which keeps it as a .jxl in the project's pictures.
- **The rest of the app**: .jxl pictures show in the Assets docker, the gallery, the media viewer, the file dialog's previews and on the timeline. The browser cannot show JPEG XL itself, so the studio sends a PNG of the picture to whatever displays it. Anything that reads the file itself gets the .jxl.
- **Generating and compiling**: a .jxl used as a reference is sent to ComfyUI as a PNG (ComfyUI cannot read JPEG XL). A .jxl still on the timeline is compiled from its PNG. A .jxl uploaded anywhere other than the project's pictures arrives as a PNG.
- Everything runs on this computer: the codecs (jxl-oxide to read, libjxl to write) are in static/vendor/jxl/.

Search in every dropdown list (2.478.0)

- Every **dropdown list with 8 or more items** - in the dockers, the image editor, the timeline, Settings and every dialog - opens with a **search box** on top, like the node search:

  – **type** and the list narrows as you type, best matches first: the start of a word, then anywhere, then the letters in order ("flx2" finds "FLUX.2 [dev]"). The matched letters are lit, and a group's name finds its items. "12 of 340" shows how many are left.

  – ↑ ↓, **PgUp** / **PgDn** (Ctrl+Home / Ctrl+End for the first / last) move the lit row; **Enter** or a **click** picks it; **Tab** picks the lit row and moves on; **Esc** or a click outside closes the list with nothing changed.

  – On a list that has the focus, **any letter** opens it already searching for that letter; **Alt+↓**, **F4**, **Space** or **Enter** open it; ↑ ↓ still step through it without opening it.
- **Settings ▸ General ▸ Dropdown lists**: "Search in lists with N or more items" (8; **0** = always the plain list). Kept in this browser.

The public beta: the repository, what is experimental, reporting a problem (2.477.0)

- bitGen Studio is now a **public beta** with its own repository: the source, the guides, the Feature Reference PDF, the changelog (**CHANGELOG.md**, every release) and the installers on the **Releases** page.
- **docs/BETA.md** says what is solid and what is **experimental**: the Unreal Engine bridge (written for Unreal 5.8 and tested against a stand-in, not yet inside a real Unreal Editor), the Daz Studio live link, transparent Unreal renders, and macOS (MiniMax H3 video needs NVIDIA CUDA).
- **Reporting a problem**: open an issue with the **Bug report** template. Give the version (package.json, or the release you downloaded), your OS / graphics card, what you did and what happened (a screenshot or a short recording), and the Console docker's last lines. For the Unreal bridge, add the `[bitGen Bridge]` lines from Unreal's Output Log. **Never** post your custom.settings (it has your folders and tokens).
- **Updating**: download a newer release, or `git pull`, then run start.bat / misc/bitgen-studio.sh again. Your projects, settings, models and styles are not in the repository, so an update leaves them alone.
- **Contributing**: CONTRIBUTING.md has the set-up, where code goes (routes/ + lib/, each docker an extension in ext/), the tests (`bash scripts/ci-units.sh` - what GitHub runs on every push) and the docs to update.

Unreal lights, volumetric lights and fog - in the image editor and the timeline (2.476.0)

- **Update the plugin** to **bitgen-ue-bridge-1.2.0.zip** (it places the lights and the fog).
- **Image editor ▸ UNREAL ▸ Lights & Fog** (or the **Lights** tab in Properties, Window ▸ Unreal Lights) for an Unreal or MetaHuman layer:

  – **Lights from a preset**: Three-point (key, fill, rim), Golden hour (a low warm sun + sky), Night street (a warm spot + a cool fill), Neon (magenta + cyan rect lights with volumetric fog), Studio softbox.

  – **Point** / **Spot** / **Rect** / **Sun** / **Sky**: a new light is placed for the camera - in front, up and to the side, aimed at the subject.

  – Each light: **colour** or a **temperature** in kelvin, **intensity** (candelas; lux for the sun), **reach**, **soft size**, **a spot's** cone**, a rect light's** size**,** Volumetric** (how strongly it lights volumetric fog - light shafts), **shadows**, its **place** (X Y Z) and **aim** (roll, pitch, yaw); ☐ switches it off, ✕ deletes it.

  - **Fog**: height fog (density, falloff, colour, where it starts) and **volumetric fog** (albedo, extinction, how far it reaches), and fog presets (light haze, morning mist, thick volumetric, cinema smoke).
  - **Keep them in the Unreal level**: otherwise bitGen's lights and fog (named bitGen_*) are only there for the picture.
  - Every change renders the layer again - **one undo step**.
- **UNREAL ▸ Lights** is the **light tool**: every light shows as a marker where it is in the picture (a spot or a rect light shows where it points). **Drag** a marker across the picture (it keeps its distance from the camera), **Shift + drag** = nearer / further, **Alt + drag** = turn its aim; Del deletes the picked light.
- **Timeline ▸ UNREAL ▸ Lights & Fog…** (an Unreal, MetaHuman or Unreal camera clip picked): the same panel, plus ⇦ **The image editor's lights** to bring the lights + fog of the image editor's layer; **Render** makes the clip again with them.

The Unreal camera in the image editor: move it, lens, f-stop, exposure, presets (2.475.0)

- **Update the plugin**: unzip **bitgen-ue-bridge-1.1.0.zip** (or newer) over the old one in your project's Plugins folder (the camera's lens, "To Unreal's viewport" and "Click to focus" need it).
- **UNREAL ▸ Camera** (with an Unreal or MetaHuman layer picked) is the **Unreal camera tool** - drag on the picture to move the camera in the scene:
  - **drag** = **Orbit** round what it looks at (the focus distance, else 5 m), **Ctrl + drag** = **Look** (turn it where it stands: yaw / pitch), **Shift + drag** = **Pan**, **Alt + drag** = **Roll** (tilt the horizon), **the wheel** = **Dolly** forward / back, **Ctrl + wheel** = **zoom** (the focal length).
  - The modes are also buttons on the options bar (Orbit, Look, Pan, Dolly, Roll, **Focus**: click what should be sharp and Unreal measures the distance).
  - While you drag, Unreal draws small previews; when you let go it renders at full size - **one undo step**. Thirds and the focus point are drawn over the layer.
- **The Camera tab** (Properties; **Window ▸ Unreal Camera**; UNREAL ▸ Lens & Exposure):
  - **Presets**: lenses from 18 mm to 200 mm, macro, an anamorphic look - and **Save** your own.
  - **Lens**: focal length (slider + number) linked to the **field of view**, the **filmback** (full frame, Super 35, APS-C, Micro 4/3, IMAX, phone).
  - **Depth of field**: **f-stop** (f/1.2 - f/22) and the **focus** distance (0 = all sharp), **Click to focus**.
  - **Exposure** in EV (-5 … +5).
  - **Place** (X Y Z, cm) and **turn** (roll, pitch, yaw, °) as numbers; ⇩ **From Unreal's viewport**, **⇧ To Unreal's viewport **(Unreal looks through this camera),** Render**.
- **Timeline**: UNREAL ▸ Keys… of an Unreal camera now has a **Lens** row (focal length, f-stop, focus, exposure) used by its render.

No tool, the selection box and several layers at once (2.474.0)

- **Esc** puts the tools down: no tool, nothing picked. (A tool's own Esc - ending a pen path, cancelling a crop or a polygon lasso - still comes first; with no tool and nothing picked, Esc does what it did before, e.g. leaves full screen.) A tool's letter (V, M, B…) picks a tool again.
- **With no tool**, hovering an object outlines it blue; a **click** picks it (**Shift** or **Ctrl** adds it to / takes it away from what is picked). **Drag** on the picture and a **dashed box** follows the pointer:
  - **left to right** (blue) picks every layer **wholly inside** the box,
  - **right to left** (green) picks every layer the box **touches**,
  - **Shift** keeps what was picked and adds to it. A click on nothing picks nothing.
- **Layers panel**: a click picks one layer; **Shift-click** picks every layer between the last one clicked and this one; **Ctrl-click** adds a layer or takes it away. Picked rows are lit (their numbers 1, 2, 3 are still the order Compose and → References use).

- With several layers picked, the **Move tool** moves them all together (one undo step), and **Del** deletes them all (one undo step). With a pixel selection, Del still clears the selected pixels.

The Feature Reference PDF (2.473.0)

- **docs/bitGen-Studio-Features.pdf** lists every feature of bitGen Studio so far - about 575 of them in ten areas (the workspace, generating video, images, music and sound, the timeline, the image editor, CAD and 3D, the bridges, delivery, keys): what each does, where it is and since which version. It opens with the areas at a glance (each a link) and has bookmarks per area and sub-area.
- It is made from **docs/FEATURES.md** for every release (`python3 scripts/make_features_pdf.py`), beside the Guide (step-by-step instructions + shortcut keys) and the brochure.

The Unreal Engine 5.8 bridge: Unreal layers, MetaHuman layers, the Unreal camera (2.472.0)

- **Install once** (2.513.0: or let bitGen do it - Load UE project in the menu bar): unzip **bitgen-ue-bridge-1.1.0.zip** (or newer) into your Unreal project's **Plugins** folder (`<Project>\Plugins\bitGenBridge`), open the project and enable the plugins it asks for (Python Editor Script Plugin, Editor Scripting Utilities, Sequencer Scripting, Movie Render Queue). The Output Log shows "[bitGen Bridge] listening". In bitGen: **Settings ▸ 3D ▸ Unreal Engine ▸ Test connection**. It works on this computer only (127.0.0.1); the port is 30020 unless you change it on both sides.
- For **clear backgrounds** (MetaHuman layers) turn on Project Settings ▸ Rendering ▸ Post Processing ▸ "Enable alpha channel support in post processing" in Unreal (the README in the zip says how; "alpha is inverted" in Settings fixes a reversed one).
- **Image editor ▸ UNREAL tab** (on the CAD toolbar, or Layer ▸ New Unreal Layer… / New MetaHuman Layer…):
  - **Unreal Layer**: a level of the open project through one of its cameras (or the level viewport as it is) at the size you choose - a picture layer you can move, scale, filter and mask.
  - **MetaHuman Layer**: a MetaHuman of the project in the pose of an animation at a time, framed **Face / Bust / Full body** and turned, on a clear background.
  - The **Unreal tab** (Properties) nudges the camera (Turn, Tilt, Dolly, Field of view) or the figure (framing, turn, pose time) and **Renders again** - one undo step. **Use Viewport** takes Unreal's viewport camera now.
- **Timeline ▸ UNREAL tab**:
  - **Unreal Layer**: a **Level Sequence** of the project (or a level through a camera / the viewport, held) rendered by Unreal's **Movie Render Queue** at the timeline's size and fps -> a video layer **UE1** at the playhead.
  - **MetaHuman Layer**: a MetaHuman playing an animation, framed and turned, on a clear background -> **MH1**.
  - **Unreal Camera**: your own camera - the first key from Unreal's viewport (or a camera of the level); move the viewport in Unreal, put the playhead later and press **Key**; **Keys…** edits them (time, place, turn, field of view - the camera moves straight between them); **Render Again** renders the level through it -> **UE Cam1**.
  - Every clip remembers how it was made: **Render Again** makes it anew after you change the Unreal project.

Parametric 3D bodies: Extrude, Revolve, Fillet, Chamfer, Shell, Split Body (2.471.0)

- **SOLID tab** on the CAD toolbar (and Layer ▸ Extrude… / Revolve… / Fillet… / Chamfer… / Shell… / Split Body… / Change Parameters…). Draw a closed sketch (SKETCH tab), then:

– **Extrude**: the sketch's closed profiles (all of them - holes cut out - or the ones you tick), a distance, One side / Symmetric / Two sides, a start offset. **New Body** makes a new **body layer**; **Join / Cut / Intersect** work on a body you pick.

– **Revolve**: round the sketch's X or Y axis or one of its lines, any angle up to 360°.

– **Fillet** / **Chamfer** the sharp edges - all, the top or bottom ones, the vertical ones, or the ones round the faces. **Shell** hollows the body, open at the top or the bottom.

– **Split Body**: a plane (XY / XZ / YZ at a distance); this layer keeps one side and the other side becomes a **new body layer**. Both stay parametric.

- **Parametric**: every number is a sum you can type - `25`, `depth`, `depth / 10 + 2`, `1 in`, `30 deg` (the value shows as you type). **ƒx Change Parameters** names your numbers (each may use the others); change one and every feature that uses it follows. Edit the **sketch** later and its bodies are built again by themselves.
- **The Body tab** (Properties) lists the **features** in order: ☑ switches one off (suppress), ✎ or a double-click edits it, ↑ ↓ reorder, × deletes. A feature that fails is shown in red with the reason; the rest of the body stays. Below: **View** (Home / Front / Back / Top / Bottom / Right / Left, Turn / Tilt / Roll, **Orbit**

  – drag the body on the canvas), **Look** (colour, metal, rough, edges, detail) and **Export** (STL, glTF .glb, STEP, → Timeline). Every change is one undo step.
- A body layer is a **smart object**: move, scale, rotate, warp, filters and masks all work on it.
- **Settings ▸ 3D** picks the engine: **In the browser (OpenCascade)** - exact solids, nothing to install - or **Blender (on this computer)**. Both build the same body.
- **The timeline**: SOLID ▸ **Body → 3D Layer** puts a body on the timeline as a 3D layer (turn it, light it, key it). Change the body in the image editor and its 3D layers update by themselves (**Update Bodies** does it by hand); **Edit in Image Editor** picks it there.

Picking like a drawing program + the CAD toolbar (2.470.0)

- **Hover / picked outlines**: with the **Move** tool, the shape, smart object, text, sketch, 3D layer or picture content under the pointer shows its outline in **blue** - it can be picked. The picked one has an **orange** outline (and the Move tool's handles). The background is the canvas, not an object.
- **Click** on an object = it is picked (press and drag = picked and moved at once). **Click on empty canvas** = nothing picked (the handles go; a drag that starts there moves nothing). **Esc** = nothing picked. A click in the Layers panel picks the layer again. With a selection active (or Auto-select on) the Move tool works as before.
- **After drawing a shape** (Shape, Custom Shape, Rectangle, Ellipse, Polygon, Star, Spiral, Graph Paper, the common shapes, 2-Point Line) the **Move tool** comes up by itself with the new shape picked. Switch it off with ⚙ on the CAD toolbar (or SHAPES ▸ AFTER DRAWING ▸ Then Move).
- **The CAD toolbar** runs along the top of the image editor, over the tool options:

  – **SKETCH**: CREATE (Create Sketch, Line, 2-Point / Center Rectangle, Center / 2-Point / 3-Point Circle, 3-Point Arc, Polygon, Slot, Spline, Point), MODIFY (Fillet, Trim, Offset, Chamfer, Extend, Mirror, Rectangular / Circular Pattern, Construction), CONSTRAINTS (12 small buttons for what is picked), INSPECT (Sketch Dimension, Measure), INSERT (DXF in, DXF / SVG out), FINISH SKETCH (+ Profiles → Shapes).

  – **SHAPES**: SELECT (Move, Pick Nothing), DRAW (the shape tools), AFTER DRAWING.

  – Big buttons show the most used tools; **GROUP ▾** lists all of them. The tab follows your work (a sketch tool or a sketch layer -> SKETCH). ⌃ folds it to its tabs; **Window ▸ CAD Toolbar** hides / shows it.
- **The timeline** has the same toolbar over the preview row: **CREATE** (Text, 3D Text, 3D Model…, Camera, Path, Move / Hand / Zoom / Fit / Select / Mask), **3D** (with a 3D layer picked: the widget, the views, ◇ Key, Bake) and **SKETCH** (**Sketch on This Frame** - the frame at the playhead opens in the image editor with a new sketch layer; **Sketch on a Blank Picture** at the timeline's size). With the Move tool the clip under the pointer is outlined blue on the preview; the picked one stays orange.

Linked smart objects, PSD smart objects kept, .atn actions (2.469.0)

- **File ▸ Place Linked…**: a picture from your computer goes into the project's **linked** folder and comes in as a **linked smart object** (in the Layers panel). Its contents live in that file: change the file (in any program, or save a new one over it) and the layer follows - when the editor gets the focus back, every 20 s, or Layer ▸ Smart Objects ▸ **Update Modified Content**. Its place, size, warp and smart filters stay.
- **Layer ▸ Smart Objects**: **Convert to Linked…** (an embedded smart object's contents saved into the linked folder, the layer and its copies linked), **Embed Linked** (the link dropped, the contents kept), **Relink to File…** (another picture of the linked folder). **Edit Contents** on a linked one: ✔ Done also writes the new contents to its file. A file that has gone shows ⚠.
- **PSD**: smart objects whose contents are embedded in the PSD (PNG, JPEG, GIF or PSD) now open **as smart objects** - their contents and transform kept, so they scale without loss; saving a PSD writes smart objects back as placed layers with their contents embedded (a warped one goes as pixels).
- **Actions ▸ Load Actions…** also takes **.atn** action files: the set and its actions come in; recorded steps become the matching steps here - Gaussian Blur, Unsharp Mask, Median, Invert, Desaturate, Equalize, Posterize, Threshold, Mosaic, Add Noise, Emboss, Brightness / Contrast, Hue / Saturation (with their settings), Duplicate / New / Delete Layer, Merge Down, Flatten, Select All, Deselect, Inverse, opacity, fill, blend mode, Rotate, Flip, Stop. A step with no match comes in **switched off**, named after the original step, so the rest plays.

Actions: brush strokes + hand-made selections recorded, conditionals, Image Processor, droplets (2.468.0)

- **Recording** (Actions tab, ●) now also keeps:

  – every **brush stroke** of the Brush, Eraser, Pencil, Smudge and the other brush tools - where it went, its size, brush, opacity, flow and colours. Played, it is drawn again at the same place on the picture (scaled with the picture's size); your own tool and colours are put back after.

  – every **selection you make by hand** (marquee, lasso, magic wand, quick selection, quick mask, Deselect ...) - its outline, so playing it on a bigger or smaller picture makes the same selection there.
- ▸ **Actions: Insert Conditional…**: If the document is landscape / portrait / square, there is a selection / none, the layer is a pixel / text / shape / adjustment layer, there are several layers - **then** play one action, **else** another. Conditionals can call actions that have conditionals of their own.
- **File ▸ Scripts: Image Processor…**: pick pictures (files or a folder); save each as **JPEG** (quality 1-12, resize to fit W x H) and / or **PNG** (resize to fit), optionally after playing an action - into the project's batch/<name>-JPEG and -PNG folders, or downloaded.
- **File ▸ Automate: Create Droplet…**: an action + where its results go (project / gallery / download, PNG or JPEG) as a **droplet tile** at the bottom of the Actions tab: drop pictures on it (or click it to choose some) and each is opened, the action played, the result saved (the Batch). ✕ on the tile deletes it.

Editing single colour channels; spot channels (2.467.0)

- **Edit one colour channel** (Channels panel, next to Layers / Paths): click **Red**, **Green** or **Blue** - from then on painting, erasing, fills, filters and adjustments change **only that channel** (the panel marks it ✎ and shows it in grey). **Shift-click** another to edit two (shown in colour without the third). **RGB** (or Ctrl+2) = all again; Ctrl+3 / Ctrl+4 / Ctrl+5 pick Red / Green / Blue the same way. The layer's transparency is never changed while only some channels are edited.
- **Spot channels**: ◍ in the Channels panel (or Select ▸ New Spot Channel…) - a name, the **ink colour** and its **solidity** (0 = a see-through varnish, 100 = it covers); the selection becomes its ink.

Its ink shows over the picture (it is not in the layers until merged); double-click its swatch for the options.

- **Paint an alpha or spot channel**: pick its row, ✎ - the brush adds to it, the eraser takes away (shown like a quick mask); ✔ (or Q) keeps it as one undo step. The selection is not touched.
- ⇣ **Merge Spot Channel** (or Select ▸ Merge Spot Channel): its ink becomes a layer on top, the channel goes. A .bgi keeps spot channels with their colour and solidity.

OpenType features, optical kerning, type to outlines, text in a shape, round corners on any path (2.466.0)

- **OpenType** (Character panel): buttons for the font's own features - Standard / Discretionary Ligatures, Contextual Alternates, Swash, Stylistic / Titling Alternates, Ordinals, Fractions, Oldstyle / Lining / Tabular Figures, Slashed Zero, the font's Small Caps, Superiors / Inferiors - and a **Stylistic set** (ss01-ss20). The text is then shaped with the font's own file from your computer and drawn from its glyphs (the panel names the file). A font the computer has no file for is drawn as before, without them (the panel says so). Character styles keep them.
- **Kerning ▸ Optical** (Character panel): the letters spaced from their shapes (the white between each pair evened out to the font's own "nn" / "oo"), not from the font's kerning pairs - also needs the font's file.
- **Layer ▸ Type: Convert to Shape**: the text layer becomes a shape layer of its letters' outlines (same name, colour, place, opacity, styles; edit the outlines with the Direct Selection tool). **Type: Create Work Path**: the outlines as the work path (the text stays). Both take everything you set - lines, paragraph text, scales, faux italic, baseline shift, underline / strikethrough, OpenType, a turned text.
- **Text in a shape**: with a shape layer picked, click **inside** its outline with the Type tool (or Layer ▸ Type: Text in the Picked Shape) - the text flows line by line inside that outline (alignment, justify, indents, spacing work there too; what does not fit is hidden). Convert to Point Text takes it out again.
- **Round corners on any path** (Direct Selection, A): ◜ **Radius** + **Round Corners** on the options bar rounds the picked corner anchors - none picked = every corner of the path. Works on a shape that was joined with another (combined, merged), on paths and on vector masks; a radius too big for a side is cut to fit.

Puppet pin depth + rotation; the Patch tool's Content-Aware mode (2.465.0)

- **Puppet Warp** (Edit ▸ Puppet Warp): the picked pin (the last one added or clicked) now has

  – **Pin Depth ⬆ / ⬇** on the options bar: where the warp folds the layer over itself, a pin set forward draws its part on top, one set back goes underneath (the number by the pin is its depth);

  – **Rotate Auto / Fixed** + an angle: Auto lets the content round the pin turn as the mesh needs (as before); Fixed holds it turned by that angle. **Alt-drag near the picked pin** (on its dashed ring) turns it - it becomes Fixed. Both are kept in a smart object's Puppet Warp (double-click it in the smart filters to change them).
- **Patch** (J group) has **Normal** and **Content-Aware** on its options bar:

  – **Normal** (as before - Source / Destination) + **Transparent** (the texture only half over what was there) + **Diffusion** 1-7 (how far the edge's light and colour spread in: low for grain or fine detail, high for smooth).

  – **Content-Aware**: the selection is rebuilt (no AI) from the place you drag it to - **Structure** 1-7 (7 = only from there, its pattern kept; 1 = from round there too) and **Color** 0-10 (0 = its colours as they are; 10 = blended into the surroundings).
- **Content-Aware Move**: **Structure** 1-7 (how thin the blended edge round the moved part is: 7 = 1 px) and **Color** 0-10 (how much the moved part's colours follow its new place).

Sketch layers: typed sizes, the Measure tool, units, DXF / SVG files (2.464.0)

The rest of the CAD side of **sketch layers**: exact sizes while drawing, measuring, and files other CAD programs read.

- **Typed sizes while drawing**: once a shape has its first point, just **type a number** - a small box opens by the pointer. **Line**: Length, **Tab** = the Angle (degrees from level, anticlockwise); **Rectangle / Center Rectangle**: Width, Tab = Height; **Circle / 2-Point Circle**: Diameter; **Polygon**: Radius. **Enter** draws it - what you did not type follows the pointer (a line toward a level / upright pointer stays level / upright) - and the typed values are kept as **dimensions** (change them later like any other). A line chain goes on: type the next length straight away. Sums work (100/3). **Esc** closes the box.
- **Measure** (**M** with a sketch tool picked, or in the Sketch tab): click a point, line, circle, arc or spline = its sizes (length, angle, radius, diameter, circumference, area, sweep...); a second click = between the two (distance and ΔX / ΔY of two points, a point to a line or circle, the angle of two lines - parallel ones: how far apart -, two circles' centre distance and gap); a click **inside a closed region** = its **area** and **perimeter** (holes taken out; arcs exact). The numbers show in a box at the top right of the picture (copies them); Esc clears.
- **Units** (Sketch tab ▸ Measure + CAD files): px, or **mm / in** with "1 px = ..." - the Measure box and the files use them.
- **File ▸ Export Sketch as DXF…**: an R12 DXF any CAD, laser or CNC program opens - the sketch's origin is 0,0, y goes up, in the units chosen; construction lines (layer CONSTRUCTION) and dimensions (layer DIMENSIONS) if you want them; splines go out as polylines. **File ▸ Export Sketch as SVG…**: the picture's size (a real size in mm / in), the lines and (when the sketch fills them) the profiles. Both are kept in the project's **cad** folder (and downloaded).
- **File ▸ Import DXF…** / **Import DXF from Project…**: lines, circles, arcs, points, polylines (their curved bulges = arcs), splines and ellipses go into the picked sketch layer (or a new one); text, hatches and blocks are left out (it says how many). **Fit to the picture** or **real size round the origin** (1 drawing unit = so many px). Ends that meet are joined, so a closed outline is a profile at once (Fill profiles, Profiles → Shapes, Measure its area).

Sketch modify tools: trim, extend, fillet, offset, mirror, patterns, polygon, slot, splines (2.463.0)

More CAD tools for **sketch layers** (the **N** tools and the Sketch tab). Everything they make is constrained, so the sketch keeps solving.

- **More drawing tools**: **Polygon** (centre, then a corner; the number of **Sides** on the options bar - a dashed construction circle keeps its corners on it, its sides equal), **Slot** (the two centres, then the width - two half-circles joined by tangent lines round a construction axis), **2-Point Circle** (the two ends of a diameter), **3-Point Circle** (three points on it), **Spline** (click points; Enter or a double-click ends it - a smooth curve through every point; drag the points later with Sketch Select).
- **Trim** (**T**): point at a piece of a line, circle or arc - the piece between the nearest crossings / points turns red - and click to cut it away (a circle becomes an arc; the cut ends stay on what they crossed).
- **Extend** (**E**): click a line near its free end to run it on to the next line or circle in that direction.
- **Fillet** (**F**) / **Chamfer**: click a corner (where two lines meet) or the two lines - a tangent arc with a **radius** dimension, or a straight cut with its distance; the value on the options bar, then type to change it.
- **Modify (what is picked)** in the Sketch tab:
  - ⇉ **Offset…**: a copy at a distance (px) to one side; a joined chain stays joined, lines stay parallel, circles / arcs stay concentric - the distance is a dimension you can change.
  - ⇋ **Mirror**: pick the entities, then the mirror line last - mirrored copies held **Symmetric** about it.

  - ▦ **Rectangular Pattern…** (columns / rows and their gaps) and ✺ **Circular Pattern…** (how many, over how many degrees; round the picked point or the origin) - copies with the same constraints as the original.
- Each one is one undo step; profiles (⬚ Profiles → Shapes, Fill profiles) see the new regions at once.

Sketch layers: a CAD sketch with constraints and dimensions (2.462.0)

A **sketch layer** is a CAD-style 2D sketch: lines, rectangles, circles and arcs held in shape by **constraints** and **dimensions**. Change a dimension and the sketch follows; drag a point and everything stays as constrained.

- **Layer ▸ New Sketch** (or just draw with a sketch tool): a sketch layer (in Layers) with a fixed **origin** (the red point) and two construction axes through the middle of the picture.
- **The sketch tools** (**N** in the tool column, N again = the next one). With a sketch tool picked the letters pick the others: **S** Sketch Select, **L** Line, **R** 2-Point Rectangle (also Center Rectangle), **C** Circle, **A** 3-Point Arc, **P** Point, **D** Sketch Dimension; **X** turns what is picked into construction (dashed, never drawn in the picture); **Delete** removes what is picked; **Esc** stops / unpicks.
  - **Line**: click point after point (a chain); **Esc**, a double-click or the chain's first point ends it. Nearly level / upright lines snap and get **Horizontal / Vertical**; ending on a point joins them (coincident), on a line / circle keeps the point on it (a line across a rectangle, ends on its sides, cuts it into two regions).
  - **Rectangle**: two corners (or the centre and a corner) - click twice or drag. **Circle**: the centre, then the edge. **3-Point Arc**: the start, the end, then a point on the arc.
  - **Sketch Dimension**: click a line (its length - placed above / below a slanted one = its horizontal size, beside = its vertical size), two points (their distance), a point and a line, two lines (their angle; parallel ones = their distance), a circle (its diameter) or an arc (its radius); then click where the number goes and type the value (sums work: 100-20). Double-click a number later to change it.
  - **Sketch Select**: click to pick (Shift adds); drag points, lines and circle edges - the constraints hold; drag a number to move it.
- **The Sketch tab** (with Properties / Adjustments / AI / ◈ Control): how many **degrees of freedom** are left (✓ **fully constrained** when none), the **constraints** for what is picked - Coincident (two points, or a point on a line / circle), Collinear, Concentric, Midpoint, Fix, Parallel, Perpendicular, Horizontal, Vertical, Tangent, Equal, Symmetric (two points about a line) - Construction, Delete, the drawing tools, and how the layer looks in the picture (line colour + width, **Fill profiles** = the closed regions filled, **Dimensions** shown).
- **Colours while sketching**: blue = still free, black = fully constrained, orange dashed = construction, grey numbers in brackets = driven (reference) dimensions. A constraint that would **over-constrain** the sketch, or that is already set by the others, is refused (a dimension becomes a driven one instead).
- ⬚ **Profiles → Shapes**: every closed region as a **shape layer** above the sketch (fill, outline, generate into it…). ✓ **Finish Sketch** goes back to the Move tool.
- Each change is one undo step; moving / scaling the layer moves its sketch; a .bgi keeps every constraint.

Poses on the timeline, and the Daz live link (2.461.0)

On the timeline: a scene sent from Daz Studio (bitGen Bridge ▸ Scene) opens in the .blend preview over the preview canvas, where the camera already moves (scene camera / free view). Its Properties now have **Pose**:

- pick the **figure** and a **bone** (type part of its name to find it) and turn it with **X / Y / Z** (degrees, on top of its own pose - or of its animation: the turn rides on the motion). The preview turns it at once; **Reset bone / Reset pose**; Ctrl+Z undoes it. **Save** and **Render to real** give the pose to Blender (keyed over the frames when the figure is animated).
- **To the image editor**: the same scene as a **3D layer** of the image editor with this pose and this view (the free view's place) - there **Generate from it** makes a new image of it.

The Daz live link (experimental) - bitGen moves the camera and poses in Daz Studio itself:

- In Daz Studio run **Scripts ▸ bitGen ▸ bitGen Live Link** (from the bitgen-daz-bridge zip, 1.1.0). A small window stays open while it is linked; it asks bitGen about once a second what to do.
- In the image editor's **3D** tab: **Camera → Daz** (Daz's viewport looks through a "bitGen Camera" placed like the 3D layer's view), **Pose → Daz** (the same-named bones of the selected figure turn by your turns), **↺** takes the turns back, **☐ as I change** sends both on every change. The line above them says whether Daz is linked and which figure is selected there.
- On the timeline: **View + pose → Daz** in the Pose section (the free view + the turns).
- Render in Daz as usual and send it back with bitGen Bridge ▸ Render this frame -> a picture.
- It was written from Daz Studio's scripting documentation and could not be tried in a real Daz Studio when it was made; Daz's bones turn in Daz's own axes, which may not match Blender's. The live link window's log says what failed.

3D layers: a Daz / Blender scene in the image editor (2.460.0)

Bring a 3D scene into a picture, move its camera, pose its figures and use the view to make a new image.

- **Send it from Daz Studio**: Scripts ▸ bitGen ▸ bitGen Bridge ▸ **Scene** - the scene (figures with their rig, props, lights, cameras) arrives as a Blender scene in the project (Blender must be installed - Blender settings). Any **.blend** works too.
- **Layer ▸ New 3D Layer (Daz / Blender)…**: pick the scene (Daz sends are marked "Daz ·") or type the path of a .blend. A **3D layer** appears: an ordinary picture layer (masks, filters, moving and blend modes work) that Blender draws again whenever you change its view. The original files are never changed. With no picture open, one opens at the scene's size.
- **The 3D tab** (beside AI / ◈ Control):

  – **Camera**: **Scene camera** (the view Daz / Blender set), **Front / Back / Left / Right / Top**, or type **Turn**, **Tilt**, **Distance** and **Lens** (the field of view in degrees). **Drag the view**: on the canvas drag = orbit round the scene, **Shift**+drag = pan, **Ctrl**+drag or the **wheel** = zoom; a wire box shows the view while you drag and Blender draws it when you let go (about a second).

  – **Pose**: pick the **figure** and a **bone** (type part of its name to find it), then turn it with the **X / Y / Z** sliders (degrees, on top of the pose it came with). **Reset bone** / **Reset pose**.

  – **Frame**: which frame of an animated scene.

  – **Render**: **Quick** (fast, flat shading - what the view uses), **EEVEE** or **Cycles**, **☐ transparent** background, **▶ Render** draws the layer at its full size (Layer ▸ Render 3D Layer does the same).

  – **Generate from it**: type what you want in the prompt docker (e.g. "a photo of a woman waving in a sunny park"), then press it - the editor switches to **Edit** and the prompt docker makes a new image from the picture as you see it.
- Every change is **one undo step** (the picture and its camera / pose together). A **.bgi** keeps the 3D layer's scene, camera and pose.

Tools named by what they do (2.459.0)

The help pages, tooltips, menus, messages, this guide and the brochure now name every tool, panel and feature by what it does - no other products' names. A few names changed on screen: the **Node Pen Tool** (F5 group), the **Shape Tool (nodes)** (F10), the **Dockers** panel group (Window ▸ Float Dockers) and **Layer ▸ Frame**. Files from other programs are still read and written the same way (PSD, Final Cut Pro 7 XML, EDL, OpenTimelineIO, AAF).

Auto, floorplan and chain dimensions (2.458.0)

- **Auto Dimension** (Layer ▸ Auto Dimension, or ⊞ **Auto** on a dimension tool's bar): the picked object's overall **width** (above it) and **height** (beside it), as two dimension groups held in a group "Size of <name>".
- **Floorplan Dimensions** (Layer ▸ Floorplan Dimensions…, or ⌂ **Floorplan** on the bar): pick a closed shape - a room or the outline of a plan drawn with the shape / line tools - and every **wall** gets a dimension on its **outside**, plus the **overall width and height** further out; all in one group "Floorplan <name>", one undo step. The dialog sets the distance from the walls and whether you want the walls, the overall size or both. (A layer that is not a shape is measured by its box.)
- **The Chain Dimension tool** (F8, with the other dimension tools): click points along a wall - they snap to nodes and corners like the other dimension tools - then **Enter** or a **double-click**, then click where the dimension line goes: a dimension between every two points on one line, and the **total** further out. It runs horizontal or vertical the way the points run; **Shift** = horizontal, **Ctrl** = vertical. **Backspace** takes the last point back, **Esc** cancels.
- ↔ on the bar = how far auto / floorplan dimensions sit from the object. Every dimension they make is an ordinary dimension group: drag its parts, change the scale.

Dimension groups (2.457.0)

A dimension is now a **group of parts** in the Layers panel: **Extension Line 1**, **Extension Line 2** and the **Dimension Line** (its two arrowheads and its number on it). Each part is its own vector line; the group moves, scales and turns as one, and the parts are always drawn again from the dimension's measurements, so the arrowheads stay on the dimension line and the extension lines keep their gaps.

Drawing one (the Parallel and Horizontal / Vertical Dimension tools):

- **Press on an object's edge and drag away** - that edge is measured and the dimension line follows the pointer.
- Or **click a node or an edge, then another node or edge** (an edge gives the point on it square to the first pick), then **click where the dimension line goes**.
- **Shift** = the dimension line horizontal, **Ctrl** = vertical (while drawing and while dragging it). Esc cancels.

Editing one (the Move tool, the Shape tool, the path tools or a dimension tool, with the group picked):

- **Drag an extension line**: its measured point slides along the dimension - the length and the number change live (on a snap point it lines up with it; Alt = free).
- **Drag an extension line's nodes** (the squares at its two ends): that extension line alone gets longer or shorter - its gap at the object or how far it runs past the dimension line.
- **Drag the dimension line** (with the Move tool: by the round dot in its middle) - nearer or further; Shift / Ctrl make it horizontal / vertical. With the Move tool the rest of the line or the number moves the whole group.
- One undo step each. A part clicked on the canvas picks its group. The size + unit on the property bar and Layer ▸ Dimension Scale… work on a group; a new scale measures every group again. Older one-layer dimensions still open and drag by their ends (now Shift = horizontal, Ctrl = vertical there too).

The StoryBoard (2.456.0)

The **video docker is now the StoryBoard** (its header and the status bar button) and it **floats**: ⇱ in its header makes it a window of its own over the app - its title drags it, every edge sizes it, it docks as a tab in a named docker, ⇲ puts it back under the timeline.

- **The StoryBoard bar** (above the scenes): the **image model** the pictures are made with (the prompt docker's image style until you pick another), and what rides with each picture as **reference images**: ☑ **Character sheets** (of the characters the scene's prompt names), ☑ **Locations** (the locations it names), ☑ **Reference images** (the scene's own pictures and the project's References). **All missing** makes every scene's picture that is not there.
- **Each scene** has a **thumbnail**: **make** (↻ again later) renders it from the scene's prompt with that model - an ordinary image job (the Queue shows it); **→ First** / **→ Last** use it as the scene's **first / last frame** (its render starts / ends on that picture); ✕ removes it. The pictures are kept in the project's storyboard folder.

Edit theme mode leaves dialogs working (2.455.0)

While **Edit theme mode** is on, **dialogs and popup menus work normally**: the Style panel's "a picture as the icon" opens the file dialog and you can browse and pick in it; the image editor's dialogs, Settings, menus, Liquify … take your clicks and keys as usual. They are not outlined and never picked for styling. Esc inside a dialog is the dialog's; elsewhere Esc closes the Style panel, then ends the mode (also with the button focused).

The panel column is a docker; every tab group floats (2.454.0)

- **Every tab group** in the editor's right column floats with its ⇱: Colour (Color / Swatches / Character / Paragraph), Properties (Properties / Adjustments / AI / ◈ Control), Layers (Layers / Paths / Channels / History / Actions / Comps) and the Dockers (Transform / Shaping / Align / Object). Window ▸ Float … does the same.
- **The column is a docker with drop zones**: drag a floating group by its title over the column - an **orange line** shows where it will go (above or below a group, or at the end) - let go and it is docked there. A floating group can still dock as a tab in a named docker (Window ▸ New Docker…).
- **Resizable**: drag the column's left edge to make it wider or narrower (200 - 640 px).
- The order and the width are kept for next time.

Frames lock their contents (2.453.0)

- **Locked contents**: what is inside a frame (Lock Contents to Frame - on by default) cannot be moved or transformed by itself in normal mode. Clicking or dragging it with the Move tool takes the **frame**, and the frame's scale / rotate handles move, scale, turn and skew the contents with it (shown clipped to the frame while you drag; one undo step).
- **Right-click** on the canvas: **Edit Frame Contents**, Extract Contents, Lock Contents to Frame (+ Duplicate / Delete Layer). While editing, the contents move / transform / paint on their own and a right-click offers **Finish Editing Frame** first (the ✓ bar on the canvas still works too). On a plain closed shape the menu offers Place Inside Frame….
- A double-click on a frame now gives the Move tool's rotate handles (like any layer).

Move tool: scale / rotate handles, transform options (2.452.0)

With the **Move tool (V)**, the active layer has handles:

- **Click once** = the **scale handles** (8 squares): a corner scales both ways from the opposite corner - **Shift** keeps the proportions, **Alt** scales from the centre; a side handle stretches one way.
- **Double-click** = the **rotate handles**: drag a corner (curved arrow) to turn round the centre - **Shift** = 15° steps; drag a side handle (↔ / ↕) to skew. Click once again for the scale handles.

- While dragging you see where it goes; letting go applies it - one undo step. A drag anywhere else still moves the layer. Pixels are redrawn once, shapes stay vector, **type stays editable** (it can be turned now), smart objects keep their contents, a group transforms all of its layers.
- **Transform options** (the options bar): ∠ angle + ↻, 90 / 90, ⇋ / ⇵ flip, W % x H % (together) + - all round the layer's centre.

The editor's Edit / Generate tabs drive the prompt docker, Del, quick prompts (2.451.0)

- **AI ▸ Edit tab**: the prompt docker switches to **Image** by itself (even from Video), its style to **Qwen Image 2.1 edit**, and its big button reads ✏ **Edit** - type the instruction there and press it. The tab's own Edit button and model row are gone. Nothing open yet = it says to open a picture first.
- **AI ▸ Generate tab**: the prompt docker's button reads **Generate** and renders into the selection of the selected layer; ✔ Apply blends it in (like Krita AI). No picture open = a new picture, the usual way.
- Another tab (Refine, Remove …): the prompt docker goes back to the mode and style it had.
- **Del** deletes the current layer (with a selection it clears the selection; Shift+Del always deletes the layer).
- **Quick prompts** - in the prompt docker's menu bar: **Save** keeps the prompt (its first line names it; the same first line again replaces it), ⚡ **Quick prompts ▾** lists them - pick one to put it in the prompt box, deletes the one picked last. They are kept in quick_prompts.json in the install folder (every project).

Warp grid + split lines, Liquify extras (2.450.0)

Edit ▸ Transform ▸ Warp - the options bar now has:

- **Grid**: Default / 3 x 3 / 4 x 4 / 5 x 5 / Custom (columns x rows, 1 - 10 each) - more cells = finer control. The warp you already made is kept.
- **Split Warp ✚** Crosswise / **┃** Vertically / **━** Horizontally: click on the warp where the split line goes - the picture does not move, you just get more points there. **Alt-click** a split line to take it away.
- Drag a grid point (where lines cross - its handles go with it), a handle, or anywhere inside to bend that cell. Enter applies; a smart object keeps the grid.

Filter ▸ Liquify… gains:

- **Smooth (E)**: evens out the distortion under the brush (hold or brush over it).
- **Push Left (O)**: drag up = the pixels go left, down = right (Alt = the other way).
- **Face-Aware Liquify**: the ☺ **Face tool (A)** - drag an oval round a face; its eyes, nose and mouth are placed inside and can be dragged where they really are. Then the sliders: Eye Size / Height / Width / Distance, Nose Height / Width, Smile, Mouth Width / Height, Forehead, Chin Height, Jawline, Face Width (-100 .. 100). Reset Face puts them back; OK applies the brushes and the sliders together. No face placed yet = a guess in the middle.

Per-character type (2.449.0)

Different letters in ONE text layer can have their own settings:

- While typing (the Type tool's box is open), **select some letters**, then change the **font, B, I, size, tracking or colour** in the options bar - or anything in the **Character** panel (vertical / horizontal scale, baseline shift, faux bold / italic, All Caps, Small Caps, superscript / subscript, underline, strikethrough). Only those letters change; the typing box stays open with them still picked. Each change is one undo step.
- **Nothing selected** (just the caret) = the whole layer, as before.
- Typing inside a styled run takes its style (the letter before the caret's).

- Letters of different sizes share one baseline; each line is as tall as its largest letters. When sizes / fonts differ, the typing box floats above the text (like warped text) so the letters can be picked reliably.
- **Layer ▸ Type: Clear Character Overrides** puts every letter back to the layer's own settings.

The layer keeps the paragraph settings (alignment, leading, the paragraph box). Text on a path, vertical, warped, 3D and text-effect-preset text keep one style.

The Content-Aware Fill workspace (2.448.0)

Select what to remove, then **Edit ▸ Content-Aware Fill…** - a workspace of its own:

- **The picture**: what is filled outlined in red, the **sampling area** (where the fill may take pixels from) in green.
- **Sampling**: **Auto** (a ring round the selection), **Rectangular** (the box round it) or **Custom** - paint it with the **Sampling Brush**: **+** adds, **−** (or Alt) takes away, its size. **↺ Reset** = Auto again; **show it** turns the green off.
- **Preview**: the result next to it, again after each change (after a short pause).
- **Colour Adaptation**: None / Default / High / Very High - the fill's colours and light blended into its surroundings.
- **Output To**: Current Layer / New Layer / Duplicate Layer. **OK** = one undo step. The choices are kept.

PatchMatch (no AI): only patches from the sampling area are used.

Artboards (2.447.0)

Several **artboards** in one picture - each a page of its own size (a post, a story, a banner …).

- **Layer ▸ New ▸ Artboard…**: a name, a size (the picture's, a preset - Instagram post, Story / Reel, Full HD, 4K, A4, YouTube thumbnail, X post - or your own), its background colour (or none), and whether the picked / active layers go into it. The first sits at the picture's top-left; the next ones to the right of the last, and the canvas grows to hold them.
- **The Artboard tool** (in the Move tool's group - press V again): drag a rectangle = a new artboard there.
- **Layer ▸ New ▸ Artboard from Group**: the active group becomes an artboard round what is in it.
- An artboard is a group: its layers show **only inside its rectangle**, over its background; its name and outline are drawn above it. Moving it (Move tool, arrows) moves its layers with it. **Properties**: X / Y / W / H, the background, ⇱ **Fit** (the rectangle round what is in it). Canvas Size and Image Size move / scale the artboards too.
- **File ▸ Export ▸ Artboards to Files…**: each artboard as its own PNG (only its own layers) - into the project's Images as <picture>-<artboard>.png, and / or downloaded. A .psd keeps them as PSD artboards; a .bgi keeps them.

Rulers, guidelines and snapping on the timeline's preview canvas (2.446.0)

in the timeline's menu bar (beside ⊞) shows **rulers** along the top and the left of the preview canvas - in the picture's pixels from its top-left corner, following the zoom and the pan.

- **A guideline**: drag from the top ruler (a horizontal one) or the left ruler (a vertical one) onto the canvas.
- **Move it** with the Move tool (the pointer turns into ↕ / ↔ over it); **drop it back onto its ruler** to remove it; **double-click it** to type its place. Guidelines belong to the timeline (saved with it) and are drawn in the guideline colour (⚙ Timeline settings ▸ Grids & Guides).

- **Snapping**: a layer moved with the Move tool snaps its edges and its centre to the guidelines, the picture's edges and centre, and - while the grid shows (⊞) - the grid's lines, within the snap distance. Hold **Alt** to move freely.

Grid buttons, the Grids & Guides settings (2.445.0)

⊞ turns the grid on and off - on the image editor's top bar (before) and in the timeline's menu bar (beside , the grid over the preview canvas).

The settings are a **Grids & Guides** tab - in the image editor's ⚙ settings and in the timeline's ⚙ **Timeline settings**:

- **Grid spacing** (picture pixels; 0 = it follows the zoom) and **Major grid every** n lines (drawn stronger; 0 = none)
- **Draw the grid as** lines or dots (a dot at each crossing, bigger on the major ones)
- **Grid colour** and how strong it is, **Guideline colour**, **Guideline snap distance** (screen pixels)

The editor keeps them with its other settings (View ▸ Grid and Snapping… opens the tab); the timeline keeps them in the settings file (the same for every browser), the ⊞ on / off too.

The image editor's Edit / Generate run from the prompt docker, ⚙ editor settings (2.444.0)

With the image editor open and its **AI** panel showing:

- **Edit** tab: the prompt docker's image style switches to **Qwen Image 2.1 edit** and its button reads ✏ **Edit**. Type the instruction in the prompt docker ("make her jacket red", "turn it into a watercolor") and press ✏ Edit - the selection is edited (no selection = the whole picture). The panel's own Edit button, model list and prompt box are gone - the prompt docker is where you type. Leaving the Edit tab puts the prompt docker's style back.
- **Generate** tab: the prompt docker's button reads **Generate** and renders INTO THE SELECTION OF THE SELECTED LAYER (no selection = the whole picture). The results come in as before (click one to compare); ✔ **Apply** blends the shown one into that layer - only inside the (feathered) selection, one undo step - like Krita's AI. On a text / shape / group layer it becomes a new layer. (⚙ ▸ AI can make Apply always add a new layer.)
- Another panel tab showing (or the editor closed): the prompt docker works as before (a picture onto the canvas as a new layer).

⚙ at the right end of the editor's menu bar (or Edit ▸ Preferences…, Ctrl+,) opens the editor's settings: AI (results, context, feather, megapixels, what ✔ Apply does) and Interface (the look, rulers, panels, the options bar).

Edit theme mode: style anything on screen (2.443.0)

in the top bar (next to ⚙ - or Settings ▸ Themes ▸ **Edit theme mode**) turns on EDIT THEME MODE: the element under the pointer is outlined, and a click PICKS it instead of using it (⚙ does not open Settings, a button does not run). The **Style** panel opens beside it:

- **Apply to** - this element only, all like it in this panel, all like it (the same kind with the same classes - every ghost button, say), or every element of its kind; the number it matches now is shown.
- **Text** - font, size, weight, colour, text shadow (x, y, blur, colour).
- **Box** - fill, border colour, corner radius, button shadow (x, y, blur, spread, colour, inset).
- **Icon** - a new icon (an emoji / a symbol, or a small picture - png, svg, jpeg, webp, gif), its position (left, right, above, below, icon only, hidden), its size and its fill (an emoji is tinted too). The words stay as they are.
- ⬆ picks the element around it, ↺ **Clear** removes the rule; drag the panel by its title.

Every change shows at once. **again** (or Esc twice) = LIVE mode: clicks work again and the styles stay. They are kept in this browser and with the theme: **Save theme** writes them into themes<name>.json, loading / importing a theme brings its own. Settings ▸ Themes lists the rules (× each, ↺ **Clear element styles**). In a dialog that is open on top, the panel opens inside it.

Frames: layers inside a shape, Edit Frame, generate into a shape (2.442.0)

A frame is any closed shape - a rectangle, an ellipse, a polygon, a closed path - with layers INSIDE it: they show only within its outline (its fill behind them, its outline on top), and in Layers they are a tree under the shape (▾ opens / closes it).

- **Put layers in**: Ctrl-click the layers in Layers (pick them), make the shape active, then **Layer ▸ Frame ▸ Place Inside Frame…** - the dialog shows the frame (any closed shape can be chosen) and can centre the contents in it. Or drag a layer onto the MIDDLE of a closed shape's row in Layers.
- **The frame moves as one**: move, turn, scale or skew the frame (the Move tool, the property bar, the Transform docker, the path tools) - everything inside moves, turns and scales with it (live while dragging; one undo step). **Lock Contents to Frame** off = the frame moves alone.
- **Edit Frame Contents** (double-click the frame with the Move tool, or the menu): the contents show whole (what is outside the frame faint) and each one can be moved, transformed, painted like any layer; the bar on the canvas - ✓ **Finish Editing Frame** - goes back to normal mode. In normal mode a click on the contents picks the frame.
- **Extract Contents**: the layers come out, above the frame, where they are.
- **Generate into a shape**: with a closed shape active, AI ▸ Generate makes the picture in the shape's box and ✔ Apply puts it INSIDE the shape (a frame). Layer ▸ Frame ▸ Generate Into the Picked Shape turns this off.
- A frame goes to a .psd as a group (the shape + its contents clipped to it) and stays a frame in the .bgi.

CAD dimensions: exact points, live ends, a scale (2.441.0)

Exact points: with a dimension tool (Parallel, Horizontal / Vertical, Angular) the pointer snaps like a CAD program - to a shape's nodes and the middles of its sides, the corners and side middles of every layer (a picture's too), the nearest point on a shape's outline, guideline crossings - the point it takes is marked and named while you hover (Alt = no snapping).

Drag an end: pick a dimension with the Move tool (or the Shape tool / the path selection tools) - its two ends and its line show handles. Drag an end: the dimension and its number follow live. **Shift** keeps the end level with the other end (horizontal), **Ctrl** straight above / below it (vertical) - since 2.457.0. The middle handle moves the dimension line. One undo step each. (With a dimension tool a click on an end starts a NEW dimension there.)

A scale: the property bar of a picked dimension shows its size and a unit. Type the real size and pick the unit (mm, cm, m, km, in, ft, yd, mi): that is the picture's **scale** - every dimension in it is measured again with it, and the ones you draw next use it too. ⚖ (or Layer ▸ Dimension Scale…) sets the decimals or clears the scale. A picture keeps its scale (in its .bgi too); each picture has its own.

Rulers, guidelines and snapping (2.440.0)

Rulers (View ▸ Rulers, Ctrl+R) count in the unit of **View ▸ Ruler Units**: pixels, millimetres, centimetres, inches or points (at the document dpi set in View ▸ Grid and Snapping…).

Guidelines: drag one out of a ruler - out of the top ruler a horizontal one, out of the left ruler a vertical one. Move one with the **Move tool** (or hold Ctrl with any tool); drag it back onto a ruler (or off the picture) to remove it. View ▸ Guidelines (Ctrl+;) shows / hides them, **Lock Guidelines** (Alt+Ctrl+;) keeps them where they are, **New Guideline…** places one by typing its position,

Guidelines… lists every one (change a position, delete, add, clear all), **Clear Guidelines**. A document keeps its guidelines (in its .bgi too). Double-click a ruler for the list.

Snapping - View ▸ Snap To: **Snap** (Shift+Ctrl+;) switches it on / off, and what to snap to: **Grid**, **Guidelines**, **Objects** (the other layers' edges and centres), **Nodes** (a shape's points), **Midpoints** (of its sides), **Intersections** (where guidelines cross, with the page edges), **Page** (its edges and centre). At first: guidelines, their crossings and nodes. The drawing, selection and path tools (the marquees, Crop, the Pens, the Shape tools, the vector lines and shapes, dimensions, connectors) snap the pointer; the **Move tool** snaps the layer's box - its left / centre / right and top / middle / bottom - onto what is near. What it snapped to is shown at the point (guideline, node, midpoint, centre, edge, intersection, grid). **Grid and Snapping…**: the snap distance (screen pixels), a fixed grid spacing (0 = the grid as drawn at this zoom), the document dpi, the guideline colour.

The property bar, corner radii, Add / Subtract into a shape, line width and style (2.439.0)

The image editor has a second bar under the tool's options - the **property bar** of the picked layer: **X, Y, W, H** in pixels (its box) - type a value and press Enter; keeps the proportions (a new width changes the height with it, and back), changes one side only. Each change is one undo step; a vector shape stays vector (its corners and outline stay sharp), text is resized, pixels are resampled.

A **rectangle** (a Rectangle-tool rectangle, F6, or a Shape-tool rectangle, U) shows its four **corner radii** ⌜ ⌝ ⌟ ⌞ - changes all four together, unlinked each corner is its own. A **vector shape or line** shows its **outline width** (type it, or pick from the list: none, hairline, 1 … 24 px) and its **line style**, picked from a picture of each (solid, dashed, dotted, dash-dot …; Custom… opens the Outline Pen, F12).

The F6 shape tools (Rectangle, Ellipse, Polygon, Star …) have a **draw mode** at the start of their bar: □ **New** (each shape a layer of its own), ⊕ **Add**, ⊖ **Subtract**, ⊗ **Intersect**, ⊘ **Exclude**. In a mode other than New, a shape drawn while a vector shape is picked becomes a part of that shape - added to it, cut out of it … - and the shape it already was stays exactly as it was (both parts can still be edited as nodes; undo takes the new part away again).

Floating dockers, named dockers, dialogs in the middle (2.438.0)

The image editor's panel groups float: ⇱ at the end of the Layers group's tabs (Layers / Paths / Channels / History / Actions / Comps) or the Colour group's (Color / Swatches / Character / Paragraph) - also Window ▸ Float Layers Panels / Float Colour Panels. The group becomes a window of its own inside the editor: drag it by its title, size it by any edge or corner; ⇲ or ✕ puts it back in the right column (Window ▸ Panels Back in the Column puts both back).

Named dockers: Window ▸ New Docker… in the image editor, or ⊞ Docker layout ▸ ▣ New docker… in the menu bar - an empty floating docker with your name for it (double-click the name to rename). Drag any floating docker by its title over it - it lights up - and let go: it is docked there as a tab (click a tab to show it; its ⇱, or dragging the tab away, floats it again). Named dockers dock in named dockers too. ⚙ on a named docker: Rename…, **Main docker** (it floats over the whole app) or **Confined to** a docker (the image editor, the timeline, another named docker … - it stays inside that docker and hides with it), Delete (what is in it goes back where it came from). Where everything is - floating, docked, sizes, names, tabs - is kept in **panel_layout.cfg** in the install folder.

▣ **Docker - an empty docker the others drop into** (2.545.0): ▣ **Docker** in the statusbar (after Queue) or **Ctrl+Alt+D** shows / hides it. Float a main docker with ⇱ in its title bar (Preview, Timeline, Image editor, ComfyUI, Music, Stems, SFX, Chat, Console; the StoryBoard has its own ⇱), then drag it by its title over ▣ Docker - an orange line shows where it will go - and let go.

- **Side by side or stacked**: ⇆ in its title bar = side by side (horizontal), click = ⇅ stacked (vertical); ⚙ ▸ Layout also has **Tabs** (one at a time, as named dockers always were). Every named docker can do this.
- Side by side / stacked, each docker in it keeps a small title bar: drag it out (or ⇱) to float it, ⇲ puts it back in the main window. Drag the bar between two to size them.

- **Double-click the title** to rename it in place - Enter keeps it, Esc leaves it as it was.
- What is in it, the order, the sizes, the layout and the name are kept in panel_layout.cfg.
- 2.549.0: **drag a main docker in by its own title bar** - press on Music's (or Preview's, Timeline's …) title and drag it over the ▣ docker; a tag follows the pointer and the orange line shows where it lands. A plain click still folds the docker. (The ⇱ on the title bars was an empty button since 2.545.0 - it shows ⇱ again.)
- 2.549.0: the statusbar button carries the docker's **name** (▣ Audio). Its title bar has **Make permanent** - it becomes a regular docker in the main column with the regular title bar (▾ fold, ⇱ float it again, , maximize), its statusbar button shows / hides it, the grip under it sets its height, floating dockers still drop into it - and **Delete** (asks first; the dockers in it go back where they came from, the docker and its button go). Ctrl+Alt+D then makes a new one.
- 2.550.0: **right-click a ▣ docker's title bar** (floating, or made permanent) for Rename…, Side by side / Stacked / Tabs, Make permanent or ⇱ Float it again, and Delete docker. Only the dockers you made have it. A permanent docker goes where the window was (before the main docker under its middle) and takes room in the column (it shrinks with the others down to 180 px - it was added at the bottom and could sit below the room the column has).
- 2.558.0: a floating ▣ docker no longer sits **above every other docker**: it is one of the app's floating dockers (with the floating prompt docker, Basic Settings and LLM tokens), and the one you clicked last comes to the front. While you drag a docker by its title bar every ▣ docker shows a **dashed orange outline**, so you can see where to drop it.
- 2.558.0: more dockers go in: **Settings, References, Assets, Queue and Gallery** (the side columns - press on their title, not their ▾ /) and the **StoryBoard** are dragged in by their title bar like the others. A side docker's column closes while it is in a ▣ docker, and ⇲ puts it back in its own place with its column open again. (The prompt docker, Basic Settings and LLM tokens dock with their own drop zones and do not go into a ▣ docker yet.)
- 2.559.0: every **folder filter** remembers the folder you picked until you pick another - the ComfyUI model dropdowns' folder filter (Settings, Basic Settings, the LoRA rows - each dropdown its own), the model browser, the character dialogs' LoRA folder and the Model manager's folder tabs (each tab its own). Kept in **folder_filters.cfg** in the install folder, so they survive closing and restarting the app.
- 2.559.0: **Basic Settings are kept in basic.cfg** in the install folder: every change (by hand, or by a style or a project) is written to it a moment later, and the app reads it back when it starts (after the open project's own settings), so you start with the Basic Settings you last had. The style card's model dropdowns are not in it (they save into the style). To start Basic Settings from scratch, close the app and delete basic.cfg.
- 2.560.0: a ▣ docker **docks like the other dockers**: drag a floating ▣ docker by its title bar near the edge of a docker

  – the same blue drop zone as Basic Settings shows ("Dock Docker above Preview") - and let go: it docks above / below a docker, left / right of it (side by side), or at the top / bottom of a side docker (Settings, References, Assets, Queue, Gallery). Its edge sizes it; drag its title bar away and it floats again (and can be dropped on another edge at once). Where it is docked comes back at the next start.
- 2.560.0: **Basic Settings, LLM tokens and the prompt docker go into a ▣ docker**: drag them by their title bar over a ▣ docker (floating or docked) - it is outlined and an orange line shows where they go - and let go. Drag their small title bar (or their tab) out of the ▣ docker, or press ⇲, and they float on their own again. They come back in their ▣ docker at the next start. (Near a docked ▣ docker's edge they dock beside it instead, as before.)
- 2.561.0: a ▣ docker can become a **column of its own**: drag it by its title bar onto the seam **between two columns** (Settings | References | Assets | the main column | Queue | Gallery - or the window's left / right edge). A tall blue strip shows ("New column: Docker between References and Assets"); let go and it is a full-height column there, with its own splitter to size its width. ▾ folds it to a narrow strip, its statusbar button hides it, and dragging its title bar away floats it again. The column, its place and its width come back at the next start.

- 2.562.0: a StoryBoard scene whose render (or upscale, or ✎ First frame) job was **interrupted by a restart** now says "interrupted by a restart - ↻ Resume it in the Queue docker (Pending tab) and it carries on here" and remembers the job. Press ▶ **Resume** on that job in the Queue docker's Pending tab and the scene takes it back: it shows as running again with its progress bar, and the take lands in the scene when it finishes. (Discard the job instead and the scene says so.)
- 2.562.0: **nothing plays by itself** - a finished job, a take opened from the Done tab or the timeline, the running job's joined preview, the lightbox, the StoryBoard's Watch and the swap dialog's preview load their video or song paused; press ▶ (or Space) to play.
- 2.563.0: **a scene's pictures can come from the other scenes**: the picture picker (on a scene, ✎ First frame's / / , and the First / Last frame slots) opens on **Scenes** - every other scene's own pictures under its number (its First / Last frames, its reference pictures, its ✎ pictures) - and **Clips** takes a frame of any rendered take, for a scene's reference pictures too. A picked picture says where it came from ("scene 1 · First frame").
- 2.563.0: ✎ **First frame ▸ Picture A**: pick the picture the scene starts from and set *Start from* to Picture A. Its style, composition, camera framing and the people's poses are kept exactly; the place is the location picture if you pick one, else the location the scene (or the board's global prompt) names, with its view picture, else Picture A's own place. **Write** has the LLM write the instruction (it names the pictures as <image1>, <image2> …) - edit it if you like; ✎ Make first frame with Picture A and an empty direction writes it first. The edit style picked in the accordion makes the frame.
- 2.564.0: **Voice chat - talk to the app.** Press **Voice** in the statusbar (or **Ctrl+Alt+V**) and just talk - there is nothing to hold: it hears when you start and stop speaking, writes down what you said (faster-whisper on this computer, kept loaded while you talk), and the local LLM answers out loud (your computer's own voices) and DOES what you asked: write or change the prompt ("write me a prompt of a fox in the snow"), the negative prompt, pick a style, change Basic Settings ("make it eight seconds", "720p", "16:9"), what Generate makes, the video mode, press Generate, make a still, stop the running job, add or render a StoryBoard scene, show a docker. What it did is listed under its answer. While it speaks it does not listen. pauses listening, / its voice (and which voice), the box at the bottom lets you type instead, ✕ (or saying goodbye) ends it. It needs faster-whisper (Settings ▸ Audio) and an LLM (Settings ▸ LLM). 2.564.0 also fixes ✎ Write (2.563.0) and the phone's chat, which showed "[object Object]" instead of the LLM's words.
- 2.565.0: **the StoryBoard is a storyboard**: ▦ (the new default) shows every scene as a card like the Model manager's - its storyboard picture (else its First frame), "Scene n", its state and its text - in a grid with an ➜ from each scene to the next. On a card: makes its picture (↻ another), ▶ renders it, → First starts it on the picture. Click a card and its full scene card opens under the grid with everything it had (the prompt, Ref / First / Cont / Trans, the frames, its pictures, ✎ First frame, the filmstrip …); ✕ (or "All scenes" in the picker) closes it. shows the list as before.
- 2.565.0: **Write storyboard** (the StoryBoard bar): give it a brief - the LLM writes the scenes (your characters, the ticked references) and every scene gets its storyboard picture, made with the bar's image model and references (☑ Character sheets / Locations / Reference images). With ☑ **Start on it** each picture becomes its scene's FIRST frame (a Ref scene turns First) - change it with ↻ or the scene's First frame picker. The writer needs at least one character.
- 2.566.0: **MOSS-Image** (OpenMOSS's open 5B image model, Apache-2.0 - strong at text in pictures and layouts) is a model type: the image styles **MOSS-Image** (SFT - the one to use) and **MOSS-Image Base** (for LoRAs). **Settings ▸ Add-ons ▸ MOSS-Image** downloads its files from Comfy-Org/MOSS-Image into ComfyUI's folders - tick SFT / Base (int8, 5.4 GB; bf16 for more precision), the text encoder (Qwen3.5 9B) and the VAE are always included - and puts the styles back if you deleted them. It needs ComfyUI from 16 Oct 2026 or later (the add-on checks the running ComfyUI and tells you to update it if it is older); about 14 GB of VRAM with the int8 files. 25 steps, cfg 2.5 (ComfyUI's own template).
- 2.567.0: the **theme editor** (Edit theme mode ▸ the Style panel) can line text up and space things out: **Text ▸ Align** ⇤ ↔ ⇥ (left / centre / right / justified) and ↕ (top / middle / bottom) - it works on buttons, rows and columns of items and blocks of words alike; **Text ▸ Padding / Margin** = the space around the words themselves (plain-text elements); **Box ▸ Corners** rounds each corner on its own (↖ ↗ ↘ ↙, over Radius); **Box ▸ Padding / Margin** = the space inside / outside the

element's edge. Four boxes each (top, right, bottom, left); = copies the first value to all four; – (or an empty box) = its own. Saved with the theme like the rest.

- 2.568.0: the **StoryBoard's pictures are written by the LLM**: sends the scene back to the LLM, which writes the picture's prompt - the scene's opening frame (who, what, where, framing, light), naming the reference pictures. The references are the **character sheets** and the **last frame of the scene before** (when that scene has a rendered take - the picture continues from that moment); with no take before it, it is the characters + the scene's words. ☑ LLM writes it / ☑ Previous last frame on the StoryBoard bar (both on). **changes a picture** - a picture of another scene, a frame of a take, any project picture or a file (with ☑ Start on it, it becomes the scene's First frame too). In the grid **the picture is the whole card**: the scene's name and two lines of its text over the bottom, the buttons on hover; the tooltip holds the full text and the picture's prompt; a click opens the scene's full card.
- 2.569.0: **Zones in the image editor** - boxes on the canvas that say what the AI puts where ("a kite" here, "a dog" there, "a bench" there). AI panel ▸ Zones ▸ **Draw boxes** (the Zones tool, with the marquees): drag on the canvas for a box (each gets its colour and number), drag a box to move it, its edges / corners to size it, Delete removes the picked one; type what goes in each box in the list. Optional: the scene (the whole picture). Then pick HOW and **Generate zones**: ▣ **One box at a time** - a Generate (Add content) per box with the fill style, all queued at once into Pending; ☑ Apply as they come puts each box's first result in as its own layer. ✎ **Edit model, one pass** - the picture with the boxes outlined in their colours goes to the Edit model once ("Put a kite inside the red box …, remove the outlines"): the light matches best, the place is looser. ▦ **Regional prompt, one pass** - a NEW picture from the boxes with a ComfyUI image style: the scene's words over the whole, each box's words only inside it (ComfyUI's area conditioning - best with SDXL-type styles; FLUX / Qwen follow it more loosely). A picture starts on ▣, a blank canvas on ▦. The boxes are kept with the document (and its .bgi) and are never painted into the picture.
- 2.570.0: **Pen tablet** (XP-Pen, Wacom, Huion … through Windows Ink) in the image editor - the **Pen** tab next to Brush (Window ▸ Pen Tablet): a **test pad** that shows the pressure, the tilt, which end of the pen and the **pressure levels** your pen sends (~8192 for an XP-Pen); the **pressure curve** like the driver's (Extra soft, Soft, Linear, Firm, Extra firm, S-curve - or drag its points), **Lightest touch** and **Full pressure at**; ☐ **Pressure → size** / ☐ **Pressure → opacity** for every brush (and its smallest size); ☑ the **eraser end** erases (the brush comes back when you turn the pen round); the **side button** = Eyedropper; ☑ **tilt** turns the tip (Brush ▸ Shape Dynamics ▸ Angle / Roundness control: Pen tilt); ☐ a mouse: pressure from speed. Seven **pen brushes** in the brush presets ▾: G-pen, Mapping pen, Brush pen, Real pencil, Fineliner, Technical pen, Calligraphy (a 45° nib that turns with the tilt). No pressure? XP-Pen's Pen Tablet settings ▸ tick **Windows Ink**, then restart the app.
- 2.571.0: **select several clips and drag them all onto the timeline** - in the Gallery, the Assets dock and the References tray: **Ctrl-click** adds / takes away one, **Shift-click** selects the range from the last one you clicked (Ctrl + Shift adds a range); a plain click works as before (one tile; the Gallery opens it in the viewer). Drag any of the selected and all of them go: dropped on the timeline (or the preview), they are laid down **oldest first** - the oldest on the top layer, the newest at the bottom, each on its own layer, one after the other from where you dropped them. **Sequential** on the timeline's bar (next to ripple) arranges every clip one after the other with no gap, down the layers from the top (V1, the picture layers, the video layers), and keeps it that way while it is on: delete a clip and the ones below close up; put in a layer with a clip and the ones below make room; trim a clip and they follow. Sound layers, text, shapes and the karaoke layer stay where they are.
- 2.572.0: **Auto transition [N] frames [type]** on the timeline's bar - at every cut between two video clips on different layers the next clip comes in over N frames (dissolve, fade, wipes, slides, circle, zoom …) **without moving anything**: every clip keeps its start, so the lip sync with the song stays frame-exact. The clip before the cut is held underneath for those frames (its own frames past its end when the file has them, else its last frame held). On = kept at every change (after Sequential); off = plain cuts again; a transition you set yourself is left alone. Clips that are all on V1 use V1's own join (Out), which overlaps them. / **Video sound** mutes or unmutes V1 and every video layer at once, so the full song on its sound layer plays with no cuts in it.

- 2.573.0: ✂ **cut duration** on the StoryBoard's song strip (the audio clip's menu bar, before Transition): the seconds the song is auto-cut into (2-60, presets 10 / 15 / 20 / 29.9). A change re-cuts the open song at once - segments you locked by typing a duration stay where they are - and the value is remembered for the next song; saves the .seg and the scenes as before. 29.9 keeps every scene inside ONE Long video run (⚙ Long video, MiniMax H3: 719 frames); 30.0 is 720 frames and would render as two joined runs.
- 2.573.0: **ComfyUI updates** (Settings ▸ ComfyUI ▸ Engine, under ▶ Start / ■ Stop): this ComfyUI's version and kind of install, the channel (Newest = master, what the portable build's update_comfyui.bat does - new models such as FastH3 Omni arrive there first; Stable = the newest release), Check and Update ComfyUI. The portable build runs ComfyUI's own updateupdate.py (it installs the new requirements itself); a git clone is fast-forwarded (or moved to the newest release tag) and pip installs requirements.txt. A ComfyUI bitGen Studio started is stopped first and started again; one started some other way has to be closed first. Node packs are not touched.
- 2.573.0: **the update check on startup** - 20 s after the app starts it asks GitHub about bitGen Studio AND ComfyUI; something newer puts a ⬆ in the status bar (a click opens the right Settings tab) and a small card once per new version (Open / Later / Don't check on startup). Settings ▸ General ▸ Updates ▸ "Check for updates on startup" turns it off or on. Offline = nothing at all.
- 2.573.0: **FastH3 Omni** - FastVideo's 8-step distillation of MiniMax H3's reference-to-video model (pictures, videos and voices in, video with sound out, several times faster). Settings ▸ Add-ons ▸ FastH3 Omni downloads it (22 GB int8; H3's text encoder and VAEs only when missing); ComfyUI 0.43 or later (Update ComfyUI, Newest). Use it as the video style **FastH3 Omni**, or the StoryBoard's Workflow ▸ **FastH3 Omni · ref2va 8-step** (renders like H3_Video_Ref: every scene ref2va, scene n carries scene n-1's take). Any H3 graph whose model is a FastH3 file (Basic Settings ▸ Override style ▸ Model) gets its recipe automatically: sigma shift 12/3, VSA sparse attention, 8 steps, cfg 1, the base model's turbo LoRA taken out; Sage / Spectrum / Veda are skipped (they would replace VSA). Omni is a reference-to-video model - First / Cont / Trans (fl2va) scenes should stay on H3's own fl2va model.
- 2.574.0: **theme edit mode ▸ Text on any words** - a docker's title bar (pick it: Text shows its title, aimed at that docker only) and any element that has words of its own beside buttons or boxes (a label with its checkbox) can be given new words; the ▾ / ▸ caret, the and the buttons stay, a collapse keeps the new title, ↺ Clear brings the old one back. Elements with no words of their own say so - pick the words themselves.
- 2.574.1: **Apply to ▸ This element only** is the first choice for every element you pick in theme edit mode, and the default - each pick starts on it (the panel no longer keeps the last element's Apply to). New words always belong to the picked element only: with a wider Apply to (All like it, Every) the rest of the style goes wide but the words stay on that one element. A text rule that finds several titles, or a bare tag (div, button), never renames them - a theme where every docker took one name heals by itself; rename the docker you meant again.
- 2.575.0: **Model manager - your own files of a model**: on a card's page, **Add...** pictures, videos or sounds (or drop them there from the gallery / Assets). Each one keeps the prompt, negative, seed, cfg and steps it was made with - read from this app's own render record, a PNG's ComfyUI graph or A1111 text, or a video's / song's tags - and ✎ edits them. A click on any picture (yours, Civitai's examples, the hero thumbnail) opens it whole in the viewer; ✎→ sends its prompt with its seed, cfg and steps to the prompt box and Basic Settings; makes it the card's thumbnail (or ✎ Edit ▸ Thumbnail ▸ Change... a picture from disk; ↺ Its own goes back). Kept in model_info with the model's page - the model folders are never touched.
- 2.575.0: ✎ **Edit card**: **Your rating** is five stars in half steps - hover to see, click (the left half of a star = ½), ← → by halves, × clears; **Model type** (Checkpoint, Diffusion model, LoRA, LyCORIS, VAE, Text encoder, ControlNet, Upscaler, Embedding ...); **NSFW** . The filter bar has **every model type** and **NSFW** (unticked = NSFW cards hidden, remembered).
- 2.575.0: ✎→ **To prompt** on gallery pictures / videos / songs, Assets, References (pictures, videos, sounds) and in the viewer's details: the file's prompt with its seed, cfg and steps (an image: Basic Settings ▸ Image's steps / cfg; a video: Basic Settings' steps / cfg; a song: the Music docker's lyrics and seed). A note at the bottom says what was set.

- 2.575.0: **The Civitai sync in the status bar**: " Civitai 12/80" while Civitai info / Sync runs (a click opens the model manager), then "✓ Civitai synced: … found · … no match" for 30 s.
- 2.575.0: **Dockers come back as they were left**: a docker put into a ▣ docker keeps its place, its tab order and the shown tab, and its status (open / collapsed / hidden) over a restart; minimized floating panels stay minimized; extension dockers reopen open or closed as left; a layout change made just before closing is saved; a saved ⊞ docker config no longer overrides the restored layout while the app starts.
- 2.576.0: **Sync finds renamed Civitai models**: a model its file name does not find is looked up by its SHA-256 (the file's content - exact, whatever it is called). Reading a big file takes a while; the status bar says " Civitai 3/9 · reading 45%". Models that were "no match" before are asked again on the next Sync, once.
- 2.576.0: **To model… in the viewer**: sends the picture, video or song you are looking at to a model's files (model_info, with its prompt, seed, cfg and steps). The model it was made with is picked first; tick **Make it the model's thumbnail** (ticked for you when the model has none).
- 2.576.0: **Ctrl+↓ = the model's thumbnail**: in the viewer on one of a model's own pictures (or a Civitai example) it becomes the thumbnail at once; on any other picture the picker opens with the thumbnail box ticked. In the model manager's page, click a picture / example, then Ctrl+↓.
- 2.577.0: **Your own pose sheets**: the six built-in pose templates are gone. Save your pose sheets (PNG, JPG or WebP; sub-folders are fine) in the **pose_sheets** folder of the install folder. In the Character creator, **Pose sheet** (where the Pose dropdown was) opens their thumbnails: click one and ✔ **Use** (or double-click it); **No pose sheet** = a natural standing stance; **Open folder** shows the folder, ↻ **Refresh** reads it again after you save new ones. The sheet keeps your pose sheet's panel layout and the pose and camera angle in each panel, with your character in them. The pick is remembered. The folder is yours - it is never in a release zip or the repository.
- 2.578.0: **SolidCraft docker** - SolidCraft (open-source parametric 3D CAD, MIT OR Apache-2.0, solidcraft.io) runs inside the app in its own docker (the SolidCraft button in the status bar), all of it: sketches with constraints, extrude, revolve, sweep, loft, fillets, chamfers, shells, holes, threads, patterns, sheet metal, plastic parts, assemblies with joints and motion studies, parameters, STEP / IGES / STL / OBJ / 3MF / glTF. **Install** it once in the docker (Install, about 20 MB from SolidCraft's GitHub releases) or Settings ▸ Add-ons ▸ SolidCraft; offline: download solidcraft-web-<version>.zip from its releases page and use **Install from a zip…**. After that it works offline; a newer SolidCraft shows **Update** in the bar. The bar connects it to the app: **Designs** opens the project's designs and parts (projects/<project>/solidcraft) or one from the computer; **Save to project** keeps the design there - after that SolidCraft's own Save (Ctrl+S) keeps the project's copy current; **→ Image editor** puts a picture of the model (transparent background, the **Size** you pick, **Edges** on or off) into the image editor at once - **Shift**+click for SolidCraft's own lit render (slower, the bar shows its progress); **View** picks the angle (Iso, Front … Bottom, or a Named View you saved in SolidCraft - the way to picture your own angle); **→ Timeline** adds the model as a 3D layer at the playhead (sent again, it updates); ✏ **Sketch → SolidCraft** turns the image editor's selected sketch layer into a SolidCraft sketch to extrude or revolve; **Export** saves STEP, STL, OBJ or 3MF into the project.
- 2.579.0: **The SolidCraft docker shows only when SolidCraft is installed** (its status-bar button too) - install it in Settings ▸ Add-ons ▸ SolidCraft (**Install**, or **Install from a zip…** offline) and the docker appears. **Send** buttons in its bar: **Send to Image editor**, **Send to Timeline**, **Send to Sketcher** (since 2.630.0 - the part comes into the Sketcher as a group), **Send to Blender** (only when Blender is found: Blender opens in its own window with the part in metres, Z up, saved as projects/<project>/blender/solidcraft/<name>.blend) and **Send to Unreal** (only when an Unreal project or editor is set: the part is imported as a Static Mesh into /Game/bitGen/SolidCraft and placed in the open level - sent again it is replaced where it stood; it needs the bitGen Bridge plugin **1.4.0** - Load UE project updates it, or unzip **bitgen-ue-bridge-1.4.0.zip** into the project's Plugins folder).
- 2.579.0: **Delete a model** in the Model manager - on a card or **Delete** on its page. A confirmation lists what goes: the model file (deleted from the disk for good - not the recycle bin), the files beside it with its name (preview pictures, .civitai.info, .bitgen.json, its .yaml …) and its model_info page, and warns which styles use it.

- 2.579.0: **Settings ▸ Dockers**: every docker - the main column's, the side dockers, the floating windows (Basic Settings, Model manager …) and every extension docker, including ones you add later - with where it is now and **↺ Reset**: back at its default place and size (out of a floating window or a ▣ docker, not maximized, shown and open, its first height / width; a window at its first place and size). **↺ Reset all** does every one.
- 2.580.0: **Model manager ▸ Model paths**: every group of paths in ComfyUI's **extra_model_paths.yaml** (the file it reads - shown at the top), each with its base_path, is_default and one row per path (folder type + path, ✓ the folder is there / ✗ it is not).

  – **New group (other_ui1)**: pick the base folder (another UI's install, a model drive …); the group is named other_ui1, other_ui2 … (the first free number) and filled with the model folders found under it (models/checkpoints, models/loras, models/vae … - an A1111 / Forge install's models/Stable-diffusion and models/Lora too).

  – **Change a path**: type in it, or to pick the folder (inside the base it is written relative to it); **+ path** adds a row, removes one; rename the group or tick is_default; **Save** writes that group (**↶** forgets the changes). Several paths for one folder type are written as a list.

  – **Delete group** removes the group from the file (the folders and models stay on the disk).

  – Only the group you save changes - every other group, comment and blank line of the file stays as it was; the previous file is kept as extra_model_paths.yaml.bak. If the file was changed elsewhere since the tab read it, the save is refused and the tab reads it again.

  – **Restart ComfyUI**: ComfyUI reads the file only when it starts - this stops the ComfyUI bitGen Studio started and starts it again (not running: starts it; started by someone else: left alone, and it says so).
- 2.581.0: **Sync and Civitai info keep a model's videos and animated pictures too** (a video model's examples are usually clips): its examples come in Civitai's order - videos (at 512 px, at most 60 MB each), GIFs and stills - and the card's thumbnail is the first one as it is, so a video thumbnail plays on the card (muted, looping) and on its page; the examples show the video playing, GIFs marked GIF, and the viewer opens a video as a video. Models synced before are fetched again by their Civitai version id on the next Sync. A video can be the card's thumbnail (✎ Edit card, Ctrl+↓ on a picked example or one of your files, To model…).
- 2.581.0: **The selected LoRAs' keywords go into what the LLM writes**: Enhance and the prompt chat tell the LLM the keywords (trigger words) of the selected LoRAs - Basic Settings' LoRA rows for the Generate mode, where the Model manager's ✓ picks land, and the Model manager's own picks - and Enhance puts a keyword the LLM left out in anyway (at the end; the console says which). The keywords are the card's (yours from ✎ Edit, else Civitai's trained words, else the LoRA file's most used tags), up to 5 per LoRA.
- 2.581.0: **Generate in the SolidCraft docker**: describe a part (and/or give a picture), and it comes into SolidCraft as its own editable features - sketches (lines, arcs, circles), extrudes, cuts, holes, fillets and chamfers in its timeline, each one you can change.

  – Install it once: Settings ▸ Add-ons ▸ **CAD generate** - CadQuery (needed: it runs the part's code) and **cadrille** (ticked: an open 2B model that writes the part from text or a picture; best with an NVIDIA GPU; its weights are CC BY-NC 4.0 - non-commercial use - downloaded from Hugging Face on the install).

  – **Engine**: Auto (cadrille when it is installed, else the app's local LLM), cadrille, or the local LLM (a picture is described by the vision model first). Code that fails to run goes back to the LLM with the error, up to 2 times. cadrille takes a description or a picture (a picture wins when both are given); it learned from clean renders of parts, so a render or drawing of the part works much better than a photo.

  – **into the open design** off (the default): a new design; on: added to the open one.

  – The rebuild is checked: SolidCraft's volume must be CadQuery's. A step it cannot rebuild yet (revolve, loft, sweep, shell, mirror …) or a rebuild that does not match = the part goes in as one solid (the STEP) instead, and it says why.

  – The part's STEP + OBJ are in the project's solidcraft folder, its code in solidcraft/ai/. ✎ **The code**: edit it and ▶ **Run again**. Ctrl+Enter in the description generates.

- 2.582.0: **Fibo 2.0 in the edit tools** (Bria AI's 8B model; its weights are CC BY-NC 4.0 - non-commercial use only). Two image styles: **Fibo 2.0 Edit Turbo** edits a picture from an instruction in 4 steps, and **Fibo 2.0 Dev** makes pictures from text and edits them in 30 steps. Both are in the image editor's AI ▸ Edit (the selection's picture is `<image_1>`), and both take up to 4 reference pictures (`<image_1>`..`<image_4>`) - the reference dock's `<Picture n>` tags and "picture 2" in the prompt are written that way for you.

  – **Low VRAM**: the styles load the w4a8 ConvRot files (6.6 GB) with the int8 Qwen3-VL 4B text encoder - about an 8 GB card, ComfyUI moving the rest to RAM. GGUF Q4_0 / Q5_0 / Q8_0 work too: pick the .gguf as the style's Model in Settings ▸ Image (the loader follows the file; it needs the ComfyUI-GGUF node pack).

  – **Install**: Settings ▸ Add-ons ▸ **Fibo 2.0** - tick the files (Edit Turbo w4a8, the text encoder and the VAE are ticked; a GGUF installs ComfyUI-GGUF too). Fibo 2.0 needs ComfyUI from 21 Oct 2026 or later.
- 2.583.0: **Team ▸ the render farm** (2.589.0: an optional add-on - Settings ▸ Add-ons ▸ Team render farm) - team members open the same project and render on each other's PCs, with the owner's permission. Settings ▸ **Team** (after Remote GPU):

  – **The team folder** is a folder on the NAS every PC can open (`\\nas\bitgen-team` or a mapped drive). The leader picks it, types the team's name and their own name and presses **Create the team**; everyone else types their name and presses **⇥ Join**. The leader sees the request and presses ✓ **Approve**; **Remove** takes someone out (their key stops working at once). **⇤ Leave the team** on any PC.

  – **Daily keys**: the leader's bitGen writes every approved member a new key in the team folder each day (it does this by itself while it runs; **↻ Issue today's keys** does it now). A key is valid until midnight and only works together with the private key that stays on the member's own PC - copying the key file gives nobody access. If the leader's bitGen does not run on a day, nobody gets that day's key (the farm waits for it).

  – **The same project**: **Use the team's projects folder** makes this PC open the projects in the team folder (restart bitGen); every member does this once.

  – **Share this PC for team renders** opens the render port (8312 by default) - only while bitGen runs and the switch is on, and only the render-farm calls are on it (the app itself stays on this PC). Same network: nothing else to set; over **Tailscale**, type this PC's Tailscale name or 100.x address in *This PC's address for the team*.

  – **Access** is the owner's: a member's **Ask for access** shows under *Asks for access* - **Unrestricted** (any time bitGen runs here), **from - to** (two dates and times; outside them the PC refuses the job) or **Deny**. *Access given* lists who may render here; **↺ Revoke** takes it back.

  – **Render on** (the status bar): **This PC**, a team member's PC (✓ with your access and its time; Ask; greyed when it is offline or not sharing) or ⚡ **Spread** (each new render to the least busy of this PC and the PCs you may use). The menu also shows every render the team's PCs are doing, with its progress. Generation jobs go (pictures, videos, songs, graphs, swaps, upscales …); installs, downloads, training, the movie export and the quick ffmpeg jobs always run here. A job on another PC shows in your queue as usual (its log comes back), and the finished file is in the shared project - it lands in your assets like any render. A job the other PC refuses (no access, outside its time, not sharing, a file outside the team's projects folder) fails with the reason in its log.
- 2.584.0: **Compare LLMs** (the prompt docker's menubar) - type a prompt and press : every ticked LLM answers it in turn (the one in the LLM dropdown first - it is loaded already; each of the others loads when its turn comes, so a round takes as long as loading them). Each reply shows in the Prompt chat under its LLM's name and time; press ✓ **Use this one** on the reply you like - it becomes the chat's answer (stored in the chat, read aloud, a song goes to the Music dock, an image / video fence sets the mode) and the others fold away (show / hide). ✗ **None of them** records that none was good; **Stop** skips the LLMs not asked yet. The replies use everything a normal chat turn does (the styles, the LoRA keywords, the reference dock, the conversation so far).

  – ▾ picks which LLMs take part (vision projectors are never in the list). With an external LLM endpoint set (Settings ▸ LLM) there is only one LLM, so is off.

– shows the best LLM for each job - screenplay, music video, song, image prompt, video prompt, chat (the job is read from the prompt's words, the same ones the buttons add) or every job: how often each LLM was picked, how often it took part, its win %, its average time, how often it failed. **Use** makes that LLM the chat LLM. Below: every round with the request and every reply (one, Clear all), **Export CSV** for a spreadsheet.

– The record is kept in **llm_arena/** in the install folder (one file per round) - this PC's own, never in a zip.

– If an old llama-server is still running on the LLM port, it would answer for every model; such a reply is refused and said ("the LLM port answered as …") instead of being recorded under the wrong name.

- 2.585.0: **train an LLM on your Compare LLMs picks** - the picks become a training set for the LoRA trainer's LLM fine-tune, which writes a new .gguf into the LLM folder.

  – Each round now keeps the **full prompt** the LLMs were sent (bitGen's instructions - styles, guides, LoRA keywords, references - the conversation so far and the request), so a model trained on it learns to answer the way bitGen asks. ▸ a round's details show it. Rounds from before 2.585.0 have only the request.

  – ▸ **Train an LLM on your picks** says what your decided rounds give - the replies you picked as **examples** and one **preference pair** (picked vs not picked) per reply you did not pick - for the job chosen at the top or every job. Pick the trainer's LLM job (or type a name for a new one), examples + pairs / only one of them, with or without bitGen's instructions, and **Send to the trainer**. Sending again adds only what is new. The longest example is said in tokens: the job's **Max length** must fit it (a longer one is left out of the run and said in its log - never cut, which would lose the answer).

  – In the trainer (an LLM job): **Job ▸ Learn from** - **Examples (SFT)**, the answers you want; **Preferences (DPO), "this, not that" against the base model;** Preferences (ORPO)**, the same without a reference model (less VRAM). Preferences use a much smaller learning rate (set for you) and **Beta** (0.1 - higher keeps it closer to the base). **Dataset ▸ Preference pairs**: the count, From Compare LLMs, prefs.jsonl, Clear the pairs. ▶ Start counts the pairs for DPO / ORPO; the log shows how often the picked answer already wins.

  – A useful run needs a few hundred picks, ideally of one job (screenplays, say). Training starts from the base model's original (Hugging Face) files - a .gguf cannot be trained - and the result is a new .gguf. Check the licence of each model whose replies you train on: some do not allow their output to train other models.

- 2.585.0: **which LoRAs' keywords the LLM writes into prompts** (Enhance and the prompt chat): with **Override style** on in Basic Settings, only the **temporary LoRAs** there (of the Generate mode - image or video); with it off, only the **selected style's LoRAs** (the image style for Image, the video style for Video and Chat). LoRAs picked in the Model manager no longer count by themselves.

- 2.586.0: **Settings ▸ Add-ons ▸ LLM** - the chat / writing LLMs in one dropdown: pick one and **Download** - the .gguf goes into the LLM folder (Settings ▸ LLM) and shows in the prompt docker's LLM dropdown and in Compare LLMs. ✓ marks the ones you have; **Delete it** removes the picked one (not the one running).

  – ★ **Qwen3.8 27B** (Alibaba, August 2026, Apache 2.0) is the base we recommend for **your own checkpoint**: UD-Q4_K_XL (17.6 GB) runs on a 24 GB card, Q5_K_XL / Q8_0 for more quality. In the trainer its base model trains as a 4-bit QLoRA on a 32 GB card (Unsloth's 4-bit checkpoint; a 16-bit LoRA needs more than 36 GB).

  – **Gemma 4 31B** (Google, Apache 2.0) - the other strong writer, the same class; **Gemma 4 12B** (QAT, 6.7 GB) - fast, the quick one to train; **Qwen3.5 9B**; and the setup wizard's Qwen3 4B / 8B / 14B.

  – Compare the ones you download with Compare LLMs on your own jobs, then train the winner on your picks.

  – Qwen3.8 and Gemma 4 need a recent llama.cpp - update it with miscget-llama.ps1 if a model will not load.

- 2.702.1: **Credits** - Dandyson EEbu (Baridakara) is credited for text effects and fonts (Settings ▸ About and the Credits section, under Text effects and fonts).
- 2.702.0: **SVG pictures** - an .svg can go into Assets (drop it, or + add pictures) and into the References (drop it, + add reference, or drag it from Assets). Nothing after an upload reads SVG (ComfyUI, the timeline and the models want pixels), so the studio draws it into a PNG as it goes in - sharp, because a vector is drawn at the size it is used (the long side at least 2048, at most 4096 pixels), with its transparent background kept. Assets then holds <name>.png. The Open dialog lists .svg with the other pictures, so Image Edit's Open and Place as Layer take one too. An SVG that needs other files beside it (linked pictures or fonts) draws without them. **The logo at 1920 x 1080** is in docs/images/bitgen-logo-1920x1080.png.
- 2.701.0: **A start-up splash** - the desktop app opens with a 15 second film of the logo (the lines drawn, the wings beat, "bitGen Studio", "together · grow · transform") and the credits **Produced and developed by Timothy Nolan Paul** and **Programmed by Claude Opus 5.5**, while the studio starts behind it. Click it or press any key to skip; the studio appears when the film ends (or after 16.5 seconds whatever happens). BITGEN_NO_SPLASH=1 turns it off. The film is also in docs/images/bitgen-studio-splash.mp4. **The setup** shows the logo too (its title bar and every page's header).
- 2.700.1: the logo is no longer cut off - the title bar shows the whole butterfly (static/logo-mark.svg, the logo cropped to its outline) and the app icon gives the wing tips room inside its rim.
- 2.700.0: **The bitGen Studio logo** - Tim's own design (made for Tiger Art): two tigers for cooperation against all odds, a tree for growth, and the butterfly they make together for change into something beautiful. It is the app icon (the window, the taskbar, the Windows installer), the browser tab's icon and the mark at the left of the title bar (in the theme's accent colour). A 10 second intro film of it is in docs/images/bitgen-studio-intro.mp4 (made from misc/intro/ - node misc/intro/render.js makes it again).
- 2.699.0: **Edit size follows Override style** - with Basic Settings' **Override style** ticked, an edit of a References picture comes out at Basic Settings' Size (as in 2.698.0); not ticked (or Size = Style's own), it comes out at the **first reference picture's own size** (in 16 px steps, at most 4096 px on the long side) - the workflow keeps that size instead of scaling to its own megapixels.
- 2.698.0: **Edits come out at Basic Settings' Size** - an edit of a References docker picture (the prompt docker's ✏ Edit or Refine with the image editor closed) now uses Basic Settings' image **Size** + **Aspect**: picture 1 is cropped to that shape (its middle kept) and the workflow keeps that size (its own "scale to N megapixels" steps follow). **Style's own** keeps the picture's own shape and size, as before. Edits inside the image editor keep the selection's size (the result is pasted back over it).
- 2.697.0: **P-H3 Image Edit works** - every P-H3 Image Edit job failed with "Return type mismatch between linked nodes [guider]": removing the unused picture slots also removed the graph's guider. Only a slot's own reference nodes are removed now (the character creator's guiders were affected the same way).
- 2.696.0: **The Sketcher's own Photo style** - a dropdown on the Sketcher's bar, just left of Photo, lists the image edit styles; Photo uses the one picked there (the prompt docker keeps its own style), and the last one used stays picked (after a restart too). When a picked style is gone it falls back to the prompt's edit style or the default edit model.
- 2.695.0: **Roof coverings** in the Sketcher. A roof's (or the Roof tool's) **Covering**: pre-painted steel - corrugated, ribbed (trapezoidal) or standing seam - concrete tiles, terracotta tiles, slate, asphalt shingles or clear polycarbonate. The roof takes the covering's colour (until you pick your own) and its pattern - ribs down the slope for sheeting, courses and staggered joints for tiles, slate and shingles. The panel gives the covering's typical minimum pitch and warns (⚠) when the roof is flatter - check the maker's own minimum. Tiles and slate weigh as heavy in the rafter, ceiling and lintel checks, sheeting and shingles as light; Photo's prompt names the covering ("a hip roof of terracotta roof tiles").
- 2.694.0: **Roof framing** in the Sketcher. A picked roof's **Roof framing**: **Rafters** - common rafters down every face at the spacing (450 / 600 / 900), a ridge board, hip / valley rafters on the sloped edges two faces share, break purlins where a gambrel / mansard changes slope, ceiling joists on

the top plates, and an underpurlin half way up any slope longer than you set (0 = never) with struts down to the ceiling joists - or **Trusses** laid out at their spacing (top chords, bottom chord, webs; the truss maker designs them). Rafters and ceiling joists get the same ✔ / ✗ guide check and ✔ **Size them** (the rafters over the slope to the underpurlin, the ceiling joists over how often they are held up). Every roof type is framed except flat and follow-the-walls. Shown with **Show framing**, which now makes the roof see-through too; the totals are in the panel and the member schedule. Hips, ridges, underpurlins and struts are laid out, not sized.

- 2.693.0: **Wall framing** in the Sketcher. A picked room's **Wall framing** sets the frame for every wall: **Timber** (stud size and grade, one or two top plates) or **Light-gauge steel** (C-section studs, one top track), the stud spacing (300 / 450 / 600), how often the nogging rows go and nominal strap braces; **Frame this room's walls** leaves one room out. Each wall gets a bottom plate (cut at doors), top plates, studs at the spacing, a jamb stud each side of an opening, jack studs over the head / lintel and under a window's sill trimmer, noggings in rows (broken at openings), a strap brace in each clear panel 1.8 m or longer and a post at each corner; a wall two rooms share is framed once. The room's totals (studs, metres of studs / plates / noggings, braces) show in the panel and in the member schedule. It is drawn with **Show framing**. Laid out by common practice - stud sizes are not checked (AS 1684's stud tables: wall height, spacing, wind) and the bracing is nominal (AS 1684 / your engineer).
- 2.692.0: **Floor systems and lintels** in the Sketcher. Pick a room and set its **Floor system**: slab on ground (edge beams), waffle pod slab (pods + ribs), stiffened raft (internal beams), suspended slab on piers, or a **raised timber floor** on concrete / steel / brick / timber stumps, with bearers and joists you size (stock sizes, MGP10 / MGP12 / MGP15 pine or F17 hardwood, 300 / 450 / 600 centres). The structure is built under the room in 3D and listed (counts and lengths); a raised floor says when there is too little clearance under the bearers. Each joist and bearer is **checked** ✔ / ✗ with plain beam sums from the grade's published strength and stiffness (a house floor's 1.5 kPa, a 1.8 kN point load, a bounce limit); ✔ **Size them** picks the smallest stock size that passes. A door or window in an outside wall gets a **Lintel** - Auto (the smallest that passes for its width, the roof load width and sheet or tile roof) or one you pick. **Show framing** makes the walls see-through so the lintels show. This is a guide, not a certificate: no supplier's or standard's span tables are copied in - confirm the sizes against AS 1684 or your supplier's span tables, and slabs / footings are your engineer's design (AS 2870; the site class is recorded with the slab).
- 2.691.0: **New walls butt up to the old ones** in the Sketcher: where a wall's free end meets another wall, the new wall stops against it instead of getting a square cap of its own. End to end (joined, or drawn there), the old wall's end is shaped round the corner so their outside faces meet - one clean corner, as if it were one wall. A wall that ends on another wall's line (a wall across a room from wall to wall, or into a room's side) is cut to that wall's face, even at an angle or thicker than it. **Wall thickness** now remembers the last one you set - in the Wall or Room tool, or on a picked wall or room - and the next walls and rooms are drawn that thick; a number over 2 is read as millimetres (200 = 0.2 m). A stored thickness stuck at the 2 m limit goes back to 150 mm.
- 2.690.0: **Walls join end to end** in the Sketcher: drawing with ▮ Wall, a point near another wall's end or corner (within about 15 cm, or 14 px when zoomed out) is pulled onto it - the tip says ⊕ **joined** (hold Shift to place it freely). A wall that ends there is **joined** to that wall: drag the new wall and the old wall's end goes with it, so the old wall grows or shrinks; drag the old wall and the new one follows the same way. A wall drawn out of an end piece is joined too, and the joins are saved with the model. Dragging both together moves them as one.
- 2.689.0: **Cupboards & shelving in the Sketcher** (Furniture ▸ Category **Cupboards & shelving**): built-in robes, walk-in robes, linen, pantry and broom cupboards, bookshelves and shelving units.

  – Pick a template, or press **Design / change…** to open the dialog: name, layout (**straight**, **L-shape** or **U-shape walk-in**), width / depth / height (and the robe's depth for L / U), board thickness, plinth, doors (none, hinged, sliding, bi-fold, glass - and whether they show in 3D), handles (bar, knob, edge pull), the body / door / handle colours (any colour or one of the Sketcher's materials).

– Each run (Back, Left, Right) has its **bays**: a width (0 shares what is left), what is in it (hanging rail, double hanging, shelves, drawers + shelves, shoe shelves, open), how many shelves / drawers; ↑ ↓ to reorder, ✕ to take one out, Add bay, ⇔ Share evenly. A front view and a plan follow every change.

– **Save as template** keeps your design (by its name - the same name replaces it) for every project; your templates are listed with ★ in the Furniture panel and under **Yours** in the dialog; Delete template removes one.

– Click inside a room and it goes with its back against the nearest wall, facing in, flush into a corner when it is close; outside a room it stands free. Picked, its panel has Width / Depth / Height, Doors in 3D, ✎ **Design…** (Apply = one ↶ step) and ↔ **Fit the wall** (as wide as the wall; a walk-in robe as deep as the room too).

– On the floorplan and in Plans: each run, its bays, hanging rails dashed, sliding tracks or door swings.

- 2.688.0: **Sketcher: bar always on screen, wall ends, Ctrl+drag, a truer Photo prompt.**

  – The Sketcher's top bar (File … Photo, Plans, Send) stays on screen: when docked, the docker can no longer be scrolled away under it, and the view takes the room that's left.

  – **Click a wall's end piece** (the square cap at an open wall's end) and the whole wall is picked. **Double-click it** to push / pull: drag the end along the wall to make the wall longer or shorter (1 cm steps, Shift: free; Esc ends it). The end's own width (a pier) is in the wall's panel: ■ **This end's width…**. Corners keep their own piece.

  – **Ctrl+drag** moves what a plain drag would, held **horizontal or vertical** by the way you drag (like Shift). Ctrl+click without dragging still adds to (or takes out of) what's picked.

  – **Photo** now writes its prompt from the model: it names what is there (rooms, doors, windows, roof types, stairs, furniture by name, people, pools, roads, the ground) and the materials you used, and asks for nothing else. No windows in the model, no windows or glass in the prompt.
- 2.687.0: **A quieter sketch plane** in the Sketcher: clicking a face still makes it the sketch plane (the bar's ✎ chip names it, the drawing tools draw in it, ⊥ Look at turns to it), but the orange square and grid are no longer drawn over it. A plane you keep with still shows its faint dashed outline.
- 2.686.0: **Change style draws the line art first**: when the prompt's image style is not an edit model (SDXL, Pony, Illustrious, SD 1.5, FLUX.1 …), Change style first turns the picture into a ✏ line-art drawing, then the style makes the look from it through a line-art ControlNet. The new picture keeps the drawing's outlines while the look, colours and rendering are new. The line art appears in the preview after the picture, and the result after it.

  – The ControlNet must suit the style's model family: SDXL, Pony and Illustrious use an SDXL one (the SDXL ControlNet Union ProMax from **Settings ▸ Add-ons ▸ ControlNet** works, and is told it gets line art); SD 1.5 and FLUX.1 need their own line-art ControlNet in ComfyUI's models/controlnet.

  – No ControlNet for that family (Krea 2, FLUX.2, Qwen, Z-Image, Anima …), or the line art can't be made (the ControlNet add-on's nodes are missing), and the picture is remade picture to picture instead - the bar says why.

  – An edit model (FLUX.2 Klein edit, Qwen Image Edit …) still follows the look's instruction itself, no line art.

  – Basic Settings' Override (steps, CFG, checkpoint, temporary LoRAs) rides the line-art render too.
- 2.685.0: **Change style uses Override style**: with an SDXL, Pony or Illustrious style in the prompt, Change style now remakes the whole picture with that style's own workflow (picture to picture). So the style's own checkpoint and LoRAs are used, and with Basic Settings ▸ Override style on, its steps, CFG, checkpoint and temporary LoRAs too. Before, these styles ran a fixed SDXL inpaint workflow that ignored all of it. While it renders, the bar beside the button says what Override put on the picture (for example "Override: steps 30, CFG 6, checkpoint …"), or why a value was left out (an SDXL checkpoint can't run inside an edit model such as FLUX.2 Klein).
- 2.684.0: **A tidy look dropdown**: in the image preview the look dropdown beside Change style is now only as wide as its words and the same height as the buttons, so the preview's bar stays on one line.

- 2.683.0: ⇄ **works again**: picking another image or video style now always makes the one before it the previous setting, so ⇄ can take you back (before, Basic Settings filling in for the new style hid the old one and ⇄ stayed greyed out). **— none —** and **— template —** count as settings too. With nothing to go back to yet, ⇄ is dimmed and a press says so beside it.
- 2.682.0: **Shift+click runs a button in edit theme mode**: with on, hold Shift and click a button and it does what it does (Alt+click still works the same way). A plain click still picks it to style.
- 2.681.0: **The look stays picked**: in the image preview the looks are now a dropdown (Photoreal, Pixar, 2.5D, Anime, Cartoon) with **Change style** beside it. The look you pick stays picked (also after a restart), and each press of Change style makes the shown picture again in that look. Its tooltip says which image style it uses.
- 2.680.0: ⇄ **the previous settings**: beside the prompt docker's image style and video style dropdowns. Change the style and the one you had before becomes the *previous* setting; ⇄ puts it back - and the one it replaces becomes the previous, so pressing ⇄ again toggles. With **Override style** on, a setting also holds that kind's Basic Settings values and its temporary LoRAs, and ⇄ brings those back too. The tooltip says what ⇄ will put back. Kept in model_settings.tmp in the app's folder, so it survives a restart.
- 2.679.0: **Restyle - the result and ⇆ Before / after**: when a Style render is done the finished picture is shown in the preview right after the original - the preview opens again on it if you closed it meanwhile. **⇆ Before / after** (beside Style) shows the original and the result side by side, labelled; click it again to go back. It is there for every restyled picture, also when you open the result later from the gallery (this browser remembers the last 200).
- 2.678.0: **P-H3 Image** (PrunaAI/P-H3-Image-Comfy - PrunaAI's image model on MiniMax H3): **Settings ▸ Add-ons ▸ P-H3 Image** downloads its LoRA (2 GB), text encoder (int8), VAE (int8) and prompt upsampler (w4a8) - about 17 GB - and uses the MiniMax H3 FL2VA transformer the video already has (it is downloaded only if it is missing). Two styles come with it: **P-H3 Image** (text to image, 2048 x 2048 by default - up to ~8 MP; your prompt goes to its upsampler as "Aspect ratio: … Request: …") and **P-H3 Image Edit** (put the picture to edit first in the reference dock, up to 7 more as references; the result keeps its shape at about 4 MP). 8 steps, no CFG, no negative prompt. P-H3 Image Edit is also an edit model for the image editor's Edit and the preview's Style. It needs a ComfyUI with the P-H3 nodes - they are on ComfyUI's master branch / the nightly until the next release (0.44); an older ComfyUI is refused with the words to update. Licence: the MiniMax H3 Community License (it excludes the EU, UK, Republic of Korea and the US without a licence from MiniMax; show "Powered by MiniMax H3"). About 21-31 GB of VRAM at 8 MP; NVIDIA Ampere or newer for the int8 / w4a8 files.
- 2.677.0: **The restyle's progress under the picture**: while a Style ▾ render runs, a thin bar under the picture (above the preview's buttons) shows it - a moving stripe while it is queued and the models load, then the bar filling with the sampler's steps (" Pixar · step 12 of 25 · 48 %"); green when it is done (it fades as the result opens), red if it failed.
- 2.676.0: **Style ▾ in the image preview**: open any picture (the gallery, the assets …) and press **Style ▾** beside Save - **Photoreal**, **Pixar**, **2.5D**, **Anime** or **Cartoon**. The picture is made again with the image style picked in the prompt: an edit model (FLUX.2 Klein, Qwen Image Edit, FLUX Kontext …) is told to change only the look and keep the composition, faces, poses and clothes; an SDXL / Illustrious / FLUX.1 style refines the whole picture with the look's words and the picture's own prompt (strength 55-72 %). With **Override style** on in Basic Settings, its steps, CFG, seed, checkpoint and LoRAs are used, as Generate does. It is an ordinary job (queue, gallery, log); when it is done it opens in the preview next to the original. A style that can do neither says so.
- 2.675.0: **Pins stay put**: a docker's pin (its fixed height, or width for a side docker) is no longer cleared when the layout changes - ⊞ Docker configs loading the sizes for the dockers you opened, switching workspace, applying a Layout, or ↺ Reset in Settings ▸ Dockers. Click the pin to pin / unpin as before; **right-click it** for a small menu: * Pinned at … px* and **↺ Reset pin** (any size again), or * Pin at this size* when it is not pinned.
- 2.674.0: **One ComfyUI folder for everything**: with Settings ▸ ComfyUI ▸ Folder empty, bitGen Studio now looks for ComfyUI in comfy, ComfyUI and ComfyUI_windows_portable beside the app

(more than one: the most recently updated), then wherever the model lists find it - and the launcher, every add-on download (MOSS-Image, FIBO, AI Fill …), the model catalog and the ComfyUI clear all use that same folder. Before, an empty folder made downloads go to comfyComfyUImodels while a ComfyUI in ComfyUI was the one in use. Setting the folder (or *Use this folder*) still decides it.

- 2.673.0: **Road tool** in the Sketcher (Ground ▸ Road): pick a type - **Dirt road**, **Street**, **Road**, **One-way road**, **Avenue** (4 lanes, a planted median) or **Highway** (6 lanes, a barrier, guard rails) - and click along the way; double-click or Enter ends the road. It follows the ground (levelled across). Clicking near a road's end carries on from it, clicking on a road makes a T (Shift: no snapping). Each type has a steepest grade (dirt 20 %, street 15 %, road / one-way 12 %, avenue 8 %, highway 6 %); steeper ground is flagged ⚠ and the panel offers **Cut the ground** (an even grade, the ground cut and filled to it with 1 : 2 batters - the m³ shown) or **Tunnel / bridge** (through high ground in a tunnel with portals, over a deep valley on a bridge with piers); *When too steep* can do either at once. Junctions are found by themselves: **traffic lights** where major roads meet (with crossings and stop lines), **stop signs** where a minor road joins a major one, an all-way stop where two streets cross, **give way** where a street ends on a street; plus speed signs, one-way and street-name signs. *Traffic keeps to the* left (Australia, UK - white centre lines) or right (US - yellow). Alt+click a road to drag its points; ➤ Select picks it (name, type, past steep ground, Grade again,). The floorplan and Plans draw the roads with their names.
- 2.672.0: **MOSS-Image stand-ins**: when ComfyUI does not have the style's small FLUX.2 VAE (flux2-vae_small) but has the full one (flux2-vae), or has another Qwen3.5 9B text encoder (w4a8 / int8 / bf16) than the one asked for, the render uses the one you have and says so in the log. Settings ▸ Add-ons ▸ MOSS-Image installs just the text encoder + VAE when no model is ticked - for your own MOSS model (a Civitai fine-tune: set it as the style's Model).
- 2.671.0: **Gutter profiles** in the Sketcher: pick a roof - **Gutters** lists the built-in sections (Quad (box), Half round, OG (ogee front), Square line (deep)), **Gutter profile** shows them (and yours) as pictures, and ✎ **Gutter profiles…** opens the profile editor with the kind *Gutter*: the fascia is the line on its left, x out from it, y up from the gutter's bottom, and the back's top hangs just under the roof's edge. Draw it the same way as a skirting or cornice (click / drag nodes, double-click a segment for a node, right-click ▸ Convert to curve / line, Delete node) and Save - the roof's gutter is then your profile (the project keeps it, curves and all), swept along every eave; the downpipes sit under its middle. The Roof tool's own options take a gutter profile too, for the roofs it makes next.
- 2.670.0: **Profiles with Bézier curves** in the Sketcher's profile editor (skirting, cornice, door / window frames, sills - ✎ New / edit profiles…): every built-in arc and ogee is now ONE curve with just two nodes on it (its start and finish), so the Cove cornice is three nodes. **Click** a node or a segment to pick it; **drag** a node (its handles move with it) or a curve's two **handles** (shown on the picked segment, or either side of a picked node); **double-click** a segment to add a node there (a curve is split without changing its shape); **right-click** a segment for *Convert to curve* / *Convert to line* and *Add a node here*, or a node for *Delete node* (Delete works too). Saved profiles keep their curves; older ones open with straight sides.
- 2.669.0: **ComfyUI cleared completely - before an update and when bitGen Studio exits.**

  – **Update ComfyUI** (Settings ▸ ComfyUI) first runs `misc\clear-comfy.bat`: every ComfyUI process is stopped (one you started yourself too - its Python holds the packages the update replaces), port 8188 is freed, and ComfyUI's `temp\`, `__pycache__` folders and ComfyUI-Manager's cache are emptied - never your models, output, input, workflows or settings. Its lines show in the update's log. A ComfyUI that bitGen Studio started is started again afterwards.

  – **When bitGen Studio exits** (the app window closed, its console window closed / Ctrl+C, or the app crashing): `misc\clear-comfy.bat` runs, every bitGen Studio process still running from its folder (render / music / LLM workers, llama-server, ffmpeg …) is ended, and the terminal window `start.bat` opened closes by itself.

  – Settings ▸ ComfyUI ▸ **Clear ComfyUI completely when bitGen Studio exits** (on by default) - it outranks *Leave ComfyUI running*; turn it off to keep ComfyUI running between sessions as before.

– `misc\clear-comfy.bat` gained `/nopause` (no "press any key" at the end) for this; run by hand it works as before.

- 2.668.0: **Eaves, fascias and gutters** in the Sketcher: the Roof panel (the tool's and a picked roof's) shows the eave as four pictures - **Simple** (just the overhang), **Open** (the rafter tails show under the overhang, 600 mm apart), **Boxed** (a flat soffit from the fascia back to the wall) and **Closed** (a lining under the rafters, on the roof's slope). **Fascia + barge boards** puts a fascia along the eaves and barge boards up a gable's verges (*Fascia height*); **Gutters** None / Quad (box) / Half round, hung on the fascia, with **Downpipes** down the wall near each run's end. A picked roof also has *Fascia / soffit colour* and *Gutter colour*. A new roof from the Roof tool starts boxed, with a fascia and quad gutters; roofs made before stay as they were. They show on the building plans' elevations too. **MOSS-Image:** it needs ComfyUI **0.42.2 or newer** - ComfyUI 0.41.x and 0.42.1 know the model but render pure noise. A MOSS-Image picture is now refused before it is queued on a ComfyUI that is too old, with how to update it (Settings ▸ ComfyUI ▸ Update ComfyUI); the add-on's install says so too. The MOSS-Image (SFT) style no longer adds skip-layer guidance (ComfyUI's own template has it off; MOSS-Image Base keeps it).
- 2.667.0: **The Sketcher's objects panel** (Outline on its bar) gains:

  – **Shapes** - every shape the drawing tools made: *Solid n* (closed), *Shape n* (flat or open), *Lines n* (loose lines) and each group under its own name. Click a row to pick the shape, then drag one of its faces in the view to push / pull it. A drag on a group's face push / pulls it too; a click still picks the whole group.

  – **Construction** - with **Planes** and **Construction lines** inside it.

  – *Planes:* click a face with ➤ Select to make it the sketch plane, then (on its chip in the bar, or on its row) keeps it. on the heading makes a ground / front / side plane through the origin. Click a plane's row to draw in it again; ⊥ looks straight at it.

  – *Construction lines:* Tape measure, hold **Ctrl** on the second click - a dashed line through both points that the drawing tools snap onto (*On Guide*, and where two cross).

  – hides a plane or line, deletes it.

  – **Rename anything** - double-click a row's name (or F2 on the picked row), type, Enter (Esc leaves it): rooms, room groups, walls, floors, ceilings, skirting, cornices, doors, windows, roofs, stairs, lights, furniture, cameras, shapes, groups, planes and construction lines. One undo step; the properties panel's heading uses the new name.
- 2.666.0: △ **Roof types** in the Sketcher: the Roof tool's panel (and a picked roof's panel) shows every type as a picture - **Gable, Hip, Skillion** (lean-to), **Dutch gable** (a hip with a small gable - a gablet - at the top of each end), **Clipped gable** (jerkinhead - a gable whose top is hipped back), **Gambrel** (barn - steep below, shallow above), **Mansard** (steep on all four sides, a shallow hip or flat top), **Saltbox** (the ridge off centre), **Pyramid**, **Butterfly** (two slopes down to a valley), **Flat** and **Follows the walls' slope**. Click a picture to change the roof. Each type has its own settings: Pitch (gambrel / mansard: Upper and Lower pitch), the Dutch gable's *hip part of the rise*, the clipped gable's *clipped part*, the gambrel / mansard *break*, the saltbox *ridge from the front*, and *Flip* (skillion / saltbox: the other side high). Ridge (along / across), Overhang, Thickness and Colour work for all. Every type shows on the building plans' elevations and roof plan.
- 2.665.0: **Building plans** in the Sketcher: **Plans** on its bar (after Photo) opens the *Building plans* dialog - drag it by its title bar. Pick:

  – **Sheet** A4 / A3 / A2 / A1, landscape or portrait, and the **scale** - *Auto (fits)* (the largest standard scale the views fit at) or 1:20 - 1:500.

  – **Pages** - *All views on one sheet* (one scale for all, e.g. everything on one A1) or *One view per page* (each page its own scale on Auto).

  – **Views** - a floor plan for every storey, the four elevations (north, east, south, west - north is up on the plan) and a roof / site plan (roofs, pools, driveways, the ground's contours).

  – **Show** - dimensions + levels (each outside wall face, the doors / windows along it, the overall size, in mm; FFL marks and the height on the elevations), room names / sizes / areas, furniture, coloured (the materials' colours).

– **Title block** - project, drawing, drawn by, date, revision and notes; it also has a north arrow, a scale bar and *Sheet n of m*. The preview shows each page (◀ ▶). **Save PDF** (Ctrl+Enter) writes it into the project's pdf folder (the image editor's File ▸ Open PDF from Project lists it) and, with *Download too* ticked, to your downloads. The dialog keeps your choices.

- 2.664.0: **Mannequin model** in the Sketcher: Furniture () ▸ Category *People* ▸ **mannequin (model)**, then click where it stands - a modelled female mannequin, as tall as the person's Height. Its panel's Pose switches between it and the posed House-builder figures. The model is by dimasalcalde and zahers on CGTrader (Settings ▸ About ▸ 3D models). Its file (ext/pushpulldock/assets/mannequin-female.json) comes in its own zip, not in the public repo; without it the standing figure is drawn instead.
- 2.663.0: **Split walls, the sketch plane and ⊥ Look at** in the Sketcher.

  – **Double-click a wall** (➤ Select - seen from above, on the floorplan, or on its face in 3D below its top) to split it there; double-click again for more. Each piece is then a wall of its own: click it, **Delete** takes just it out, and **dragging it** across its line moves it - when the walls beside it run straight on, a new wall each side joins the moved piece to the room (a bay or a recess). Double-clicking a wall's top in 3D still cuts its top (a slope).

  – The **Wall tool** catches a room's wall anywhere along it (*On Wall*): start and finish walls on it, or on a split point.

  – Click a room's **corner** or a wall's **end piece** (Select): its panel sets its **Width** (wider than the wall = a pier) and, for an end, its **Length past the end**; Delete puts it back as the wall. Dragging from it still draws a new wall.

  – **Sketch plane**: the face you click with Select (a wall's face, a floor, a face of the model) becomes the sketch plane - the bar's chip names it (2.687.0: nothing is drawn over the face any more). Line, Rectangle, Circle and Polygon then draw in that plane. × on the chip, Esc or a click on nothing ends it. ⊥ **Look at** (the bar) turns the view straight at it.
- 2.662.0: **Walls end square** in the Sketcher, and **new walls dragged out of corners and wall ends**.

  – Each wall is now a rectangle from corner to corner - picked, hidden or raised on its own it ends square, not in a point. The corners are pieces of their own that fill the gap, and a free wall's ends get a square end piece.

  – With ➤ Select, point at a room's outside corner or a wall's end piece - it lights up. **Drag out of one of its sides** and a new wall comes out that way (from the end face: on along the wall; from a side: across it). Drag from its **top** (seen from above) and the new wall goes any way. It has the wall's height, thickness and wall type, and its own ends drag out more walls the same way. Ctrl+Z takes it back.
- 2.661.0: **Wall types** docker in the Sketcher (its button in the bar).

  – It lists the model's wall types - Brick veneer, Double brick, Timber frame (weatherboard / fibre cement), Rendered block, Interior stud, Interior steel stud and Wet-area stud to start with - each with its thickness and how many walls use it.

  – Pick one to see its cut drawn to scale and its **layers from the outside to the inside**: brick, air gap / cavity, wrap / sarking, timber or steel frame (tick *batts* for insulation between the studs), insulation board, plasterboard (gyprock), render, weatherboard, fibre cement, block, concrete, stone, tiles ... Change a layer's kind or thickness (mm), ↑ ↓ to move it, × to take it out, * Add a layer*. **Wall thickness** adds the layers up.

  – **Use it for**: the picked wall, the picked room's walls, every exterior wall, every interior wall (× None = the room's own thickness again). / / × at the top make, copy and delete types.

  – The walls are built as thick as their type, round their centre line, and follow when you edit the type; the outside layer (brick, render, cladding ...) colours the wall's outside face, and the layers' seams show on the wall's top and on the floorplan. A wall's own panel has **Wall type** too.
- 2.660.0: **One wall out of a room** and **interior / exterior walls** in the Sketcher.

– ⌫ **Eraser** on a wall (or **Delete** with the wall picked, or **Take this wall out** in its panel) takes out just that wall - the floor and ceiling stay and that side is open. Its doors / windows go with it. A wall two rooms share goes out of both, so the rooms open into each other (their floors meet). **Shift+Eraser** still erases the whole room.

– The room's panel: ↺ **Put back** brings its walls back (↶ works too).

– A wall's panel: **Interior / exterior** - *Auto* (a room on both sides = interior, else exterior), *Interior* or *Exterior*; its heading says which. An interior wall's outside face takes its inside colour (its own *Outside* colour still wins); the floorplan fills interior walls lighter than the outside ones.

- 2.659.0: **Driveway** in the Sketcher (under *Ground*, no key).

  – Click its points along the way - it curves smoothly through them - and double-click (or Enter) to end it.

  – Its curve shows with its points (□) and their handles (○): drag a **handle** and the curve bends (the other handle turns with it; **Alt** = this side only), drag a **point** to move it, **Delete** removes the point you clicked. Click another driveway to edit its curve; type a width in the box.

  – It lies on the ground (4 cm over it) or at a level, 3 m wide by default, with optional **kerbs**.

  – ➤ Select a driveway for its panel: width, surface material (concrete, stone … or a picture), kerbs, on the ground / its level, Smooth the curve, Delete. The floorplan draws it with its centre line.

- 2.658.0: **maximize for the Sketcher, the Timeline and the Image editor** depends on where the docker is.

  – **Docked** (in the main column): makes it as tall as the column; the dockers below keep their heights and are pushed down (scroll the column to reach them), the column scrolls to the docker's top. / Esc puts it back.

  – **Floating** (outside the column): fills the whole window between the title bar and the status bar and closes the other dockers; / Esc (the Sketcher: ⇲ Dock) brings the layout back as it was.

  – **The Sketcher docked** now has its own height and a bottom handle (drag it to make it taller / shorter) and a pin like the other dockers; the height is remembered.

- 2.657.0: **Pool** in the Sketcher - under *Ground*, beside Terrain (no key - every letter is taken).

  – Click a corner and the opposite corner (or type `8,4` + Enter) - a rectangle pool; **Shape: Free shape** = click its corners, back on the first (double-click / Enter). Its panel: **Shallow end** / **Deep end** (the floor slopes between them along its length), **Edge**: edge tiles (coping), decking, edge tiles + decking or none, their widths, the water's edge level (the ground at its middle, or typed) and **Level the ground round it**.

  – On ground the ground is cut away under it and levelled round it (a cut / fill pad named "Pool 1 surround").

  – ➤ Select a pool for its own panel: Length / Width, the level, the depths, ⇄ swap the deep end, the edge and its materials (tiles, timber … or a picture), the lining, the water and its colour, the m³ of water; drag it, Delete it.

  – The floorplan draws the water, the edge, its name, size and depths.

- 2.656.0: ✂ **Cut / fill** in the Sketcher's Terrain - a level pad cut into / filled onto the ground.

  – Terrain ▸ ✂ **Cut / fill**: click the pad's corners on the ground, back on the first (or double-click / Enter). Inside, the ground is flat at the pad's **level** - the ground's mean inside it (balanced cut and fill), or untick *Level = the ground's mean inside* and type one. Outside, **batters** slope back to the natural ground at **1 : n** (metres across for each metre up / down). **Earthworks**: cut and fill, cut only or fill only.

  – The status line and the panel give the **cut** and **fill** in m³ and the net to take away / bring in.

  – Click a pad to pick it: change its level, batter or earthworks (cut and filled again), **Apply again** after sculpting, **Forget** / Delete removes its outline (the ground keeps its shape). Type a level in the box.

  – The floorplan shows each pad's dashed outline, name, level and cut / fill.

- 2.655.0: **starts by itself** - **Ctrl+click** the button: the restore is set up AND started (✏ Edit pressed for you a moment later); **Shift+click** (or Ctrl+Shift+click): the restore that also **colourises** a

black-and-white photo, started the same way. A plain click still only sets it up. With no photo to take (the References docker empty and the image editor closed) nothing starts and the note says what to do.

- 2.654.0: **Restore / fix a photo** - a button in the prompt docker's left column (under).

  – One click: Generate mode **Image**, the ▾ operation ✏ **Edit**, and the restore instruction leads the prompt - scratches, dust, spots, stains, creases and tears taken out, noise and grain reduced, faded colours and contrast fixed, soft detail sharpened, the same people, faces, clothes, pose and framing kept. Words you type after it are added ("her dress was dark green"). A second click adds nothing.

  – **Shift+click**: the same, and the photo colourised (for a black-and-white photo) - since 2.655.0 it also starts.

  – The photo: with the image editor closed, put it in the **References docker** (one picture - more are used together); with the editor open, the **selected layer** (or the selection on it). A note beside the button says which it will use. Then press ✏ **Edit** - the result lands in the gallery (the editor: Pending).

  – The edit model is the prompt docker's image style - an edit-capable one (Qwen Image Edit, FLUX.2 / FLUX 3 Klein) is picked for you while the editor is on screen.
- 2.653.0: **Terrain in the Sketcher** - the ground the building stands on, with hills, slopes and contour lines.

  – **Terrain** is under *Ground* in the Sketcher's tools (**Alt+G** - G is Group). The first click on a model without ground makes one: 40 m square or 10 m past the building, a height every 0.5 m, just under the lowest floor, grass.

  – The panel picks what a press does: **⬆ Raise** / **⬇ Lower** (press and drag; **Shift** the other way), **≈ Smooth**, **▬ Flatten** (toward the Flatten height - **Alt+click** the ground takes it). **Brush size** and **Strength**; **[/]** make the brush smaller / bigger, or type a size. Each stroke is one ↶ step.

  – **Contour**: set the **Elevation**, click, click … a contour line (it is drawn at its height); click back on its start to close it, or double-click / Enter to end an open one. With **Shape the ground after each line** on (the default) the ground follows every line drawn: each line's points held at its height and a smooth slope between them; the next line's elevation goes up by **Next line +**. A click on a drawn line picks it - change its elevation, open / close it, **Delete** erases it. **Shape to the contours** does it any time; **Clear the lines** keeps the shape. Tip: in the floorplan the lines are drawn straight down on the plan.

  – **Heightmap picture…**: a grey picture - black = **Lowest**, white = **Highest**, its top the north (+Y) edge.

  – ⌂ **Level under the rooms**: the ground under every room just below its floor, blended back over the **Level margin**. ⊙ Centre on the building**,** Width / Length / Detail **(its shape kept), its** Material** (a colour, a material or a picture), the **Contour interval** and every-n-th darker line, **Contour lines on the ground**, **Earth sides**, ▬ Flatten all**,** Remove the ground**.

  – The ground is saved with the model, goes to .glb / Blender / Unreal / the image editor with it, is in the real photo, and the floorplan draws its contour lines (the darker ones labelled) on the lowest storey.
- 2.652.0: **The real photo uses your materials** - **Photo** in the Sketcher's toolbar (or the prompt docker's ✏ Edit with the Sketcher linked) sends the edit model the clean render (your backdrop / sky in it), its depth map and a **material board**: one labelled tile for every picture the model uses ("Kitchen floor, Bed wall 1 outside", "outside walls", "Background (backdrop)") - the words tell the model to use exactly those materials there and keep the background. The prompt box's words are added. Edit models that take only two pictures get it without the board (the message says so - Qwen Image Edit 2511 / FLUX.2 Klein take three). The photo goes to the project's images (and the image editor when it is open).
- 2.651.0: **The Sketcher's material tools** - every part of the building its own material, pictures as textures, a picture as the background.

  – **A wall's inside and outside**: a wall's panel has **Inside** and **Outside**; a window has **Frame** and **Glass**; floors, ceilings, skirting, cornices, roofs, stairs and floors-without-walls have theirs.

Each opens the **material picker**: a colour, a built-in material (brick, stone, timber floors, tiles, marble, carpet …) or a** picture**.

– **Pictures as textures**: from the Refs docker's pictures, the project's images or a file. The picture is saved in the project and repeats every **Size** metres (turn it with **Turn °**). The same pictures can paint faces you drew.

– **The paint bucket** (B, the Materials button opens the same picker) now paints **one wall's side** - the inside or the outside you click; **Shift+click** paints every wall on that side (a room's insides / all the outsides). Glass takes its own paint.

– **Background** (the toolbar): **Backdrop** - a picture behind the model, filling the view without stretching - or **360° sky** - a panorama picture round the scene (turn it); None = the theme's.

- 2.650.0: **The Sketcher highlights what is picked and what is under the pointer** - the picked thing (a wall, a room, a door, a piece of furniture …, or several with Ctrl+click) is tinted in the theme's accent colour, and the thing under the pointer is tinted blue - unless it is already picked. On the floorplan the same: the picked symbol is filled in the accent, the hovered one in blue.
- 2.649.0: **The Sketcher's floorplan keeps the app's theme colours** - the background is the theme's (no more white paper), the walls, doors, stairs and labels are drawn in the theme's text colour, the floors in its panel colour (with a touch of each room's floor colour), glass and lights in its accent, the grid in its line colour. Changing the theme repaints the plan.
- 2.648.0: **Zoom with Shift + right-drag, ⊡ Zoom to selection and the pointer** - in the Sketcher, the timeline's canvas and the image editor. **The Sketcher's Esc and Ctrl+click.**

  – **Shift + right-drag**: forward (up) zooms in, back (down) zooms out, round the point you pressed. The plain right drag still pans in the Sketcher; a right-click with no drag still opens the menus. In Artist mode the drag zooms and a Shift+right-**click** (no drag) still takes you out.

  – **⊡ Zoom to selection**: the Sketcher (beside - what is picked: faces, a room, a wall, a door, a piece, a camera …, or several), the timeline (beside - the picked layer) and the image editor (under on the tool column, View ▸ Zoom to Selection, Ctrl+Alt+0 - the selection, or else the picked layer's pixels).

  – **The pointer**: the Sketcher's Select and the timeline's and image editor's Move tools are drawn as a pointer arrow.

  – **Sketcher Esc**: in Orbit, Esc goes to ➤ Select; any other time it drops the tool and the pick and goes to Orbit. It never docks the maximized Sketcher any more (⇲ Dock / do).

  – **Sketcher pick**: a click picks one thing; **Ctrl+click** adds more - faces and lines, and building parts (rooms, walls, doors, furniture, lights, cameras …; Ctrl+click a picked part takes it out). Drag one of them and they all move (one undo), Delete erases them all, ⊡ zooms to them; a click on one of them keeps just that one.
- 2.647.0: **The Sketcher's Floorplan view, Esc = Orbit, a room's Width and Length.**

  – **Plan** (the toolbar, or View ▾ Floorplan): the view looks straight down, on paper, one storey at a time (a list picks the storey when there are more). The walls are cut through and drawn solid; doors show their leaf and swing (single, double, sliding; an opening is dashed), windows their glass and sill, stairs every tread with the walking line and UP, furniture its outline and name, lights a circle and a cross, and every room its name, Width × Length and area. The building's tools work as in 3D - a room, a wall, a door … drawn on the plan is drawn with these symbols - and a click picks a door, a light, a piece, the stairs, a wall or a room. Orbit turns the plan round (it stays flat). again, another view or a camera's view: the 3D view back as it was.

  – **Esc**: one press drops the tool and what is picked and goes to **Orbit** - in the 3D view, the floorplan and a camera's view (a wall being drawn is kept). With nothing left to drop it puts a maximized docker back.

  – **Width and Length** for every room: its panel has them (along its first wall and across it); a rectangle keeps its corner, any other shape is stretched (its doors and windows stay in place along the walls). The Room tool has a Width and a Length too: set both and one click puts a room that size; 0 = draw it with two corners as before.
- 2.646.0: **Artist mode: Alt+P colours, Ctrl+P the prompt docker, Alt+G layers + pending** - generate from the layer you are painting on without leaving Artist mode.

– **Alt+P** - the colour palette at the pointer (it was Ctrl+P). Alt+P again closes it.

– **Ctrl+P** - the prompt docker over the full-screen canvas, its prompt box ready to type in (the same key as everywhere else in the app).

– **Alt+G** - the Layers panel on the right and Pending beside it, over the canvas (the canvas keeps its size, the other panels stay hidden). Pick the layer, Ctrl+P, write the prompt, ▾ Edit (or Generate): the picture comes back to Pending - keep it as a layer from there. Alt+G again hides them.

– Leaving Artist mode puts the editor's panels column back as it was (a narrow editor used to lose it).

- 2.645.0: **Ctrl+P brings the prompt docker to the front** - anywhere (typing in a box too): it opens, unfolds, comes out of a closed ▣ docker and over a floating window or a maximized docker (the Sketcher, the timeline …), with the prompt box ready to type in. (2.646.0: in Artist mode too - its colour palette moved to Alt+P.) The image editor's **File ▸ Print** moved to **Ctrl+Shift+P**.
- 2.644.0: **The guides say Sketcher** - the README, this guide, the Feature Reference, the beta page, the help and the shortcut list no longer describe the House builder (the Sketcher replaced it in 2.630.0); the Sketcher section starts with what it does now; a new Sketcher screenshot.
- 2.643.0: **Sketcher: move / turn a camera while looking through it** - ✥ Move / Rotate in the camera's panel show its widget in front of the view; Esc deselects it.
- 2.642.0: **Credits** - Settings ▸ **About** shows the version, "Produced and Developed by Tim Paul", "Programmer - Claude Opus 5.5" and the credits to everyone whose code, models, tools and ideas bitGen Studio uses; the same credits are the last section of this guide (and on its cover).
- 2.641.0: **Sketcher: cameras picked, moved and turned like objects** - no box round a picked camera; it shows X / Y / Z move arrows (drag the camera or an arrow), a double-click swaps them for the rotate rings, and clicking anything else picks that instead.
- 2.640.0: **Sketcher: each part's own properties, and profiles** - a clicked wall / floor / ceiling / skirting / cornice (view or outline) shows its own panel (a wall's length, height and inside colour); skirting, cornices, door frames, window frames and window sills take a profile from a picture dropdown, and ✎ Profiles… draws your own.
- 2.639.2: **Sketcher: the 3D view never goes black silently** - a bad number in a model (a corner, a light, a piece) no longer blacks the whole view: that piece is left out, the camera is put back, and a red note at the bottom of the view says what was wrong (a render error or a lost graphics context says so too).
- 2.639.1: **Sketcher fix** - docked in a narrow column the new outline took the 3D view's width (it showed black); there the outline now starts off and floats it over the view, and the view never gets narrower than 160 px.
- 2.639.0: **Sketcher: Outline docker** (rooms ▸ walls, floor, ceiling, skirting, cornice, doors, windows; hide / show; room groups), skirting + cornice, Alt+drag a wall freely, Shift+click = the whole room.
- 2.638.0: **SolidCraft maximized = the whole window** - fills bitGen's window between the title bar and the status bar and closes every other docker; (or Esc) puts the layout back exactly as it was (which dockers were open, which were folded, their sizes).
- 2.637.0: **Shift+right-click leaves Artist mode** - anywhere on screen, no context menu (Ctrl+Esc opens the Start menu on Windows; Ctrl+Esc and Ctrl+Shift+F still work).
- 2.636.0: **Ctrl+Alt+S opens Settings; the bitGen Default theme** - Ctrl+Alt+Shift+T (or bitGen Default in Settings ▸ Themes, or `?theme=default` in the address) puts bitGen's own look back - Dark, its accent, no element styles, UI size 100 %; ↶ The theme before undoes it. The built-in bitGen Default can't be overwritten or deleted.
- 2.635.0: **Assets: Project | Global | All** - project assets in `projects/<name>/Assets`, global ones in `projects/Assets`, a folder = a group; uploads go into Assets; right-click Move to Global / Move to Project.
- 2.634.0: **Setup in colour** - done green, installing light green, warnings orange, errors red; a quiet step says it is still working (and why it takes long) so setup never looks stalled - the setup window and the install scripts.

- 2.633.0: **Sketcher: rotate rings (yaw / pitch / roll) for cameras + furniture, roll, lens presets, focal length / field of view**.
- 2.632.0: **Sketcher: cameras picked inside rooms, moved, turned (R); Esc / double-click deselect everything; the Orbit cursor**.
- 2.631.0: **Sketcher: camera views, open doors / windows, auto roof + floor** - View ▾ lists the cameras (the view from one); a door / window's Open % + direction; the Room tool's Auto roof / Auto floor.
- 2.630.0: **The House builder is gone** - its last parts in the Sketcher (people, File ▸ its saved houses as Sketcher models); SolidCraft sends to the Sketcher.
- 2.629.0: **Sketcher: materials, cameras, 360° locations, Furnish** - the House builder's materials with textures, cameras (look through, shot + depth + normal maps, a 360° Location), Furnish a room; the furniture exported.
- 2.628.0: **Sketcher: Furniture** - the House builder's furniture library and the project's 3D models placed in the Sketcher; maximized closes every other docker (⇲ Dock opens them again).
- 2.627.0: **The Sketcher's real photo** - a clean render + its depth map go to the edit model, the prompt led by "Turn this 3D render into a photorealistic photograph of a building ..." (it came back as a shaded sketch before).
- 2.626.0: **PushPull is the Sketcher** - with the prompt docker: Image + an edit model, "create a real photo of the model in view", Generate = a real photo in the Image assets; → Image editor / → Timeline send a Sketcher layer.
- 2.625.0: **Sketcher: faces, edges and corners edited** - a face's Extrude (+ / - a cut) and Twist, dragged like Blender, Ctrl+drag twists; edges moved, Length, Extrude edge, Fillet, Shift+click more, Ctrl+click the face's loop; a corner moved; doors and windows resized / moved by their grips, a window's Elevation; keeps the status bar.
- 2.624.0: **Sketcher: walls on a floor, sloped walls** - bitGen's theme colours; Offset from floor edge; drag one wall, the floor's snap regions, onto its edge; double-click a wall's top = a cut to drag up / down, a top edge's slope, Delete; Ceiling Flat / Follows the walls' slope; a roof that follows them with an Eaves width; double-click empty = Orbit; / ⇲ Dock and × in its title bar.
- 2.623.0: **Edit theme mode: every button and its words** - double-click any words to change them, dropdown options, placeholders and tooltips, the floating tool panels, Alt+click to use the app without leaving the mode.
- 2.622.0: **ComfyUI found by itself** - an installed ComfyUI that Settings does not name yet is found, and its checkpoints and diffusion models are in the Model manager and Basic Settings' model lists (the video Model list had stayed empty - it read the list the wrong way; fixed). Settings ▸ ComfyUI ▸ Folder offers Use this folder.
- 2.621.0: **Speech & dialog in the SFX dock** - text typed, pasted or written by the LLM, read with emotion by OmniVoice or Ming-3.0 TTS in one voice or a voice per speaker, saved as .wav / .mp3 into the Audio assets with the date, time, description and text in the file's metadata.
- 2.620.0: **Sketcher builds** - rooms with walls, floors and ceilings that snap together and share a wall when dragged, doors and windows through any wall, roofs (gable, hip, shed, flat), stairs, floors, and lights (downlights, ceiling lights, pendants, chandeliers, spots, wall lights, floor and table lamps) with a night view; the docker maximizes to the whole window, has its own File menu and / buttons.
- 2.619.0: **Sketcher** - a new docker for quick 3D modelling: draw a shape, push / pull it into a solid (pockets, holes through, walls from an offset), exact sizes typed in, snapping to corners / edges / faces / the axes, groups, paint, undo; models kept in the project; sent on to the image editor, the timeline, Blender and Unreal, or exported as .glb / .obj / .stl.
- 2.618.0: **A bridge layer deleted or hidden closes its bridge** - in the image editor: a Blender layer hidden or deleted closes the scene tools (no new layer is made) and the scene itself (the timeline's preview of it too); a 3D (Daz / Blender) layer ends its scene's Blender and drops what was waiting for Daz's live link; an Unreal / MetaHuman layer stops the Unreal View, clears its picture and hides it. In the timeline: an Unreal clip whose layer is hidden () or that is deleted closes its

window. Shown again (or undone), the layer still has its picture - pick it to open its scene / view again.

- 2.617.0: **Text effects from a zip** - Presets ▾ Import PSD text effects or a zip… takes a zip of text effects (every PSD in it becomes a preset, the fonts in it are added to the studio) or several PSDs at once.
- 2.616.0: **Install with one file** - `install_all.bat` (Windows), `install_all.sh` (Linux) or `install_all.command` (macOS) on its own installs Git and Node.js when they are missing (Windows: Python too), clones the repository next to it and starts the studio (the first start installs the rest). Run it again any time to open the studio.
- 2.615.0: **Basic Settings follow the style** - each style remembers its own Basic Settings in misc/basic_settings.tmp. Changing the style anywhere (the prompt docker, Basic Settings' style card, a board …) first saves every Basic Settings field + the style's name for the style you are leaving, then puts back what you last had for the new one. The app starts on the last session's styles with their settings.
- 2.614.0: **Pick ▼ ranges, ✂ Cut cuts them; Ctrl taps while it plays always count** - a click on a ▼ picks its range (Ctrl / Shift + click: more) and ✂ Cut cuts the picked ones (none picked: all). Ctrl taps while the preview plays now count with ▶, the speed list or a ticked box focused, and while the preview docker's player plays a timeline clip.
- 2.613.0: **▼ Ranges with Ctrl while it plays, ✂ Cut all / Speed up, hover time, playback speed** - tap Ctrl while the preview plays to mark a range's start and again for its end (as many as you like, each its own colour, on the highlighted layer); drag a ▼ to move it (the preview shows that frame), right-click removes it. ✂ Cut all cuts every range; Speed up plays them 2× - 10× faster (fast-forward a render). The time under the pointer shows in a small box; the preview plays at 0.125× - 8×.
- 2.612.0: **A cut clip stays one clip / ✂ to new layer** - ✂ Cut and Del on frames join the parts before and after the cut into one continuous clip again (a video or sound: one new "-join" file; a picture: just shorter), one ↶ undoes it. The new "✂ to new layer" box puts the part after a cut (or a Ctrl+K split) on a new layer right above. The preview loads the next clip a second early - no black frame at a cut.
- 2.611.0: ▯ **Frame - click a frame, Del removes it** - the ▯ tool beside ▸ ⇤ ⇹ ⇆ ↔ on the timeline (or F over the layers): a click on a layer selects the one frame under the pointer on that layer, a drag several, Shift+click up to here, ← / → a frame on / back. Del / ⌫ cuts them out of that layer and closes the gap; ↶ puts them back.
- 2.610.0: **Morph on the timeline** - the Morph tab beside Properties (the timeline's layers panel): write Character 1 (what it starts as, e.g. "a monkey") and Character 2 (what it turns into, e.g. "a man"), pick Layer 1 (the clip it starts from - its LAST frame) and Layer 2 (the clip it ends on - its FIRST frame), the length (1 - 10 s). The prompt is written for you (edit it if you like - ↺ writes it again). Make the morph renders it with the video style picked in the toolbar from those two frames (as the canvas shows them), puts it on a new Morph layer between the two layers starting where layer 1's clip ends, and moves layer 2's clip to start where the morph ends. Its sound is off unless you tick "keep its sound". ↶ undoes it all. Works best with the two characters in a similar framing.
- 2.609.0: **Dock any floating docker in the image editor's panel column** - drag a floating docker by its title over the column where Layers is: an orange line shows where it goes (above / below each docker there, or at the end); let go = it is docked there. Works for a main docker floated with ⇱ (Assets, Gallery, References, Queue, Preview …), a ▣ docker, Basic Settings, LLM tokens, the Camera panel, the prompt docker and Pending. Docked, each has a slim bar: drag it to another place in the column or out of it (it floats again), ▾ folds it, ⇱ floats it, ⚙ (a ▣ docker) its menu; its bottom edge sizes it. Where each one is docked, the order, its height and fold are kept and come back at the next start; docked in the editor they hide and show with it.
- 2.608.0: **Font lists show each font in itself** - every font dropdown writes each font's name in that font (type to search as before), and the closed list shows the picked font in its own lettering.
- 2.608.0: **Fill a shape with text** - image editor: right-click a shape layer (or the Shape tool's bar, or Layer ▸ Fill Shape with Text…) and pick how: **Flow inside** (a text layer whose lines wrap inside the

outline; Fit = the biggest size that fits), **Stretch to fill** (the letters bend and stretch to fill the outline - a heart, a circle, a star) or **Repeat as fill** (the shape's fill becomes rows of the text, at an angle - it stays a shape). Flow / Stretch take a text preset and stay text you can type into. Timeline: a shape layer's ◆ Shape ▸ Text fill (rows), and a text layer's In a shape (Stretch to fill / Flow inside a shape layer; Place on it).

- 2.608.0: **5 more warp presets** - Shell Lower, Shell Upper, Fisheye, Twist and Wedge, next to Arc … Squeeze: in Shape / 3D…, Edit ▸ Transform ▸ Warp and the timeline's Warp ▸ Preset.
- 2.608.0: **A warp grid on text and shapes** - Warp Grid (the Type and Shape tools' bars, right-click, Edit ▸ Transform ▸ Warp): drag the grid's points or anywhere inside, pick a preset, Grid 3 x 3 / 4 x 4 / 5 x 5, split lines - Enter applies. The layer stays text (type into it - the warp follows) or a shape (change its colour - still warped); Layer ▸ Remove Warp Grid takes it off. Timeline: Warp ▸ Preset turns into a mesh you drag with (Shift+W).
- 2.607.0: **Text presets stay on through shapes and 3D** - a text with a preset (Gold, Chrome, a PSD text effect …) keeps it when you warp it, put it round a circle or along a path, make it vertical or 3D (image editor ▸ Shape / 3D…); the 3D sides take the preset's colour, darker (or the side colour you set). On the timeline a preset text now gets Text Animators and On a path too, and 3D text (▸ 3D text) has its own Preset ▾ - the preset's look is the front face, the sides its colour.
- 2.607.0: **New text starts with your preset** - on the timeline, with nothing selected Properties shows New text ▸ Preset; the T tool's next text starts with it (picking a preset on any text sets it too).
- 2.607.0: **Every font on your computer** - each font list (image editor Type tool, Properties, Character panel; timeline text, 3D text, captions; karaoke style) keeps the usual fonts at the top under "Common" and lists every font installed on this computer under "This computer (N)".
- 2.606.0: **A click on a text picks it and switches to the Move tool** - also with the Type tool in hand (it used to open the text for typing). **Double-click** a text to edit it - the caret goes where you double-clicked. Picking a layer in the Layers panel with a drawing or Type tool in hand also switches to the Move tool. The rotate / skew handles: the new Scale / ↻ Rotate / skew button in the Move tool's options bar, or double-click on one of the handles (double-clicking inside a text now edits it).
- 2.605.0: **Hover shows the Move tool's thin frame** - in the image editor and on the timeline's preview canvas, an object under the pointer (a text, a shape, a picture layer…) shows the same thin dashed frame the Move tool uses, not a thick outline - with the Move tool and also with the drawing tools (and the Type tools over a text). A **click** on an object with a drawing tool picks it and switches to the Move tool; a **drag** still draws a new one. A text is picked anywhere in its box, also between its letters.
- 2.605.0: **Fixed: a second shape lost** - drawing a shape while another shape was picked did nothing (an error); it now draws a new shape (the Ops buttons still combine shapes when you pick one).
- 2.604.0: **Image editor ▸ Save to Images** - pick the format beside (PNG, JPEG, WebP, JPEG XL, AVIF, GIF, TIFF, BMP, TGA). Saving first keeps a temp copy of everything visible, then flattens it (JPEG / BMP on white - not black) and saves it in that format into the project's Images (Assets) AND the Gallery (a format the Gallery does not list - WebP, JPEG XL, … - shows there as a PNG of the same picture). Your layers stay as they are.
- 2.604.0: **Skew fixed** - with the Move tool, double-click for the rotate handles and drag a side handle: that side slants and the opposite side stays put (it used to swing the other way). Alt slants round the centre. Text skews too and stays editable text.
- 2.604.0: **Right-click ▸ Rasterize Text** - on a text layer in the image editor and on a text on the timeline's preview canvas: the text becomes a picture (undo brings it back).
- 2.604.0: **No thick box round a picked object** - the image editor shows only the thin handle frame; the timeline canvas a thin dashed frame with small handles.
- 2.603.0: **Timeline: a range on one layer** - drag the red position marker to where the range starts, then Shift+click on a layer where it ends (on a clip or between them; Shift+drag lets the end follow the pointer). The range is drawn on that layer's row and the layer is highlighted. ✂ **Cut** then

removes the whole range from that layer only - the other layers are untouched (with ripple the gap closes on that layer). A drag across the ruler is still a plain selection.

- 2.603.0: **Timeline undo** - ↶ / ↷ in the timeline's menu bar, Ctrl+Z / Ctrl+Shift+Z (or Ctrl+Y) with the pointer over the layers. Every edit is a step (a drag is one step); the selection is not.
- 2.603.0: **ComfyUI left running is restarted** - if a ComfyUI on this computer is already running when the app starts (left from before, or started by hand), the app ends it and starts it again itself, so its console, Restart and memory are the app's. Settings ▸ ComfyUI ▸ "Restart a ComfyUI left running when the app starts" (on).
- 2.602.0: **Image editor ▸ Type tool** - the dashed frame round the text you are typing now fits the letters exactly and grows with every key (centred text grows both ways). With the Type tool the pointer is the text I-beam over a text and the new-text cursor (an I-beam in a dotted box) elsewhere. A click on a text starts editing right where you clicked - between the two letters nearest the click - instead of selecting it all; a click inside the text you are typing moves the caret there. The text's box for the Move tool's handles, Align and clicking is now as wide as the letters (it was measured too small).
- 2.601.1: **Fixed: Ming-3.0 TTS ""mul_cuda" not implemented for 'BComplex32'"** - Ming runs its whole model in bfloat16, and the last step that turns its output into sound does maths with complex numbers, which PyTorch cannot do in bfloat16. bitGen now does that one step in full precision (float32), the rest stays as fast as before. Nothing to reinstall.
- 2.601.0: **Reinstall, and every install button says what it is doing** - an add-on, package or model that is already installed shows **Reinstall** (a downloaded model: **Download again**), and pressing it really fetches everything again instead of skipping what is "already there". The moment an install or download starts its button is disabled and a status right beside it says what is happening - the step it is on, a percentage, the seconds - then ✓ **installed** or ✗ **why**. Settings stays open while it runs. This covers Settings ▸ Add-ons, faster-whisper, OmniVoice, sd-cli, Comfy Kitchen, Sage / Spectrum / the node packs, the model catalog, Civitai downloads, the LoRA trainer, Viggle (native), the package upgrade, extensions from a folder, and in the dockers demucs (Stems), the AI matte, SAM 2, Select ▸ Subject and Segment Anything.
- 2.601.0: **Settings ▸ Audio ▸ voice clone: ▶ Play** (was "Play selection") - plays the selection when there is one, the whole clip when there is none; it turns into ■ Stop while it plays and a white line follows on the waveform. A click on the waveform (no drag) clears the selection.
- 2.601.0: **Fixed: Ming-3.0 TTS "TorchCodec is required for load_with_torchcodec"** - a new torchaudio hands audio reading to a separate package (TorchCodec) that also needs FFmpeg's libraries. bitGen now reads and writes the voice clips for Ming-3.0 TTS (and OmniVoice) itself, so nothing needs reinstalling.
- 2.600.1: **Fixed: "transcription failed: open() got an unexpected keyword argument 'metadata_errors'"** - when a voice clone's transcript (or any faster-whisper transcription) was made. A new version of PyAV (the video / audio reader faster-whisper uses) dropped an option faster-whisper still passes. bitGen now reads the audio itself and hands faster-whisper the sound directly, so it works whichever PyAV is installed - nothing to reinstall. A new install of faster-whisper also keeps a PyAV that still has the option.
- 2.600.0: **Social media in Basic Settings ▸ Format** - for quick access, without the timeline's Export. The Format dropdown has a **Social media** group: **Reels / Stories 9:16**, **Facebook feed 4:5**, **Square 1:1**, **Landscape 16:9** and **the render's own shape**.

  – Picking one sets **Aspect** to match, so the model renders that shape (the feed's 4:5 renders 3:4 and is cut to 4:5). The line under Format says what will happen.

  – When the render finishes, and before it lands in the gallery and assets, the MP4 is made ready for the sites with the same settings as **Export ▸ MP4 · Social media**: H.264, AAC sound at 48 kHz stereo (a silent track if it has none), a steady frame rate, faststart, at most 1080 on the short side. The job's log says "ready for Facebook / Instagram / TikTok", or what is wrong. If it cannot be converted, the plain MP4 is kept and the log says why.

  – Works for MiniMax H3, Wan and LTX renders and the music video.

- 2.599.0: ↺ **Reset project shows a lock screen.** After you confirm, a cover over the whole window says **"Resetting Project..."** - a spinning ring, the dots running, the step it is on (stopping jobs, deleting and creating the folder again, reopening) and the seconds. Nothing behind it can be clicked or typed into. It stays through the page reload and goes only when the empty project is open (✓ **Project reset**). If the reset is refused, the cover goes and the reason is shown.
- 2.598.0: **The seed has a Random / Fixed toggle** - in Basic Settings (video and image) and in every style editor (image, video, music).

  – **Random**: every render gets a new seed (the number box is greyed). **Fixed**: every render uses the number beside it - the same picture again with the same settings. Typing a number, or ♻ (the last seed used), makes it Fixed; switching back to Random keeps the number for next time.

  – **In a style**: Fixed means that style always renders with its own seed. Which seed a render uses: Basic Settings' Fixed seed first; with **Override style** on, Basic Settings' Random stays random; otherwise a style's Fixed seed; otherwise a random one. Styles saved before have Random (their old seed is not used, as before).

  – The image settings' seed is sent while Override style is on.
- 2.598.0: **The Model manager finds the models and LoRAs the running ComfyUI has**, also when they are not in the ComfyUI folder set in Settings ▸ ComfyUI (the ComfyUI Desktop app keeps them in its own folder, or a yaml is given on ComfyUI's command line). It asks ComfyUI for its model folders and for the names in its dropdowns; a model ComfyUI lists whose file this computer cannot open still gets a card (its size says so). Press ↻ in the manager after starting ComfyUI.
- 2.597.0: **MP4 · Social media** in the timeline's **Export** - for Facebook, Instagram and TikTok. Pick it as a format, or **a pack… ▸ social media** for four at once: **9:16** (Reels / Stories, a blurred fill), **4:5** (the Facebook feed), **1:1** and **16:9**, each 1080 on the short side.

  – It is made the way the sites want it: H.264 (High, 8-bit), AAC sound at 48 kHz stereo, a steady frame rate (a movie above 60 fps comes out at 30), the index at the front of the file (faststart), no timecode track, at most 1080 on the short side, with a bitrate ceiling so files stay a sensible size. A movie with no sound gets a silent track (some uploaders refuse a video without one).

  – After it is made, the file is checked and the log says "ready for Facebook / Instagram / TikTok" - or which point is wrong. A 9:16 longer than 90 seconds gets a note: it may post as a plain video instead of a Reel.

  – Why other files were refused: AVI (an old video codec with uncompressed sound) is not for social media; the 10-bit, H.265 and AV1 MP4s are for editing and newer players; and the plain MP4 · H.264 export adds a timecode track when the timeline has a start timecode. Use MP4 · Social media for anything you upload.
- 2.596.0: **Update llama.cpp** in **Settings ▸ LLM**. The line under the buttons says which llama.cpp you have and whether a newer build is out ("llama.cpp b10456 · the newest is b10601 - Update available").

  – **Update llama.cpp** stops the LLM server and the vision model, then puts the newest build over the engine folder set above (Windows: the CUDA build your driver supports). Your models and settings stay. Its progress shows in the small log right there and the Settings window stays open. ⬇ **Install engine** shows instead while none is installed.

  – Every button in the LLM tab shows what it is doing while it does it: ▶ **Start server** says "Starting the server…", then "Loading <the model>…" until the model is ready, then "✓ Model loaded"; ■ **Stop**, ↻ **Rescan**, **Scan folder** and Settings' **Save** say "Stopping / Scanning / Saving..." and then ✓ - or ✗ with the reason.
- 2.595.0: **WeVisOne-VL** (Tencent, built on Qwen3-VL) as a vision model. **Settings ▸ Add-ons ▸ WeVisOne-VL** has a dropdown: ★ **4B Q4_K_M** (recommended), 4B Q6_K, 4B Q8_0, 2B Q8_0 and 2B Q4_K_M - each with its projector.

  – It downloads into **your vision models folder** (the one set in Settings ▸ LLM; without one, the LLM folder's vision subfolder), in its own WeVisOne-VL-4B or WeVisOne-VL-2B folder, so it pairs with the right projector.

  – Then pick it in the prompt docker's vision dropdown: pictures in the prompt are read and described for the LLM. It is good at text in pictures, counting and saying where things are.

– It needs a recent llama.cpp (miscget-llama.ps1). Its segmentation and matting are not part of the GGUF files - in bitGen it reads and describes pictures.

- 2.594.0: **Convert to 3D from the right-click menu.** Right-click a picture in the **Gallery**, **Assets** or **References**, or the **image editor's canvas** (it adds Convert Picture to 3D… / Convert Layer to 3D… to its menu; the Layers panel and the Layer menu have Convert Layer to 3D… too).

  – Pick the engine: **Hunyuan3D 2.1** (the shape, quick) or **TRELLIS.2** (shape + texture, slower). Both run in ComfyUI, as the buttons do.

  – Pick where the model goes: **Image editor** (a 3D layer - Blender puts the model in a lit scene with a camera framed on it; orbit it, render it and generate from it in the 3D tab - needs Blender), **SolidCraft docker** (opens as a new design) or **Just save it** (the project's models3d/ folder).

  – The job runs in the queue; when it is done the model opens where you said. A References or editor picture is kept in the project's images first.

- 2.594.0: **Ming-3.0 TTS** - a second speech engine. Install it in **Settings ▸ Add-ons ▸ Ming-3.0 TTS** (its code, its own Python and the 0.5B model, in the ming-tts folder), then pick it in **Settings ▸ Audio ▸ Speech engine**.

  – A cloned voice needs its clip **and** the exact transcript; a designed voice (a description) uses Ming's voice design. The chat , the Speech dock and Test voice all use the picked engine.

  – Its authors test it on Linux with CUDA 12.4. On Windows it is new ground: an RTX 50-series gets the CUDA 12.8 build of PyTorch. Without the text normaliser (pynini, usually missing on Windows) write numbers as words.

- 2.594.0: **Bark 3.0 speech-to-text** (Underdog). Install it in **Settings ▸ Add-ons ▸ Bark 3.0 speech-to-text**, then pick it in **Settings ▸ Audio ▸ Speech-to-text engine**.

  – It is **English only** and gives **no word times**, so it does the plain transcripts: the Transcribe button, voice clone transcripts, the phone remote and the voice chat. Lyrics, captions, word times and other languages always use faster-whisper (keep it installed for those).

  – Long files are cut into pieces of up to 30 seconds at quiet moments, so a transcript still has rough times.

  – License CC-BY-4.0 (its NOTICE file stays beside the model).

- 2.593.0: **Face fix and ControlNet are add-ons now.** Switch them on in **Settings ▸ General ▸ Add-ons in the studio** (both start off).

  – **Off**: Basic Settings hides the Face fix / ControlNet section (ControlNet also hides its Settings tab), Settings ▸ Add-ons hides its card, and no render uses it.

  – **On**: they show again, and bitGen checks everything the add-on needs and downloads whatever is missing into the right ComfyUI folder by itself - right away, and again every time it starts while the add-on is on. Settings ▸ Add-ons shows the progress.

  – **Face fix** needs the ComfyUI-Impact-Pack and ComfyUI-Impact-Subpack node packs and the face detector face_yolov8m.pt (models/ultralytics/bbox).

  – **ControlNet** needs comfyui_controlnet_aux, its four preprocessors' models (DWPose, PiDiNet, Lineart, Depth Anything V2 - in custom_nodes/comfyui_controlnet_aux/ckpts) and the SDXL ControlNets (xinsir Union ProMax for every type, plus OpenPose, Scribble and Depth - in models/controlnet, about 10 GB). They are for an SDXL / Pony / Illustrious second model.

  – Each card in **Settings ▸ Add-ons** lists every download link (the GitHub node packs and the Hugging Face files, with the folder each one goes in), so you can also fetch them yourself.

  – After a node pack is added, restart ComfyUI once.

- 2.592.0: **Two characters, each with their own LoRA, in one picture** (from Winnougan's Nougan Nodes - Settings ▸ Add-ons ▸ **Nougan Nodes** installs it into ComfyUI; restart ComfyUI after). In the image editor's AI panel ▸ Zones ▸ ▦ **Regional prompt**, every box line has a button: pick the character's LoRA from ComfyUI's loras and its strength. Each LoRA works only inside its own box(es), so two people keep their own faces instead of blending into one; the scene's words still cover the whole picture, and boxes without a work as before.

  – Two different character LoRAs at most per picture (a third is refused before anything renders). Two boxes may use the same LoRA.

– Best on FLUX.2 / Krea 2 / Qwen Image styles, with LoRAs trained for that model. On Qwen Image **Edit** the picture stays the model's own input: the LoRAs keep to their boxes and the boxes' words become area prompts.

- 2.592.0: **Timed lines in an LTX prompt (Prompt Relay)** - give each stretch of an LTX 2.3 / 2.5 clip its own words in ONE render. Lines without a time are about the whole clip; a line that starts with a time is about that stretch:

a red fox in the snow, cinematic [0-2s] the fox sniffs the ground [2-4s] the fox looks up at the camera [4s] it runs off to the left

- Each line runs until the next one starts (the first from 0 s, the last to the end of the clip). Times are seconds: `[2s]`, `[2-4s]`, `[2.5 - 4 s]`, `[2 to 4]`.
- beside the LTX mode in the prompt docker adds the next line (2 s each). It works the same in a scene on the StoryBoard and in the timeline's AI clip.
- Needs Settings ▸ Add-ons ▸ **Nougan Nodes** (Kijai's ComfyUI-PromptRelay works too). Without it the clip renders all the lines as one prompt, and the job's log says so. The timing is set in the first pass; the template's second (detail) pass keeps the whole-clip words.
- 2.591.3: **GitHub's tests are green again.** Git on Windows can store .bat files with Linux line endings, and a new test read them as they were checked out and failed. A .gitattributes file now makes every .bat / .cmd check out with Windows line endings, and inside a git checkout the test checks that rule instead of the bytes.
- 2.591.2: **push-to-github.bat is back in the app folder** (beside start.bat) and it is **never in the repository**: GitHub does not get a copy (it is ignored and left out of the repo zip). It comes with the flat zip. Dropping the repo zip on it no longer deletes it from the folder. A copy left in misc by 2.591.0 / 2.591.1 moves back up by itself the next time bitGen starts.
- 2.591.1: **GitHub's tests after the misc move.** A repository updated by unzipping over it (or by an upload that only adds files) still had the old root copies, and GitHub's tests stopped on them. Now those copies are not an error, and **push-to-github.bat** removes them before it commits, so GitHub gets the new layout.
- 2.591.0: **A tidy app folder.** The bitGen Studio folder now shows only **start.bat** (plus package.json / package-lock.json, which npm needs beside node_modules, and README.md / LICENSE for the GitHub page). Everything else that sat beside them - server.js, the Python workers, default.settings, quick_prompts.json, requirements.txt, the setup scripts (get-comfy.ps1, get-llama.ps1, get-sd-cli.ps1, get-yue2.ps1, get-qwen3vl.ps1, get-electron.ps1, download-models.ps1), clear-comfy.bat, reset-layout.bat and the notes - is in **misc**.
- Start as before: double-click **start.bat**.
- Updating from a zip over the old folder: the old copies left in the folder are tidied away by themselves the first time bitGen starts (your own quick prompts are kept - they move into misc).
- push-to-github.bat: (2.591.2: back in the app folder, beside start.bat, and never in the repo.)
- The folders and files bitGen makes while it runs (projects, styles, models, custom.settings, the layout files …) stay where they were.
- 2.590.0: **Artist mode** is the new name of the full-screen canvas (2.588.0's ⬚ focus mode): a labelled **Artist mode** button right after the image editor's menus (File … Help), View ▸ Artist Mode, Ctrl+Shift+F. Only the canvas shows; Ctrl+P colours (2.646.0: Alt+P), Ctrl+B brushes, Ctrl+Esc back. The editor's menu bar now wraps onto a second line when the editor is narrow, so no button is cut off any more.
- 2.590.0: **Every tab of the image editor is a docker.** Drag a tab (Layers, Color, AI, Unreal, Brush …) by its name:
- **onto another docker** (its tabs or its content, in the panel column or floating) - it becomes a **tab** of it (the docker lights up orange);
- **onto the panel column at the top or bottom edge of a docker** (or below the last) - a docker of its own there (an orange line shows where);

- **anywhere else in the editor** - it **floats**: drag it by the empty part of its tab strip, size it by its corner, ⇲ puts it in the panel column, ✕ hides it.
- at the start of a docker's tabs drags the WHOLE docker the same way (onto another = all its tabs join it).
- A click on a tab still just opens it. Where every tab is, is kept. **Window ▸ Show All Dockers** shows the hidden ones, **Window ▸ Reset Docker Tabs** puts every tab back where it started.
- 2.590.0: **Unreal View** - what the selected Unreal / MetaHuman layer's camera sees in Unreal, live (a few frames a second):
- **Image editor**: a floating docker that opens when you pick an Unreal / MetaHuman layer (☑ show it when an Unreal layer is selected) and follows the layer's camera as the camera tool, the Unreal tab, the lens and the lights change it. Picking another layer stops it. It is a tab like the others - drag it onto a docker to keep it there.
- **Timeline**: a window over the preview row that opens when you pick an Unreal clip (UNREAL ▸ View ▸ Unreal View shows / hides it): the clip at the **playhead** - an Unreal camera between its keys, a MetaHuman in its pose, a Level Sequence at that frame (Sequencer opens it there).
- Its bar: ● live · frames a second, the size (S / M / L - smaller is faster), , and ⚡ **Keep Unreal fast**.
- **Do you have to keep Unreal open?** The Unreal Editor must be RUNNING with your project open (the bitGen Bridge lives inside it) - but it does not have to be in front or on screen: minimize it or keep it behind bitGen. By default Unreal slows to a few frames a second when it is not the window in front; ⚡ **Keep Unreal fast** turns that off (Editor Preferences ▸ General ▸ Performance ▸ Use Less CPU when in Background), so stills, renders and the live view stay quick.
- Needs the **bitGen Bridge plugin 1.5.0** - Load UE project updates it (the 1.4.0 plugin shows the view more slowly, and not for sequence clips).
- 2.589.0: **The Team render farm is now an optional add-on** (it is no longer part of bitGen). It comes as its own zip, **bitgen-ext-teamdock-v<version>.zip** - the install, repo and flat zips do not carry it. ⚠ **It may trigger virus checkers**: it makes signing keys, listens for the team's PCs on a network port and writes to the team folder every 15 s, which Malwarebytes has flagged as ransomware. Install it only on the PCs that render for each other:
- **Settings ▸ Add-ons ▸ Team render farm ▸ Install from a zip…** (pick the zip), or put the zip in the app folder or Downloads and press **Install**. **Restart bitGen** - Settings ▸ Team and the status bar's Render on appear.
- **Remove** takes it out again (restart). This PC's team identity and keys stay in the app's team/ folder - delete that folder to forget the team. If your antivirus stops it, allow bitGen's folder or remove the add-on.
- Without it every job renders on this PC; a projects folder already pointed at the team's NAS folder keeps working.
- 2.588.0: ⚡ **Live generation in the image editor** (Krita's live mode) - ⚡ in the editor's menu bar (beside) or View ▸ ⚡ Live Generation opens the ⚡ **Live** docker. Paint: every change (a stroke, an undo, a layer shown / hidden or added, the prompt, a setting) is rendered again a moment later - the whole picture as a Refine at the live strength.
- **One render at a time**; changes made while it renders are rendered next (the newest change wins - a waiting live render is dropped when a newer one comes). The model stays loaded between live renders (no VRAM clear) and live renders go to the front of the queue; they never show in the queue docker, the gallery or the assets.
- The docker: the result (hold it = full size on the canvas), the **style** (a fast one feels live: turbo / lightning / LCM / FLUX schnell / Z-Image Turbo at 4-8 steps), **strength** (30-50 % keeps your drawing), **steps** (0 = the style's own), **size** (MP - smaller is faster), **seed** (the same seed = the result only changes where you paint; a new one), ▣ **on canvas** (the result over the picture with a mix), pause, ↻ render now, ✔ **Apply as layer** (one undo step), ✕ live off.
- The prompt is the AI panel's box, or the main prompt box when that is empty.

- 2.588.0: ⬚ **Focus mode in the image editor** (2.590.0: called Artist mode) - ⬚ in the menu bar (or View ▸ Focus Mode, Ctrl+Shift+F): only the canvas, full screen and frameless - bitGen's bars and the editor's menus, tools, panels, rulers and dockers are hidden. Painting, [ ], zoom, Space-pan and undo work as always.
- **Alt+P** (Ctrl+P until 2.646.0) - the colour palette at the pointer: a saturation / brightness square and a hue strip (drag), the foreground / background colours (⇄ swaps), the hex value, the palette's chips and your recent colours - left click = foreground, right click = background. Alt+P again, Esc or a click outside closes it.
- **Alt+G** (2.646.0) - the Layers panel + Pending over the canvas; **Ctrl+P** (2.646.0) - the prompt docker.
- **Ctrl+B** - the brushes at the pointer: every brush preset (click = paint with it) + size, opacity and flow.
- **Ctrl+Esc** - back to normal (Ctrl+Shift+F too - some Windows keyboards keep Ctrl+Esc for the Start menu). The browser's own Esc only leaves the browser's full screen; the canvas stays frameless until Ctrl+Esc.
- **Shift+right-click** (2.637.0) - back to normal too, anywhere on screen (Windows uses Ctrl+Esc for its Start menu); no context menu opens.
- 2.588.0: **docker ▸ RefMods** - the top of the LoRA & fine-tune trainer docker switches between Training and RefMods: MiniMax H3 reference mods (no training) kept in the app's **refmod/<name>/** folder, so **every project** uses them.
- **New RefMod**, then its **type**: Character reference · Location · Vehicle · Object · Prop · Flora · Animal, and a short description.
- **Media…** (or drop files): pictures go into images/, videos into videos/, sounds into audio/ - each with its caption box (a picture.txt uploaded next to a picture is its caption) and ✕.
- **Autocaption** - the vision model writes the empty captions (all = every one again); a video's caption comes from one of its frames; sounds keep the caption you type.
- **Extract RefMod** - a queue job through ComfyUI (the ComfyUI-MiniMaxH3Mod pack): the pictures, then frames spread over each video (16 in all) -> <mod>.safetensors, kept in the RefMod's folder (and copied back into ComfyUI's models/refmods whenever a render needs it). Sounds stay with the dataset - the RefMod node takes pictures.
- **Use in the prompt** puts **@name** in the prompt box: a MiniMax H3 render (References / First-last frame) loads the mod before the sampler; the @ is taken out of the words the model reads.
- 2.588.0: **Settings ▸ Add-ons ▸ SolarRenderer** - render a STRUCTURE video (grey boxes, a semantic map, skeletons, depth, normals, a body mesh, a sketch) plus an optional first frame and a six-part prompt into the finished clip in 4 steps (Enderfga's control + four-step LoRAs on MiniMax H3; a causal pair for long clips). The add-on puts the SolarRenderer node pack into ComfyUI's custom_nodes and downloads the LoRAs you tick; it renders on MiniMax H3's ref2va model, text encoder and VAE (Settings ▸ Setup). Then **Settings ▸ ComfyUI ▸ Graphs ▸ ☀ SolarRenderer** imports the pack's workflows through Import & check (inputs counted, missing files said, a test run, saved when it works). The weights carry the MiniMax H3 Community License - the EU, the UK, South Korea and the US are excluded territories.
- 2.587.0: **Settings ▸ ComfyUI ▸ Graphs ▸ Import & check** - bring in any ComfyUI workflow (a saved workflow or an API export) and bitGen checks it, draws it, test-runs it and keeps it when it works:
- **The inputs it takes** - pictures · videos · sounds (its Load Image / Load Video / Load Audio nodes); the reference dock fills them in order, and a loader with nothing in the dock is bypassed for that render.
- **The output** - a Preview node is made a Save node (a preview writes no file), a video combine that does not save is turned on; a workflow that saves nothing is refused.

- **Missing nodes** (ComfyUI does not have them - install the node pack, restart ComfyUI, import again) and **missing model files** (a loader names a file ComfyUI does not list) are reported. A saved (UI) workflow needs ComfyUI running to be read.
- **The controls it gets**, already wired: Prompt (empty = the prompt box), Negative (empty = the negative prompt), Seed (bitGen's), Steps, CFG, Width / Height, Length - shown in Basic Settings when the graph is picked.
- **The graph is drawn** below: every node and link, the inputs, the output, missing nodes and model files coloured; the mouse wheel zooms, dragging pans, Fit shows it all. A workflow loaded in the tab (↻ Load, File, a graph picked) shows its inputs and its graph the same way.
- ▶ **Test run** queues it once through ComfyUI with a test picture / sound / video in every input. **It worked** = it is saved into comfy_workflows/ and as a graph (picked in the tab - Generate ▸ ⚙ Graph runs it; the test picture is in the gallery). **It failed** = ComfyUI's errors are listed and nothing is saved. A workflow that needs a real face or a particular picture may fail on the test pattern - add it by hand with File then.
- 2.587.0: **Settings ▸ Dockers ▸** beside each docker's name turns it **off**: the docker and its status-bar button are hidden (so it cannot be opened by accident) - shows it is off, a click brings both back. Floating windows too.
- 2.587.0: **Settings ▸ Dockers ▸ Layouts** - a layout is the whole docker arrangement (where each docker is, its size, shown / hidden / off, the docker configs, the floating windows, the workspaces) + the theme: **Save layout** (layouts/<name>.json in the install folder), ↺ **Apply** (the page reloads into it), **Export**, **Import**, ×. ★ **Save as the default** writes layouts/default.json and themes/default.json - copy both into the repo: a fresh install starts with that layout, and a browser that never picked a theme takes that theme. (Themes keep their own Save / Export / Import in Settings ▸ Themes - themes/<name>.json.)
- 2.581.0 fix: **Designs ▸ open and Open from the computer** in the SolidCraft docker opened nothing (SolidCraft reads its file storage only when it starts) - a design or part now goes in as if dropped on it.
- 2.574.0: **step n of total for every generation** - pictures through ComfyUI now report their sampler steps live (they only said "rendering… Ns" before), sd-cli pictures and videos too; and every place that shows a running generation says it: the status bar (step n/total), the queue, the Console beside Cancel, the Song tab and the Music docker (they showed "[object Object]"), the image editor's result tiles and Pending cards, From the picture, the 3D / Unreal real pictures, the timeline's fill, AI take, camera depth and karaoke overlay, the StoryBoard (grid cards, the scene picker, upscaling, ✎ First frame, storyboard pictures, the environment / character creators, sheet takes, RefMod extraction), the swap dialog (takes and first frames), VR world, the style doctor and the Sketcher.

The StoryBoard (the docker under the timeline with the scenes) is called the StoryBoard everywhere - its title, its statusbar button, Docker configs, the messages - and collapsing it keeps the name (2.545.0).

Dialogs and message boxes open in the middle of the screen. Move one by its title and it opens where you left it next time (kept in **dialog_layout.cfg** in the install folder, by the dialog's name); if the window has become smaller it is pulled back on screen. Delete the file to have them all in the middle again.

Tools & Properties stays inside the timeline docker: over its preview canvas row (so the tracks and the menu bar stay free; under the menu bar when the preview row is hidden) - dragged, sized or when the docker changes size it stays within it - never outside it (a small docker makes it smaller, its contents scroll) - and it hides while the timeline docker is collapsed.

Shift+Del deletes the selected layer in the image editor.

Generate fill in a shape or a mask, the pattern library (2.437.0)

Generate Fill in Shape… (Layer menu, the layer's right-click menu, or fill in shape in Properties): generates inside the picked layer's outline - a shape, a text, a pixel layer's content, a smart object (or its vector mask's path). Type what should fill it (empty = the prompt docker's prompt) and how many results; the AI panel generates, click a result to compare, ✔ **Apply** (Enter) puts it on a NEW layer right above the shape, **clipped** to it (↳) - the shape stays a live shape, the fill can be moved, masked or generated again (a second fill goes above the first). **Generate Fill in Mask…** does the same inside a layer's mask (white = where): the result gets a copy of that mask. One undo step each.

The pattern library - patterns/<name>.png in the install folder, shared by every project (copy .png files in to share them). **Edit ▸ Define Pattern…** makes one from the selection's box of the picture as it looks (or the whole picture). **Edit ▸ Pattern Library…** shows them: ▦ = a new Pattern fill layer, × = delete, Import PNG…, Define Pattern…. **Layer ▸ New Fill Layer ▸ Pattern…** is a live fill layer (Properties: the pattern, Scale %, an offset); **Edit ▸ Fill… ▸ Contents: Pattern** fills the selection with one; **Layer Style ▸ Pattern Overlay** lists them as user:<name> beside the built-in patterns. A .bgi keeps the patterns it uses inside and adds them to the library where it is opened.

Project files: .bgi, .bgv, .bgp (2.436.0)

bitGen has its own files now - a picture, a video edit and a whole project, each kept exactly as it is so it opens again ready to go on working.

.bgi - the image editor's project (File ▸ **Save Project (.bgi)** Alt+Ctrl+S, **Save Project As (.bgi)…**, **Open Project (.bgi)…**, **Open Project from Project…**; Open… / Ctrl+O and a .bgi dropped on the editor open one too). Everything stays editable: layers and groups, masks, text / shape / fill / adjustment layers, layer styles, blend modes, locks and labels, smart objects with their contents, their smart filters and filter masks (liquify included), paths, alpha channels, the selection and layer comps. The undo history starts fresh. It is a zip: doc.xml (the document as XML), preview.png (the picture) and each layer / mask as a PNG - any unzip opens it. Kept in the project's **bgi** folder (tick "also download it" for a copy); Save Project saves to the same file again with no questions, the first time it asks for a name.

.bgv - the video project (on the timeline's toolbar). **Save** writes the timeline open now - every layer, clip, keyframe, effect, caption, marker and setting - as XML, with the nested timelines its nest clips use and a list of the media it plays (and whether each was there). Kept in the project's **bgv** folder (+ a download when asked). **Open a .bgv** (from the computer or the project's list) shows what is in it first - clips, layers, nested timelines, media this project does not have - then **Open it** makes it a named timeline of this project and switches to it. Nested timelines are added when the project has none of that name (yours are never written over). A clip whose file the project lacks stays on the timeline and plays once the file is added to the assets.

.bgp - the whole project (in the menu bar, beside). XML: the project's state, timelines, chats and settings readable; the media as base64. **Self-contained** puts everything inside, the media too - one file to move the project to another computer. **Manifest only** keeps the project's files and lists its media where they are on this computer - small, for a backup of the work. The dialog shows what each will weigh; the caches and previews bitGen re-makes are left out unless ticked (the recycle bin always). **Save…** asks where (the Save As dialog). **Open a .bgp…** makes it a NEW project (never over an existing one; the name is yours to change) - **Open it now** switches to it. Media a manifest lists that are not on this computer are named. Both ways stream, so a project of gigabytes is fine.

Saved themes and PDF fonts (2.435.0)

Settings ▸ Themes ▸ Customize: a colour for each part of the theme (background, panels, raised panels, lines, text, dim text, OK green, error red, hover, canvas, selected) - the app (and the image editor) change at once; a changed one is marked; ↺ **The theme's colours** undoes them.

Saved themes: type a name, **Save theme** - it is written to **themes<name>.json** in the install folder (the colours, the accent, the UI size, the theme it started from) and shows as a card under it. A click on a card loads it, × deletes the file. **Export** downloads the theme as a .json. **Import** uses one

and saves it at once as themes<its name>.json; the theme you had is kept first as **themes<its name>_old.json**, and ↶ **Revert** (it shows after an import) puts that one back under its own name. Copy .json files into the themes folder to share them between computers.

PDF ▸ Save PDF… / Save as New PDF… (the image editor's Pages strip, or File): the Save dialog has **Embed fonts** (on by default, kept): each line of type is written as real text in its own font, embedded in the PDF with only the letters it uses. The font file comes from this computer's fonts (Windows, macOS, Linux font folders) or the install's **fonts** folder (drop a .ttf / .otf there). Any alphabet stays real text. A font that is not on this computer, or that lacks a letter, falls back to the PDF's standard fonts (Latin letters) or into the page's picture, as before. Off = the standard fonts only.

Themes and the UI size (2.432.0)

Settings ▸ Themes:

- **Theme**: Dark (bitGen's own), Darker (OLED black), Slate (cool blue-grey), Light, High contrast. The whole app changes - the dockers, the dialogs, the image editor (its bitGen theme is built from the same colours; Window ▸ Theme in the editor still picks Pro / Studio / Classic for it alone).
- **Accent colour**: any colour for the buttons, the lit tabs and the sliders instead of the theme's own; Reset = the theme's.
- **UI size**: one slider for every font and every panel together, 60 % to 175 % (it moved here from General; the slider in the left dock's corner is the same one).

The theme, the accent and the size are kept in this browser and are on the page before it paints, so a light theme never flashes dark.

PDFs: the guide and the brochure (2.392.0)

`docs/bitGen-Studio-Guide.pdf` is this guide plus a one-page view of every shortcut key. `docs/bitGen-Studio-Brochure.pdf` is a 3-page brochure for bitGen Studio. Both use the studio's dark theme.

3D text and 3D models on the timeline canvas

- (under **T** in the canvas toolbar) adds **3D text** or a **3D model** (.fbx or .obj — choose an OBJ's .mtl and textures together with it), or one of the project's models. Dropping .fbx / .obj files on the canvas works too.
- Move, scale, turn and fade it like any layer; ◆ keyframes work. Its own settings: text, font, depth, bevel, colour, metal, rough, light, Rot X / Y / Z (keyframeable) and Spin (degrees per second), its picture size and length.
- The canvas shows it live; for ⚙ **Compile** it is baked into a see-through video automatically (Compile waits if a bake is still due).

Generate progress

Above the **Generate** button: the running job's step count (hover it for the details). Under it: a progress bar, only while a job runs.

Projects, queue and gallery

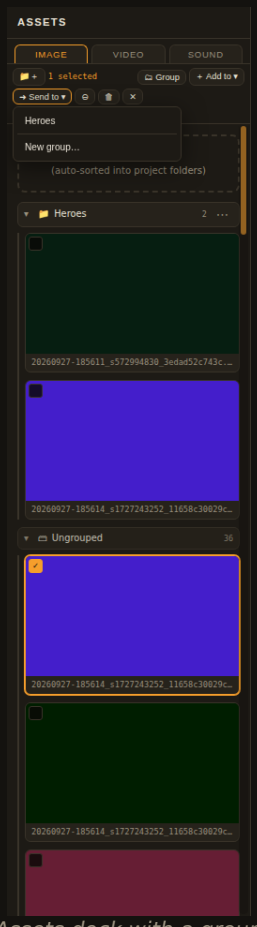
- **When a theme goes wrong** (2.636.0) - **Ctrl+Alt+S** opens Settings from anywhere (even while typing, even when the ⚙ is gone); **Ctrl+Alt+Shift+T** puts the built-in **bitGen Default** theme on (Dark, its own accent and colours, no element styles, UI size 100 %), as does bitGen Default in Settings ▸ Themes or opening the studio with `?theme=default` after its address. The theme you had is kept: Settings ▸ Themes ▸ ↶ The theme before.
- **Assets: Project | Global | All** (2.635.0, the dropdown in the Assets dock's head) - project assets live in `projects\<name>\Assets\`, global assets (every project has them) in `projects\Assets\` (next to

your projects; it is not a project itself). New uploads go into the project's Assets folder (the global one while the dock shows Global). **A folder in an Assets folder is a group** - make one in Explorer or with ; ➜ Send to moves the file into the folder, Add to puts a link there too, ⊖ puts it back, deleting a group moves its files back (nothing is deleted). **Right-click a tile** (or the selection): **Move to Global** / **Move to Project**, New group, Delete. Each asset is also linked into the project's `images\` / `video\` / `sound\` folder (a hard link - no extra disk space), so everything that uses an asset by name keeps working; files that are only there (older projects, renders) show as ungrouped project assets, and the old dock groups became folders.

- **New** / / (the menu bar at the top of the window) — projects. Everything lives in `projects\<name>\` (`output\`, `video\`, `images\`, `sound\`, characters, scenes).
- **Queue** — one job at a time on the GPU; pause, abort (the Abort dialog) and retry; every job keeps its log. **Clear** (in the Queue's header, on both tabs) removes every finished job from the list — done, failed and cancelled; the number on it says how many. Nothing queued or running is touched and no files are deleted.
- **Gallery** — every render; click to preview, drag onto the references, the timeline or a scene.

Grouping and deleting assets

The Assets dock can sort a project's files into **groups** — folders that live in the dock only. The files themselves never move, so every scene, timeline clip and reference that uses them keeps working.



Assets dock with a group

- **Select** — tick the box in a tile's corner, Ctrl-click (⌘ on a Mac) or Shift-click for a range. Once something is selected a plain click adds or removes tiles; Ctrl+A selects the whole tab, Esc clears. With nothing selected a click still opens the lightbox.
- makes an empty group. **Group** makes a new group holding the selection.
- **Add to ▾** puts the selection in a group *as well* — an asset can sit in several groups.
- **➜ Send to ▾** *moves* it: out of the group it was picked in, into the new one. You can also drag tiles onto a group's header (hold Ctrl while dropping to add instead).

- ⊖ takes the selection out of the group it was picked in.
- ⋯ on a group header: Rename, Select all, Delete group (its assets stay). Click a header to fold it away.
- One group can hold pictures, videos and sounds — each tab shows that group with its items of that kind.
- (or the Delete key) moves the selection to `projects/<name>/recycle_bin` — never a hard delete. It asks first when *Settings → General → Confirm delete* is on, or when the project still uses one of the files.

Groups are saved per project in `projects/<name>/asset_groups.json`.

Adding files with the + tiles (2.394.0)

A dashed **+ add** tile opens the file dialog below, so you can add files without dragging them in:

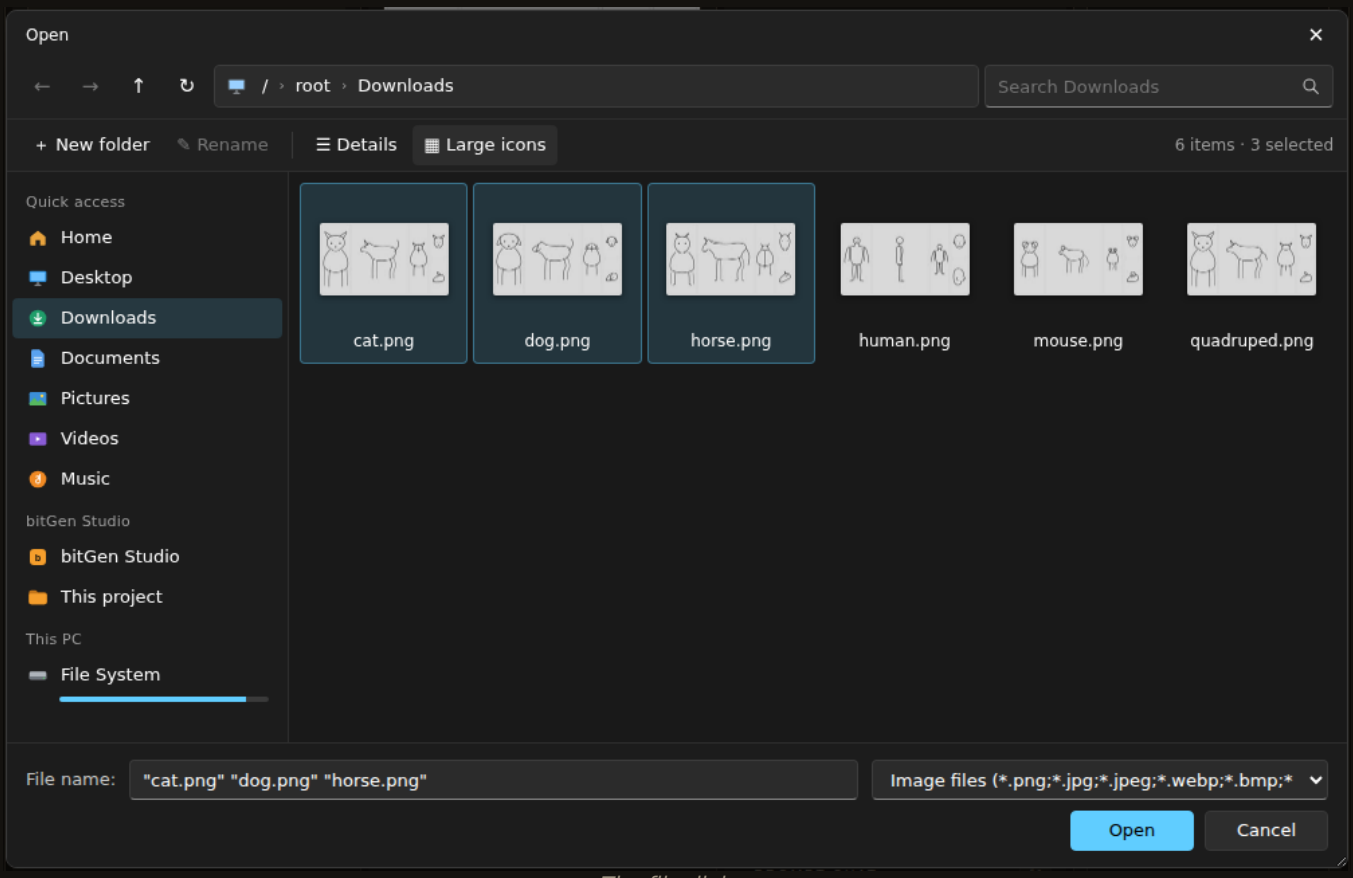
- **References → Video**: **+ add videos**, under the reference videos.
- **References → Sound**: **+ add sounds**, under the reference sounds.
- **Assets → Image / Video / Sound**: **+ add pictures / videos / sounds**, at the top of the list.

Pick several at once, as in Windows 11: **Ctrl-click** adds or removes one file, **Shift-click** picks a range, **Ctrl+A** picks them all. Click **Open** to add them.

Everything you pick goes into the project's Assets. From a References tile, the files also become references, up to the model's limit of three videos and three sounds. Any that don't fit stay in Assets, and the line above the tabs tells you. The tile shows how many places are left, for example **+ add videos (2 free)**, and disappears when the list is full.

You can also focus a tile and press **Enter**, or drop files onto it.

The file dialog



The file dialog

Every beside a path and every *Choose file* button opens bitGen Studio's own Open / Select Folder dialog — the same on Windows, macOS and Linux:

- **Navigation pane** — Quick access (your real Desktop, Downloads, Documents, Pictures, Videos, Music), bitGen Studio (the app folder, this project, ComfyUI) and This PC with each drive's free space.
- **Address bar** — click a folder name to jump there, or click the empty part and type a path. ← → ↑ ↻ go back, forward, up and refresh.
- **Search** (top right) — finds names in the folder and all its subfolders; `*` and `?` work as wildcards.
- **Details** / ▦ **Large icons** — sortable Name / Date modified / Type / Size columns, or thumbnails of pictures and videos.
- **New folder** and ✎ **Rename** (F2); right-click for Open, Rename, Copy path, New folder and Refresh.
- **File name** — type a name, a full path, a folder name (to go into it) or a pattern such as `*.png` (to filter); the file-type list sits beside it.
- **Several files** where a button takes several: Ctrl-click and Shift-click.
- Keys: Enter opens, Backspace / Alt+↑ goes up, Alt+← / Alt+→ back / forward, F5 refreshes, typing jumps to a name, Esc cancels.

The dialog reads files from **this** computer, so it only replaces the file pickers in the app window or a browser on the same machine — the phone remote keeps the phone's own picker. **Settings → General → Use bitGen Studio's file dialogs** switches the *Choose file* buttons back to the browser's picker.

The guide and help in Chinese and Japanese (2.703.0)

The guide (the **Help** window and the guide entries **Ctrl+K** finds), the help pages and the shortcut list are also in **Simplified Chinese** (, for mainland China), **Traditional Chinese** (, for Taiwan and Hong Kong - 2.704.0) and **Japanese** (). The app's own buttons, menus and dockers stay in English, so the translations name them in English - what you read is what you see on screen.

- **Pick the language** in the dropdown at the top of the Help window or in a help page's title bar (**English** / / /). One setting for both, kept. Until you pick one it follows your computer's language.
- **Ctrl+K** searches the guide in that language, and still finds a section by its English name.
- English is the original. When a section changes in English after it was translated, the translated one says so at its top, with a button that shows the English one. A section not translated yet is shown in English, with a note.
- **The Guide PDF** in each language: docs/bitGen-Studio-Guide-zh-CN.pdf, docs/bitGen-Studio-Guide-zh-TW.pdf and docs/bitGen-Studio-Guide-ja.pdf.
- The translations are files beside the English ones: docs/i18n/zh-CN/, docs/i18n/zh-TW/ and docs/i18n/ja/ (USAGE.md, shortcuts.json) and static/help/zh-CN/, static/help/zh-TW/, static/help/ja/. The Traditional Chinese one is made from the Simplified one (`python3 scripts/i18n-zhtw.py`, Taiwan's words) and then read over. `node scripts/i18n-help.js status zh-CN` lists what is out of date.

The app itself in Chinese and Japanese (2.705.0)

The dropdown on the menu bar, just left of ⊞ (docker layout), changes the language of the app itself: **English** / / / . Buttons, menus, docker titles, tabs, Settings, tooltips and the hints in empty boxes change at once - nothing restarts, and back to English puts every word back.

- It is the same setting as the language of the Help guide and the pages: change one and the other follows.
- This release translates the main window. The image editor, the timeline, the Sketcher and the CAD docker are still in English (they come next); so are words the app builds while it runs (a status with a number or a file name in it).

- The translated guides name the buttons the way the translated app shows them; the sections about the editors keep the English names those editors still show.
- The words come from static/i18n/ui/<language>.json. `node scripts/i18n-ui.js status ja` says how many are translated.

Language packs (2.715.0)

Every language besides English is a **language pack**: the app's own words, the guide, the help pages, the shortcut list and the feature list. , and are built in; more can be added without a new version of the app. **Settings ▸ Languages** lists them, with how much of each is translated.

- **Install a pack** adds a language from a pack's .zip. It is checked first; a language that is already there asks before it is replaced (a pack for a built-in language is used instead of it until you remove the pack).
- The new language is in the dropdown next to ⊞ at once. Anything the pack has not translated shows in English.
- × **Remove** takes an installed pack away (a built-in language comes back).
- **Master pack** is everything in English, with TRANSLATING.txt saying how to fill in each file - to translate bitGen Studio into a new language. Put the language's code and name in pack.json, translate what you can (the app's words in ui.json make the biggest difference), zip it and install it.
- **Export for translating** gives a pack back with the parts added to the app since, its translations kept - to carry on, or to improve a built-in language.
- Installed packs are kept in the langpacks folder of the install. A pack's help pages keep only plain formatting.
- **Shared by the community** (2.716.0) lists the packs other people translated, from the bitGen Community on tigerart.com.au - an admin checks each one before it shows. **Install** puts one in with one click.
- ⇪ **Share** on an installed pack sends it to the community as you (sign in to Community first), with a note for the admins. It shows for everyone once an admin has checked it; **Your shared packs** says where each one is, and × takes one off. The website's Language packs page (tigerart.com.au/languages.php) lists the same packs, offers the master pack and takes uploads too.

Faster first frames and the speed-up fixes (2.717.0)

A tester measured bitGen's speed-ups on one RTX 3090 (same picture, same seed, two blind judges). What changed:

- ✎ **First frame on the scene cards**: the style list starts with **Auto**. With Qwen Image Edit 2511 installed (the full package - a 20 GB model), Auto makes the frame with it in 4 steps (its Lightning LoRA) instead of Qwen Image 2.1's

  – A frame that needs more than three pictures (the base, the cast, the props) still goes to Qwen Image 2.1 edit, and the note says so. Pick a style in the list to always use that one.
- **Qwen Image 2.1 edit Turbo8**: a new image style - Qwen Image 2.1 in 8 steps with the community Turbo8 LoRA (Settings ▸ Models ▸ Model catalog ▸ Qwen Image 2.1 Turbo8 LoRA). Small, dense text is weaker than at 25 steps; titles and labels hold.
- **Licences**: Qwen Image 2.1 and the Turbo8 LoRA are under the Qwen Research License - non-commercial use only. Qwen Image Edit 2511 and its Lightning LoRA are Apache 2.0 - commercial use is fine. That is why Auto prefers 2511.
- **Veda** skips a shot whose prompt describes a fight, a transformation, an explosion or a chase: the test found it saved only 22 % there and softened the picture. **Veda on action shots too** in the video style keeps it on every shot.
- **Veda with Long video**: Veda now takes the place of Long video's low-VRAM attention (a 12 s long shot ran 65 % faster). If a very long shot runs out of memory, set Veda off for it.

- **Spectrum** on a run of 12 steps or more keeps the first 2 and the last 3 steps real - the last steps draw faces and fine detail, which blurred when they were forecast.
- Long video still leaves Sage attention out: Sage on top of it measured only 17 % faster.

Models not found - checked before a render (2.719.0)

Before a picture, a video or a song starts, bitGen checks that every model file it will load is in one of ComfyUI's model paths: the workflow's own model, text encoder, VAE and upscaler, and the style's model, VAE, text encoders and LoRAs (and Basic Settings' model and LoRAs when Override style is on).

- The check is quick: it reads a list of every model file bitGen has found, kept in **misc/models_found.json** (the `models` folder of ComfyUI and every path in its extra_model_paths.yaml). The list is made again when a name is not on it (you may have just added the file), when ComfyUI's folder changes, and when you press in Models. A file found by its name in another subfolder counts as found.
- When files are missing, a **Models not found** box lists each one: its name, the folder it goes in and where it came from (the workflow, the style or Basic Settings). A file in bitGen's model catalog shows its size and where it downloads from; **⬇ Download** fetches those (with the Hugging Face token saved in Settings ▸ Models), a bar for each, then checks again - when all are there the render starts by itself. A file not in the catalog says so: download it yourself (Civitai, Hugging Face) into the folder shown.
- **Cancel** (or Esc, or Enter) starts no render. A download already running keeps going - Settings ▸ Models shows it. Renders waiting in the queue for the same missing files are stopped without asking again.
- Every render is checked, wherever it is started: the prompt docker's Generate, the image editor's AI (Edit, Compose, Regenerate - also from the References docker's pictures), the StoryBoard, the characters, the timeline and the music docker. When you cancel, the place you started from says it was not rendered. The Style doctor is not stopped: it reports a missing model in its table.
- A render that still stops (2.720.0) - its worker exits with an error - is checked too: the model names ComfyUI refused and the files its style loads. When one is not in a model path, a box asks **The render stopped: … Download it?** with the same ⬇ Download / Cancel. After the download, press ↻ Retry on the job in the Queue docker (or Generate again). This catches what the check before the render cannot see: a custom node's model, a scene the StoryBoard renders by itself, Swap, LTX and Wan's own picks.
- Not checked: a ComfyUI on another computer (Remote GPU), no ComfyUI folder set, sd-cli styles, and the files a Wan or LTX style picks for itself.

Free VRAM only when the model changes (2.722.0)

Settings ▸ ComfyUI ▸ Free VRAM before a generation now has **Free it**: **when the model changes** (the default) or **before every generation** (as before).

- When the model changes: a queue of renders that use the same model keeps it on the card - no unload and no reload between them, so pictures and clips in a row start sooner. When the next render loads other model files, the card is freed first, and the console says which happened ("VRAM kept: the same model(s) …" or "the model changed").
- It does not keep adding to the VRAM: ComfyUI holds one copy of a loaded model and reuses it, a render's working memory is released when it ends, and ComfyUI frees room by itself when the next graph needs it.
- After an out-of-memory error (or ComfyUI stopping), the next render frees the card whatever model it uses.
- Before every generation is the old behaviour: every render starts from an empty card (the safest for long clips after a mixed session). Untick the switch to leave it all to ComfyUI.

Floating dockers: the one you used last is on top (2.724.0)

Every floating docker now shares one order, above the rest of the app: Community, Basic Settings, LLM tokens, the ▣ dockers, the dockers you made with ⊞ **Docker configs** ▸ ▣ **New docker…**, a main docker floated with ⇱, the timeline's stage docker and the voice chat.

- **Click a floating docker** anywhere (its title bar, a button, a box inside it) and it comes to the top. Clicking into it with the keyboard does the same.
- **Opening one** (Community on the status bar, S for Basic Settings, a ▣ docker's button) puts it on top too. Before, a docker made with ▣ New docker… always covered Community.
- Floating dockers sit over the docked dockers and the status bar, and under the menus, pop-up menus and message boxes.
- A docker you dock again (beside, inside or in a column) leaves the order, and comes back on top when you float it.
- The prompt docker keeps its own rule: Ctrl+P lifts it over whatever covers it.

A song in the preview: a visualization, and Next plays the next song (2.725.0)

Click a song or a sound in the Gallery, Assets ▸ Sound or References and it opens in the preview, the same window that shows a picture when you click one. **▶ on a song's own player in the Gallery's Audio tab** opens it there too, already playing (2.725.1).

- **▶ starts a visualization** picked at random: spectrum bars, mirrored bars, a waveform, a ring, a flight through stars or ripples. It moves with the music. Each song gets a different look from the last one.
- beside the player picks another look while it plays. Paused, the picture holds still and dims.
- **Next ›, ‹ Prev, the arrows on the picture and ← →** go to the next or previous **song in the docker you clicked it in**, and play it. Pictures and videos in between are skipped, and after the last song it starts again at the first.
- On a picture or a video they page through the list as before, and nothing starts playing by itself. The strip at the bottom still opens anything.
- Closing the preview stops the song. The Preview docker (its button on the status bar) is not affected.

The Neural blue theme and the bitGen brain picture (2.726.0)

- **Settings ▸ Themes ▸ Neural blue** turns the studio midnight navy with an electric-blue accent: the colours of the bitGen brain picture. Dark (tiger orange) stays the default, and your own accent colour still works on top of it.
- **At start-up** the app shows one of two splash screens at random: the intro film, or the brain picture with the credits for 8 seconds. A click or any key skips either.
- **A new install** puts the brain picture in the first project's Assets ▸ Images, so the image editor, AI edits and image-to-video have a picture to try straight away. Later projects start empty, and you can delete it like any other picture.
- The website (tigerart.com.au) has the same Neural blue look under **Colours** at the bottom of every page.

The brain comes alive; upscales that came back as the source (2.727.0)

- **The splash's brain picture is a film now**: lightning strikes the bitGen brain, a flash, and the big bitGen Studio title, for 15 seconds with the credits over it. The picture still shows if the film cannot play. A click or any key skips it, as before. The website's home page shows the same film.

- **Fixed: an upscale could hand back the video it was given.** Since ComfyUI's August 2026 update, the Load Video node shows the file it loaded in ComfyUI's results, ahead of the real result. bitGen took that as the finished video, so a SeedVR2 upscale to 1080p could come back at the source's own size with *_seedvr2_1080p* in its name. bitGen now skips what ComfyUI only shows back. The same fix covers RTX Video, DLSS 5, LTX, Viggle and workflow renders.
- **SeedVR2 checks the size it got.** The file is named by the size it really is. If ComfyUI hands back the source's size, the job stops and says so instead of writing a copy. Run the upscale again to get a true 1080p version of a video made before this fix.

1920 × 1080 exactly, and RelayVSR (2.728.0)

- **1920 × 1080 (16:9 exact)** is a new size in SeedVR2, RelayVSR and the video docker's ⚙ **Upscale resolution**. A clip that is not quite 16:9 (an LTX render of 960x544) is upscaled until it covers the whole 1920 × 1080 frame, then the few extra rows are trimmed off the top and bottom. 1080p still keeps the whole picture (1906x1080 from 960x544). DLSS 5, RTX VFX and the in-render upscaler treat 1920 × 1080 as 1080p.
- **RelayVSR** in the preview bar upscales the video you are previewing with RelayVSR, a fast video upscaler that works in one step. It runs in ComfyUI's own nodes, so nothing else is installed. Pick a size, then **Upscale**; the result lands in the gallery with the original sound.
  - **Model**: RelayVSR comes as two pairs of files. **x2** (LightX2V's, trained on MiniMax H3 clips) suits an H3 or LTX render. **x4** reaches 4K from a small clip. **auto** picks the smaller pair that reaches the size you chose.
  - **Detail**: 1 keeps everything the model adds; lower is closer to a plain upscale.
  - The video docker's ⚙ **Upscaler** has **RelayVSR** too: a scene's (and **Upscale every render**) uses it, at the ⚙ **Upscale resolution**.
- **Setting it up**: RelayVSR needs ComfyUI newer than the v0.44.0 release (Settings ▸ ComfyUI ▸ Update ComfyUI on the "latest" channel, then restart ComfyUI). Settings ▸ Add-ons ▸ **RelayVSR** downloads the files (x2 is ticked; about 5.3 GB for each pair). The panel says when either is missing.

NitroFusion2: pictures in one step (2.729.0)

- **NitroFusion2** is Z-Image-Turbo trained to finish a picture in one or two steps, with sharp text and fine detail. It runs in ComfyUI's own nodes. Three image styles use it:
  - **NitroFusion2**: one step. The fastest, and the one to start with.
  - **NitroFusion2 2-step**: two steps with the authors' de-distillation LoRA on the first, which removes some artifacts. Keep its steps at 2: the two steps are one schedule split in half.
  - **NitroFusion2 GGUF**: one step from a GGUF file, for smaller cards (Q4_K_M by default; pick another as the style's Model).
- cfg stays at 1 and there is no negative prompt. About 1 megapixel (1024 x 1024) is what it was trained for.
- Your own LoRAs for the style go in its four LoRA slots, as with the other image styles; the de-distillation LoRA stays as it is.
- **Setting it up**: Settings ▸ Add-ons ▸ **NitroFusion2** downloads the files you tick: the model (12.3 GB) and the de-distillation LoRA are ticked, the GGUF files are optional, and the Qwen3 4B text encoder and the FLUX.1 VAE come with any of them. Ticking a GGUF file installs ComfyUI-GGUF as well.

Motion from a reference video: FlowHMR (2.730.0)

> **Non-commercial use only.** FlowHMR's code is free (Apache-2.0), but it cannot run without the SMPL-H body model and > Meta's SAM 3D Body and VGGT-Omega, whose licences do not allow commercial use. Motion captured with it is for personal > or research use.

- **Motion** in the preview bar takes the video you are previewing and captures the motion of the person in it, in 3D. The result is an animated skeleton (**.fbx**) and the same motion as SMPL-H (**.npz**), saved in the project's **models3d** folder as *<video>_motion*. The timeline's 3D layers, Blender, Daz and Unreal can use the .fbx.
- **Camera**: **moving** works out how the camera moved, so the person's path through the scene is right; **still** is for a camera on a tripod and is the lightest. **Camera every** 1 frame is the most exact but needs about 30 GB of video memory; every 10 frames is much lighter.
- **Setting it up**: Settings ▸ Add-ons ▸ **FlowHMR motion capture**. Install puts FlowHMR in its own Python environment and downloads its model, but three things need you first:

  – the **SMPL-H body model**: register on mano.is.tue.mpg.de, download the Extended SMPL+H model and put its neutral *model.npz* where the add-on says;

  – **VGGT-Omega** and **SAM 3D Body**: request access on their Hugging Face pages and set your Hugging Face token in Settings. The links are on the add-on's card. Install again once you have them: it says what is still missing.
- It needs an NVIDIA card and the app's Python 3.10 - 3.12. Without the FBX SDK (it installs only for Python 3.10) you get the .npz alone.

Sheet music on the staff, saved as a PDF (2.731.0)

- The Music docker's **Sheet Music** tab draws the score as real sheet music: the staves with their clefs, the key and time signatures, the notes with their stems, flags and beams, the rests, accidentals, ties and slurs, the chord symbols over the melody and the song's sections (Verse, Chorus …) as boxed marks. A YuE2 score has two staves, the vocal melody and the instrument line, joined by a bracket.
- Where a score comes from: **Write sheet from lyrics**, **Transcribe audio**, the **Score** list (every .abc in the project, newest first; each song saves its score beside the audio, and the newest is drawn when the tab opens) or **Open .abc** for a file on this computer. ↻ looks for new scores.
- **Read** opens the pages full size. ‹/› **ABC text** shows the notation the drawing is made from, and **Copy** copies it.
- **Save PDF** saves the pages as an A4 PDF in the project's **pdf** folder, named after the song (the image editor's File ▸ Open PDF from Project lists it too). With **download too** ticked it is downloaded as well. The PDF uses only the fonts every PDF reader already has, so nothing is embedded; a title in letters those fonts lack (Chinese, Japanese) shows ? marks.
- The engraver is part of bitGen Studio, not an outside library. Besides YuE2's own scores it reads ordinary ABC tunes: chords, triplets, dotted rhythms, repeats and endings, key and time changes, bass, alto and tenor clefs. Grace notes and lyric lines are not drawn.

The paper under the sheet music (2.732.0)

- **Paper** in the Sheet Music tab opens a row of choices for the paper the score is drawn on. The pages on screen, **Read** and the saved PDF all use it.
- **Paper colour**: any colour, or one of the round swatches (White, Cream, Antique, Stone, Pale blue, Kraft, Night).
- **Material**: Plain, or a texture repeated across the page like tiles: Parchment, Linen, Laid paper, Recycled, Watercolour, Canvas or Kraft. Under **Pictures** you can use any picture in the project, or **From this computer…**, repeated the same way (its shape kept).
- **Tile size**: how big one repeat is on the page, from 40 to 600 pt (72 pt = 1 inch). **Texture**: how strongly the material shows over the colour.
- **Ink**: the colour of the notes, lines and words. **Auto** uses near-black on light paper and warm white on dark paper (Night); Black, Sepia, Navy and White are fixed.

- **in the PDF too**: untick it to save the PDF on plain white with black ink (better for printing) while the screen keeps your paper. A textured PDF holds one small picture of the tile, repeated by the PDF reader, so the file stays small.
- ↺ **Plain white** goes back to the start. Your choice is remembered on this computer.

Enhance on a picture: upscale an image (2.733.0)

- Click a picture - in the gallery's **Images**, the **Image** assets or the reference dock - then press **Enhance** in the prompt docker. Its tooltip names the picture it will upscale. When the last thing you clicked was a video, Enhance still upscales the video with SeedVR2.
- The panel shows the picture and what it will make, for example 320×180 → 640×360:
  - **Size**: ×2, ×3, ×4, or a short side of 1080, 1440 or 2160 px (the shape kept, at most 8192 px a side).
  - **Way**: **SeedVR2** restores detail (ComfyUI, the SeedVR2 model and VAE - the same files as the video upscaler); **Upscale model** uses a file in ComfyUI's models/upscale_models (4x-UltraSharp, RealESRGAN ...), fast and sharp; **Lanczos** needs no ComfyUI at all but adds no new detail.
  - **Model**, **Colour match** and **Denoise** are for SeedVR2 (Model also picks the upscale model).
- **Upscale** queues the job. The result is a PNG in the gallery's Images named *<picture>_upscaled_<w>x<h>.png*. The panel remembers your choices for the project.

▶ Playing the sheet music, the notes lit as they sound (2.733.0)

- Under the score in the **Sheet Music** tab: back to the start, back one bar (press it twice quickly for the bar before), ▶ **/** play and pause, forward one bar, to the end. The time and the bar number show beside them, and **Read** has the same buttons.
- While it plays, the notes sounding light up in orange and the bar gets a soft band. With **follow** ticked, the score scrolls to keep the bar in view. Click any note to jump to it.
- **Sound**: **the song** plays the song the score belongs to (the audio file with the same name beside the .abc - a song made in the Music docker has one; the Score list marks those scores with). **the notes** plays the written notes in the browser, for a score with no song (Write, Open .abc) or to check what is written.
- **Sync (s)**: the seconds of the song before bar 1, such as an intro. Nudge it until the lights match the singing. It is remembered for each score.
- The speed comes from the score's tempo (Q:); a score with none plays at 120 beats a minute.

The words under the sheet music (2.734.0)

- Tick **words** in the Sheet Music tab and the lyrics in the Song tab's lyrics box - the song you are working on - go under the notes of the vocal staff, one syllable to a note, with a hyphen between the syllables of a word.
- They follow the sections: the lyrics' **[verse]**, **[chorus]** ... tags are matched to the score's Verse, Chorus ... in order, and a chorus that comes back sings its words again. Lyrics or a score without sections are laid from the first note on.
- A note tied over from the one before, or inside a slur, holds the syllable before. Type hyphens yourself (se-ven, Hal-le-lu-jah) where the automatic split is wrong. Chinese and Japanese lyrics get one character a note.
- The status line says how many sung notes got words, and how many syllables were left over. Edit the lyrics and the words are drawn again a moment later; untick to remove them. The words go into Read and the saved PDF too (the PDF's fonts have no Chinese or Japanese characters, so those show as ?).

The words lit as they are sung (2.735.0)

- With **words** ticked, press ▶ in the Sheet Music tab: the syllable being sung lights up orange under the vocal staff, together with its note.
- A syllable stays lit for as long as it is sung - through a note tied on from it, through the notes of its slur, and over the barline - and goes dark in a rest.
- Click a word to play from it, the same as clicking a note. Read lights the words too.
- It follows the same clock as the notes: with the song beside the score, set **Sync** so the notes and the words land on the singing.

The Sheet Music tab's layout (2.736.0)

- Two toolbars: **making a score** (Write from lyrics, Audio file… to pick a recording - its name shows on the button - and Transcribe), and **the score** (the Score list, ↻, Open .abc, Read, Save PDF with download too, words, Paper, ‹/› ABC text, Copy). Write and Save PDF are outlined in orange: the main actions.
- The player is the score viewer's header: ▶ , the time, the bar, Sound, Sync and follow. ▶ turns into an orange while it plays. Read has the same buttons.

Parts and the mix: each instrument and the vocals (2.737.0)

- A score with more than one staff gets **tabs** on top of the score: **Full score**, then one for each staff - the vocal one marked. A tab shows that staff alone on its own sheet, its name at the top: the piano player's part, the singer's part (with the words when **words** is ticked). Read and Save PDF use the part you are on, and the PDF is named after it (Night Drive - Piano.pdf). Changing tabs keeps your place in the song.
- **Play** in the player turns things on and off while you listen:

  – with **the notes**, a button for each staff - click one to mute or unmute it, **Shift+click** to hear that one alone (Shift+click it again for all of them);

  – with **the song**, **Vocals** and **Instruments**, once the song has been separated into its stems. A song that has not been separated yet shows ✂ **Separate vocals**: it runs the Stems docker's separation (demucs, on this computer) and the two buttons appear when it is done.
- The song can only be split into the singing and everything else (that is what the separation makes), so the separate instruments can be turned on and off with the notes, not inside the recording.

The song in its layers, as waveforms (2.738.0)

- The sheet music plays the **real song**. **Sound** in the player lists every song in the project: a score finds its own (the song it was made with, or the one with the same title), and what you pick is remembered for that score. **the notes (no recording)** plays the written notes in the browser, as a soft piano - for a score that has no song.
- A song plays in its **layers**, each a lane under the player: its **waveform**, the playhead running along it, and a button to turn it on or off (**Shift+click** = that layer alone; Shift+click again for all of them). Click anywhere on a waveform to play from there.
- A song that has not been split shows one **Song** lane and ✂ **Split into layers**: it splits the song on this computer (demucs) into **Vocals, Drums, Bass** and **Other** (everything else), which takes a minute or two. A song already split in the Stems docker shows its **Vocals** and **Instruments** until you split it into four.
- **Sync (s)** still lines the song up with the notes when the song has an intro before bar 1.

Create: a song's layers and its sheet music in one press (2.739.0)

- In the Sheet Music tab pick the **Song** - any song in the project (the one with the Song tab's title is picked for you), or **A song from this computer…**, which copies it into the project - and press **Create**. Three steps run, each ticked off beside the button:
  - **Split** - the song into Vocals, Drums, Bass and Other (on this computer); their waveforms show under the player straight away. A song that is already split skips this.
  - **Lyrics** - the Song tab's lyrics; when that box is empty, the words are heard in the vocal layer and put in it.
  - **Sheet music** - written from the song itself. It opens with the words under the notes, and plays that song in its layers.
- If a step cannot run (the separation is not installed, ComfyUI is not running…), Create stops there and says why.
- **Write from lyrics** is still there for a score with no recording: it composes one from the lyrics and the style.

⬇ Missing parts download when they are needed (2.740.0)

- When something needs a part that is not installed yet - the song separation (demucs) for **Create**, ✂ **Split into layers** or the Stems docker's **Separate**, or speech to text (Whisper) for hearing the words - a box says what is missing, what it does and how big it is, with **⬇ Download** and **Cancel**. You do not have to find an Install button.
- **⬇ Download** installs it right there, with a progress bar and what it is doing; when it is done the box closes and the job carries on by itself (Create goes on with the split, then the words, then the sheet music).
- **Cancel** (Enter and Esc too - Cancel has the focus, so a big download is never one stray key away) stops that job and says so. For the words, Cancel just makes the sheet music without them. **Close** while it downloads leaves the install running in the Queue.
- Models that ComfyUI renders with already ask the same way (the models-not-found box, 2.719.0).

ACE-Step 1.5, and the music styles on the Audio tab (2.741.0)

- **ACE-Step 1.5 Turbo** is a new music model: text and lyrics to a whole song with vocals, from 10 seconds to 10 minutes, with lyrics in 50+ languages, and fast (8 steps). Its licence (MIT) lets you use the songs commercially. Pick the **ACE-Step 1.5** style in the Music docker; the first song asks to download its four model files (about 9.3 GB) if ComfyUI does not have them yet.
- **Settings ▸ Audio** now holds every sound setting: the **Music styles** (they were the Music tab), then **Speech**, then **Speech to text**. The buttons at the top jump to each.
- An ACE-Step style has two extra parts:
  - **BPM, Key, Beats a bar and Language** steer the song. Leave one blank (auto) for the workflow's own value.
  - **LoRAs**: LoRA picks one from ComfyUI's loras folder, the slider sets its strength. A LoRA trained for ACE-Step 1.5 changes its sound or its voice. LoRAs are only offered for ACE-Step: whether MiniMax Music3 and YuE2 take them is not known.
- The caption is still the style (genre, instruments, mood, singer) and the lyrics are still the prompt.

Every Basic Settings tab keeps its own settings (2.742.0)

- In **Basic Settings**, each of **Ref**, **F/L**, **Text** and **Swap** remembers its own settings: Format, Resolution, Aspect, Width and Height, Seconds, Max seconds per segment, Steps, CFG, Seed and Model. Change tab and the one you leave is saved; go back and its settings return. A tab you have not opened before starts with what is showing. They are kept in this browser, so they last after a restart.
- Only your own click changes them. When a project opens, or the app picks a tab for you (adding pictures, a screenplay), the settings showing stay as they are.

- **Character chain** keeps what you typed when you go to **Standard** and back, even before you press **Save chain settings**.

☁ Rendering on a RunPod Serverless endpoint (2.743.0)

Settings ▸ Remote generation ▸ RunPod Serverless sends every render to a RunPod Serverless endpoint. There is no machine to start or stop: each render is one job, and you pay only while a worker runs it.

Setting up the endpoint (once, on runpod.io)

1. Make a **network volume** and put your models in it under `models/` (the same folders as ComfyUI's: `models/checkpoints`, `models/diffusion_models`, `models/loras`, `models/text_encoders`, `models/vae` ...). The worker reads them from `/runpod-volume/models/…`.
2. Make a **Serverless endpoint** from RunPod's ComfyUI worker image, `runpod/worker-comfyui:<version>-base` (version 5.x). If your styles use custom nodes (the MiniMax H3 nodes, VHS, KJNodes ...), build your own image from it with those nodes installed (`RUN comfy-node-install <node pack>` in a Dockerfile - see the worker's customization guide). Attach the network volume.
3. Copy the endpoint's **ID** and an **API key** (Settings → API Keys).

In bitGen Studio

- Pick **RunPod Serverless**, paste the API key and the endpoint ID, and put what a worker costs per hour (RunPod lists it per second - multiply by 3600) so Usage can add it. ✓ **Test** shows the workers that are ready or running and the jobs waiting. **Use it** switches every render to the endpoint; **This computer** switches back.
- The first job after a quiet spell waits for a worker to start (a cold start can take minutes). There are no live steps while a job runs - the progress shows the time instead.
- Pictures, videos and songs all come back. The worker only returns pictures and Save Video files, so on the way videos saved with Video Combine are saved with ComfyUI's own Save Video, and a song travels inside a tiny silent video and is taken out again on this computer.
- A job and its input files must be under **10 MB** (RunPod's limit). Long reference videos or songs are better on a pod or this computer; the job says so before anything is sent.
- The model check before a render is skipped: the models are on the endpoint's volume, not on this computer. If one is missing, the job fails with the endpoint's own message.

☁ On RunPod Serverless, only the styles it can render (2.744.0)

- While **Settings ▸ Remote generation ▸ RunPod Serverless** is in use, the style lists (the prompt docker's image styles, the video styles, the Music docker's) show only the styles the endpoint can render. The last line of a list says how many are hidden; pointing at a hidden style in Settings says why.
- An endpoint can't be asked which nodes it has, so the studio goes by what its image carries. RunPod's worker image 5.10.0 has **ComfyUI 0.34.0** and no node packs. A style works when every node of its workflow is in that ComfyUI. Styles whose worker needs ComfyUI's node lists are hidden too: LTX 2.3 / 2.5 and Wan 2.1 / 2.2.
- In the RunPod Serverless part of Remote GPU:
  - **ComfyUI in the endpoint's image:** pick **A newer ComfyUI** for an image you built with one (Fibo 2, YuE2, Qwen Image 2.1 ... then show).
  - **My image has extra node packs - show every style:** the studio can't see which packs, so every style shows.
  - A line says how many styles the endpoint can render, and a list gives the others with the reason.
- **This computer** shows every style again.

- Fixed: pressing **Save** in Settings while a remote GPU was in use put ComfyUI's address back to this computer's, so the next render ran here. Now the Remote GPU tab decides which ComfyUI renders. An address typed in the ComfyUI tab is this computer's, and **This computer** uses it.

Basic Settings ▸ GPU: what runs on the remote GPU (2.745.0)

- **Basic Settings ▸ GPU** has three choices:
  - **Remote:** every job on the remote GPU set up in Settings ▸ Remote generation (the same as its Use it).
  - **Local:** everything on this computer (the same as This computer). The remote stays set up for later.
  - **Both:** you pick what goes to the remote GPU, and the rest runs on this computer.
  - **LLM:** the chat / prompt model.
  - **Vision LLM:** the model that describes pictures.
  - **Image:** pictures, upscaling, graphs, 3D.
  - **Video + songs:** video jobs and songs. Jobs that don't use ComfyUI (exporting, karaoke, training) always run here.
- A line under it says where each kind runs now. Remote and Both can only be chosen once a remote GPU is set up; a button opens Settings ▸ Remote generation.
- With Both, ComfyUI's address stays this computer's, so your own ComfyUI starts as usual. Each job of a ticked kind is sent to the remote GPU when it starts; its log says where it rendered, and Usage counts its time there.
- **The LLMs on RunPod:** Settings ▸ Remote generation ▸ **LLM on RunPod (vLLM)** takes the endpoint ID of RunPod's vLLM worker (runpod.io: Serverless ▸ New Endpoint ▸ Deploy LLM API, with the Hugging Face model it should run).
  - The vision LLM needs a vision-language model, such as Qwen2.5-VL. One such endpoint can serve both - leave the vision endpoint blank.
  - Leave a model name blank and the studio asks the endpoint which model it serves. ✓ Test checks each one.
  - Without an endpoint, LLM and Vision LLM can't be ticked and run here.
- With Both, the RunPod Serverless style filter (2.744.0) applies only to the kinds sent to the endpoint.
- Fixed: Basic Settings' remembered settings (basic.cfg and per style) don't include the GPU choice.

Remote styles: each style says where it runs (2.746.0)

- Every style editor has **Runs on** under the style's name: Settings ▸ Image, Settings ▸ Video and Settings ▸ Audio ▸ Music styles.
  - **This computer:** the style renders with the models on this computer.
  - **Remote GPU:** the style renders on the remote GPU, so its models (and any node packs it needs) must be there. A box asks for its **RunPod endpoint ID**. Leave it blank to use the endpoint in Settings ▸ Remote generation, or give the style its own endpoint - for example one endpoint with SDXL models and another with FLUX.
- Save starts a remote style's name with **R_** (R_SDXL) if it doesn't already. Set it back to This computer and the R_ comes off again.
- **Basic Settings ▸ GPU** now decides which styles the lists show:
  - **Local:** only the styles for this computer. The last line of a list says how many remote styles are hidden.
  - **Remote:** only the remote (R_) styles. On a RunPod Serverless endpoint, a remote style it can't render is hidden too (2.744.0).

  - **Both:** every style. The style decides where the job renders: an R_ style on the remote GPU, the others here. A job with no style (upscaling, graphs and so on) follows the Image and Video + songs boxes.
- A job's log says where it rendered and why, for example "this image job (R_SDXL - a remote style) renders on RunPod Serverless ohkr3evlck2bxn".
- Settings ▸ Remote generation ▸ RunPod Serverless counts your remote styles only: "This endpoint can render K of your N remote styles".
- Your styles start as This computer, so after updating set Runs on for the styles you want on the remote GPU.

GPU on Local by default, and an update check that finishes (2.747.0)

- **Basic Settings ▸ GPU** is now the first item in Basic Settings, above Override style. It starts closed and its header shows the current choice (GPU · Local). Open it and it stays open, like the other sections.
- **Local is the default** and the first choice. Updating to this version switches it to Local once, because Remote GPU's Use it had put it on Remote, and Remote lists only R_ styles - with none yet the image and video lists were empty. The remote GPU stays set up; choose Remote or Both again whenever you want.
- With Remote chosen and no R_ style yet, the style list's last line says so: set Basic Settings ▸ GPU to Local, or a style's Runs on to Remote GPU.
- **Settings ▸ General ▸ Updates:** in a git install, Check for updates downloaded the whole update (about 13 MB of guide PDFs each release) inside one request. The studio stops waiting for a request after 5 minutes, so on a slower line, or a few versions behind, the check failed with "Failed to fetch". It now asks GitHub only for its newest commit, so it finishes in seconds, and Update now downloads the rest in the background (up to an hour allowed). If the studio stops answering, the message says so instead of "Failed to fetch".

Your API keys are kept, encrypted, in the settings folder (2.748.0)

- The API keys and tokens you add are saved in **settings/keys.json** inside the bitGen folder, each one encrypted (AES-256-GCM). They are:
  - the Hugging Face token,
  - the Civitai token,
  - the RunPod API key (Settings ▸ Remote generation),
  - the remote-access key,
  - your bitGen Community sign-in.
- **settings/keys.master** holds the key that opens them. It is made the first time a key is saved.
- **Updates leave them alone.** The settings folder is not part of the app's files, so `git pull`, Update now and a new zip never touch it. You add a key once.
- **custom.settings no longer holds any key.** Keys already in it move to settings/ the first time 2.748.0 starts. The same happens to custom.settings.bak and to the Community sign-in token in community.json.
- **Settings ▸ General** shows where the keys file is and which keys are saved (never a key itself). ↺ Reset settings to defaults keeps your keys.
- **Moving to a new folder or computer:** copy the whole settings folder across, keys.json and keys.master together. keys.json alone cannot be opened.

- **What the encryption does:** the keys are no longer plain text in a settings file, a backup of it or a file you send to someone. The app has to open them without asking you anything, so anyone who can read your files on this computer could open them too, as with any app that remembers a key.
- Fixed: misc/default.settings now lists every default the app uses (five newer ones were missing from the file; the values were already in use).

☁ Ready-made remote styles for RunPod's worker images (2.749.0)

- Seven remote styles come with the studio, one or two for each of RunPod's stock worker-comfyui images. Each image holds one set of models:

Style	RunPod image (runpod/worker-comfyui:5.10.0-...)
R_SDXL, R_SDXL img2img	sdxl
R_FLUX.1 schnell	flux1-schnell
R_FLUX.1 dev	flux1-dev
R_FLUX.1 dev fp8	flux1-dev-fp8
R_SD3 Medium	sd3
R_Z-Image Turbo	z-image-turbo

They use only ComfyUI's own nodes and the model files RunPod puts in that image, so they render with nothing uploaded.

- **Settings ▸ Remote generation ▸ RunPod Serverless ▸ Your endpoint's image:** the studio asks RunPod which image your endpoint runs (Ask RunPod, and by itself the first time) and lists only the ready-made styles for it. With the sdxl image you see R_SDXL and R_SDXL img2img; the others are hidden, and their tooltip says why. Pick the image yourself if RunPod cannot say. "base" (models on a network volume), "Your own image" and "Not known yet" show every remote style.
- Like any remote style, they show when Basic Settings ▸ GPU is on Remote or Both. On Local they are hidden.
- **The log now says where a job rendered.**
  - – On RunPod Serverless the VRAM step says the endpoint's worker loads the models, instead of "VRAM kept" (which was about this computer).
  - – The [power] line starts with "rendered on RunPod Serverless ..." and an estimate of what the job cost there (from the price per hour in Settings ▸ Remote generation). Then it gives what this computer used while it waited.
- Your own style can't take the name of a ready-made one: saving SDXL as a remote style is refused because R_SDXL already exists. Use R_SDXL, or give yours another name.

The splash is the bitGen brain, with the logo (2.752.0)

- **The desktop app's start-up splash is always the bitGen brain film** now: lightning strikes the brain, then the big bitGen Studio title, with the credits under it. The orange logo film is no longer one of the two at random.
- **The official logo sits in its top left** - small, clear of the brain. It stays there when the brain picture stands in for a film that cannot play.
- **The brain film is sharper**: it was 960 × 544 and is now 1920 × 1080, upscaled with an AI upscaler, so the bitGen Studio titles and the labels are crisp.
- A click or any key still skips the splash.
- The website's home page plays the same brain film, with the logo in the same place.

The LLM on OpenRouter; "Remote GPU" is now "Remote generation" (2.753.0)

- **Settings ▸ Remote GPU is now Settings ▸ Remote generation.** Everything in it works as before.
- **The LLM can run on OpenRouter** (openrouter.ai): one API key gives you hundreds of models, some of them free. Nothing to install. It helps most when your graphics card is small - the LLM no longer has to share it with the picture models.
  - – Make a free account at openrouter.ai and create a key (Keys).
  - – Settings ▸ Remote generation ▸ LLM: pick **OpenRouter** and paste the key. It is kept encrypted in the settings folder, with your other keys.
  - – Pick the **LLM**. **Free models only** is ticked at first; untick it to see the paid ones with their price per million tokens.
  - – Pick a **Vision LLM** for describing pictures - only the models that take pictures are listed (). Leave it blank to use the LLM, when it takes pictures.
  - – ✓ **Test** checks the key and the model.
  - – Basic Settings ▸ GPU: pick **Both** and tick **LLM** and **Vision LLM**. This works without a remote GPU - pictures, videos and songs still render on this computer.
- Free models allow about 20 requests a minute and 50 a day; 1000 a day once the account has bought $10 of credits. OpenRouter can change these limits.
- Your prompts and pictures go to OpenRouter and the model's provider. Keep private material on this computer.

☁ Use remote generation, the Remote generation docker and pod shortcuts (2.754.0)

- **Settings ▸ Remote generation ▸ Use remote generation** is the first thing in the tab. Unticked, the rest of the tab, Basic Settings ▸ GPU and the Remote generation docker are hidden, and everything runs on this computer. A new install starts unticked; an update starts ticked when something remote was already set up.
- **The Remote generation docker:** in Basic Settings ▸ GPU, press ☁ **Remote generation docker**. It floats and docks like the others, and it has no button in the status bar. It shows:
  - – where the work runs, the remote GPU in use and where the LLM runs;
  - – **My pods**: your RunPod pods (your API key - an "All" key), with ▶ **Start**, **Stop** and **Use it** (render on that pod). A stopped pod still pays for its storage until you terminate it on RunPod;
  - – **Pod shortcuts**: RunPod templates - "public pods" - that you deploy as your own pods.
- **Add a pod shortcut:**
  - – On RunPod, open **Explore** (the docker's **Explore RunPod templates ↗**) and find a template - pick one that runs ComfyUI.
  - – Copy its page's link (runpod.io/console/explore/…), paste it into the docker with a name, and press **Add**.
  - – **Deploy ↗** opens RunPod's deploy page with the template picked (bitGen's referral link). Pick the GPU and deploy.
  - – Back in the docker, **My pods** lists the new pod - **Use it**.

☁ Pods keep working after 15 November; loading pod, pod loaded, stopping pod (2.755.0)

- RunPod retires the API bitGen used for pods (REST v1) on 15 November 2026. bitGen now uses RunPod's new API (REST v2) for My pods, ▶ Start / Stop and the Serverless endpoint's image. Nothing to change on your side. Your API key needs access to pods - an "All" key.
- **The Remote generation docker shows what a pod is doing:**

– ▶ **Start** - "loading pod…" while RunPod starts it and then its ComfyUI, then "pod loaded" once ComfyUI answers. Then it shows how long the pod has been running and about what that has cost.

– **Stop** - "stopping pod…", then "stopped". A stopped pod still pays for its storage until you terminate it on RunPod.

Remote servers, RunPod Serverless or Pods, and the PODS docker (2.756.0)

- **Basic Settings ▸ GPU ▸ Remote servers:** pick **RunPod** or **OpenRouter**. OpenRouter runs the LLM there, and pictures, videos and songs render on this computer. For RunPod, pick **Serverless** (an endpoint, paid only while a job runs) or **Pods** (your own GPU machine, paid by the hour while it runs). With nothing set up yet, these choices show under Local, so you can pick Pods first - it unlocks Remote.
- **The PODS docker** (the **PODS** button in Basic Settings ▸ GPU) shows only when RunPod ▸ Pods is picked:

  – RunPod's public templates - **Official**, **Verified** and **Community** - with a search and **ComfyUI only** (on: the templates bitGen can render with). Each name opens its page on RunPod.

  – **Create pod…:** pick the GPU (RunPod's price per hour now, its memory and how many are free; "none free" can't be picked), the cloud (**Secure** - RunPod's own data centres - or **Community** - usually cheaper), how many GPUs, the container disk (wiped when the pod stops) and /workspace's size (kept while stopped, billed for storage). The hourly estimate shows below. **Create pod** makes it on RunPod; it then shows "loading pod…" and "pod loaded".

  – **My pods:** every pod on your account. marks the ones bitGen created or started. ▶ **Start** / **Stop**, **Use it** (render on it, through its ComfyUI port) and **Terminate** - it asks first, because the pod and every file on it are deleted; only a network volume survives.

Every RunPod template as a card (2.756.1)

- The PODS docker now lists **every** public template RunPod has at once. **All** shows Official, Verified and Community together, or pick one group; each button says how many templates it holds. RunPod gives at most 100 templates a group.
- Each template is a **card**, like ComfyUI's template browser: a coloured tile with its initials, the kind of machine it is made for (NVIDIA, AMD or CPU) and a **ComfyUI** badge when it runs ComfyUI; below it the name, the image, the group it comes from, its disk and /workspace sizes and its web ports. **Create pod…** opens the form above the cards, and ↗ opens the template's page on RunPod. RunPod's list has no pictures or descriptions for templates, so bitGen draws the tile.
- **Search** as you type: every word must be found in the name, the image, a port, the kind or the group - for example `comfy 3000` or `pytorch rocm`. **Every kind** narrows the cards to NVIDIA, AMD or CPU templates.
- **ComfyUI only** is now off at the start: it hid all 100 Official templates, because none of them run ComfyUI. Tick it to see only the templates bitGen can render with.
- If RunPod doesn't answer for one group, the others still show and the line under the buttons names the missing one.

Joining the Community is for people (2.756.3)

- Bots were joining the Community with random names and other people's email addresses, so the Community emailed codes to strangers. Joining now turns them away before any email goes out.
- **Community ▸ Join** in bitGen Studio does a short automatic check first. It takes about a second and you don't see it. Older versions of bitGen Studio can't join any more (they say so): update, or join on the website. Signing in is the same as before.
- On the website, **Join** has a slider: slide it to the end before pressing Join.

- A first or last name that looks machine-made (random capital letters inside one word) is refused. Type your name the way it is written.

Settings ▸ API - render on an online service (2.757.0)

- **Settings ▸ API** (right after Remote generation) lets bitGen render without a good graphics card: an online service does the work and charges your account for each render. Seven services are built in - **OpenRouter** (pictures, video, the LLM), **fal.ai** (pictures, video, songs), **Replicate** (pictures, video, songs), **OpenAI** (pictures, the LLM), **Google Gemini** (pictures, the LLM), **Claude** from Anthropic (the LLM) and **RunningHub** (ComfyUI workflows in the cloud: pictures, video, songs).
- To add one: paste its **API key** - **Service** on **Auto-detect** recognises it from the key's shape (or pick it) - type a **Name**, leave **Use** ticked and press **+ Add**. **Get a key** under the boxes opens the service's key page.
- **Use** makes the API's styles: `API_<name> pictures`, `API_<name> video` and `API_<name> songs` (only the kinds the service makes). They are in the picture, video and song style lists like any other style and render on the service - nothing runs on this computer's graphics card and ComfyUI is not needed. Untick **Use** and the styles it made go again; a style you made yourself stays.
- **Models and key** shows the model each kind starts with (for example `google/gemini-2.5-flash-image` on OpenRouter, `fal-ai/flux/schnell` on fal.ai) - type another of the service's models, or a new key, and **Save models and key**. For **RunningHub** the model is the **workflow ID** - the number in the workflow's address on runninghub.ai. bitGen reads the workflow and fills in your prompt, the size, the seed and your reference pictures.
- Any style can run on an API: in Settings ▸ Image, Video or Audio ▸ Music, set the style's **Runs on** to **API**, pick the API, its **Model** and, if the model has options of its own, **Extra settings** as JSON (for example `{"num_inference_steps": 4}`; for RunningHub a list of `{"nodeId", "fieldName", "fieldValue"}`). The name then starts with **API_**.
- **Claude** is the LLM only - it makes no pictures, video or songs. Add its key (it starts with `sk-ant-`, from the Claude Console) and **Use** ticks **Use for the LLM** for you: the screenplay writer, prompt help, picture descriptions and the shot list then run on `claude-sonnet-5-5` (change it in **Models and key**, for example to `claude-opus-5-5` or `claude-haiku-5-5`). bitGen leaves out the settings Claude refuses and gives it room to think before it answers.
- ✓ **Test** asks the service whether it takes the key. **Use for the LLM** (OpenRouter, OpenAI, Gemini, Claude) sends every LLM call - the prompt LLM, picture descriptions, the shot list - to that service; one API at a time. ✕ asks first, then removes the API and its styles.
- The key is kept **encrypted** on this computer (settings/keys.json) and only sent to its service; the page shows only its last four characters. A render's job file carries only the key it needs.
- Video: OpenAI ended its video API (Sora) in September 2026, and Gemini's video models are not offered through the API keys bitGen uses - use OpenRouter, fal.ai, Replicate or RunningHub for video.

P_ pod styles and a pod's loading bar (2.758.0)

- **Only P_ styles on Pods.** With Basic Settings ▸ GPU on **RunPod ▸ Pods** (Remote or Both), the style lists show only **P_** styles - styles that render on your pod with the models already on it. On Local or Serverless the P_ styles are hidden, because they need a pod.
- **Choosing the model: Styles.** Once a pod says *pod loaded*, press **Styles** on it in the PODS docker. bitGen reads the pod's ComfyUI and lists:

  – **The pod's models**, each with the bitGen style it fits - for example `qwen_image…` → qwen2_1, `z_image_turbo…` → Z-Image Turbo, `krea2…` → Krea 2, `flux1-krea…` → FLUX.1 dev. A model bitGen has no workflow for (a custom one such as Boogu) can't be ticked - use one of the pod's own workflows for it.

  – **The pod's saved workflows** - the ones the template ships in its ComfyUI. They already use its models and custom nodes. Tick what you want and press **Make P_ styles**. A model becomes

`P_<model file>`: bitGen's own workflow for that model type, with the pod's model, text encoder and VAE put in. A workflow becomes `P_<workflow name>`, exactly as the template made it. Picture styles only for now - a video or song workflow is listed with the reason. Making them again updates the same styles.

- Any style can run on a pod too: Settings ▸ Image ▸ the style's **Runs on** ▸ **RunPod pod**, with the pod's ID (blank = the pod you pressed **Use it** on). The name then starts with **P_**.
- **Basic Settings with a P_ style.** Picking RunPod ▸ Pods ticks **Override style**, so Basic Settings' **steps, seed, CFG, size and aspect ratio** go with every pod render. The **checkpoint** box and the **LoRA** rows are switched off while a P_ style is picked - the pod's own model and LoRAs are used. With Override style off, the pod's workflow keeps its own numbers.
- **The loading bar.** While a pod starts, the PODS docker and the Remote generation docker show where it is: 1/4 *waiting for a GPU machine* → 2/4 *starting the container* (the template's image is downloaded) → 3/4 *ComfyUI starting* (a template like this one downloads its models now - often the longest part) → *pod loaded in …*. RunPod doesn't report download percentages, so the bar is the time so far against how long this pod - or another pod of the same template - took last time ("3 min of about 7 min"); the first time it moves by stage.
- **Your pods and pod shortcuts are in the PODS docker now.** The Remote generation docker keeps where the work runs and a **PODS docker** button. A pod shortcut is the ☆ on a template's card: press it and bitGen keeps the template

  – its link is filled in for you. ★ **Shortcuts** lists them, each with **Deploy ↗** (RunPod's deploy page with the template picked) and **Create pod…**; press ★ again to remove one. A template RunPod doesn't list (a private one) can still be added on that tab by pasting its link.

⬇ Models onto a pod, and pods for a style (2.759.0)

- **A pod for a style.** In the PODS docker, **For …** (beside ComfyUI only) picks one of your bitGen styles: the templates whose name or image carries that style's model family come first, marked ✓ **fits** (Krea, Qwen, Z-Image, FLUX, Wan, SDXL …). A pod you create this way gets that style's P_ style by itself once it has loaded, when the pod has the model.
- **⬇ Models** on a running pod downloads models into its ComfyUI. Pick **a style** - its files from bitGen's download list are ticked, with their total size - or search bitGen's list and tick single files, then **⬇ Download ticked**. A bar shows each file; then **Make P_ styles from the pod**. Files go to the folder ComfyUI loads them from; when that is not under `/workspace` bitGen says so, because those files go when the pod stops.
- ComfyUI has no "download a file" call of its own, so bitGen uses what the pod has:

  – **The pod's Jupyter** - any file. Pods bitGen creates get a Jupyter password of bitGen's own (kept encrypted), so this just works when the template runs Jupyter. For a pod made on RunPod's site, paste its Jupyter password (RunPod ▸ the pod ▸ Connect) in the box.

  – **ComfyUI-Manager** - when there is no Jupyter. It only downloads models in its own list (Qwen Image, FLUX.1, Wan 2.2, SDXL … - not Z-Image or Krea 2); bitGen names the files it can't fetch that way.
- A gated model on Hugging Face uses your Hugging Face token (Settings), sent only to your own pod with the download.
- **A file from a link or from this computer** (in ⬇ Models): pick the models folder it goes in (loras, checkpoints …), then paste a link - Civitai, Hugging Face or any web address - and press **⬇ From the link**, or press **⬆ From this computer…** and pick a file from your disk. Both go through the pod's Jupyter. A file from your computer is sent in parts; your upload speed decides how long it takes, so a model that is on Hugging Face or Civitai is quicker by its link. Civitai links use your Civitai token from Settings.

The Audio docker, prices in your currency, and My pods (2.760.0)

- **The Audio docker.** Music, Stems and SFX are now the tabs of one docker, **Audio**, and the statusbar has one **Audio** button instead of three. The button shows or hides the Audio docker on the tab you used last. Its title bar reads **Audio** with the tabs **Music · Stems · SFX**: a tab shows that one and puts the others away. Each tab keeps its own height, ▾ fold, maximize and ⇱ float. Anything that opens one of them - a workspace, a saved docker layout, voice chat - opens it as the Audio docker's tab. A tab whose extension is turned off is not offered.
- **Currency** (Settings ▸ General): RunPod, OpenRouter and the other services list their prices in US dollars. Pick your currency (AUD, USD, EUR, GBP, CNY, JPY …) and bitGen shows them in it: the PODS docker's GPUs, the price an hour and your pods, Remote generation's pods, and OpenRouter's models. Hover a price to see the dollars and the rate. The rate is the day's reference rate (from Frankfurter, which publishes the central banks' rates - no account, no key), checked every 12 hours and kept for when you are offline. Type a rate in **1 USD = …** to use your own (your bank's, say); clear it to go back to the day's rate. A remote GPU's cost goes on the Usage bill in the Energy cost currency.
- **My pods** (the PODS docker): a button beside the template tabs, with how many pods you have. It shows your pods alone - no template cards, no search. The template tabs (All, Official …) show the templates alone. **Create pod…** shows the form alone (× Close or a tab goes back), and once the pod is created the docker switches to My pods, where its loading bar runs.

Pods as items: what a pod is for, files onto it, its cost, and pods made again (2.761.0)

- **Each pod in My pods is an item.** Press its name to open it (▸ / ▾). The name line shows the GPU, the price an hour, the state and what RunPod has billed for it today and this month.
- **What the pod is for**: under its name tick **Video**, **LLM**, **vLLM** (the picture-describing model), **Image** and **Image edit**. **Save** is off until you change something - a tick, a field, a file sent or a folder made - and off again once saved. Save makes the pod's P_ styles: for every model file on the pod that bitGen has a workflow for, a P_ style of each ticked kind (P_<file>, P_<file> edit, P_<file> video). A kind you untick removes the P_ styles Save made for it (the ones Styles made stay).
- **LLM / vLLM on a pod**: ticked and saved, bitGen's prompt LLM (or the picture-describing model) talks to an OpenAI-compatible server running on the pod - vLLM, Ollama and the like - through RunPod's proxy. Give its port (8000 by default) and model; leave the model blank and bitGen asks the server which one it serves. One pod at a time holds each.
- **New folder** makes a folder inside the pod's checkpoints or loras. **Checkpoint** and **LoRA** send a file from this computer into models/checkpoints or models/loras on the pod - or into the folder picked under **into**. They go through the pod's Jupyter (pods bitGen creates have it ready; for others paste the password in ⬇ Models).
- **The cost** (Basic Settings ▸ GPU, while RunPod ▸ Pods is picked): how many pods run and what they cost an hour now, plus RunPod's own bill for today and this month (GPU, CPU and disk) - asked again every minute. Usage keeps the pods day by day beside the electricity, in the Energy cost currency, and adds them to each day's bill.
- **A P_ style remembers its pod**: the template, the GPU, Secure or Community, the disk sizes and the files bitGen put on it (and where each came from). When you render with a P_ style whose pod is stopped, bitGen starts it; when RunPod no longer has the pod, bitGen creates a new one from the same template on the same GPU, waits for its ComfyUI, and puts the files on it - from this computer when the file is still there, else from its link, else from bitGen's download list. The job waits in the queue meanwhile ([pod] lines in its log), then renders on the new pod; the style moves to it.
- With Pods picked in Basic Settings, only P_ styles are offered.
- **The Audio docker** keeps one tab on screen only in the main column: Music, Stems or SFX floated (⇱) or put in a ▣ Docker shows beside the others.

One checkpoint per pod - the pod becomes the style (2.762.0)

- **Checkpoint** (in a pod's item in My pods) is now the same dropdown as Basic Settings' Override style - a folder filter, the file and - listing this computer's ComfyUI models. Pick ONE checkpoint for the pod: the pod becomes that style. **Save** makes **P_<the pod's name>** (and P_<pod> edit / P_<pod> video for those ticks) with bitGen's own workflow for the model's family - a Krea checkpoint gets bitGen's Krea workflow, an SDXL one the SDXL workflow - and the pod's text encoder and VAE closest to the workflow's.
- When the pod already has that model (the template brought it), it is simply picked - nothing is sent. Otherwise it is sent once from this computer, into the same sub-folder it has here (checkpoints/Krea/…), in the background; the line beside Save says how far it is.
- **LoRAs**: adds a row with the same dropdowns. They go to the pod with the checkpoint and into its P_ style.
- **Override style + Pods**: with a P_ style picked, Basic Settings' checkpoint box is open again. A checkpoint picked there is used on the pod: if the pod lacks it, bitGen sends it before the render (the job waits, [pod] lines in its log) and then uses the name it has on the pod. The temporary LoRA rows stay off for P_ styles.
- New folder and the file dialogs are gone from the pod item - the dropdowns replace them.

Your styles on your pods (2.763.0)

- **No more P_ copies.** On RunPod Pods (Basic Settings ▸ GPU ▸ Remote or Both ▸ RunPod ▸ Pods) the style lists show your own styles again, and each one renders on a pod. The P_ styles you made before still work. RunPod Serverless (R_) styles are hidden on Pods.
- **Which pod a style uses:** first the style's own pod (a pod's **Styles** in My pods), then the pod ticked for its kind (Image, Image edit, Video), then the pod in use. Tick **Image**, **Image edit** and **Video** on different pods for a pod per kind, or tick them all on one pod to use one pod for everything. Each kind is on one pod at a time - ticking it on a pod takes it from another.
- **A pod's Styles** (in its item in My pods) replace the Checkpoint and LoRA dropdowns of 2.762.0: the dropdown lists your styles for the Generate mode (Image, Video, Music); **All** lists every style. Pick one and press ; ✕ on a style takes it off. **Save** links them to the pod and, when the pod is running, makes them ready on it.
- **Made ready on the pod** means: the style's workflow, model, text encoders, VAE and LoRAs are put on the pod - files in bitGen's download list are downloaded by the pod itself (fast), other files are sent from this computer - and node packs the workflow needs are installed through the pod's Jupyter, then ComfyUI is restarted through ComfyUI-Manager. The line beside Save says how far it is; anything it could not do (a node no pack bitGen knows adds, a file that is nowhere) is named there.
- **A style with no pod yet:** press Generate and the render waits while a box asks: **Create pod** from RunPod's "ComfyUI CUDA 13" template (on hosts with CUDA 13 or newer) - pick Secure or Community, the GPU (is bitGen's pick: the cheapest with enough memory for the style's model, with its price an hour in your currency) and the disk - or **Use this pod** for one of your pods (a stopped one is started). Either way the style is linked to that pod, so next time it goes there by itself. **Cancel** cancels the render.
- **Before every render** on a pod, bitGen starts the pod when it is stopped, makes it again from how it was made when RunPod no longer has it, waits for its ComfyUI and makes the style ready on it once per run of the app. The job's log says what happens in [pod] lines.
- **Override style** still works on Pods: Basic Settings' checkpoint is sent to the pod before the render when the pod lacks it.
- **LLM on a pod:** ComfyUI is not a chat server, so the LLM runs as its own server on the pod (vLLM, llama.cpp or Ollama); tick LLM / vLLM on that pod and give its port, as before.

Your custom nodes on a pod, and what the pod is doing (2.763.1)

- **While a render waits for its pod** a line over the status bar says what is happening: starting the pod, making it again, waiting for its ComfyUI, putting the style's files and node packs on it, or waiting for you to answer the pod question. The job's log has every step in [pod] lines.
- **A pod that cannot be used stops the render** with the reason in its log - it is no longer rendered somewhere else instead. Cancel in the pod question cancels the render.
- **Basic Settings ▸ GPU ▸ Both:** a style with its own pod, or whose kind is ticked on a pod, goes to that pod. Any other style renders on this computer unless Image (or Video) is ticked - the job's log says so and how to change it.
- **Your custom nodes go to the pod:** a pod bitGen creates for a style gets every custom node pack of this computer's ComfyUI, then its ComfyUI is restarted. Packs from GitHub (or another git address) are cloned on the pod at the same version you have; other packs - and ones the pod cannot clone, like a private one - are zipped here (without __pycache__ or files over 50 MB) and uploaded. Packs the pod already has are left alone, and packs you switched off (.disabled) are not sent. **⬆ My custom nodes** in a pod's item in My pods does the same for any pod.
- When a style needs a node the pod lacks, bitGen asks this computer's ComfyUI which pack it comes from and sends that pack. Restarting uses ComfyUI-Manager when the pod has it, else RunPod restarts the pod.
- **2.763.2:** a render that goes to a pod no longer starts this computer's ComfyUI or checks the style against its model lists first - the render is sent straight away (before, it could fail on this computer without reaching the pod). When a render fails, the lines that say why ([pod], [gpu], errors) are printed after "that one failed" in the log.

The PODS docker opens for a style with no pod (2.763.3)

- **Generate with a style that has no pod** now opens the PODS docker (it replaces the small question box). A banner names the render that is waiting, and the create form is already open on RunPod's "ComfyUI CUDA 13" template with the **Style** and its kind (**For**: Image, Image edit or Video) picked, bitGen's GPU pick (the cheapest with enough memory for the style's model), **GPUs** 1 and the disk filled in. Change the GPU, how many GPUs, the cloud or the disk, then press **Create pod**: the style is linked to the new pod, the pod is the one for that kind, and the render goes there once the pod has loaded and has the style's files.
- **Or use one of your pods:** while a render waits, every pod in My pods shows **▶ Use for <style>**. ▶ Start, Stop and Terminate are there as before. **Cancel the render** in the banner cancels it.
- **Renders reach the pod now:** RunPod's proxy refused the render with "HTTP Error 403: Forbidden" because of the way the render worker introduced itself; it now gives its own name. The live step count also works over the pod's https address.
- **The style's files are waited for:** the downloads on the pod are followed until they finish. If a file the style needs is still not on the pod, the render stops and its log names the file - it is not sent to a pod that cannot render it.

The PODS docker: what a pod is doing, and every detail before it is made (2.763.4)

- **How busy a pod is:** in My pods, each running pod shows bars beside its name for how busy its GPU is and how full its GPU memory is. Open the pod to see all of them: the GPU, its memory, the RAM, the CPU and both disks (the container disk and /workspace). They update every few seconds while the docker is on screen.
- **Every detail before Create pod:** the form lists what the pod will be - the template and its image, the GPU x how many (and the memory in all), the price an hour, the cloud, both disks, a CUDA 13 host, the ports, the style it renders and the kinds it is the pod for, and what goes on it after it loads (this computer's custom nodes, the style's files). It follows every change you make in the form.
- **The prompt docker's style is picked:** the docker's style dropdowns (For, Style, and a pod's Styles) start on the style picked in the prompt docker for the Generate mode.

- **A pod that is gone is never made again by itself:** when RunPod no longer has a style's pod, the render opens the PODS docker, says the pod is gone, and fills the new one in the way the old one was made (its template if RunPod still lists it, its GPU, how many, the cloud, the disk) - check it and press Create pod, or use one of your pods.
- **Sending a file from this computer works now:** RunPod's Jupyter refused the upload ("refused part 1 (HTTP 400)") because bitGen gave the file a hidden name while it was being sent; it uses a normal name now. This covers Override style's checkpoint, LoRAs and custom nodes sent from this computer. A big checkpoint takes as long as your upload speed allows.

The RunPod docker, and dockers inside your own dockers (2.763.5)

- **RunPod docker:** a docker of its own with your RunPod balance, what it is spending now and how long the balance lasts at that, your spend limit, what your pods cost (running now, today, this month, and each pod - a terminated pod stays while it is on the bill), and RunPod's GPU prices (Secure or Community cloud, the memory, the price an hour, how many are free, the most one pod can have; search it and sort it by price, memory or name). Open it with RunPod in Basic Settings ▸ GPU or in the PODS docker. It is asked again every minute while it is on screen, and asks at once.
- **The cost line left Basic Settings:** the line under GPU (2.761.0) now lives in the RunPod docker.
- **Any docker of yours takes other dockers:** a docker made with ⊞ Docker configs ▸ ▣ New docker… (floating, or confined to a docker) now takes PODS, RunPod, Basic Settings, LLM tokens and the other docker windows - drag one by its title bar onto it and let go on the orange line - as many as you like, as tabs or side by side. A main docker's title bar dragged onto it goes in too. They come back in it after a restart.
- **The floating dialogs stay in front of the dockers:** PODS, RunPod, Basic Settings, Community and the other floating dialogs are always over the ▣ dockers and your own dockers, even after you press a docker. A docker confined to another docker no longer covers them (it was put over everything). Move a dialog aside, or dock it, to reach what is under it.
- **Custom nodes sent from this computer:** a node folder is packed a file at a time, so bitGen keeps working while a big one is packed; weights, sounds and archives are stored as they are, and a Python environment inside a pack is left out (the pod installs its own). The log says how far it is: the files packed, then each part sent and its size. A pack over 2 GB is not uploaded and the log says why - put it on the pod with git, or make it smaller.

Pod presets, and several styles on one pod (2.764.0)

- **Several styles share one pod:** a pod can hold as many styles as its disk takes - only one style's models are loaded at a time, so the first render after switching styles takes a little longer.
- **Generate with a style that has no pod:** a box opens with your pods - each with its GPU, whether it is running, its price an hour, the styles it holds and whether its GPU memory is enough for this style - and <style> on each: the style goes on that pod (its files are sent first; a stopped pod is started) and the render goes there. Your presets whose pod is not on RunPod now are listed too: <style> & ▶ adds the style and makes the pod. Create pod… opens the PODS docker's form, filled in for the style; Cancel render drops the render. The box only opens when you press Generate.
- **Presets tab (PODS):** every preset is a file in your bitGen folder, runpod/pods/presets/<name>.pod, with the template, the GPU, how many GPUs, the cloud, both disks, a CUDA 13 host, the kinds and the styles. Each one is an item you open: edit any detail, × takes a style off, Add a style puts one on, then Save changes. Styles change on a running pod at once; the GPU, template and disks the next time the pod is made.
- **The title bar:** ▶ starts the pod (a terminated one is made again with every detail and all its styles' files - workflows, models, text encoders, VAE and LoRAs - plus your custom nodes); stops it (the GPU charge ends, /workspace stays); terminates the pod - everything on it is deleted, the preset stays so ▶ can make it again; × deletes the preset (the pod is left as it is). and × ask first.

- **Under the details:** the loading sequence while it starts (creating, the pod loading, ComfyUI starting, your custom nodes, each style's files, ready - a step that could not finish says why), the pod, its cost an hour, the hours used and the total cost of every pod the preset has had (with RunPod's bill for this month).
- **When its GPU is not free:** ▶ lists the GPUs with as much memory or more that are free now - pick one and the preset keeps it.
- **Making presets:** a pod created for a style is kept as a preset by itself; Save as preset on any pod in My pods; New preset starts one from scratch. A style put on a preset's pod anywhere joins the preset.

Basic Settings ▸ GPU on Local shows nothing remote; a model sent to a pod shows how far it is (2.764.1)

- **Local is quiet:** with Local picked, Basic Settings ▸ GPU shows only Local · Remote · Both and "Every job runs on this computer." - the Remote servers and RunPod choices, ☁ Remote generation docker, PODS, RunPod and ⚙ Settings ▸ Remote generation are hidden, and the Remote generation, PODS and RunPod dockers close when you pick Local.
- **Remote and Both show them again.** With nothing set up yet, picking Remote (or Both) waits: the choices show so you can pick RunPod ▸ Pods (or set one up in Settings ▸ Remote generation), and it is saved as soon as there is somewhere to render.
- **A render waiting for its pod says why:** the Queue card no longer says "waiting for the GPU" then - it says what the pod is doing: starting, loading, putting your custom nodes on it, downloading a file on the pod, or sending a model from this computer with how far it is: "sending model.safetensors to the pod - 1.2 GB of 6.5 GB (18%) · 3.4 MB/s · about 26 min left". The line over the status bar and the job's log say the same.

Only the custom nodes your workflows use go on a pod (2.764.2)

- **Not every custom node any more:** a pod gets only the custom node packs its styles' workflows use and it lacks - found from this computer's ComfyUI (which folder each node comes from), cloned at the same version or uploaded. A pod made for a style, a preset's ▶, and a style added to a pod all work this way; a pack none of your workflows use stays here.
- **⬆ Nodes its styles need** (My pods ▸ a pod) puts on the packs the pod's styles' workflows use - put a style on the pod first.
- **The log says each step:** a small pack shows its size in KB, then "<pack> is on the pod", "asking the pod to unpack and install …", and "installing <pack> on the pod (1 of 2 done, 40 s)". If the pod cannot report how the install went for 3 minutes, the step ends with that reason instead of waiting.
- **A preset's loading sequence** has one step per style - its nodes and its files.

A style's custom nodes come from their git address (2.764.3)

- **Cloned on the pod, not sent from your computer:** a node pack a style's workflows need is cloned on the pod from its git address (GitHub to RunPod is fast). A pack you installed with git is cloned at the same commit. A pack without a git checkout here is looked up first - bitGen's own node list, its pyproject.toml, ComfyUI-Manager's node list on your computer, the Comfy Registry - and cloned at its version's tag when there is one (else the newest, and the log says so). A pack that lives inside a bigger repository (like ComfyUI-H3-NativeAudioLock) is cloned aside and linked in.
- **Uploaded only when there is no other way:** a pack with no address anywhere (one you made yourself), a single .py, or one the pod cannot clone (a private repository) is uploaded from your computer - the log names it and says why.

Joining the community: why the email code (2.764.4)

- **You don't have to be a member to use or download bitGen Studio** - you only need to be a member to talk to the community (the Help room, chat and friends, and posting in the Discussions without waiting for an admin; guests can still post there with an admin's approval).
- **New members confirm their email address** with a 6-digit code sent to it - to prevent spam sign-ups and to make sure a real person is joining. Until the code is typed, a new account can sign in but not post, chat or add friends; an account never confirmed is removed after a week. The join forms in Community and on the website say so.

Dock a docker above or below another one; floating dockers stay in front (2.764.5)

- **Above or below:** drag a floating docker over a docker that sits inside one of your dockers (side by side, or the tab showing) - near its top edge an orange bar shows across its top, near its bottom edge across its bottom. Let go and the two are stacked in that place (a new stacked docker named after them - ⚙ renames it); the others stay beside them. This works in a floating docker and in a ▣ docker docked in the main column, for main dockers, Basic Settings, PODS and the other docker windows.
- **Into a stack:** over a stacked docker the bar shows between the dockers in it - drop there to add one between them. Near its left or right side it goes beside the stack instead. The middle of a docker still adds it beside (side by side) or as a tab, as before.
- **Floating dockers in front:** a floating docker is always in front of a docker docked inside another docker, even when you press the docked one; the floating dialogs (Basic Settings, PODS, Community …) stay in front of both.

Pod downloads that stop at 0%, and gated models (2.764.6)

- **Downloads keep going:** since 2.763.4 the pod's downloads and node installs could stop at 0% / "installing" right after they started. They now run as their own job on the pod, so they finish even after bitGen has closed its connection to the pod's Jupyter.
- **Gated models are said:** before a pod downloads a Hugging Face file, bitGen asks Hugging Face whether it may. A gated file (its license not accepted, or no Hugging Face token) or one that is not there any more is not started - the render's pod note, the log and ⬇ Models say which file and what to do: open its page on huggingface.co, accept the license, and add your Hugging Face token in Settings ▸ Models. Krea-2 is not gated.
- **A job that stopped, or no data:** a download with no data for 3 minutes says so next to its bar; a job the pod stopped (no sign of it for 90 seconds - the pod restarted or ran out of memory) is marked failed with the reason instead of waiting.

Checkpoints and LoRAs from Civitai straight onto the pod (2.765.0)

- **Found on Civitai, downloaded by the pod:** when a pod needs a checkpoint or LoRA from this computer (a style's model, the Override style's checkpoint, a preset started again), bitGen first looks for the very same file on Civitai. The pod then downloads it from civitai.com or civitai.red - far faster than sending it from your computer. Only this computer's own file sends when it is not on Civitai, or the pod could not get it.
- **How it is found:** its Model manager card (Civitai look-up) names its Civitai version; otherwise bitGen reads the file once to get its SHA-256 (kept in its card, so only the first time takes a moment) and asks Civitai for it. Only a file with the same SHA-256 is used - never a different version or size.
- **civitai.red:** the site picked in Settings ▸ General ▸ Civitai is asked first, then the other one; mature models are on civitai.red and need your Civitai API key. The key goes only to Civitai with the pod's download - it is not kept in the pod's file list.

- The render's pod note shows "the pod is downloading <file> from civitai.red - 1.2 of 6.5 GB (18%) · 45 MB/s"; if Civitai refuses it (no key, a mature model on civitai.com) it says why and sends it from this computer instead.

Remote models: the list your pods use, ticked pods and a bar for each transfer (2.766.0)

- **The list:** runpod/pods/remote_models.json holds the models your pods use - for each one its label and group ("Krea raw", "Krea"), the ComfyUI folder and file, and the link a pod downloads it from (Hugging Face, Civitai, your own Hugging Face repo or any link). It fills itself: the styles your pods use, a model found on Civitai, a model put in your Hugging Face repo, a Civitai in remote mode. Keys and tokens are never written in it.
- **Remote mode:** with remote generation on and RunPod pods picked in Basic Settings, Model manager and Basic Settings' Override dropdowns (the checkpoint and the LoRA rows) list THESE models instead of this computer's. Picking one that a pod does not have yet: the pod downloads it from its link before the render. Local mode is as before.
- **Tick pods:** a ☐ left of each pod's name in PODS ▸ My pods - tick as many as you like. ⇪ on a model's card in Model manager sends it to every ticked pod; a ✓ count on the card shows how many pods have it.
- **A model on your hard drive with no link:** Model manager ▸ From this computer… - search for it and press ⇪ Add. bitGen looks for the very same file on Civitai first; if it is not there it puts it ONCE in a private repo on your own Hugging Face account (<you>/bitgen-models - needs a Hugging Face token that can write, in Settings ▸ Models), and every ticked pod downloads it from there in minutes. With no such token it is sent from this computer to each ticked pod (slow). The same happens by itself when a render needs a model of yours on a pod.
- **Link…** adds a download link with its folder; **↻ My styles' models** adds the files of the styles your pods use.
- **A progress bar for each transfer:** PODS docker and Model manager show every transfer - a pod downloading, this computer sending to a pod, a file going to your Hugging Face repo, a file read once to find it on Civitai - with %, size, speed and time left; a failed one is red with the reason. ✕ Clear takes the finished ones off.
- The render's LoRAs (Override's temporary LoRAs too) now go to the pod the same way; a LoRA that cannot be fetched is said and the render goes on.

Sending to a pod that is busy, Override in remote mode, Civitai filters (2.766.1)

- **"The pod's ComfyUI is not answering yet":** bitGen now finds the port the pod's ComfyUI really answers on (the one it knows, 8188, then the pod's own ports from RunPod) and remembers it. A pod that is busy for a moment (loading a model) is waited for up to 3 minutes - its bar says "waiting for the pod to answer"; a stopped pod is said at once (▶ Start it). When ComfyUI is busy but the pod's Jupyter answers, downloads go ahead anyway.
- **Override in remote mode:** Override style's checkpoint and LoRAs were checked on Settings' port instead of the pod's own port - they now go to the pod the render uses.
- **Civitai filters (Model manager ▸ Civitai):** All / Checkpoints / LoRAs (and other types: VAE, Embedding, ControlNet, Upscaler), the model family - every base model of a family (All Krea = Krea 2 + Flux.1 Krea, All Flux …) or just one, from Civitai's own list - Sort by (highest rated, most downloaded, most liked, newest) and the period (all time … today). A change searches again; your filters are kept.

Local or Remote in Basic Settings switches everything over (2.766.2)

- **Local** (Basic Settings ▸ GPU): local mode - the model lists and the folder filter show this computer's model folders again, and ComfyUI on this computer is checked: not running = started. Everything else works as before.
- **Remote** (with RunPod pods): remote mode - the ComfyUI bitGen started on this computer is stopped (one you started yourself is left open; bitGen says so), the model lists show the models in runpod/pods/remote_models.json, and the folder filter shows their model families (Krea, Flux, Pony …) instead of folders - pick one to see only that family. Model manager's tabs show the families too.
- What happened is said under the choice ("Local mode - starting ComfyUI on this computer", "Remote mode - ComfyUI on this computer was stopped"), and the page's lists change at once.

Files the pod's ComfyUI can see, and an old ComfyUI on the pod updated (2.766.3)

- **Downloaded but "not on it":** some RunPod templates keep their models outside the ComfyUI folder (extra_model_paths.yaml, --base-directory). bitGen now asks the pod's ComfyUI where it reads each kind of model from and puts files there; a file already downloaded to the old place is moved, not downloaded again.
- **An old ComfyUI on the pod:** when a style's workflow uses a node that is built into a newer ComfyUI (for example TextEncodeQwenImage21 for Qwen Image 2.1), the pod's ComfyUI is updated before the render - git pull, its requirements, then ComfyUI restarted. The render's pod note says each step. When the pod's ComfyUI is its main process, the update is done but the pod needs a restart (PODS ▸ My pods) - said. A ComfyUI that is not a git checkout is not changed - update it with ComfyUI-Manager.

A pod that still lacks a node stops the render first (2.766.4)

- When the pod's ComfyUI still lacks a node the style's workflow uses (its ComfyUI could not be updated, or a node pack bitGen does not know), the render now stops before it is sent, with the reason - instead of failing on the pod. The next render tries to make the pod ready again.
- Files just put on the pod are looked for again (up to 30 seconds) before bitGen says they are missing.

The LLM, Vision LLM and video style in remote mode (2.767.0)

- **Where the LLM runs:** Basic Settings ▸ GPU (Remote or Both) has two new rows - **LLM** and **Vision LLM** - each Local, OpenRouter or RunPod:

  – **Local** - llama-server on this computer, as before.

  – **OpenRouter** - OpenRouter's model; pick it right there from a searchable list (or in Settings ▸ Remote generation ▸ LLM ▸ OpenRouter, which has a search box too). **Reasoning** is on: the model thinks before it answers, and its thinking (reasoning_details) is given back to it unchanged on the next turn of a chat. Untick Reasoning there to turn it off.

  – **RunPod** - with Pods: your pod runs the same GGUF as your LLM / Vision dropdowns. The first time you press Enter to send a prompt, bitGen sets it up there and the prompt waits: it builds llama.cpp for the pod's GPU (once - kept in /workspace), downloads the GGUF (and the vision model's mmproj), adds a small route to the pod's ComfyUI (ComfyUI restarts once) and starts the server. The rows show how far it is. A GGUF bitGen has no link for goes once into your private Hugging Face repo. With Serverless: the RunPod vLLM endpoint (Settings ▸ Remote generation).
- **The video style:** when Remote is picked (and when you change the Video style), the picked video style's models, LoRAs and node packs are put on its pod in the background - a progress bar each - so the first video does not wait for them.
- In remote mode with the LLM on RunPod or OpenRouter, changing the LLM dropdown no longer loads it on this computer.

- **⬆ Update ComfyUI** (PODS ▸ My pods, on each pod): updates the pod's ComfyUI to its latest version now - git pull (a pinned or edited checkout goes to ComfyUI's own master), its requirements, ComfyUI restarted - and says how far. A failed automatic update is tried again on the next render and says why (with the end of the pod's log); after a successful one that still lacks a node, bitGen says to restart the pod.

Remote options in their docker, and the prompt docker's model lists in remote mode (2.768.0)

- **Basic Settings ▸ GPU** now holds only Local / Remote / Both, the line saying where each kind of job runs, and **☁ Remote generation docker**. Everything else for remote work is in that docker: Remote servers (RunPod / OpenRouter), RunPod Serverless / Pods, the Both boxes, the **LLM** and **Vision LLM** rows (Local / OpenRouter / RunPod) and RunPod. Picking Remote or Both before anything is set up opens the docker so you can pick RunPod ▸ Pods there.
- **The prompt docker's LLM and Vision lists in remote mode** list the remote host's models instead of the GGUFs on this computer:

  – **OpenRouter** (the default when nothing is picked) - OpenRouter's **free** models; the Vision list shows the free ones that take pictures. The list loads without a key, but sending a prompt needs your OpenRouter API key (free to make) - ☁ Remote generation ▸ ⚙ Settings ▸ Remote generation ▸ LLM ▸ OpenRouter.

  – **RunPod with Pods** - the GGUFs bitGen can put on your pod; the one you pick is set up there when you next send a prompt.

  – **RunPod Serverless** - the vLLM endpoint's model. Picking a model saves it; nothing is loaded on this computer. Switch back to Local and the lists show this computer's models again (Settings ▸ LLM always lists them).
- **⬆ Update ComfyUI** on a pod now installs ComfyUI's requirements with the same Python the pod's ComfyUI runs with, and says the real error if a step fails.

A new pod's ComfyUI is updated automatically; missing files are looked for (2.769.0)

- **A pod bitGen makes** (PODS ▸ Create pod, or the pod a style asks for) has its **ComfyUI updated once, automatically**, when it first loads - the same as ⬆ Update ComfyUI. A render sent to the new pod waits for it (the job's line says "updating ComfyUI on the new pod first"), and PODS ▸ My pods shows it on the pod. A ComfyUI that is already the latest is left running - no reinstall, no restart. To turn this off, set `"pod_comfy_auto": false` under `rgpu` in the config.
- **⬆ Update ComfyUI** installs ComfyUI's requirements once per version, and restarts ComfyUI only when its code is newer than the one running.
- **Files that do not show up after a download:** when the pod's ComfyUI still does not list a file, bitGen looks for it on the pod. A file it finds outside the folders ComfyUI reads is linked into the right folder. Otherwise the job's line says why: only partly downloaded (the download stopped), not anywhere on the pod, in the right folder but not listed, or the pod's ComfyUI did not answer its model or node lists.

Pods bitGen makes set themselves up (2.769.1)

- **A pod bitGen makes is remembered as bitGen's** - on this computer and on the pod itself (RunPod keeps the mark), so it is known again even after a reinstall. PODS ▸ My pods marks it .
- **It sets itself up once it loads** - no render or button needed: its ComfyUI is updated, the styles linked to it get their files and node packs, and if no pod is in use yet it becomes the one in use. Its line in My pods says how far it is. If bitGen is closed while the pod loads, it carries on when bitGen starts again.

- **⬆ Update ComfyUI** now updates the ComfyUI that is actually running (found by the port it answers on), not another copy on the pod, and restarts it. A failed automatic update is tried again (3 times at most).
- A pod job's error line now shows the whole reason, with what is known about missing files first.

Only your live pods in the "no pod yet" box (2.769.2)

- When you render with a style that has no pod, the box that asks which pod to use now lists **only the pods in My pods** - running or stopped. Pods that were terminated (or are being terminated) and presets whose pod is gone are no longer offered there; PODS ▸ Presets still makes a preset's pod again. If RunPod could not list your pods, the box says why.

LLM Local / Remote, OpenRouter lists and All (2.770.0)

- **☁ Remote generation docker:** LLM and Vision LLM are now just **Local** or **Remote**. Remote uses the platform picked in Settings ▸ Remote generation ▸ LLM - OpenRouter by default, or RunPod. Rendering on RunPod no longer moves the LLM there.
- **Settings ▸ Remote generation ▸ LLM ▸ OpenRouter:Show free** and **Show paid** decide which OpenRouter models the lists offer (paid ones show their price per million tokens).
- **Prompt docker, remote mode:** the LLM and Vision lists show **10 models at a time** - type in the box before each list to search (name, maker, free, paid, vision). **All** opens every OpenRouter model in a dialog: search, Free / Paid / Vision filters, sort by name, price, context or newest, and each model's details. Pick one, choose LLM or Vision LLM and press **Use** - the list switches to it and it is the model used.
- **A pod's ComfyUI that lists no nodes** (usually newer code without the packages it needs) is repaired before a render: its requirements are installed and it is restarted. A pod's reason is written one part per line, so nothing is cut off.

A pod whose ComfyUI crashed (2.770.1)

- **ComfyUI not running on the pod:** a web page on ComfyUI's port (a template's placeholder while ComfyUI is down) no longer counts as "ComfyUI answers". When bitGen finds no ComfyUI running, it starts it from its folder with its own Python; if it does not come up, the job's line gives the end of its log (or which program holds its port - then restart the pod).
- **Transfers in the PODS docker:** a file already on the pod gets no progress bar, and a file sent again replaces its old row.

Costs docker and Statistics (2.771.0)

- **Costs** (the RunPod docker, renamed): **All costs** lists what everything cost - **This computer** (the energy while the app ran + the daily supply charge, at Settings ▸ General ▸ Energy cost), **☁ RunPod** (your pods, from RunPod's own bill, + Serverless endpoints), **OpenRouter** (the LLM and vision LLM), each **Settings ▸ API** service you use and a **remote GPU** with a $/hour - each on its own row, with the **Total** in your energy bill's currency. Pick the days at the top: **Today**, **7 days**, **This month**, **Last month** or **Everything**.
- **Click a row** (or Enter on it) for that platform's **detailed list**; **← All costs** goes back:
  - – This computer: energy and supply, day by day, and every job it ran (minutes, kWh, cost).
  - – RunPod: the balance, the pods (today / this month), **By day** (pods + Serverless) and RunPod's GPU prices - as before.
  - – OpenRouter: OpenRouter's own figures for your key (today, this week, this month, since the key was made, the limit left) and every call bitGen made, by model and one by one (tokens in / out, the cost OpenRouter reported; "-" = no price said).
  - – An API service: its renders (when, what, how long, the price the service gave) and its LLM calls.
  - – Remote GPU: day by day, hours and cost.

- **Open it** with Costs in the ☁ Remote generation docker or the PODS docker (straight to RunPod's list), **Costs…** in Settings ▸ General ▸ Energy cost, or the beside "⚡ since start" in the System stats. Picking Local no longer closes it.
- **Statistics** (in the Costs docker, or **Statistics…** in Settings ▸ General ▸ Energy cost) opens a dialog with **every generation on its own**: its **startup** (from the head of the queue until ComfyUI is handed the work - a pod started or made ready, files sent or downloaded, node packs installed, ComfyUI updated or restarted, the workflow built), its **generation** (the render itself; an online service: its whole run) and the **total**, plus the time it waited behind other jobs. The cards on top give the counts and the averages.
  - A pod render that waited for its pod shows ⚙ - click it for that **pod preparation** and its phases (downloads, node packs, restart …).
  - **Pod preparations** lists every time a pod was started or made ready: how long, each phase, the styles, any problems.
  - **By place** (this computer, RunPod pods, an API …) and **By mode** give the averages for each. **CSV** saves the list.
  - Kept on this computer only (the usage folder), from 2.771.0 on - earlier renders are not in it.

ComfyUI installed on a blank pod, and what a pod is doing (2.772.0)

- **New pods start blank.** A pod bitGen makes for a style is now made from RunPod's own **PyTorch** template (Python, PyTorch, CUDA and Jupyter - no ComfyUI), and bitGen installs ComfyUI on it itself. RunPod's ready-made ComfyUI templates keep their own ComfyUI copy in their own state (too old for a style, missing its nodes, crashed); on a blank pod bitGen knows exactly what is there. bitGen picks a PyTorch template with CUDA 12.8 (it runs on RTX 50-series and older cards) and the pod lands on a host whose driver runs it. RunPod's ComfyUI CUDA 13 template is used only when RunPod lists no PyTorch one.
- **Any template without ComfyUI works the same way:** pick one in the PODS docker (All, Official …) and press Create pod… - the summary says **ComfyUI: bitGen installs it on the pod**, and the pod gets port **8188** opened for it. A ComfyUI template keeps its own ComfyUI, as before.
- **What the install does,** on the pod, step by step (each one said in the PODS docker):
  - **Finding** what the pod has - Python, PyTorch and its CUDA, the GPU, the free space (git is installed if missing).
  - **Downloading** ComfyUI from GitHub into /workspace/ComfyUI - /workspace is kept while the pod is stopped.
  - **Creating** ComfyUI's own Python environment that **reuses the pod's PyTorch** (no second download of several GB).
  - **Installing** what ComfyUI needs (its requirements) and ComfyUI-Manager.
  - **Starting** ComfyUI on port 8188 and **Loading** it until it answers. The first time takes a few minutes. Then the pod is set up as any pod bitGen makes - its styles' files and node packs go on it, and it becomes the pod in use if none is. It is not updated again straight away (it is the newest ComfyUI already).
- **After the pod is stopped and started,** the same steps run again, but everything already there is skipped - only ComfyUI is started (a blank template does not start it by itself). A render waiting for the pod waits for this too.
- **A step that fails** stops the install with the reason in words (git's or pip's own error line, or the end of ComfyUI's log), shown on the pod and in the render's line. Statistics lists each install as a pod preparation ("ComfyUI installed / started").
- **What a pod is doing - on the pod itself.** In PODS ▸ My pods, under each pod's name (open or closed), one line says what is being done **now**: its verb - **⬇ Downloading**, **Installing**, **Loading**, **Finding**, **⇪ Sending**, **▶ Starting**, **Restarting**, **⬆ Updating**, **Linking**, **Checking**, **Preparing** … - what, and how long so far. It covers the ComfyUI install, a new pod's setup, ⬆ Update ComfyUI, making styles ready (files, node packs, downloads) and a render waiting for its pod.

  - The **finished tasks** go into a dropdown under it - ✓ **7 done · 2 already there · ✗ 1 failed** - closed, so the pod stays small. Open it for the list, newest first: ✓ done, – already there, ✗ failed, each with how long it took. It stays open or closed as you leave it while the list updates.
  - When nothing is running the line gives the result: ✓ **Done** (ready) or ✗ **Stopped** with the reason, in red.
  - The list is kept while bitGen runs (60 tasks a pod at most).
- **A closed pod item is really closed now** - before, its buttons and details showed even with ▸.

Remote mode: the pod's stats, one mode's styles, uploads that finish (2.773.0)

- **The left docker's System stats show the pod in remote mode.** With Basic Settings ▸ GPU on **Remote**, RunPod ▸ Pods and a pod in use, the System heading says **System · ☁ <your pod>** and the charts are the pod's: its CPU load, RAM, each GPU (load, temperature, VRAM) and its disks as bars - **Disk /** (the container disk, wiped when the pod stops) and **Disk /workspace** (kept while it is stopped). The queue, the progress bar and ⚡ since start stay this computer's (that is your electricity). On Local the charts are this computer's again. In remote mode with no pod in use (or Serverless / OpenRouter) the heading says **System · this computer** - hover it for why.
- **Only the styles of the mode you are in go on a pod.** In an image mode (Image, ⚙ Graph) bitGen uploads and checks only your **image** styles - the Image style you picked is made ready on its pod; in Video (or Music) only the **video** styles. A pod setting itself up, Save in My pods and a preset do the same: the other mode's styles are said on the pod ("left for Video mode: ...") and made ready when you switch to that mode or render one of them. Speech and Chat make no style ready. The ☁ Remote generation docker's line shows the style of your mode.
- **A LoRA sent from this computer no longer stops at the end with "[Errno 2] No such file".** The file arrived whole, but on some pod templates Jupyter keeps the upload in another folder than the one bitGen looked in to move it into ComfyUI's models folder. bitGen now finds it where Jupyter put it; if it really cannot, it says so in words. Each upload's part file has its own name, so a leftover from an older upload is never taken for it.

A render that failed on its pod goes again; a template's ComfyUI waited for (2.774.0)

- **A render that failed because its pod could not be used goes again by itself** once that pod is ready. That covers a setup that stopped, files not on the pod yet, or a node its ComfyUI still lacked. The render is held, and the Queue says **failed · goes again when its pod is ready**. It goes again when something new is done to that pod:
  - its setup finishes;
  - ComfyUI is installed or updated on it;
  - its styles are made ready;
  - a file you send from this computer lands on it. bitGen first waits until nothing else is running on the pod. The new render has the same settings and its log says why it went again; the old one then says **failed · went again**.
  - At most 3 tries per render, within 12 hours. After that, press Generate when the pod is fixed.
  - Clear the failed render from the Queue and it is not tried again.
  - It is kept while bitGen runs (not over a restart).
- **(2.773.1) A pod's ComfyUI is waited for before a render uses it.** A ComfyUI template still setting itself up is waited for, and the pod says "its ComfyUI is being set up by the template". A pod whose RunPod image has no ComfyUI gets bitGen's install even when this computer did not make it.
- **Step-by-step tutorials** for setting up RunPod (sign up, an **All** API key, a pod made by bitGen) and OpenRouter (sign up, a key, the LLM on it) are on tigerart.com.au/tutorials.php.

Text to speech, speech to text and song separation on a pod (2.775.0)

On a mini PC or a small graphics card, speech, transcripts and stems are slow or do not run. They can now run on your RunPod pod, like pictures, video and songs.

- **Tick them when you make the pod.** PODS ▸ Create pod… has an **Audio** row:
  - **Text to speech** - OmniVoice: speech, your cloned and designed voices, Speech & dialog;
  - **Speech to text** - faster-whisper: transcripts, lyric timing, a new clone's words, Create's lyrics;
  - **Song separation** - demucs: ✂ Split into layers, the Stems docker, the sheet music's song layers. bitGen installs the ticked ones on the pod when it is set up, under /workspace/bitgen-audio. That folder is kept while the pod is stopped. They reuse the pod's PyTorch, so it is a few minutes the first time.
- **Or tick them on a pod you have.** Open the pod in PODS ▸ My pods. The **Audio** row is under "what it is for". A tick is saved at once and installed now if the pod is running. A stopped pod is never started for it: the engine is installed when the pod next runs.
- **Where the jobs run.** In remote mode (Basic Settings ▸ GPU **Remote** or **Both**, RunPod ▸ **Pods**), each of these jobs goes to the pod ticked for it - the pod in use first. On **Local** they run on this computer, as before. Nothing needs installing on this computer for the ones a pod takes: Settings ▸ Audio and the install box count the pod's engine as installed.
- **Your voices stay on this computer.** A cloned voice (its clip and its words) stays in the project's voice store, where you made it. Its clip goes to the pod only when you speak with that voice, and only once - it is kept there under a name made from its contents. The speech comes back to the project's Sound folder, as before.
- **How it works.** The job's files go up through the pod's Jupyter. bitGen's own helper runs there with the same job. The results come back through the pod's ComfyUI, to where this computer would have saved them. The pod's activity line says what is being done, and the job's log carries the pod's own lines.
- **YuE2 (its own program) no longer asks for a pod.** It renders on this computer in every mode; before, remote mode asked for (or started) a pod and then rendered here anyway. **YuE2 ComfyUI**, ACE-Step 1.5 and MiniMax Music3 render on the pod.
- Ming-3.0 TTS and Bark STT stay on this computer.

Every type Local or Remote, in sections (2.776.0)

- **The ☁ Remote generation docker is in sections:**
  - **☁ Remote servers** - RunPod or OpenRouter, Serverless or Pods;
  - **Generation** - **Image**, **Video** and **Music**;
  - **Language models** - **LLM** and **Vision LLM**;
  - **Audio** - **Text to speech**, **Speech to text** and **Song separation**. Each type is **Local** (this computer) or **Remote**, in one column, so you see at a glance what runs where. A line under a Remote type says where it goes: its style's pod, the remote GPU, or for audio the pod that has it.
- **Music is its own type now.** Songs used to follow Video; now they can stay on this computer while video renders on a pod, or the other way round.
- **Audio on Remote** runs on the pod ticked for it (PODS ▸ Audio). If no pod has it yet, the pod in use gets the tick and the install. On RunPod Serverless, audio stays on this computer (it needs a pod).
- The first time you change a type here, the others keep what they were doing. Picking **Remote** in Basic Settings ▸ GPU still means "everything remote" until you set a type on its own.
- **The look is the theme's.** Each section's background, border, corners and heading colour come from the theme (`--rt-sec-bg`, `--rt-sec-border`, `--rt-sec-radius`, `--rt-sec-head`). edit theme mode styles a section like any other element.

Use model and Use temporary LoRAs (2.777.0)

With **Override style** on, Basic Settings has two new ticks for pictures and two for video:

- **Use model** sits above the Checkpoint (pictures) and the Model (video). Ticked, the model you picked replaces the style's. Unticked, the style's own model renders, and Override's other settings (size, steps, CFG, seed) still apply. Your pick is kept for next time.
- **Use temporary LoRAs** sits above each Temporary LoRAs list. Ticked, those LoRAs are added to whichever model renders, the style's or the override. Unticked, none are added, and the list is kept.

So one style can render with its own model plus your LoRAs, with another model and no LoRAs, or with both. Both ticks are on by default, which works as before. An unticked field is dimmed.

The pod docker no longer has the **⬆ Nodes its styles need** and **⬆ Update ComfyUI** buttons. bitGen does both when it sets up a pod, and the pod's activity line shows how far it has got.

A file that did not get onto the pod: Try again? (2.778.0)

When bitGen makes a style ready on a RunPod pod and a file it needs still isn't on the pod at the end, a box asks whether to try again. That covers three cases: a file sent from this computer whose upload failed, a download on the pod that failed, or a file found nowhere. The box lists each file, says how it failed and why, and names the styles that need it. There is one box per pod.

- **↻ Try again** makes those styles ready on the pod again. Files already there are skipped. When the files arrive, the render that was waiting for the pod goes again by itself.
- **Not now** closes the box. It isn't asked again unless the file fails again.

Before a file is called missing, bitGen also looks for it online by its name: first your remote models list (`runpod/pods/remote_models.json`), then the model's Civitai info card, then a Civitai search for that exact file name. If it's found, the pod downloads it from Civitai. A mature model needs your Civitai API key (Settings ▸ General ▸ Civitai).

When a pod cannot be created (2.779.0)

If RunPod won't create a pod, a dialog says why:

- a title, for example **The GPU you picked is busy**;
- what to do (another GPU, the other cloud, fewer GPUs, or try again in a few minutes - nothing was charged);
- RunPod's own words, with its HTTP status;
- what was asked for: the template, GPU, cloud, host and disks.

Copy copies all of it, for a bug report. It comes up from PODS ▸ Create pod, Presets ▸ ▶ and the box that asks for a pod when a style has none.

Fixed: creating a pod from one of RunPod's own templates (such as its PyTorch template, which bitGen uses for a style's pod) said "pick a template first". bitGen's own check refused RunPod's template ids, so RunPod was never asked.

A pod is a bitGen pod (2.780.0)

The PODS docker no longer lists RunPod's templates. It has three tabs:

- **My pods** lists the pods bitGen made or started. Any other pods on your RunPod account sit in one line under them, showing how many there are and what the running ones cost an hour. Open that line to see each one, with Stop on any that is running, so nothing runs unseen. They still count in Costs.
- **Presets** is as before. A preset's template is shown but can't be changed.

- **Create pod** creates a bitGen pod. Pick the GPU, cloud, disks, styles, what it is for and audio. One line says what it's made from: RunPod's PyTorch template, which bitGen installs ComfyUI, the custom nodes your styles use and their files on. That takes a few minutes the first time; after that it's kept in /workspace. If RunPod stops listing that template, bitGen uses its ComfyUI CUDA 13 one.

A render waiting for a pod opens Create pod, filled in for its style.

Tips

- **Tags beat descriptions** — `@anna` in a prompt brings Anna's sheet and voice along automatically.
- **Cont** keeps a character across a cut much better than **First** when the frame you start from does not show a face.
- **Free VRAM before every generation** (Settings → ComfyUI) avoids most out-of-memory failures when you mix images, music and video.
- Hover anything — every button has a tooltip. Turn on *Expert mode* in Settings → General to hide the help paragraphs once you know your way around.
- A failed render always says why on its scene card and in the Queue log.

<!-- credits:start -->

Credits

Produced and Developed by Tim Paul

Programmer - Claude Opus 5.5

bitGen Studio - a local AI video, image, music and editing studio. Everything runs on your own computer: no sign-ups, no cloud.

bitGen Studio stands on the work of many people. Thank you to everyone below - the projects whose code, models, ideas and tools we use or learned from.

Generation engines

- **ComfyUI** (Comfy Org and comfyanonymous) - the generation engine - every image, video and music style runs as a ComfyUI graph; its own frontend's graph conversion is mirrored for workflows
- **comfy-kitchen** (Comfy Org) - the int8 ConvRot weight layout, re-implemented in the native engine (Apache-2.0)
- **llama.cpp** (ggml-org) - the local LLM (llama-server) behind Enhance, the prompt chat, screenplays and the vision captioner
- **stable-diffusion.cpp** (leejet) - the first image and video backend (sd-cli)
- **Hugging Face diffusers** (Hugging Face) - the MiniMax H3 weight conversion and modular pipeline the native engine follows (Apache-2.0)
- **Hugging Face Hub** (Hugging Face) - where the model files come from
- **FFmpeg** - compiling, exporting, proxies and every audio / video conversion

Video models

- **MiniMax H3** (MiniMax) - the main video model - text, references, first / last frame, native audio
- **Viggle-Animate** (Viggle AI) - character animation; its inference code (Apache-2.0) guided the native engine
- **LTX-Video** (Lightricks) - the LTX video styles and the official templates
- **Wan** (the Wan team) - the Wan video styles, camera paths (ATI) and VR worlds
- **SeedVR2** (ByteDance Seed) - upscaling takes
- **FastH3 Omni** - the FastH3 recipe for MiniMax H3 graphs
- **Veda** - the optional sparse-attention accelerator for MiniMax H3

- **MiniMax-H3-NativeAudio-MusicVideo-Workflow** (Shrek3OnVH5) - the music-video lip-sync workflow the MMH3_NativeAudio_MV style is built from (MIT)

Image models

- **Qwen Image and Qwen Image Edit** (the Qwen team, Alibaba) - picture styles, ✏ Edit, fills and the Sketcher's real photo
- **FLUX.2 and FLUX 3 Klein** (Black Forest Labs) - picture styles, edits and LoRA training bases
- **Krea 2** (Krea) - picture styles (Turbo and Raw)
- **FIBO 2.0** (Bria) - the Fibo 2.0 Edit styles
- **Anima, MOSS-Image, SDXL and Pony models** - more picture styles
- **SAM 2 (Segment Anything)** (Meta) - Select Subject, cut-outs through a shot
- **Hunyuan3D 2.1** (Tencent) - picture to 3D (under its own community licence)
- **TRELLIS.2** (Microsoft) - picture to 3D
- **cadrille** (col14m) - Generate in SolidCraft writes part code from a description or a picture (weights CC BY-NC 4.0)
- **CadQuery** - runs that part code

Language, speech and music

- **Qwen3, Qwen3.5, Qwen3.8 and Qwen3-VL** (the Qwen team, Alibaba) - the recommended local LLMs, the vision captioner and fine-tune bases
- **Gemma 4** (Google) - local LLMs and fine-tune bases
- **WeVisOne-VL** - the vision add-on
- **faster-whisper** (SYSTRAN) - transcription, karaoke words, edit by transcript, voice chat
- **Whisper** (OpenAI) - the speech-recognition models faster-whisper runs
- **OmniVoice** - text-to-speech and voice cloning
- **OpenAudio / fish-speech** (Fish Audio) - the first text-to-speech engine
- **Ming-3.0 and Bark** - the speech add-ons
- **YuE** - song generation (YuE2), as its own engine and in ComfyUI
- **Demucs** (Meta) - stem separation (vocals / instrumental)

Training

- **ai-toolkit** (Ostris) - the LoRA and checkpoint trainer
- **Unsloth** - LLM and vision fine-tunes

ComfyUI node packs

- **ComfyUI-KJNodes** (Kijai) - the MiniMax H3 long-video nodes and many helpers
- **ComfyUI-GGUF** (city96) - GGUF model loading
- **ComfyUI-VideoHelperSuite** (Kosinkadink) - loading and saving video and audio in graphs
- **comfyui-inpaint-nodes** (Acly) - the image editor's selection fill
- **comfyui_controlnet_aux** (Fannovel16) - ControlNet preprocessors
- **ComfyUI-MiniMaxH3Mod, Spectrum-MiniMax-H3, ComfyUI-H3-NativeAudioLock** - character RefMods, MiniMax H3 speed-ups, the song lock for music videos
- **Nougan Nodes, RH DLSS 5, SolarRenderer** - regional character LoRAs and Prompt Relay, ⚡ DLSS 5, ☀ solar rendering

Built into the app

- **three.js** - every 3D view (MIT)

- **opentype.js** - 3D text (MIT)
- **pdf.js** (Mozilla) - opening PDFs in the image editor (Apache-2.0)
- **pdf-lib and fontkit** - saving PDFs with embedded fonts (MIT)
- **ag-psd** (Agamnentzar) - PSD files and their text effects (MIT)
- **polygon-clipping** - the Sketcher's walls (MIT; with splaytree and robust-predicates)
- **replicad and OpenCascade** - parametric 3D bodies (MIT; OpenCascade LGPL-2.1)
- **jxl-oxide, libjxl and @jsquash/jxl** - JPEG XL open and save (MIT / Apache-2.0 / BSD-3-Clause)
- **Node.js, Express and Multer** - the app's server
- **Electron** - the desktop app
- **Python, NumPy, OpenCV, soundfile and pyaaf2** - the workers, tracking, beats, sound and AAF export
- **OpenTimelineIO** - timeline exchange (OTIO, FCP XML, EDL)
- **uv** (Astral) - the setup's Python
- **ReportLab, pypdf, jsdom and Playwright** - the Guide PDFs and the test suite

3D models

- **Female mannequin model** (dimasalcalde (CGTrader)) - the Sketcher's mannequin (model) - People in the Furniture tool
- **Female mannequin model** (zahers (CGTrader)) - the Sketcher's mannequin (model) - People in the Furniture tool

Text effects and fonts

- **Dandyson EEbu (Baridakara)** - text effects and fonts

Programs bitGen Studio works with

- **Blender** (the Blender Foundation) - the Blender add-on: scenes and renders both ways, 3D layers, poses
- **Unreal Engine and MetaHuman** (Epic Games) - the bitGen Bridge plugin: Unreal layers, cameras, lights, the live view
- **Daz Studio** (Daz 3D) - the Daz bridge and the live link
- **SolidCraft** (Tangency Labs) - open-source parametric CAD in its own docker
- **Civitai** - model details, examples, search and download
- **RunPod** - rendering on a remote GPU
- **DaVinci Resolve and Final Cut Pro** - timelines out and back in

Ideas and inspiration

- **Krita AI Diffusion** (Acly) - the image editor's selection Generate / Fill / Remove follows its technique per model family
- **Wan2GP and mmgp** (deepbeepmeep) - the idea of streaming model blocks to the GPU (no code used)
- **The Sketcher's furniture library** - the furniture, people and materials - first made for the House builder of earlier versions
- **The professional creative tools** - the photo editors, vector and drawing programs, video editors, motion-graphics tools, 3D modellers and parametric CAD programs whose ways of working the image editor, the timeline, the CAD tools and the Sketcher follow
- **Everyone who tests bitGen Studio** - and reports what to fix

bitGen Studio is MIT licensed. Each project above keeps its own licence - misc/NOTICE.md lists what ships, what is downloaded, and the model licences. Model weights are downloaded by you and carry

their makers' terms. <!-- credits:end -->